

FPGA 与 VHDL 期末考试

——20 套参考答案与评分标准

2026 年 6 月 16 日

目录

第一章 第 1 套试卷——参考答案与评分标准	1
1.1 单项选择题（共 6 小题，每小题 3 分，共 18 分）	1
1.2 填空题（共 4 小题，每小题 3 分，共 12 分）	3
1.3 程序改错题（共 5 分）	4
1.4 程序阅读分析题（共 5 分）	6
1.5 设计题（共 60 分）	8
1.5.1 设计题 1: 4 位比较器（20 分）	8
1.5.2 设计题 2: 模 10 同步计数器（20 分）	10
1.5.3 设计题 3: Moore 型“101”序列检测器（20 分）	12
第二章 第 2 套试卷——参考答案与评分标准	19
2.1 一、选择题（每题 3 分，共 18 分）	19
2.2 二、填空题（每题 3 分，共 12 分）	20
2.3 三、程序改错题（5 分）	22
2.4 四、程序阅读分析题（5 分）	24
2.5 五、综合设计题（共 60 分）	26
2.5.1 设计题 1: 组合逻辑设计——8 位数值比较器（20 分）	26
2.5.2 设计题 2: 时序逻辑设计——模 6 计数器（20 分）	29
2.5.3 设计题 3: 状态机设计——“101”序列检测器（Mealy 型）（20 分）	32
第三章 第 3 套试卷——参考答案与评分标准	39
3.1 一、选择题（每题 3 分，共 18 分）	39
3.2 二、填空题（每题 3 分，共 12 分）	42
3.3 三、程序改错题（共 5 分）	44
3.4 四、程序阅读分析题（共 5 分）	46
3.5 五、综合设计题（共 60 分）	48
3.5.1 设计题 1: 组合逻辑设计——8 线-3 线优先编码器（20 分）	48
3.5.2 设计题 2: 时序逻辑设计——模 10 BCD 计数器（20 分）	52
3.5.3 设计题 3: 状态机设计——Moore 型“101”序列检测器（20 分）	56

第四章 第 4 套试卷——参考答案与评分标准	63
第五章 第 5 套试卷——参考答案与评分标准	83
5.1 一、选择题答案 (18 分)	83
5.2 二、填空题答案 (12 分)	84
5.3 三、程序改错题解析 (5 分)	84
5.4 四、程序阅读分析题解析 (5 分)	86
5.5 五、综合设计题参考答案 (60 分)	86
5.5.1 设计题 1: 8 线-3 线优先编码器设计 (20 分)	86
5.5.2 设计题 2: 模 60 计数器设计 (20 分)	89
5.5.3 设计题 3: “101” 序列检测器 (Moore 状态机) (20 分)	93
第六章 第 6 套试卷——参考答案与评分标准	97
6.1 一、选择题答案 (18 分)	97
6.2 二、填空题答案 (12 分)	98
6.3 三、程序改错题解析 (5 分)	98
6.4 四、程序阅读分析题解析 (5 分)	100
6.5 五、综合设计题答案 (60 分)	100
6.5.1 设计题 1: BCD 码-七段数码管译码器 (20 分)	100
6.5.2 设计题 2: 4 位双向移位寄存器 (20 分)	102
6.5.3 设计题 3: “1011” 序列检测器 (Moore 型和 Mealy 型对比) (20 分)	105
第七章 第 7 套试卷——参考答案与评分标准	111
7.1 一、选择题答案 (18 分)	111
7.2 二、填空题答案 (12 分)	112
7.3 三、程序改错题解析 (5 分)	112
7.4 四、程序阅读分析题解析 (5 分)	114
7.5 五、综合设计题参考答案 (60 分)	115
7.5.1 设计题 1: BCD 码转 7 段数码管译码器 (20 分)	115
7.5.2 设计题 2: 4 位双向移位寄存器 (结构化设计) (20 分)	117
7.5.3 设计题 3: “1011” 序列检测器 Moore 状态机 (20 分)	120
第八章 第 8 套试卷——参考答案与评分标准	125
8.1 一、选择题答案 (18 分)	125
8.2 二、填空题答案 (12 分)	126
8.3 三、程序改错题解析 (5 分)	127
8.4 四、程序阅读分析题解析 (5 分)	128
8.5 五、综合设计题参考答案 (60 分)	129

8.5.1	设计题 1: 8 位算术逻辑单元 (ALU) (20 分)	129
8.5.2	设计题 2: N 级二分频器链 (20 分)	132
8.5.3	设计题 3: 数字密码锁控制器 (20 分)	136
第九章	第 9 套试卷——参考答案与评分标准	141
9.1	一、选择题答案 (18 分)	141
9.2	二、填空题答案 (12 分)	142
9.3	三、程序改错题解析 (5 分)	143
9.4	四、程序阅读分析题解析 (5 分)	144
9.5	五、综合设计题参考答案 (60 分)	145
第十章	第 10 套试卷——参考答案与评分标准	159
10.1	一、选择题答案 (18 分)	159
10.2	二、填空题答案 (12 分)	160
10.3	三、程序改错题解析 (5 分)	160
10.4	四、程序阅读分析题解析 (5 分)	162
10.5	五、综合设计题参考答案 (60 分)	163
第十一章	第 11 套试卷——参考答案与评分标准	177
11.1	一、选择题答案 (18 分)	177
11.2	二、填空题答案 (12 分)	178
11.3	三、程序改错题解析 (5 分)	179
11.4	四、程序阅读分析题解析 (5 分)	180
11.5	五、综合设计题参考答案 (60 分)	181
第十二章	第 12 套试卷——参考答案与评分标准	193
12.1	一、选择题答案 (18 分)	193
12.2	二、填空题答案 (12 分)	194
12.3	三、程序改错题解析 (5 分)	195
12.4	四、程序阅读分析题解析 (5 分)	197
12.5	五、综合设计题参考答案 (60 分)	198
第十三章	第 13 套试卷——参考答案与评分标准	211
13.1	一、选择题答案 (18 分)	211
13.2	二、填空题答案 (12 分)	212
13.3	三、程序改错题解析 (5 分)	212
13.4	四、程序阅读分析题解析 (5 分)	214
13.5	五、综合设计题参考答案 (60 分)	215

13.5.1 设计题 1: BCD 码至 7 段数码管译码器 (20 分)	215
13.5.2 设计题 2: N 位双向移位寄存器 (20 分)	217
13.5.3 设计题 3: “1011” 序列检测器 (20 分)	220
第十四章 第 14 套试卷——参考答案与评分标准	227
14.1 一、选择题答案 (18 分)	227
14.2 二、填空题答案 (12 分)	228
14.3 三、程序改错题解析 (5 分)	229
14.4 四、程序阅读分析题解析 (5 分)	230
14.5 五、综合设计题参考答案 (60 分)	232
14.5.1 设计题 1: BCD 码转 7 段数码管译码器 (20 分)	232
14.5.2 设计题 2: 4 位双向移位寄存器 (20 分)	233
14.5.3 设计题 3: 自动售货机控制器——Mealy 型 (20 分)	235
第十五章 第 15 套试卷——参考答案与评分标准	241
15.1 一、选择题答案 (18 分)	241
15.2 二、填空题答案 (12 分)	242
15.3 三、程序改错题解析 (5 分)	242
15.4 四、程序阅读分析题解析 (5 分)	244
15.5 五、综合设计题参考答案 (60 分)	246
15.5.1 设计题 1: BCD 码—7 段数码管译码器 (20 分)	246
15.5.2 设计题 2: 使用 GENERATE 语句设计 N 位双向移位寄存器 (20 分)	248
15.5.3 设计题 3: “1011” 序列检测器——Moore 型与 Mealy 型对比 (20 分)	252
第十六章 第 16 套试卷——参考答案与评分标准	257
16.1 一、选择题答案 (18 分)	257
16.2 二、填空题答案 (12 分)	258
16.3 三、程序改错题解析 (5 分)	259
16.4 四、程序阅读分析题解析 (5 分)	261
16.5 五、综合设计题参考答案 (60 分)	264
16.5.1 设计题 1: N 位桶形移位器 (Barrel Shifter) 设计 (20 分)	264
16.5.2 设计题 2: 数字钟 (Digital Clock —HH:MM:SS) 设计 (20 分)	268
16.5.3 设计题 3: 交通灯控制器——Moore 型与 Mealy 型对比设计 (20 分)	273

17.5.3 设计题 3: SPI 从机通信控制器——Moore 型与 Mealy 型对比设计 (20 分)	288
17.6 本套试卷评分总表	301
第十八章 第 18 套试卷——参考答案与评分标准	303
第十九章 第 19 套试卷——参考答案与评分标准	343
19.1 一、选择题答案 (18 分)	343
19.2 二、填空题答案 (12 分)	344
19.3 三、程序改错题解析 (5 分)	346
19.4 四、程序阅读分析题解析 (5 分)	349
19.5 五、综合设计题参考答案 (60 分)	351
19.5.1 设计题 1: 8 位多功能桶形移位器与优先级编码器 (20 分)	351
19.5.2 设计题 2: 参数化可编程分频器与 Johnson 计数器 (20 分)	358
19.5.3 设计题 3: 双向交叉路口交通灯控制器 (20 分)	365
第二十章 第 20 套试卷——参考答案与评分标准	373
20.1 一、选择题 (共 6 小题, 每小题 3 分, 共 18 分)	373
20.2 二、填空题 (共 4 小题, 每小题 3 分, 共 12 分)	376
20.3 三、程序改错题 (共 5 分)	378
20.4 四、程序阅读分析题 (共 5 分)	383
20.5 五、综合设计题 (共 60 分)	385
20.5.1 设计题 1: 8 位桶形移位器 (Barrel Shifter) 设计 (20 分)	385
20.5.2 设计题 2: LFSR 可编程伪随机序列发生器 (20 分)	391
20.5.3 设计题 3: 双向十字路口交通灯控制器 (20 分)	397

第一章 第 1 套试卷——参考答案与评分标准

1.1 单项选择题（共 6 小题，每小题 3 分，共 18 分）

1. 答案：C

解析：GAL（Generic Array Logic）属于简单可编程逻辑器件（SPLD）。SPLD 主要包括 PLA、PAL 和 GAL 三种类型。FPGA 和 CPLD 属于复杂可编程逻辑器件（CPLD 广义上也可归类为高密度 SPLD，但题目中与 GAL 并列时，GAL 是典型的 SPLD），ASIC 是专用集成电路，不属于可编程逻辑器件。

- A. FPGA ——现场可编程门阵列，属于高密度可编程器件
- B. CPLD ——复杂可编程逻辑器件，密度高于 SPLD
- C. GAL ——通用阵列逻辑，是典型的 SPLD
- D. ASIC ——专用集成电路，不可编程

2. 答案：B

解析：FPGA 中实现组合逻辑的基本单元是查找表（LUT，Look-Up Table）。LUT 本质上是一个小容量 SRAM，通过预先存储真值表来实现任意组合逻辑函数。

- A. Flip-Flop ——用于实现时序逻辑，非组合逻辑
- B. LUT ——查找表，实现组合逻辑的基本单元
- C. Multiplier ——专用硬核乘法器，非基本单元
- D. PLL ——锁相环，用于时钟管理，与逻辑实现无关

3. 答案：B

解析：在 VHDL 中，实体（ENTITY）的端口定义使用关键字 PORT。PORT 后跟端口名称、方向和数据类型。

- A. ARCHITECTURE ——定义实体的实现（结构体）
- B. PORT ——定义实体端口的关键字
- C. SIGNAL ——声明内部信号的类型
- D. COMPONENT ——声明待例化的元件

4. 答案：C

解析：CPLD 与 FPGA 的核心区别在于：

- CPLD 基于乘积项（Product Term）结构，互连线是连续的（基于开关矩阵），因此布线延迟可预测
- FPGA 基于查找表（LUT）结构，互连线为分段式，延迟不可预测
- CPLD 逻辑密度通常低于 FPGA
- CPLD 更适合组合逻辑密集型电路，FPGA 更适合大规模时序电路

选项分析：

- A. CPLD 逻辑密度低于 FPGA，错误
- B. CPLD 基于乘积项而非 LUT，FPGA 基于 LUT 而非乘积项，描述颠倒，错误
- C. CPLD 互连为连续式、延迟可预测，正确
- D. CPLD 更适合组合逻辑，FPGA 更适合大规模时序电路，错误

5. 答案：C

解析：在 VHDL 的 PROCESS 中，SIGNAL 和 VARIABLE 的关键区别如下：

- A. 正确：SIGNAL 使用 $\leq$ （信号赋值），VARIABLE 使用 $:=$ （变量赋值）
- B. 正确：VARIABLE 只能在 PROCESS 内部定义和使用，其作用域仅限于所在 PROCESS
- C. 错误：SIGNAL 赋值在进程结束时（或遇到 WAIT 语句时）才生效，而非立即生效；VARIABLE 赋值则是立即生效。题干描述恰恰说反了
- D. 正确：SIGNAL 用于进程间通信，VARIABLE 用于进程内部临时计算

6. 答案：C

解析：一个 n -输入 LUT 最多可以实现 n 个变量的任意组合逻辑函数。4-输入 LUT 有 $2^4 = 16$ 个存储单元，可存储并实现最多 4 个变量的任意逻辑函数。

若变量数超过 4 个（如 5、6、7、8 个变量），则需要多个 LUT 级联实现。

1.2 填空题（共 4 小题，每小题 3 分，共 12 分）

1. 答案： IEEE, STD_LOGIC

解析：VHDL 标准逻辑库名称为 IEEE，其中最常用的数据类型是 STD_LOGIC（9 值逻辑类型，可综合）。另一个常用类型是 STD_LOGIC_VECTOR（总线类型）。

评分标准：每空 1.5 分。第一空写成 ieee（小写）亦给分；第二空必须为 STD_LOGIC（或 std_logic）。

2. 答案： <=, ON

解析：

- 并发信号赋值语句使用关键字 <=。注意：在结构体中直接使用（不在 PROCESS 内）的信号赋值即为并发赋值。
- PROCESS 的敏感信号列表关键字是 ON，语法为 WAIT ON 信号名，或在 PROCESS 后直接列出敏感信号：PROCESS(signal1, signal2)。

评分标准：每空 1.5 分。第一空写 <= 或写 WHEN...ELSE（并发条件赋值）亦可；第二空必须为 ON。

3. 答案： IOB（输入输出模块）, 互联资源（Interconnect/PI）

解析：FPGA 的基本结构由三部分组成：

- (a) CLB（Configurable Logic Block，可配置逻辑块）——实现逻辑功能
- (b) IOB（Input/Output Block，输入/输出模块）——与外部引脚接口
- (c) 互联资源（Interconnect / Programmable Interconnect，可编程互连）——连接各 CLB 和 IOB

评分标准：每空 1.5 分。答出 IOB/IO 模块（或输入输出块）即给分；答出互联资源/互连/PI/布线资源即给分。

4. 答案： /=, >=

解析：VHDL 关系运算符：

- 等于：=
- 不等于：/=
- 大于：>
- 小于：<

- 大于等于: >=
- 小于等于: <=

注意 <= 在 VHDL 中既是“小于等于”又是信号赋值符号，通过上下文区分：在条件表达式（IF、WHEN 等）中为关系运算符，在信号赋值语句中为赋值符。

评分标准：每空 1.5 分。必须为 /= 和 >=。

1.3 程序改错题（共 5 分）

原程序：

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY mux2to1 IS
5      PORT(
6          a, b : IN  STD_LOGIC;
7          sel  : IN  STD_LOGIC;
8          y    : OUT STD_LOGIC
9      );
10 END mux2to1;
11
12 ARCHITECTURE behavioral OF mux2to1 IS
13 BEGIN
14     PROCESS(a, b, sel)
15     BEGIN
16         IF sel = '0' THEN
17             y := a;
18         ELSE
19             y := b;
20         END IF;
21     END PROCESS;
22 END behavioral;

```

错误分析与修改

本题共 5 处错误（含中文分号），每处 1 分，共 5 分。**评分说明：**指出错误位置并给出正确写法即得分；若指出错误但未写正确写法，得 0.5 分。

错误 1：关键字拼写错误： ENTIFY → ENTITY

第 3 行：“ENTIFY mux2to1 IS” 应为 “ENTITY mux2to1 IS”。

解析：ENTITY 是 VHDL 中声明实体（设计单元接口）的关键字，不可拼错。

错误 2：数据类型拼写错误： STD_LOGIG → STD_LOGIC

第 5 行：“STD_LOGIG” 应为 “STD_LOGIC”。

解析：IEEE 标准库 1164 中定义的逻辑类型为 STD_LOGIC。

错误 3：关键字拼写错误： ARCHITE_CTURE → ARCHITECTURE

第 11 行：“ARCHITE_CTURE behavioral OF mux2to1 IS” 应为 “ARCHITECTURE behavioral OF mux2to1 IS”。

解析：ARCHITECTURE 是定义结构体（描述实体内部实现）的关键字。

错误 4：中文分号错误： 多处 ； → ；

第 1 行 ieee；、第 2 行 ALL；、第 5 行 STD_LOGIG；、第 8 行)；、第 9 行 ；、第 16 行 a；、第 18 行 b；——所有中文全角分号（；）均应改为英文半角分号（;）。

解析：VHDL 仅识别 ASCII 字符集中的半角标点。中文全角分号会导致语法解析失败。

错误 5：信号赋值操作符错误： := → <=

第 16 行 y := a； 和第 18 行 y := b； 均应改为 y <= a； 和 y <= b；。

解析：y 是 OUT 模式的端口，在结构体中表现为 SIGNAL 语义，必须使用信号赋值操作符 <=。:= 仅用于 VARIABLE（变量）赋值。

修正后的完整程序

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY mux2to1 IS
5      PORT(
6          a, b : IN  STD_LOGIC;
7          sel  : IN  STD_LOGIC;
8          y    : OUT STD_LOGIC
9      );
10 END mux2to1;
11

```

```
12 ARCHITECTURE behavioral OF mux2to1 IS
13 BEGIN
14     PROCESS(a, b, sel)
15     BEGIN
16         IF sel = '0' THEN
17             y <= a;
18         ELSE
19             y <= b;
20         END IF;
21     END PROCESS;
22 END behavioral;
```

1.4 程序阅读分析题（共 5 分）

程序内容

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY decoder38 IS
5      PORT(
6          a  : IN  STD_LOGIC_VECTOR(2 DOWNT0 0);
7          en : IN  STD_LOGIC;
8          y  : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)
9      );
10 END decoder38;
11
12 ARCHITECTURE behavior OF decoder38 IS
13 BEGIN
14     PROCESS(a, en)
15     BEGIN
16         IF en = '0' THEN
17             y <= (OTHERS => '1');
18         ELSE
19             CASE a IS
20                 WHEN "000" => y <= "11111110";
21                 WHEN "001" => y <= "11111101";
22                 WHEN "010" => y <= "11111011";
```

```
23      WHEN "011"  => y <= "11110111";
24      WHEN "100"  => y <= "11101111";
25      WHEN "101"  => y <= "11011111";
26      WHEN "110"  => y <= "10111111";
27      WHEN "111"  => y <= "01111111";
28      WHEN OTHERS => y <= (OTHERS => '1');
29  END CASE;
30  END IF;
31  END PROCESS;
32  END behavior;
```

分析与答案

(1) 电路类型：3 线-8 线译码器（3-to-8 decoder）。（2 分）

(2) 功能说明：（3 分）

- 输入 a 为 3 位二进制选择信号（地址），en 为高电平有效使能信号
- 当 en = '1' 时，根据输入 a 的值选中 8 个输出中的 1 路，该路输出为低电平（'0'），其余 7 路输出为高电平（'1'）——即输出低电平有效（active-low）
- 当 en = '0' 时，译码器禁止工作，所有 8 路输出均为高电平 '1'（全禁止状态）
- 例如：a = "000" 且 en = '1' 时，y(0) 输出 '0'（被选中），y(7 downto 1) 输出 '1'

评分标准：

- 答出“3-8 译码器”得 2 分
- 答出“使能高电平有效”得 1 分
- 答出“输出低电平有效”或“被选中的输出为 0，其余为 1”得 1 分
- 答出“en=0 时全输出高电平/禁止状态”得 1 分

1.5 设计题（共 60 分）

1.5.1 设计题 1：4 位比较器（20 分）

1. 端口图（5 分）



端口说明：

- a[3:0]：4 位输入 A（比较操作数 1）
- b[3:0]：4 位输入 B（比较操作数 2）
- equal：输出，当 a = b 时为 '1'
- greater：输出，当 a > b 时为 '1'
- less：输出，当 a < b 时为 '1'

评分标准（5 分）：

- 模块框绘制正确（1 分）
- 输入端 a[3:0]、b[3:0] 标注正确（1.5 分）
- 输出端 equal、greater、less 标注正确（1.5 分）
- 端口方向箭头正确（1 分）

2. 完整 VHDL 代码（15 分）

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;          -- 用于 unsigned 比较
4
5  ENTITY comp4 IS
6    PORT(
7      a      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
8      b      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
  
```



```

9      equal    : OUT STD_LOGIC;
10     greater  : OUT STD_LOGIC;
11     less     : OUT STD_LOGIC
12 );
13 END comp4;
14
15 ARCHITECTURE behavioral OF comp4 IS
16 BEGIN
17     PROCESS(a, b)
18     BEGIN
19         -- 默认值，避免推断锁存器
20         equal    <= '0';
21         greater  <= '0';
22         less     <= '0';
23
24         IF a = b THEN
25             equal <= '1';
26         ELSIF a > b THEN
27             greater <= '1';
28         ELSE
29             less <= '1';
30         END IF;
31     END PROCESS;
32 END behavioral;

```

评分标准（15 分）：

- 库声明正确，包含 ieee 和 std_logic_1164（1 分）
- 包含 numeric_std 库用于比较（1 分）（若未使用该库但用 unsigned(a)>unsigned(b) 转换方式也不扣分）
- ENTITY 声明正确，端口名称、方向、位宽正确（4 分）
- ARCHITECTURE 结构正确（1 分）
- PROCESS 敏感信号列表包含 a 和 b（1 分）
- 比较逻辑正确： $a=b \rightarrow equal = 1$ $a > b \rightarrow greater = 1$ $a < b \rightarrow less = 1$ 3
- 默认值赋值避免锁存器推断（1 分）
- 代码可综合，语法正确，结构完整（3 分）

补充说明：也可使用并发条件信号赋值（WHEN-ELSE）实现，同样给分：

```

1 ARCHITECTURE dataflow OF comp4 IS
2 BEGIN
3     equal    <= '1' WHEN a = b ELSE '0';
4     greater  <= '1' WHEN a > b ELSE '0';
5     less     <= '1' WHEN a < b ELSE '0';
6 END dataflow;

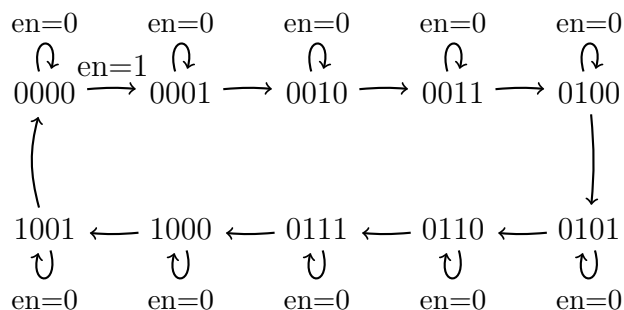
```

1.5.2 设计题 2：模 10 同步计数器（20 分）

设计需求回顾

- 端口：clk（时钟），rst（异步复位，低电平有效），en（使能），q[3:0]（计数输出）
- 模 10 计数：计数值 0 ~ 9，到 9 后回 0
- 同步计数：在时钟 clk 上升沿触发

1. 状态转移图（5 分）



简化版状态转移图（推荐答题版本）：

- en=1 时转移标注正确（1 分）
- en=0 时保持（自环）标注正确（1 分）
- 图面清晰，箭头、标注齐全（1 分）

2. 完整可综合 VHDL 代码（10 分）

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY mod10_counter IS
6      PORT(
7          clk : IN  STD_LOGIC;
8          rst : IN  STD_LOGIC;          -- 异步复位，低电平有效
9          en  : IN  STD_LOGIC;          -- 使能，高电平有效
10         q   : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
11     );
12 END mod10_counter;
13
14 ARCHITECTURE behavioral OF mod10_counter IS
15     SIGNAL count : UNSIGNED(3 DOWNTO 0);    -- 内部计数寄存器
16 BEGIN
17     -- 异步复位、同步计数进程
18     PROCESS(clk, rst)
19     BEGIN
20         IF rst = '0' THEN                    -- 异步复位，低电平有效
21             count <= (OTHERS => '0');
22         ELSIF rising_edge(clk) THEN          -- 时钟上升沿
23             IF en = '1' THEN                -- 使能有效
24                 IF count = 9 THEN            -- 计数值到9
25                     count <= (OTHERS => '0');    -- 回0
26                 ELSE
27                     count <= count + 1;        -- 递增
28                 END IF;
29             END IF;
30             -- en = '0' 时保持当前值 (count <= count 隐含)
31         END IF;
32     END PROCESS;

```

```

33
34   q <= STD_LOGIC_VECTOR(count);           -- 类型转换输出
35 END behavioral;

```

评分标准（10 分）：

- 库声明正确（ieee + numeric_std）（1 分）
- ENTITY 端口定义正确：clk、rst、en 输入，q[3:0] 输出（2 分）
- 内部信号 count 用 UNSIGNED 类型（或 INTEGER，或 STD_LOGIC_VECTOR + 手动逻辑）（1 分）
- 异步复位实现正确：IF rst='0' THEN 在 ELSIF rising_edge(clk) 之前（2 分）
- 使能逻辑正确：en=1 时才计数，en=0 时保持（1 分）
- 模 10 边界条件正确：count=9 时下一状态为 0（1 分）
- 输出类型转换正确（1 分）
- 代码可综合，无语法错误（1 分）

补充说明：若使用 INTEGER 类型实现计数，逻辑等同，也同样给分。示例：

```

1  SIGNAL count_int : INTEGER RANGE 0 TO 9;
2  ...
3  IF count_int = 9 THEN
4      count_int <= 0;
5  ELSE
6      count_int <= count_int + 1;
7  END IF;
8  q <= STD_LOGIC_VECTOR(TO_UNSIGNED(count_int, 4));

```

1.5.3 设计题 3：Moore 型”101”序列检测器（20 分）

设计需求回顾

- 输入：clk（时钟），rst（同步复位，高电平有效），x（串行输入）
- 输出：y（当检测到”101”序列时输出 1，否则输出 0）
- 类型：Moore 型状态机（输出仅取决于当前状态）
- 状态定义：

- S_0 : 初始状态（未匹配任何有效序列/刚复位）
 - S_1 : 已匹配 “1”
 - S_2 : 已匹配 “10”
 - S_3 : 已匹配 “101”（检测成功）
- 编码风格：2/3-进程描述方式

1. 状态转移图（8 分）

- 因此检测为“非重叠”模式 (non-overlapping): 检测到“101”后返回 S_0 重新开始。若需重叠检测 (overlapping), 则 S_3 遇到 $x = 0$ 时不是回到 S_2 而是另一个分支——但本题采用标准 Moore 实现, $S_3 \xrightarrow{x=0} S_2^1$ 可支持重叠检测。

2. 完整 VHDL 代码——3 进程 Moore 型 (12 分)

方案一: 3 进程描述 (推荐——最规范)

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY seq_detector_101 IS
5      PORT(
6          clk : IN  STD_LOGIC;
7          rst : IN  STD_LOGIC;           -- 同步复位, 高电平有效
8          x   : IN  STD_LOGIC;           -- 串行输入
9          y   : OUT STD_LOGIC            -- 检测输出 (Moore型)
10     );
11 END seq_detector_101;
12
13 ARCHITECTURE moore_3proc OF seq_detector_101 IS
14     -- 枚举类型定义状态
15     TYPE state_type IS (S0, S1, S2, S3);
16     SIGNAL current_state, next_state : state_type;
17
18 BEGIN
19     -- =====
20     -- 进程1: 状态寄存器 (同步时序逻辑)
21     -- =====
22     state_reg: PROCESS(clk)
23     BEGIN
24         IF rising_edge(clk) THEN
25             IF rst = '1' THEN
26                 current_state <= S0;           -- 同步复位到初始状态
27             ELSE
28                 current_state <= next_state;
29             END IF;
30         END IF;

```

¹注意: Moore 机中 S_3 到 S_2 的转移允许重叠检测, 如输入序列“10101”可检测出两次“101”。若要求非重叠检测, 则 $S_3 \xrightarrow{x=0} S_0$, 考生作答时两种方案均给分, 只需逻辑自洽。

```

31  END PROCESS state_reg;
32
33  -- =====
34  -- 进程2: 次态逻辑 (组合逻辑)
35  -- =====
36  next_state_logic: PROCESS(current_state, x)
37  BEGIN
38      -- 默认值, 避免推断锁存器
39      next_state <= S0;
40
41      CASE current_state IS
42          WHEN S0 =>
43              IF x = '1' THEN
44                  next_state <= S1;
45              ELSE
46                  next_state <= S0;
47              END IF;
48
49          WHEN S1 =>
50              IF x = '1' THEN
51                  next_state <= S1;           -- 连续收到 "1", 保持在 S1
52              ELSE
53                  next_state <= S2;           -- 收到 "0", 进入 S2 (匹配 "10")
54              END IF;
55
56          WHEN S2 =>
57              IF x = '1' THEN
58                  next_state <= S3;           -- 收到 "1", 进入 S3 (匹配 "101"
59                  )
60              ELSE
61                  next_state <= S0;           -- 收到 "00", 回到初始
62              END IF;
63
64          WHEN S3 =>
65              IF x = '1' THEN
66                  next_state <= S1;           -- 重叠检测: 新 "1" 可作为下一序
67                  列起点
68              ELSE
69                  next_state <= S2;           -- 收到 "0", 可视为 "10" 的前缀

```

```

68         END IF;
69
70         WHEN OTHERS =>
71             next_state <= S0;
72     END CASE;
73 END PROCESS next_state_logic;
74
75 -- =====
76 -- 进程3: 输出逻辑 (组合逻辑——Moore型特征)
77 -- =====
78 output_logic: PROCESS(current_state)
79 BEGIN
80     CASE current_state IS
81         WHEN S3 =>
82             y <= '1';                -- 仅S3输出1
83         WHEN OTHERS =>
84             y <= '0';                -- 其余状态输出0
85     END CASE;
86 END PROCESS output_logic;
87
88 END moore_3proc;

```

方案二：2 进程描述（次态逻辑与输出合并）

```

1 ARCHITECTURE moore_2proc OF seq_detector_101 IS
2     TYPE state_type IS (S0, S1, S2, S3);
3     SIGNAL state : state_type;
4
5 BEGIN
6     -- 进程1: 状态寄存器
7     state_reg: PROCESS(clk)
8     BEGIN
9         IF rising_edge(clk) THEN
10             IF rst = '1' THEN
11                 state <= S0;
12             ELSE
13                 CASE state IS
14                     WHEN S0 =>
15                         IF x = '1' THEN state <= S1; ELSE state <= S0; END IF;
16                     WHEN S1 =>

```



```

17         IF x = '1' THEN state <= S1; ELSE state <= S2; END IF;
18     WHEN S2 =>
19         IF x = '1' THEN state <= S3; ELSE state <= S0; END IF;
20     WHEN S3 =>
21         IF x = '1' THEN state <= S1; ELSE state <= S2; END IF;
22     WHEN OTHERS =>
23         state <= S0;
24     END CASE;
25     END IF;
26 END IF;
27 END PROCESS state_reg;
28
29 -- 进程 2: Moore 输出逻辑 (仅取决于当前状态)
30 output_logic: PROCESS(state)
31 BEGIN
32     IF state = S3 THEN
33         y <= '1';
34     ELSE
35         y <= '0';
36     END IF;
37 END PROCESS output_logic;
38
39 END moore_2proc;

```

评分标准 (12 分):

- 库声明和 ENTITY 端口定义正确 (1 分)
- 状态枚举类型定义 (TYPE state_type IS...) 和使用 (1 分)
- 进程 1——状态寄存器: 时钟上升沿触发 + 同步复位 $\text{rst}=1 \rightarrow \text{S0}$ (2 分)
- 进程 2——次态逻辑: CASE 语句覆盖全部 4 个状态, 转移逻辑 100% 正确 (4 分)
- 进程 3——输出逻辑: Moore 型特征——仅取决于 current_state/state, 与 x 无关 (2 分)
- 输出逻辑正确: S3 时 $y=1$, 其余 $y=0$ (1 分)
- 代码风格规范: 缩进、注释、避免锁存器 (1 分)

常见错误及扣分:

- 将输出逻辑写成 `y <= '1' WHEN state=S3 AND x='...' ELSE '0'`——Moore 型输出不应依赖 `x`（扣 2 分）
- 忘记 `WHEN OTHERS` 分支——可能推断锁存器（扣 1 分）
- 复位逻辑写成异步（`IF rst='1' THEN ... ELSIF rising_edge(clk)`）——若题目明确要求同步复位，扣 1 分
- 状态转移表中有错误——每个错误转移扣 1 分

第二章 第 2 套试卷——参考答案与评分标准

2.1 一、选择题（每题 3 分，共 18 分）

1. 答案：D

解析：GAL（Generic Array Logic）的与阵列是可编程的，或阵列是固定不可编程的。选项 D 声称两者均可编程，与 GAL 的实际结构不符，故为错误项。GAL 采用 EEPROM 工艺，可重复编程；其输出逻辑宏单元（OLMC）可灵活配置为组合输出或寄存器输出。

评分标准：选 D 得 3 分，其他选项得 0 分。

2. 答案：B

解析：FPGA 的基本可编程逻辑单元是查找表（LUT, Look-Up Table）。LUT 本质上是一个小容量 SRAM，通过存储真值表来实现任意组合逻辑函数。PIP 是可编程互连资源，IOB 是输入/输出模块，Block RAM 是块存储器。

评分标准：选 B 得 3 分，其他选项得 0 分。

3. 答案：A

解析：选项 A 的语法完全正确：关键字 IS 在实体名后、PORT 前，分号在 END adder 后。选项 B 缺少 END adder 后的分号。选项 C 缺少 IS 关键字。选项 D 中端口列表的最后一个信号后不应使用逗号，应使用分号。

评分标准：选 A 得 3 分，其他选项得 0 分。

4. 答案：B

解析：在 VHDL 的 PROCESS 中，信号（SIGNAL）赋值具有“事务延迟”特性——赋值值在当前进程执行完毕后（即进程挂起时）才更新到信号驱动器中，因此当前进程中后续读取的仍是旧值。而变量（VARIABLE）赋值是立即生效的，赋值后即可读取新值。选项 B 正确描述了这一关键区别。

评分标准：选 B 得 3 分，其他选项得 0 分。

5. 答案：B

解析：CPLD 与 FPGA 在结构层面的本质区别在于基本逻辑单元的实现方式：CPLD 基于乘积项（Product Term）结构，由可编程的与阵列经固定或阵列输出；FPGA 基于查找表（LUT）结构，通过 SRAM 存储逻辑函数真值表。两者均可实现时序逻辑。

评分标准：选 B 得 3 分，其他选项得 0 分。

6. 答案：C

解析：一个 n 输入的 LUT 可以实现任意 n 个及以下变量的组合逻辑函数。4 输入 LUT 即可以处理最多 4 个变量的任意逻辑函数。选项 D “8 个及以下”是错误的，因为 4 输入 LUT 只有 $2^4 = 16$ 个存储单元，无法存储 $2^8 = 256$ 行的真值表。

评分标准：选 C 得 3 分，其他选项得 0 分。

2.2 二、填空题（每题 3 分，共 12 分）

1. 答案：IEEE

解析：VHDL 标准库声明语句为 `LIBRARY IEEE;`，用于引入 IEEE 标准化库，其中包含 `STD_LOGIC_1164` 等常用设计包。注意关键字 IEEE 需大写，且语句以分号结尾。

评分标准：填写“IEEE”得 3 分，大小写不敏感。填“ieee”亦可。填错得 0 分。

2. 答案：复合

解析：VHDL 中数据类型分为标量类型（Scalar）和复合类型（Composite）。标量类型如 `STD_LOGIC`、`BIT`、`INTEGER` 等表示单一元素；复合类型如 `STD_LOGIC_VECTOR`、`BIT_VECTOR`、`ARRAY` 等表示一组元素的集合。`STD_LOGIC_VECTOR` 是由多个 `STD_LOGIC` 元素组成的一维数组，属于复合数据类型。

评分标准：填写“复合”得 3 分。填“标量”得 0 分。

3. 答案：&

解析：VHDL 中的拼接运算符为 `&`（ampersand），用于将多个信号或位拼接成一个更宽的向量。例如：`sig_a & sig_b` 将 `sig_a` 和 `sig_b` 拼接为一个新的向量，`sig_a` 在高位，`sig_b` 在低位。

评分标准：填写“&”得 3 分。符号错误得 0 分。

4. 答案：并行（并发）

解析：VHDL 的结构体（ARCHITECTURE）是一个并发域，其内部的所有语句本质上都是并发执行的，用于描述硬件固有的并行特性。常见并发语句包括：并发信号赋值语句（<=）、PROCESS 语句、组件实例化语句（PORT MAP）、BLOCK 语句、GENERATE 语句等。这些语句在结构体中同时有效，共同描述了硬件的并发行为。

评分标准：填写“并行”或“并发”得 3 分。填错得 0 分。

2.3 三、程序改错题（5 分）

错误分析

原代码中共有 6 处错误（本题要求找出 5 处即可得满分，以下逐一列出）：

- 错误 1：** 第 7 行：敏感信号列表不完整。PROCESS 的敏感信号列表中仅包含 sel，但进程中同时读取了 en 信号。VHDL 综合规则要求：组合逻辑进程的敏感信号列表必须包含所有被读取的输入信号，否则可能导致仿真与综合行为不一致（产生锁存器）。应将 PROCESS (sel) 改为 PROCESS (sel, en)。
- 错误 2：** 第 7 行：信号 sel 的类型定义不当。sel 被声明为 INTEGER RANGE 0 TO 7，虽然语法上合法，但在实体端口使用 INTEGER 不利于顶层综合与布线，且与 en 使用的 STD_LOGIC 类型体系不一致。建议将 sel 改为 STD_LOGIC_VECTOR(2 DOWNT0 0)，同时 CASE 分支改为二进制字面量。
- 错误 3：** 第 9 行：输出端口 y 的下标范围错误。y 被声明为 STD_LOGIC_VECTOR(7 DOWNT0 1)，即只有 y(7) 至 y(1) 共 7 个元素。而 CASE 语句中尝试使用 y(0)（第 10 行：WHEN 0 => y(0) <= '1';），y(0) 元素不存在，属于下标越界错误。同时，3 线-8 线译码器应有 8 个输出（y(0) 至 y(7)），应将 y 定义改为 STD_LOGIC_VECTOR(7 DOWNT0 0)。
- 错误 4：** 第 18 行：END CASE 语句缺少分号。END CASE 后必须以分号；结尾。应改为 END CASE;。
- 错误 5：** 第 19 行：END IF 语句缺少分号。END IF 后必须以分号；结尾。应改为 END IF;。
- 错误 6：** 第 20 行：END PROCESS 语句缺少分号。END PROCESS 后必须以分号；结尾。应改为 END PROCESS;。

修正后的完整代码

```

1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY decoder3t8 IS
5     PORT (
6         sel : IN  STD_LOGIC_VECTOR(2 DOWNT0 0);  -- 修正 1: 使用
              STD_LOGIC_VECTOR
7         en  : IN  STD_LOGIC;

```

```

8      y      : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)    -- 修正2: 改为 (7 DOWNT0
          0);
9
10     END decoder3t8;
11
12     ARCHITECTURE behav OF decoder3t8 IS
13     BEGIN
14         PROCESS (sel, en)                          -- 修正3: 敏感列表加入 en
15         BEGIN
16             y <= (OTHERS => '0');
17             IF en = '1' THEN
18                 CASE sel IS
19                     WHEN "000" => y(0) <= '1';
20                     WHEN "001" => y(1) <= '1';
21                     WHEN "010" => y(2) <= '1';
22                     WHEN "011" => y(3) <= '1';
23                     WHEN "100" => y(4) <= '1';
24                     WHEN "101" => y(5) <= '1';
25                     WHEN "110" => y(6) <= '1';
26                     WHEN "111" => y(7) <= '1';
27                     WHEN OTHERS => y <= (OTHERS => '0');
28                 END CASE;                          -- 修正4: 添加分号
29             END IF;                                -- 修正5: 添加分号
30         END PROCESS;                               -- 修正6: 添加分号
31     END behav;

```

评分标准（5 分）

- 每正确指出一处错误并给出正确写法得 1 分，最多 5 分。
- 错误 3（敏感信号列表）、错误 3（y 范围）、错误 3（END CASE 分号）、错误 3（END IF 分号）、错误 3（END PROCESS 分号）为 5 个核心错误，答出其中任意 5 个即可得满分。
- 仅指出错误但未写出正确修改方法者，每处扣 0.5 分。
- 指出错误 3（sel 类型）并合理修正，可作为替代得分项。

2.4 四、程序阅读分析题（5 分）

(1) 该代码描述的是一个什么电路？（1 分）

答案：4 选 1 多路选择器（4-to-1 Multiplexer, MUX）。

评分标准：答出“4 选 1 多路选择器”或“4-to-1 MUX”得 1 分。仅答“多路选择器”未指明规模得 0.5 分。

(2) 该电路的输入信号有哪些？输出信号有哪些？（1 分）

答案：

- 输入信号：a、b、c、d（均为 1 位的 STD_LOGIC 型数据输入端），sel（2 位的 STD_LOGIC_VECTOR 型选择信号）。
- 输出信号：y（1 位的 STD_LOGIC 型输出端）。

评分标准：完整列出 4 个数据输入、1 个选择输入、1 个输出得 1 分。遗漏任何一项扣 0.5 分。

(3) 当 sel = "10" 时，输出 y 的值由哪个输入信号决定？（1 分）

答案：由 c 决定。sel = "10" 对应 CASE 分支 WHEN "10" => y <= c;。

评分标准：答“c”得 1 分，其他答案得 0 分。

(4) 如果在 Vivado 等综合工具中综合该代码，sel 被综合成什么物理资源？（1 分）

答案：sel 被综合成 LUT 的地址线/选择输入端（LUT address inputs / select lines），即用于选择 LUT 中存储的对应数据项的控制信号。

评分标准：答出“LUT 地址线”或“LUT 选择输入”或“查找表地址输入端”得 1 分。仅答“连线”或“控制信号”得 0.5 分。

(5) 该电路的行为描述使用了哪种 VHDL 语句？它与 IF 语句在使用场景上的主要区别是什么？（1 分）

答案：代码使用了 CASE 语句。CASE 语句与 IF 语句的主要区别：

- CASE 语句适用于多分支互斥的选择场景（如多路选择器、译码器），所有分支条件值互不相同，综合器可生成并行判断结构。
- IF 语句适用于具有优先级的判断场景（如优先编码器），条件按书写顺序依次判断，综合器生成级联优先结构。
- 在互斥场景下使用 CASE 比 IF 更高效：CASE 生成并行的 MUX 结构，延迟小；IF 可能生成不必要的优先级链，增加组合逻辑延迟。

评分标准：指出使用了“CASE 语句”得 0.5 分；正确说明区别（互斥 vs 优先级）得 0.5 分。

2.5 五、综合设计题（共 60 分）

2.5.1 设计题 1：组合逻辑设计——8 位数值比较器（20 分）

评分细则

表 2.1: 8 位数值比较器评分标准

评分项目	分值	评分要点
端口定义	4 分	实体声明语法正确（关键字 IS、PORT、端口模式）； 端口信号名 a, b, agtb, aeqb, altb 及类型、位宽正确； 每个端口占 1 分。
功能实现	10 分	比较逻辑正确： $a > b \rightarrow agtb = '1'$ ； $a = b \rightarrow aeqb = '1'$ ； $a < b \rightarrow altb = '1'$ ； 三种关系互斥（同一时刻仅一个输出为 1）； 使用行为描述（IF 或条件信号赋值或 WHEN-ELSE）； 逻辑完整无遗漏。
设计思路	3 分	注释说明了比较器的功能； 说明了实现方式（如“逐位比较”或“利用关系运算符”）； 注释位置合理，不影响代码可读性。
代码规范	3 分	缩进统一（2 或 4 空格）； 信号命名有意义，不随意使用 x、y 等； 关键字大写/小写风格统一； 库声明 LIBRARY 和 USE 语句完整； 注释使用恰当（不过多不过少）。

参考代码

```
1  -- =====
2  -- 设计：8位数值比较器 (8-bit Comparator)
3  -- 功能：比较两个8位无符号数a与b的大小关系
4  --      输出agtb (a>b), aeqb (a=b), altb (a<b)
5  -- 实现方式：使用IF-ELSIF行为描述，三种结果互斥
6  -- =====
7
8  LIBRARY ieee;
```

```

9  USE ieee.std_logic_1164.ALL;
10 USE ieee.numeric_std.ALL;  -- 用于无符号数比较
11
12 ENTITY comparator_8bit IS
13     PORT (
14         a      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 8位输入 a
15         b      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 8位输入 b
16         agtb   : OUT STD_LOGIC;                      -- a > b 输出
17         aeqb   : OUT STD_LOGIC;                      -- a = b 输出
18         altb   : OUT STD_LOGIC;                      -- a < b 输出
19     );
20 END comparator_8bit;
21
22 ARCHITECTURE behav OF comparator_8bit IS
23 BEGIN
24     -- 比较进程：对 a 和 b 的变化敏感
25     PROCESS (a, b)
26     BEGIN
27         -- 默认值：所有比较输出初始为 '0'，避免生成锁存器
28         agtb <= '0';
29         aeqb <= '0';
30         altb <= '0';
31
32         -- 利用 VHDL 关系运算符进行大小比较
33         IF unsigned(a) > unsigned(b) THEN
34             agtb <= '1';          -- a > b
35         ELSIF unsigned(a) = unsigned(b) THEN
36             aeqb <= '1';          -- a = b
37         ELSE
38             altb <= '1';          -- a < b
39         END IF;
40     END PROCESS;
41 END behav;

```

替代实现：条件信号赋值语句（同样满分）

```

1  ARCHITECTURE behav_alt OF comparator_8bit IS
2  BEGIN
3      -- 使用条件信号赋值语句（WHEN-ELSE）实现

```

```
4   agtb <= '1' WHEN unsigned(a) > unsigned(b) ELSE '0';  
5   aeqb <= '1' WHEN unsigned(a) = unsigned(b) ELSE '0';  
6   altb <= '1' WHEN unsigned(a) < unsigned(b) ELSE '0';  
7 END behav_alt;
```

2.5.2 设计题 2：时序逻辑设计——模 6 计数器（20 分）

评分细则

表 2.2: 模 6 计数器评分标准

评分项目	分值	评分要点
端口定义	4 分	端口 <code>clk</code> 、 <code>rst</code> 、 <code>en</code> 、 <code>q[3:0]</code> 、 <code>cout</code> 定义完整； 类型（ <code>STD_LOGIC/STD_LOGIC_VECTOR</code> ）与方向（ <code>IN/OUT</code> ）正确； 每个信号定义合理。
功能实现	10 分	时钟边沿检测语法正确：IF <code>rising_edge(clk)</code> THEN 或 IF <code>clk'event AND clk='1'</code> THEN（2 分）； 异步复位优先级最高：IF <code>rst='1'</code> THEN <code>q <= "0000"</code> ；（2 分）； 使能逻辑：ELSIF <code>en='1'</code> THEN（1 分）； 计数 0~5 循环，5 后回到 0（2 分）； 进位输出逻辑： <code>cout <= '1' WHEN q="0101" AND en='1' ELSE '0'</code> ；（2 分）； 综合无锁存器，所有信号在进程内有完整赋值（1 分）。
设计思路	3 分	注释说明计数器工作流程：复位 → 使能判断 → 加 1 计数 → 循环判断 → 进位输出； 说明了模 6 的含义（计数范围 0~5）； 注释适度、准确。
代码规范	3 分	<code>rising_edge()</code> 函数使用正确（或 <code>clk'event</code> 写法正确）； 信号类型使用恰当（ <code>STD_LOGIC/STD_LOGIC_VECTOR</code> 而非 <code>BIT</code> ）； 进程标签、信号命名合理； 库声明 <code>ieee.std_logic_1164</code> 及 <code>ieee.numeric_std</code> （如需）完整。

参考代码

```
1  -- =====
2  -- 设计：模 6 计数器 (Mod-6 Counter)
3  -- 功能：在时钟上升沿计数，计数范围 0~5，循环往复
4  --      rst=1 异步复位 q="0000"
```

```

5  --      en=1  使能计数
6  --      cout  计数值为5且en有效时输出高电平
7  --  工作流程:
8  --      1) 检查异步复位 rst=1 → q=0
9  --      2) 时钟上升沿且 en=1 → q+1
10 --      3) q=5 时下一周期回0 (模6循环)
11 --      4) 同步进位: q=5且en=1时 cout=1
12 -- =====
13
14 LIBRARY ieee;
15 USE ieee.std_logic_1164.ALL;
16 USE ieee.numeric_std.ALL;
17
18 ENTITY mod6_counter IS
19     PORT (
20         clk    : IN    STD_LOGIC;           -- 时钟信号
21         rst    : IN    STD_LOGIC;           -- 异步复位 (高有效)
22         en     : IN    STD_LOGIC;           -- 计数使能
23         q      : OUT   STD_LOGIC_VECTOR(3 DOWNTO 0); -- 4位计数输出
24         cout   : OUT   STD_LOGIC            -- 进位输出
25     );
26 END mod6_counter;
27
28 ARCHITECTURE behav OF mod6_counter IS
29     SIGNAL count_reg : unsigned(3 DOWNTO 0) := "0000"; -- 内部计数寄存器
30 BEGIN
31     -- 同步输出连接
32     q <= STD_LOGIC_VECTOR(count_reg);
33
34     -- 主计数进程
35     cnt_proc : PROCESS (clk, rst)
36     BEGIN
37         IF rst = '1' THEN                    -- 异步复位, 优先级最高
38             count_reg <= "0000";             -- 复位到0
39         ELSIF rising_edge(clk) THEN          -- 时钟上升沿
40             IF en = '1' THEN                 -- 使能有效
41                 IF count_reg = 5 THEN        -- 计数值为5时 (最大值)
42                     count_reg <= "0000";    -- 回到0, 实现模6循环

```

```

43         ELSE
44             count_reg <= count_reg + 1;           -- 否则正常加1
45         END IF;
46     END IF;
47 END IF;
48 END PROCESS cnt_proc;
49
50 -- 进位输出：组合逻辑
51 -- 当计数值为5且使能有效时，cout输出1
52 cout <= '1' WHEN (count_reg = 5 AND en = '1') ELSE '0';
53
54 END behav;

```

替代实现：使用 STD_LOGIC_VECTOR 作为计数寄存器

```

1 ARCHITECTURE behav_alt OF mod6_counter IS
2     SIGNAL count_reg : STD_LOGIC_VECTOR(3 DOWNT0 0) := "0000";
3 BEGIN
4     q <= count_reg;
5
6     cnt_proc : PROCESS (clk, rst)
7     BEGIN
8         IF rst = '1' THEN
9             count_reg <= "0000";
10        ELSIF rising_edge(clk) THEN
11            IF en = '1' THEN
12                IF count_reg = "0101" THEN           -- 二进制5
13                    count_reg <= "0000";
14                ELSE
15                    count_reg <= STD_LOGIC_VECTOR(unsigned(count_reg) + 1);
16                END IF;
17            END IF;
18        END IF;
19    END PROCESS cnt_proc;
20
21    cout <= '1' WHEN (count_reg = "0101" AND en = '1') ELSE '0';
22 END behav_alt;

```

2.5.3 设计题 3：状态机设计——“101”序列检测器（Mealy 型）（20 分）

功能说明（2 分）

当输入端 din 连续收到序列“101”时，输出端 $dout$ 在检测到完整序列的同一时钟周期输出高电平“1”。其余情况输出“0”。检测允许重叠（overlapping）：输入序列“10101”可检测出两次“101”。

状态图（4 分）

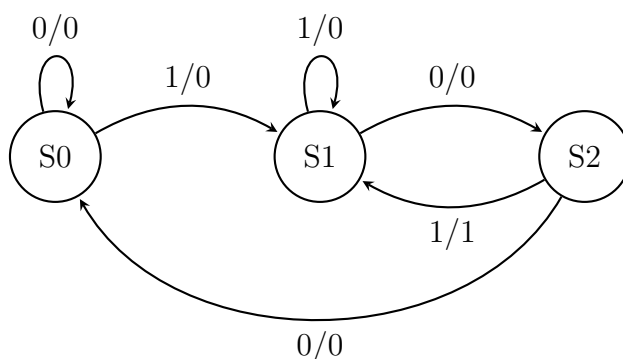


图 2.1: Mealy 型“101”序列检测器状态转换图

状态说明：

- **S0（初始状态）**：尚未检测到任何有效位。若 $din=1$ 则进入 S1（记住“1”）；若 $din=0$ 则保持 S0。
- **S1（检测到“1”）**：已收到第一位“1”。若 $din=0$ 则进入 S2（记住“10”）；若 $din=1$ 则保持 S1（重叠检测：新“1”作为下一序列的起始）。
- **S2（检测到“10”）**：已收到前两位“10”。若 $din=1$ 则输出为 1（此时检测到完整“101”）且下一状态为 S1（重叠：该“1”同时作为下一个“101”序列的首位）；若 $din=0$ 则输出为 0 且返回 S0（序列中断）。

重叠检测示例：

评分标准（状态图 4 分）：

- 状态数量正确（3 个状态 S0,S1,S2）且命名清晰：1 分
- 所有转换弧线（6 条）正确标注“输入/输出”格式：2 分（每条弧线 0.33 分，汇总）
- 重叠检测弧线正确（S1 自环 1/0，S2→S1 标注 1/1）：1 分
- 若使用文字描述代替绘图，需完整列出所有状态转移条件及输出，每遗漏一条转移扣 1 分

表 2.3: 输入序列 “10101” 检测过程

时钟周期	1	2	3	4	5	6 (之后)
din	1	0	1	0	1	—
当前状态	S0	S1	S2	S1	S2	S1
dout	0	0	1	0	1	0
下一状态	S1	S2	S1	S2	S1	—

端口定义 (3 分)

```
1 ENTITY seq_detector_101 IS
2   PORT (
3     clk   : IN   STD_LOGIC;      -- 时钟信号
4     rst   : IN   STD_LOGIC;      -- 异步复位 (低电平有效)
5     din   : IN   STD_LOGIC;      -- 串行数据输入
6     dout  : OUT  STD_LOGIC       -- 检测输出 (Mealy型, 与输入同周期)
7   );
8 END seq_detector_101;
```

评分标准 (端口定义 3 分):

- clk、rst、din、dout 四个端口齐全: 1 分
- 端口模式 (IN/OUT) 与类型 (STD_LOGIC) 正确: 1 分
- 复位极性注释清晰 (低电平有效): 1 分

VHDL 代码——双进程结构 (8 分)

```
1  -- =====
2  -- 设计: Mealy型 "101" 序列检测器
3  -- 功能: 检测串行输入序列 "101"
4  --      输出dout在检测到完整序列的同一周期置1
5  --      支持重叠检测 (如 "10101" 检测出两次)
6  --
7  -- 状态编码:
8  --   S0 = "00"  初始状态 (未匹配任何位)
9  --   S1 = "01"  已匹配 "1"
10 --   S2 = "10"  已匹配 "10"
11 --
12 -- 双进程结构:
13 --   进程1 (state_reg): 时序逻辑——状态寄存器与异步复位
```

```

14  -- 进程2 (output_proc): 组合逻辑——下一状态计算与Mealy输出
15  -- =====
16
17  LIBRARY ieee;
18  USE ieee.std_logic_1164.ALL;
19
20  ENTITY seq_detector_101 IS
21  PORT (
22      clk   : IN   STD_LOGIC;    -- 时钟信号
23      rst   : IN   STD_LOGIC;    -- 异步复位，低电平有效
24      din   : IN   STD_LOGIC;    -- 串行数据输入
25      dout  : OUT  STD_LOGIC      -- 序列检测输出 (Mealy型)
26  );
27  END seq_detector_101;
28
29  ARCHITECTURE fsm OF seq_detector_101 IS
30
31      -- 自定义枚举类型定义状态，可读性强
32      TYPE state_type IS (S0, S1, S2);
33      SIGNAL current_state, next_state : state_type;
34
35  BEGIN
36
37      -- =====
38      -- 进程1: 状态寄存器 (时序逻辑)
39      -- 功能: 在时钟上升沿完成状态切换，处理异步复位
40      -- =====
41      state_reg : PROCESS (clk, rst)
42      BEGIN
43          IF rst = '0' THEN                -- 异步复位，低电平有效
44              current_state <= S0;         -- 回到初始状态
45          ELSIF rising_edge(clk) THEN      -- 时钟上升沿
46              current_state <= next_state; -- 状态更新
47          END IF;
48      END PROCESS state_reg;
49
50      -- =====
51      -- 进程2: 下一状态逻辑与输出逻辑 (组合逻辑)
52      -- 功能: 根据当前状态和输入信号，计算下一状态与Mealy输出

```

```

53  -- 注意: Mealy型输出同时依赖当前状态和输入 din
54  -- =====
55  output_proc : PROCESS (current_state, din)
56  BEGIN
57      -- 默认赋值: 避免生成锁存器
58      next_state <= S0;
59      dout <= '0';
60
61      CASE current_state IS
62
63          WHEN S0 =>
64              -- 状态 S0: 等待第一个 "1"
65              IF din = '1' THEN
66                  next_state <= S1;      -- 收到 "1", 进入 S1
67              ELSE
68                  next_state <= S0;      -- 收到 "0", 保持 S0
69              END IF;
70              dout <= '0';                -- Mealy输出: 未完成序列
71
72          WHEN S1 =>
73              -- 状态 S1: 已匹配 "1", 等待 "0"
74              IF din = '1' THEN
75                  next_state <= S1;      -- 又是 "1", 保持 S1 (重叠检测)
76              ELSE
77                  next_state <= S2;      -- 收到 "0", 形成 "10", 进入 S2
78              END IF;
79              dout <= '0';                -- Mealy输出: 未完成序列
80
81          WHEN S2 =>
82              -- 状态 S2: 已匹配 "10", 等待 "1"
83              IF din = '1' THEN
84                  next_state <= S1;      -- 收到 "1", 完整序列 "101" !
85                                      -- 回到 S1实现重叠检测 (最后 "1"复用)
86                  dout <= '1';          -- Mealy输出: 同一周期输出 1
87              ELSE
88                  next_state <= S0;      -- 收到 "0", 序列中断, 回初始状态
89                  dout <= '0';
90              END IF;
91

```

```

92     WHEN OTHERS =>
93         -- 安全分支：处理未定义状态
94         next_state <= S0;
95         dout <= '0';
96
97     END CASE;
98 END PROCESS output_proc;
99
100 END fsm;

```

VHDL 代码——三进程结构（同等有效，8 分）

```

1  -- 三进程结构：将下一状态逻辑（组合）与输出逻辑（组合）分离
2  ARCHITECTURE fsm_3proc OF seq_detector_101 IS
3
4      TYPE state_type IS (S0, S1, S2);
5      SIGNAL current_state, next_state : state_type;
6
7  BEGIN
8
9      -- 进程1：状态寄存器（时序逻辑）
10     state_reg : PROCESS (clk, rst)
11     BEGIN
12         IF rst = '0' THEN
13             current_state <= S0;
14         ELSIF rising_edge(clk) THEN
15             current_state <= next_state;
16         END IF;
17     END PROCESS;
18
19     -- 进程2：下一状态逻辑（组合逻辑）
20     next_state_proc : PROCESS (current_state, din)
21     BEGIN
22         next_state <= S0; -- 默认值
23         CASE current_state IS
24             WHEN S0 =>
25                 IF din = '1' THEN next_state <= S1;
26                 ELSE next_state <= S0; END IF;
27             WHEN S1 =>

```

```

28         IF din = '1' THEN next_state <= S1;
29         ELSE               next_state <= S2; END IF;
30     WHEN S2 =>
31         IF din = '1' THEN next_state <= S1;
32         ELSE               next_state <= S0; END IF;
33     WHEN OTHERS =>
34         next_state <= S0;
35     END CASE;
36 END PROCESS;
37
38 -- 进程3: 输出逻辑 (Mealy型: 组合逻辑, 依赖 current_state 和 din)
39 output_proc : PROCESS (current_state, din)
40 BEGIN
41     dout <= '0'; -- 默认值
42     IF current_state = S2 AND din = '1' THEN
43         dout <= '1';
44     END IF;
45 END PROCESS;
46
47 END fsm_3proc;

```

评分细则 (代码 8 分 + 设计思路与规范 3 分 = 11 分)

常见错误提示

- **Moore 型错误:** 将输出 dout 放在时序进程中赋值 (与时钟同步), 导致输出延迟一个周期——这是 Moore 型 FSM 而非 Mealy 型, 扣 3 分。
- **复位极性错误:** 题目明确要求 rst 低电平复位 (IF rst = '0' THEN), 若写成高电平复位扣 1 分。
- **重叠检测缺失:** S1 状态下 din=1 时未保持 S1, 或 S2 状态下 din=1 时未回到 S1——无法检测重叠序列, 扣 1.5 分。
- **敏感列表遗漏:** 组合逻辑进程的敏感信号列表中缺少 din 或 current_state——综合时可能产生锁存器, 扣 1 分。
- **默认赋值缺失:** 组合进程中未对 next_state 或 dout 进行默认赋值——可能综合出不必要的锁存器, 扣 1 分。

表 2.4: 序列检测器评分标准

评分项目	分值	评分要点
VHDL 代码（8 分）		
库与实体	1 分	库声明完整（ieee, std_logic_1164）；实体端口定义正确。
状态定义	1 分	状态类型定义（TYPE state_type IS）或常量编码；至少包含 S0/S1/S2 三个状态。
状态寄存器	2 分	时钟边沿检测正确（rising_edge(clk)）；异步复位逻辑正确（rst='0' 复位到 S0，低电平有效）。
下一状态逻辑	2 分	S0→S1(当 din=1)，S0→S0(当 din=0)； S1→S2(当 din=0)，S1→S1(当 din=1，重叠)； S2→S1(当 din=1 且输出 dout=1)，S2→S0(当 din=0)； 每错一处转移扣 0.5 分。
Mealy 输出	2 分	输出逻辑同时依赖当前状态和输入 din（Mealy 特征）； 仅在 S2 且 din=1 时输出 dout=1； 输出在组合逻辑进程中计算（与时钟无关）。
设计思路说明与代码规范（3 分）		
设计思路	1.5 分	注释说明了状态含义和转移条件； 说明了 Mealy 型输出特点（同周期输出）； 说明了重叠检测机制。
代码规范	1.5 分	双进程/三进程结构清晰； 信号命名合理（current_state/next_state）； 使用枚举类型或清晰的状态编码； 进程有标签、缩进一致。

第三章 第 3 套试卷——参考答案与评分标准

3.1 一、选择题（每题 3 分，共 18 分）

参考答案与解析

1. 答案：D

解析：

- A 项正确：CPLD 确基于乘积项（Product Term）的与或阵列结构，适合实现宽输入的复杂组合逻辑（如地址译码）。
- B 项正确：FPGA 基于 SRAM 工艺的查找表（LUT），掉电后配置丢失，但可重复编程（在线重配置）。
- C 项正确：FPGA 的基本可编程逻辑单元是 CLB（Configurable Logic Block），每个 CLB 包含若干个 Slice，每个 Slice 包含 LUT 和 Flip-Flop。
- D 项错误：CPLD 的延时是可预测的（由乘积项阵列的固定走线决定），这是 CPLD 的优势而非劣势。FPGA 的延时反而不如 CPLD 可预测（取决于布局布线）。因此 CPLD适合 时序要求严格的电路。

评分标准：选 D 得 3 分；选其他不得分。

2. 答案：B

解析：FPGA 中的查找表（LUT，Look-Up Table）本质上是一个小容量 SRAM，存储了组合逻辑函数的真值表。一个 n 输入 LUT 可以实现任意 n 变量的组合逻辑函数（ 2^n 种可能）。例如，4 输入 LUT 可存储 16 位真值表数据，实现任意 4 输入 1 输出的组合逻辑。

评分标准：选 B 得 3 分；选其他不得分。

3. 答案：B

解析：VHDL 中 ENTITY（实体）声明用于描述设计单元与外部世界的接口，包括：

- 端口名（Port Name）
- 端口方向（Mode：IN、OUT、INOUT、BUFFER）
- 端口数据类型（Type：如 STD_LOGIC、STD_LOGIC_VECTOR）

内部实现细节由 ARCHITECTURE（结构体）描述。仿真激励由 Testbench 中的 PROCESS 生成。

评分标准：选 B 得 3 分；选其他不得分。

4. 答案：C

解析：Signal 与 Variable 的核心区别：

属性	SIGNAL（信号）	VARIABLE（变量）
声明位置	可在 ARCHITECTURE 声明区（PROCESS 外部）或 PROCESS 内部	只能在 PROCESS 内部
赋值符号	<code><=</code>	<code>:=</code>
生效时刻	进程挂起时（或经过 δ 延时）	立即生效
作用范围	全局（跨进程/模块）	局部（仅本进程）
历史信息	可携带（如 <code>s'event</code> ）	不可携带

- A 项错误：VARIABLE 只能在 PROCESS 或子程序内部声明，不能在外部。
- B 项错误：Signal 赋值在进程结束时（或下一仿真周期）生效，不是立即；Variable 才是立即生效。
- C 项正确：Variable 只能在 PROCESS 或 SUBPROGRAM 内部声明，使用 `:=` 赋值且立即生效。
- D 项错误：Signal 可以在 ARCHITECTURE 声明区（PROCESS 外部）声明，也可以在多进程间共享。

评分标准：选 C 得 3 分；选其他不得分。

5. 答案：B

解析：PROCESS 的敏感信号表（sensitivity list）定义了触发进程执行的信号集合。当敏感表中的任意信号发生事件（值发生变化）时，进程被激活并从上到下顺序执行一次。

- 组合逻辑 PROCESS: 敏感表应包含所有输入信号, 避免生成锁存器。
- 时序逻辑 PROCESS: 敏感表通常包含时钟信号 (可能还有异步复位信号)。

注意: VHDL-2008 支持 PROCESS(ALL), 编译器自动推断所有读入信号加入敏感表。

评分标准: 选 B 得 3 分; 选其他不得分。

6. 答案: C

解析:

- A 项: LIBRARY std; USE std.standard.ALL——std.standard 包定义了 VHDL 的基本类型 (BIT、BOOLEAN、INTEGER 等), 但不含 STD_LOGIC。
- B 项: LIBRARY work; USE work.types.ALL——work 库是用户工作库, 默认不存在 types 包, 非标准定义。
- C 项正确: LIBRARY IEEE; USE IEEE.STD_LOGIC_1164.ALL——IEEE 1164 标准定义了九值逻辑系统, 包括 STD_LOGIC 和 STD_LOGIC_VECTOR。
- D 项: LIBRARY IEEE; USE IEEE.NUMERIC_STD.ALL——该包定义了 UNSIGNED 和 SIGNED 算术类型及其运算, 不含 STD_LOGIC 定义。

评分标准: 选 C 得 3 分; 选其他不得分。

3.2 二、填空题（每题 3 分，共 12 分）

参考答案

1. ENTITY（实体）和 ARCHITECTURE（结构体）

解析：VHDL 中最基本的设计单元称为 Design Entity（设计实体），由两部分构成：

- ENTITY：声明设计的对外接口（端口名、方向、类型），即“黑盒”的外部视图。
- ARCHITECTURE：描述设计的内部功能实现（行为、数据流或结构描述），即“黑盒”的内部实现。

一个 ENTITY 可以对应多个 ARCHITECTURE（如行为级和结构级），通过配置（CONFIGURATION）选择。

评分标准：每空 1.5 分，顺序可互换。填写中文“实体”和“结构体”也给分。

2. <=、:=、&

解析：

- 信号赋值：<=（如 `q <= d;`）
- 变量赋值：:=（如 `tmp := a + b;`）
- 并置运算符：&（如 `"1010" & "0011"` 结果为 `"10100011"`）

评分标准：每空 1 分，符号必须完全正确。

3. IEEE.STD_LOGIC_1164 和 IEEE.NUMERIC_STD

解析：

- IEEE.STD_LOGIC_1164 包：定义了九值逻辑系统（'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-'）以及 STD_LOGIC 类型和 STD_LOGIC_VECTOR 类型及基本逻辑运算函数。
- IEEE.NUMERIC_STD 包：定义了 UNSIGNED（无符号数）和 SIGNED（有符号数补码）两种算术类型，提供了算术运算符（+、-、* 等）、比较运算符和类型转换函数。

注意：IEEE.STD_LOGIC_ARITH（Synopsys）和 IEEE.STD_LOGIC_UNSIGNED 虽被一些旧教材使用，但已不被 IEEE 标准推荐。考试中应使用 IEEE.NUMERIC_STD。

评分标准：每空 1.5 分。注意包名的完整写法。

4. INOUT 和 BUFFER

解析：VHDL 的四种端口模式：

端口模式	方向	可读（内部）	典型用途
IN	输入	是	数据输入、控制信号
OUT	输出	否（VHDL-93 前）	数据输出
INOUT	双向	是	数据总线（如 I ² C 的 SDA）
BUFFER	输出	是	计数器输出需内部回读时

补充说明：VHDL-2008 后 OUT 端口也可内部回读，但考试中以经典教材（VHDL-93）为准。

评分标准：每空 1.5 分，顺序可互换。

3.3 三、程序改错题（共 5 分）

参考答案

以下逐行指出错误并给出正确写法（找到任意 5 处错误即可得 5 分，全部 6 处写出不扣分）：

错误 1（第 1 行）：LIBRARY 改为 LIBRARY
 错误 2（第 14 行）：) 后缺少分号，应改为);
 错误 3（第 21 行）：y <= d(2) 后缺少分号，应改为 y <= d(2);
 错误 4（第 23 行）：y = '0' 赋值符号错误，应改为 y <= '0';
 错误 5（第 24 行）：END CASE 后缺少分号，应改为 END CASE;
 错误 6（第 26 行）：END behavior 结构体名称不匹配，应改为 END behav;

详细解析

1. 第 1 行：关键字拼写错误。LIBRARY 缺少字母 R，正确为 LIBRARY。这是最基础的语法错误。
2. 第 13–14 行：PORT 声明结束缺少分号。ENTITY 的 PORT 声明是一个完整的语句块，在闭合括号后必须加分号。修正后为：

```

1  PORT(
2      d : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
3      s : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
4      y : OUT STD_LOGIC
5  );
  
```

3. 第 21 行：CASE 分支语句缺少分号。WHEN "10" => y <= d(2) 是一个完整的顺序语句，必须以分号结束。
4. 第 23 行：赋值符号错误。在 VHDL 中，信号（SIGNAL）赋值必须使用 <=，不能使用 =（= 仅用于相等比较和初始化赋值，不用于信号赋值语句）。这一错误极为常见，考查学生对两类赋值符号的掌握。
5. 第 24 行：END CASE 缺少分号。与 END IF、END PROCESS 类似，END CASE 后面必须加分号表示该语句的结束。
6. 第 26 行：结构体名称不匹配。ARCHITECTURE 声明为 behav，结尾必须对应 END behav（或 END ARCHITECTURE behav），不能写成 END behavior。

修正后的正确代码

```

1  LIBRARY IEEE;                                -- 修正 1
2  USE IEEE.STD_LOGIC_1164.ALL;
3  ENTITY mux4to1 IS
4      PORT(
5          d : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
6          s : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
7          y : OUT STD_LOGIC
8      );                                          -- 修正 2
9  END mux4to1;
10 ARCHITECTURE behav OF mux4to1 IS
11 BEGIN
12     PROCESS(d, s)
13     BEGIN
14         CASE s IS
15             WHEN "00" => y <= d(0);
16             WHEN "01" => y <= d(1);
17             WHEN "10" => y <= d(2);          -- 修正 3
18             WHEN "11" => y <= d(3);
19             WHEN OTHERS => y <= '0';         -- 修正 4
20         END CASE;                             -- 修正 5
21     END PROCESS;
22 END behav;                                     -- 修正 6

```

评分标准：

- 正确找出并修正 1 处错误得 1 分，最多 5 分（满分）。
- 仅指出错误但未给出正确写法的，每处扣 0.5 分。
- 找出 6 处不扣分（第 5 处之后不再额外加分）。
- 错误位置的行号因原题行号可能随格式变化，只需指出类似于哪一行即可。

3.4 四、程序阅读分析题（共 5 分）

参考答案

(1) (2 分) 电路类型与基本功能:

该程序描述的是带异步复位的 D 触发器 (D Flip-Flop with Asynchronous Reset)。

基本功能:

- 当复位信号 `rst` 为高电平 ('1') 时, 输出 `q` 被立即强制清零 (`q <= '0'`), 此操作与时钟无关。
- 当复位信号无效 (`rst = '0'`) 时, 在时钟 `clk` 的上升沿, 将输入数据 `d` 的值存入输出 `q`, 即 $q(n+1) = d$ (功能表: $Q_{n+1} = D$)。

评分标准: 准确回答“带异步复位的 D 触发器”得 1 分; 正确描述功能得 1 分 (提到异步复位清零 + 上升沿采数据)。

(2) (2 分) 信号作用与复位类型判断:

- `clk`: 时钟输入, 上升沿有效。提供系统时序基准, 控制数据采样的时刻。
- `d`: 数据输入。在时钟上升沿时, 该信号的值被采样并传递至输出 `q`。
- `rst`: 异步复位输入, 高电平有效。当 `rst='1'` 时无论时钟为何状态, 输出 `q` 立即被置为 '0'。
- `q`: 数据输出。输出当前存储的值 (在复位时为 '0', 否则为最近一次时钟上升沿采样到的 `d` 值)。

该电路是异步复位。理由:

- 从代码结构看: `rst` 同时出现在 `PROCESS` 的敏感信号表中 (`PROCESS(clk, rst)`) 且 `IF rst = '1'` 优先级最高 (写在 `ELSIF rising_edge(clk)` 之前)。
- 从行为特性看: 复位操作不依赖时钟沿——只要 `rst` 变为 '1', 输出 `q` 立即被清零, 无需等待时钟上升沿。这正是异步复位的本质特征。
- 补充对比: 若为同步复位, 则敏感表中不会 (也不需要) 包含 `rst`, 且复位语句应放在时钟沿判断内部 (如 `IF rising_edge(clk) THEN IF rst='1' THEN...`)。

评分标准：正确说明 4 个信号作用得 1 分（每个 0.25 分）；正确判断为异步复位并充分说明理由得 1 分。

(3) (1 分) `rising_edge(clk)` 与 `clk'event AND clk='1'` 的比较：

功能上不完全相同。

- `rising_edge(clk)` 是 IEEE 1164 定义的函数，其内部判断条件更为严格：
 - 它要求信号发生了事件 (`clk'event`)，且当前值为 '1' (`clk='1'`)，**同时**上一值必须为 '0' 或 'L'（弱低电平）。这保证了真正的 “'0' → '1' 上升沿”。
 - 它不将 'Z' → '1'、'U' → '1' 等非正常跳变视为有效上升沿。
- `clk'event AND clk='1'` 仅检查信号发生了事件且当前值为 '1'，不检查上一值。这意味着从 'Z'、'U'、'X' 等状态跳变到 '1' 也会被当作有效时钟沿——这在实际硬件中是不合理的，且可能造成仿真与综合结果不一致。
- 对于**综合工具**而言，两者通常被映射为相同的硬件结构（上升沿触发的 D 触发器），因此在**综合后功能基本等价**。但在**仿真中**，`rising_edge()` 更加严谨，是推荐的标准写法。

评分标准：回答不完全相同并指出 `rising_edge()` 更严格（检查从 '0' 或 'L' 跳变）得 1 分；仅回答“功能相同”不得分。

3.5 五、综合设计题（共 60 分）

3.5.1 设计题 1：组合逻辑设计——8 线-3 线优先编码器（20 分）

(1) 真值表（8 分）

说明：输入 $\text{din}(7\sim0)$ 低电平有效（0= 请求）， $\text{din}(7)$ 优先级最高， $\text{din}(0)$ 最低。表中“X”表示任意值（don't care），gs 为组选通信号。

表 3.1: 8 线-3 线优先编码器真值表（低电平有效）

输入 $\text{din}(7\sim0)$								输出			gs
din7	din6	din5	din4	din3	din2	din1	din0	dout2	dout1	dout0	gs
0	X	X	X	X	X	X	X	1	1	1	1
1	0	X	X	X	X	X	X	1	1	0	1
1	1	0	X	X	X	X	X	1	0	1	1
1	1	1	0	X	X	X	X	1	0	0	1
1	1	1	1	0	X	X	X	0	1	1	1
1	1	1	1	1	0	X	X	0	1	0	1
1	1	1	1	1	1	0	X	0	0	1	1
1	1	1	1	1	1	1	0	0	0	0	1
1	1	1	1	1	1	1	1	0	0	0	0

说明：

- 第 1 行： $\text{din}(7)=0$ （第 7 路有效），无论其他输入为何， $\text{dout}="111"$ （二进制 7）。此时 $\text{gs}='1'$ ，表示至少有一路输入有效。
- 第 2 行： $\text{din}(7)=1$ （无效）且 $\text{din}(6)=0$ （有效），输出"110"（二进制 6）。
- 依此类推，按优先级从高到低逐行判定。
- 最后一行：所有输入均为'1'（无效）， $\text{dout}="000"$ ， $\text{gs}='0'$ 表示无请求。

评分标准：

- 真值表格式正确（包含输入、输出、gs 列）得 2 分。
- 正确体现“低电平有效”与优先级顺序（din7 最高 $\rightarrow$ din0 最低）得 3 分。
- 覆盖至少 7 种关键情况（包括无请求）得 2 分。
- 逻辑关系正确得 1 分。

(2) 逻辑表达式 (3 分)

由真值表可导出各输出的逻辑表达式 (卡诺图化简或直接推导):

gs (组选通信号):

$$gs = \overline{din(7)} \cdot \overline{din(6)} \cdot \overline{din(5)} \cdot \overline{din(4)} \cdot \overline{din(3)} \cdot \overline{din(2)} \cdot \overline{din(1)} \cdot \overline{din(0)}$$

即: $gs = '1'$ 当且仅当至少一个输入为 '0' (低电平有效)。等价写法: $gs = \text{NOT} (din(7) \text{ AND } din(6) \text{ AND } \dots \text{ AND } din(0))$ 。

dout(2) (最高位):

$$dout(2) = \overline{din(7)} + din(7) \cdot \overline{din(6)} + din(7) \cdot din(6) \cdot \overline{din(5)} + din(7) \cdot din(6) \cdot din(5) \cdot \overline{din(4)}$$

即: $dout(2) = '1'$ 当最高优先级的有效输入位于 $din(7,6,5,4)$ 之中 (编号 4~7)。

dout(1):

$$dout(1) = \overline{din(7)} + din(7) \cdot \overline{din(6)} + din(7) \cdot din(6) \cdot \overline{din(5)} + din(7) \cdot din(6) \cdot din(5) \cdot \overline{din(4)} + din(7) \cdot din(6) \cdot din(5) \cdot din(4) \cdot \overline{din(3)}$$

即: $dout(1) = '1'$ 当最高优先级的有效输入位于 $din(7,6,3,2)$ 之中 (编号的二进制中间位为 1)。

dout(0):

$$dout(0) = \overline{din(7)} + din(7) \cdot \overline{din(6)} + din(7) \cdot din(6) \cdot \overline{din(5)} + din(7) \cdot din(6) \cdot din(5) \cdot \overline{din(4)} + din(7) \cdot din(6) \cdot din(5) \cdot din(4) \cdot \overline{din(3)}$$

即: $dout(0) = '1'$ 当最高优先级的有效输入位于 $din(7,5,3,1)$ 之中 (编号的二进制最低位为 1)。

评分标准:

- gs 表达式正确得 1 分。
- $dout(2)$ 、 $dout(1)$ 、 $dout(0)$ 表达式各 0.5 分, 合计 1.5 分。
- 逻辑关系正确即可得满分, 表达式的等价形式 (如 NOR-NAND 形式) 也给分, 共 0.5 分。

(3) VHDL 代码——条件信号赋值语句实现 (9 分)

方案一: WHEN-ELSE 语句实现 (推荐, 符合优先级编码逻辑)

```

1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3 USE IEEE.NUMERIC_STD.ALL;           -- 若不需要算术运算可省略

```

```

4
5 ENTITY priority_encoder_83 IS
6   PORT(
7     din   : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 低电平有效
8     dout  : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);  -- 二进制编码输出
9     gs    : OUT STD_LOGIC                      -- 组选通信号
10    );
11 END priority_encoder_83;
12
13 ARCHITECTURE behav OF priority_encoder_83 IS
14 BEGIN
15
16    -- 条件信号赋值：从高优先级到低优先级依次判断
17    dout <= "111" WHEN din(7) = '0' ELSE          -- 优先级7（最高）
18           "110" WHEN din(6) = '0' ELSE          -- 优先级6
19           "101" WHEN din(5) = '0' ELSE          -- 优先级5
20           "100" WHEN din(4) = '0' ELSE          -- 优先级4
21           "011" WHEN din(3) = '0' ELSE          -- 优先级3
22           "010" WHEN din(2) = '0' ELSE          -- 优先级2
23           "001" WHEN din(1) = '0' ELSE          -- 优先级1
24           "000";                                -- 优先级0（最低）或无请求
25
26    -- 组选通信号：当任意输入有效时，gs = '1'
27    gs <= '0' WHEN din = "11111111" ELSE '1';
28
29 END behav;

```

方案二：WITH-SELECT-WHEN 语句实现

```

1 ARCHITECTURE dataflow OF priority_encoder_83 IS
2 BEGIN
3    -- 使用 WITH-SELECT，需枚举所有 din 取值组合（部分简化为含 don't care 的
4    -- 写法）
5    -- 注意：WITH-SELECT 中 WHEN 的选择值必须互斥且完备
6    WITH din SELECT
7       dout <= "111" WHEN "0-----",          -- din(7)=0
8       "110" WHEN "10-----",                 -- din(7)=1, din(6)=0
9       "101" WHEN "110-----",                 -- din(7..6)=11, din(5)=0
10      "100" WHEN "1110----",                   -- din(7..5)=111, din(4)=0
11      "011" WHEN "11110---",                   -- ...

```

```
11         "010" WHEN "111110--",
12         "001" WHEN "1111110-",
13         "000" WHEN "11111110",
14         "000" WHEN OTHERS;           -- 全 1 或 全 0 等
15
16     gs <= '0' WHEN din = "11111111" ELSE '1';
17 END dataflow;
```

说明：方案二中的 don't care 写法（如"0-----"）在 VHDL-2008 及部分综合工具中支持，但在严格 VHDL-93/87 中可能不被接受。考试中推荐使用方案一（WHEN-ELSE），清晰且可综合性强。

评分标准：

- ENTITY 声明正确（含端口名、方向、数据类型）得 2 分。
- 库和包引用正确得 1 分。
- 条件信号赋值语句使用正确（WHEN-ELSE 或 WITH-SELECT）得 2 分。
- 优先级顺序正确（从 din(7) 到 din(0) 依次判断）得 2 分。
- gs 信号逻辑正确得 1 分。
- 代码格式规范、无语法错误得 1 分。

3.5.2 设计题 2：时序逻辑设计——模 10 BCD 计数器（20 分）

(1) 状态转换图（4 分）

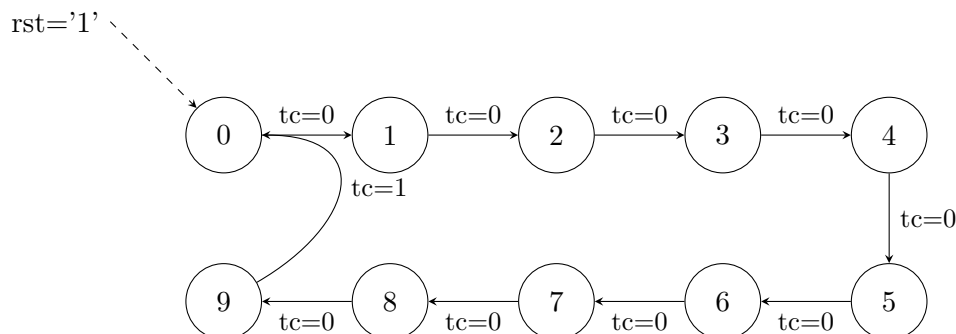


图 3.1: 模 10 BCD 计数器状态转换图

说明：

- 计数范围：0 → 1 → 2 → ... → 9 → 0（循环），每个时钟上升沿计数值 +1。
- 除 9→0 转换时 $tc='1'$ 外，其余转换 $tc='0'$ 。
- 复位信号 $rst='1'$ 时（同步复位，在时钟上升沿采样），计数器回到 0 状态。

评分标准：

- 状态节点完整（0~9，10 个状态）得 1 分。
- 状态转换方向正确得 1 分。
- 转换条件标注清晰得 1 分。
- 复位关系和 $tc=1$ 的跳变标注正确得 1 分。

(2) tc 信号产生逻辑（3 分）

tc （Terminal Count，进位/终端计数）信号：

tc 是一个组合逻辑输出，仅当当前计数值为 9 ($count = "1001"$) 时输出高电平 ('1')，其余计数值时输出低电平 ('0')。

逻辑表达式：

$$tc = count(3) \cdot \overline{count(2)} \cdot \overline{count(1)} \cdot count(0)$$

即： $tc = count(3) \text{ AND } (\text{NOT } count(2)) \text{ AND } (\text{NOT } count(1)) \text{ AND } count(0)$ 。

等价于： $tc = '1' \text{ WHEN } count = "1001" \text{ ELSE } '0';$

工作原理：

- 当计数器处于状态 9 (count="1001") 时, tc='1', 持续一个完整的时钟周期。
- 在下一个时钟上升沿, 计数器从 9 跳变到 0 (若 rst='0'), 此时 tc 由 '1' 变为 '0', 形成一个高电平脉冲 (宽度为一个时钟周期)。
- 该脉冲可用于级联多级计数器时的进位使能, 或作为状态机的外部触发信号。

注意: 虽然 tc 从 9→0 的转换中看似产生了“脉冲”, 实际上 tc 是纯组合逻辑 (仅取决于当前计数值), 其脉宽为一个时钟周期是因为计数值在该周期内保持为 9。

评分标准:

- 正确说明 tc='1' 的条件 (count=9) 得 1 分。
- 给出逻辑表达式或等效描述得 1 分。
- 说明 tc 在 9→0 时产生一个时钟周期脉冲得 1 分。

(3) VHDL 代码——含内部信号 count (13 分)

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY bcd_counter_mod10 IS
6      PORT(
7          clk : IN  STD_LOGIC;           -- 时钟
8          rst : IN  STD_LOGIC;           -- 同步复位, 高有效
9          q   : OUT STD_LOGIC_VECTOR(3 DOWNTO 0); -- BCD计数值输出
10         tc  : OUT STD_LOGIC             -- 终端计数 (进位) 输出
11     );
12 END bcd_counter_mod10;
13
14 ARCHITECTURE behav OF bcd_counter_mod10 IS
15
16     -- 内部信号: 保存当前计数值
17     SIGNAL count : UNSIGNED(3 DOWNTO 0) := (OTHERS => '0');
18     -- 也可用 STD_LOGIC_VECTOR 或 INTEGER RANGE 0 TO 9
19
20 BEGIN
21
22     -- 时序进程: 计数逻辑 (同步复位)

```

```

23  PROCESS(clk)
24  BEGIN
25      IF rising_edge(clk) THEN
26          IF rst = '1' THEN                -- 同步复位
27              count <= (OTHERS => '0');      -- 计数值清零
28          ELSE
29              IF count = 9 THEN              -- 到达最大值9
30                  count <= (OTHERS => '0');  -- 回到0
31              ELSE
32                  count <= count + 1;        -- 否则+1
33              END IF;
34          END IF;
35      END IF;
36  END PROCESS;
37
38  -- 输出赋值：内部count信号输出到端口 q 产生tc信号
39  q <= STD_LOGIC_VECTOR(count);            -- 类型转换（若count为
      UNSIGNED）
40  tc <= '1' WHEN count = 9 ELSE '0';        -- 计数值为9时tc=1
41
42  END behav;

```

代码说明：

1. 内部信号 **count**：使用 UNSIGNED 类型存储当前计数值，初始化为 0。
2. 同步复位：IF rst = '1' 写在 IF rising_edge(clk) 内部，复位操作在时钟上升沿采样。敏感表中仅含 clk，不含 rst。
3. 计数逻辑：count = 9 时回 0，否则 count + 1。
4. tc 输出：条件信号赋值（组合逻辑），count=9 时 tc='1'，否则 tc='0'。
5. 类型转换：STD_LOGIC_VECTOR(count) 将 UNSIGNED 类型转换为 STD_LOGIC_VECTOR 类型输出。

评分标准：

- ENTITY 端口声明正确得 2 分。
- 库引用正确（IEEE + NUMERIC_STD）得 1 分。

- 内部信号 count 声明正确得 1 分。
- 复位为同步复位（写在 IF rising_edge(clk) 内部）得 2 分。
- 计数逻辑正确（0→9→0 循环）得 2 分。
- tc 信号组合逻辑正确得 2 分。
- 输出 q 与内部 count 信号连接正确得 1 分。
- PROCESS 语法正确、代码格式规范得 2 分。

3.5.3 设计题 3：状态机设计——Moore 型“101”序列检测器（20 分）

(1) 状态转换图（6 分）

状态定义：

- S0：初始/复位状态，尚未检测到有效位。输出 $z = '0'$ 。
- S1：已检测到序列第 1 位 '1'。输出 $z = '0'$ 。
- S2：已检测到序列 '10'。输出 $z = '0'$ 。
- S3：已检测到完整序列 '101'。输出 $z = '1'$ 。

状态编码方案：使用用户自定义枚举类型，由综合工具自动分配编码（通常为二进制编码：S0="00", S1="01", S2="10", S3="11"），也可手动指定（如 One-Hot 编码）。

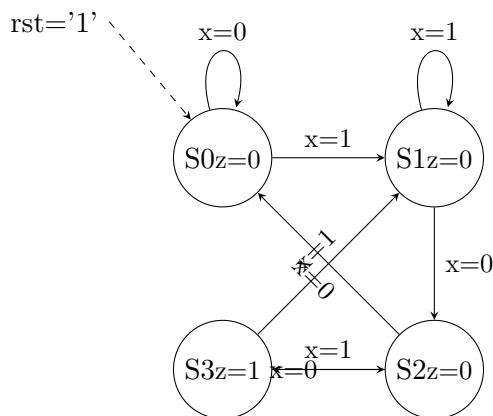


图 3.2: Moore 型“101”序列检测器状态转换图（重叠检测）

重叠检测示例：输入序列"10101"：

- 时刻 1: $x=1$, $S0 \rightarrow S1$, $z=0$
- 时刻 2: $x=0$, $S1 \rightarrow S2$, $z=0$
- 时刻 3: $x=1$, $S2 \rightarrow S3$, $z=1$ （第 1 次检测）
- 时刻 4: $x=0$, $S3 \rightarrow S2$, $z=0$ （重叠：S3 检测到序列后， $x=0$ 将状态引回 S2 而非 S0）
- 时刻 5: $x=1$, $S2 \rightarrow S3$, $z=1$ （第 2 次检测）

评分标准：

- 4 个状态定义正确（S0 初始、S1 得 1、S2 得 10、S3 得 101）得 2 分。
- 状态输出正确（仅 S3 的 $z=1$ ，其他 $z=0$ ）得 1 分。

- 状态转换条件标注完整正确得 1 分。
- 正确体现重叠检测（S3 出 $x=1 \rightarrow S1$, $x=0 \rightarrow S2$ ）得 1 分。
- 异步复位箭头标注正确得 0.5 分。
- 整体图清晰、标注规范得 0.5 分。

(2) 状态转换表（4 分）

表 3.2: Moore 型“101”序列检测器状态转换表

当前状态	输入 x	次态	输出 z
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	1
S3	1	S1	1

说明：

- 当前状态 S3 且输出 $z=1$ 时：若 $x=0$ ，次态跳转至 S2（为序列“10”的下一可能“101”做准备，重叠）；若 $x=1$ ，次态跳转至 S1（当前“1”可作为下一序列的起始位）。
- Moore 型状态机：输出 z 仅依赖于当前状态，与输入 x 无关。故 S0~S2 输出恒为 0，S3 输出恒为 1。

评分标准：

- 状态转换表格式完整（当前状态、输入、次态、输出四列）得 1 分。
- 8 行转换关系全部正确得 2 分（每错一行扣 0.25 分，直至 0 分）。
- 输出 z 仅取决于当前状态（Moore 型特征）得 1 分。

(3) VHDL 代码——三进程描述风格 (10 分)

代码特点:

- 用户自定义枚举类型 `state_type` 定义状态。
- 三进程描述风格: 进程 1 为时序状态寄存器 (含异步复位), 进程 2 为组合次态逻辑, 进程 3 为组合输出逻辑。清晰分离三大功能模块。
- 异步复位: `rst` 既出现在敏感表中, 又作为时序进程中 IF 判断的最高优先级条件。

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY seq_detector_101 IS
5      PORT(
6          clk : IN  STD_LOGIC;      -- 时钟
7          rst : IN  STD_LOGIC;      -- 异步复位, 高电平有效
8          x   : IN  STD_LOGIC;      -- 串行数据输入
9          z   : OUT STD_LOGIC        -- 检测结果输出
10     );
11 END seq_detector_101;
12
13 ARCHITECTURE moore OF seq_detector_101 IS
14
15     -- 用户自定义枚举类型: 4个状态
16     TYPE state_type IS (S0, S1, S2, S3);
17
18     -- 状态信号 (当前状态和次态)
19     SIGNAL current_state : state_type;
20     SIGNAL next_state    : state_type;
21
22 BEGIN
23
24     -- =====
25     -- 进程1: 时序进程——状态寄存器 (含异步复位)
26     -- =====
27     PROCESS(clk, rst)
28     BEGIN
29         IF rst = '1' THEN                -- 异步复位, 最高优先级

```

```

30      current_state <= S0;                -- 复位到初始状态 S0
31      ELSIF rising_edge(clk) THEN         -- 时钟上升沿
32          current_state <= next_state;    -- 状态更新
33      END IF;
34  END PROCESS;
35
36  -- =====
37  -- 进程 2: 组合进程——次态逻辑
38  -- =====
39  PROCESS(current_state, x)
40  BEGIN
41      -- 默认赋值 (避免锁存器)
42      next_state <= S0;
43
44      CASE current_state IS
45
46          WHEN S0 =>                      -- 初始状态
47              IF x = '1' THEN
48                  next_state <= S1;        -- 检测到第 1 位 '1'
49              ELSE
50                  next_state <= S0;        -- 保持等待
51              END IF;
52
53          WHEN S1 =>                      -- 已检测到 "1"
54              IF x = '0' THEN
55                  next_state <= S2;        -- 检测到 "10"
56              ELSE
57                  next_state <= S1;        -- 仍是 '1', 等待 '0'
58              END IF;
59
60          WHEN S2 =>                      -- 已检测到 "10"
61              IF x = '1' THEN
62                  next_state <= S3;        -- 检测到 "101", 进入成功状态
63              ELSE
64                  next_state <= S0;        -- 断序, 回到初始
65              END IF;
66
67          WHEN S3 =>                      -- 已检测到 "101" (成功)
68              IF x = '1' THEN

```

```

69         next_state <= S1;           -- 重叠：当前 '1' 作为新序列起
           始
70     ELSE
71         next_state <= S2;           -- 重叠：当前 '0' 延续为 "10" 的
           第二位
72     END IF;
73
74     WHEN OTHERS =>
75         next_state <= S0;           -- 安全处理
76
77     END CASE;
78 END PROCESS;
79
80 -- =====
81 -- 进程3：组合进程——输出逻辑（Moore型）
82 -- =====
83 PROCESS(current_state)
84 BEGIN
85     -- Moore型：输出仅取决于当前状态
86     CASE current_state IS
87     WHEN S3 =>
88         z <= '1';                   -- 仅在 S3 状态输出 1
89     WHEN OTHERS =>
90         z <= '0';                   -- 其他状态输出 0
91     END CASE;
92 END PROCESS;
93
94 END moore;

```

三进程风格说明：

1. 进程 1（时序——状态寄存器）：负责在时钟上升沿（或异步复位）时将次态值打入当前状态寄存器。敏感表含 clk 和 rst。异步复位判断的优先级最高。
2. 进程 2（组合——次态逻辑）：纯组合逻辑，根据当前状态和输入 x 计算次态。敏感表必须包含所有输入信号（current_state, x），且建议添加默认赋值（如 next_state <= S0）以避免推断锁存器。
3. 进程 3（组合——输出逻辑）：纯组合逻辑，根据当前状态产生输出 z。Moore 型 FSM 中，输出与输入 x 无关，仅取决于当前状态。

评分标准：

- ENTITY 声明正确得 1 分。
- 枚举类型 `state_type` 定义正确得 1 分。
- 异步复位实现正确（敏感表含 `rst` + IF `rst` 优先 + 时序进程）得 2 分。
- 次态逻辑完全正确（含重叠检测的 S3 出转移）得 2 分。
- Moore 型输出逻辑正确（`z` 仅取决于 `current_state`）得 1 分。
- 三进程风格清晰、结构规范得 1 分。
- 组合进程敏感表完整、无锁存器隐患得 1 分。
- 代码整体格式整洁、注释合理得 1 分。

本套试卷总分汇总

题型	满分
一、选择题 (6×3)	18 分
二、填空题 (4×3)	12 分
三、程序改错题	5 分
四、程序阅读分析题	5 分
五、综合设计题	60 分
设计题 1: 8 线-3 线优先编码器	20 分
设计题 2: 模 10 BCD 计数器	20 分
设计题 3: Moore 型序列检测器	20 分
合计	100 分

第四章 第 4 套试卷——参考答案与评分标准

一、选择题参考答案（每题 3 分，共 18 分）

评分标准：每题 3 分，选择正确得 3 分，错误或不选得 0 分。本题共 6 题，满分 18 分。

1. 答案：C（PAL）

解析：PAL（Programmable Array Logic）的典型结构是与阵列可编程、或阵列固定。GAL 在 PAL 基础上增加了输出逻辑宏单元（OLMC），但基本原理相同。CPLD 基于乘积项结构，FPGA 基于查找表结构。

2. 答案：B（定义电路的输入输出端口）

解析：ENTITY（实体）是 VHDL 设计的基本单元，其核心作用是对外定义电路的输入输出端口（PORT）。内部逻辑功能由 ARCHITECTURE（结构体）描述；信号在 ARCHITECTURE 内部声明；仿真激励由 Testbench 提供。

3. 答案：C（SRAM 存储单元和译码器）

解析：FPGA 中查找表（LUT）的本质是一个小容量 SRAM 加上译码电路。 n 输入 LUT 中有 2^n 个 SRAM 存储单元，输入信号经译码器选择对应的 SRAM 单元输出。上电时从外部配置存储器加载真值表到 SRAM 中。

4. 答案：B（信号赋值在当前进程结束时才更新，进程内读到的是旧值）

解析：VHDL 中 SIGNAL 的赋值（使用 $\leq$ ）具有 δ 延时特性：在一个 PROCESS 内部，所有信号赋值语句都要等到进程执行完毕（挂起）后才统一更新。在进程内读取信号将得到更新前的旧值。与之相对，VARIABLE 的赋值（使用 $=$ ）立即生效。

5. 答案: B (4)

解析: n 输入 LUT 可以实现的任意组合逻辑函数的最大输入变量数为 n 。4 输入 LUT 有 $2^4 = 16$ 个配置位, 可表示任意 4 变量逻辑函数的完整真值表。实现更多变量需多个 LUT 级联。

6. 答案: A (一个查找表和一个触发器)

解析: 典型 FPGA 的逻辑单元 (Logic Element, LE) 通常包含一个 n 输入 LUT (实现组合逻辑) 和一个 D 触发器 (实现时序逻辑), 外加一些选择器和进位链。乘法器、RAM 块、处理器核等属于 FPGA 中的专用硬核资源, 不在基本逻辑单元中。

二、填空题参考答案 (每题 3 分, 共 12 分)

评分标准: 每空 1.5 分 (每道题含两空时每空各 1.5 分, 含一空时该空 3 分), 答对得满分, 拼写错误酌情扣 0.5 分。本题共 4 题, 满分 12 分。

1. 答案: SIGNAL; CONSTANT

解析: VHDL 中 SIGNAL 关键字用于声明电路内部互连信号 (相当于物理连线), 具有 δ 延时特性; CONSTANT 关键字用于声明常量, 其值在编译时确定且不可更改。

2. 答案: std_logic_1164

解析: ieee.std_logic_1164 是 IEEE 标准库中最核心的程序包, 定义了 9 值逻辑类型 STD_LOGIC (包括 'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', '-') 以及其向量 STD_LOGIC_VECTOR。

3. 答案: WHEN-ELSE; WITH-SELECT-WHEN

解析: VHDL 并行信号赋值语句的三种形式:

- 简单信号赋值: $y \leq a \text{ AND } b;$
- 条件信号赋值: $y \leq a \text{ WHEN } sel='0' \text{ ELSE } b;$ (WHEN-ELSE 形式)
- 选择信号赋值: WITH sel SELECT $y \leq a \text{ WHEN } '0', b \text{ WHEN } '1';$ (WITH-SELECT-WHEN 形式)

三者均为并行语句, 放置在结构体的 BEGIN-END 之间, 不在 PROCESS 内。

4. 答案: Complex; 与 (AND)

解析: CPLD 全称为 Complex Programmable Logic Device (复杂可编程逻辑器件)。其内部结构主要由可编程的与阵列和宏单元 (Macrocell) 组成。宏单元通常包含一个或门、一个触发器和输出选择电路。CPLD 基于乘积项结构, 与 FPGA 的查找表结构不同。

三、程序改错题参考答案 (共 5 分)

评分标准: 找出并正确改正 5 处错误即得满分 5 分 (每题 1 分)。若找出全部 6 处, 仍按 5 分计。仅指出错误位置但未说明改正方法者, 每处扣 0.5 分。

以下逐行列出 6 处错误:

错误 1: 第 1 行: library ieee 缺少分号

改正: 在行末添加分号, 即 library ieee;

错误 2: 第 2 行: use ieee.std_logic_1164.all 缺少分号

改正: 在行末添加分号, 即 use ieee.std_logic_1164.all;

错误 3: 第 10 行: 变量声明缺少分号

改正: 在变量声明行末添加分号, 即:

```
1 variable temp : std_logic_vector(3 downto 0);
```

错误 4: 第 9 行: PROCESS 敏感信号表不完整

改正: 敏感信号表中缺少 d0, d1, d2, d3。当这些输入信号变化时, 进程也需要重新执行。应改为:

```
1 process(d0, d1, d2, d3, sel)
```

或使用 process(all) (VHDL-2008) 自动包含所有读取的信号。

错误 5: 第 13–18 行: 变量 temp 使用了信号赋值符号 <=

改正: VHDL 中变量的赋值符号为:=, 而非 <=。应将 5 处赋值语句中的 <= 全部改为:=:

```

1 when "00" => temp := d0;
2 when "01" => temp := d1;
3 when "10" => temp := d2;
4 when "11" => temp := d3;
5 when others => temp := (others => '0');

```

错误 6: 第 20 行: 输出端口 y 使用了变量赋值符号:=

改正: y 是 OUT 端口信号, 必须使用信号赋值符号 <=。应改为:

```

1 y <= temp;

```

修正后的完整正确代码:

Listing 4.1: 修正后的 4 选 1 多路选择器

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity mux4to1 is
5     port (
6         d0, d1, d2, d3 : in  std_logic_vector(3 downto 0);
7         sel             : in  std_logic_vector(1 downto 0);
8         y               : out std_logic_vector(3 downto 0)
9     );
10 end mux4to1;
11
12 architecture behavioral of mux4to1 is
13 begin
14     process(d0, d1, d2, d3, sel)
15         variable temp : std_logic_vector(3 downto 0);
16     begin
17         case sel is
18             when "00"  => temp := d0;
19             when "01"  => temp := d1;
20             when "10"  => temp := d2;
21             when "11"  => temp := d3;
22             when others => temp := (others => '0');
23         end case;
24         y <= temp;

```

```

25     end process;
26 end behavioral;

```

四、程序阅读分析题参考答案（共 5 分）

评分标准：第（1）问 2 分，功能描述正确得 2 分，部分正确得 1 分；第（2）问 3 分，判断正确得 1 分，依据充分得 2 分（每条依据 1 分）。本题共 5 分。

(1) 该电路实现的功能（2 分）：

该电路是一个 4 输入优先级编码器（4-to-2 Priority Encoder）。

输入 `din[3:0]`（4 位），输出 `dout[1:0]`（2 位编码值）和 `v`（有效标志）。

优先级规则：`din(3)` 优先级最高，`din(0)` 优先级最低。即：

- 若 `din(3)='1'`，则输出 `dout="11"`（编码 3），`v='1'`
- 若 `din(3)='0'` 且 `din(2)='1'`，则输出 `dout="10"`（编码 2），`v='1'`
- 若 `din[3:2]="00"` 且 `din(1)='1'`，则输出 `dout="01"`（编码 1），`v='1'`
- 若 `din[3:1]="000"` 且 `din(0)='1'`，则输出 `dout="00"`（编码 0），`v='1'`
- 若 `din="0000"`，则输出 `dout="00"`，`v='0'`（无效）

(2) 是组合逻辑还是时序逻辑？判断依据（3 分）：

答：该电路是组合逻辑电路。

判断依据如下：

- 无时钟信号（1.5 分）：**代码中没有 `clk` 信号，也没有使用 `if rising_edge(clk)` 或 `wait until` 等边沿检测语句。时序逻辑必须有时钟信号。
- 敏感信号表仅包含输入信号（1.5 分）：**`PROCESS` 的敏感信号表为 `(din)`，仅包含所有输入信号。当且仅当输入信号变化时进程才会被触发，输出随输入即时更新。进程内所有输出信号（`dout`, `v`）在每次执行时均被赋值，不会产生锁存。

五、综合设计题参考答案（共 60 分）

设计题 1：8 位二进制比较器（20 分）

评分标准：

- 端口框图（5 分）：框图和接口标注正确清晰得 5 分；缺少位宽标注或信号遗漏扣 1-2 分。
- VHDL 代码（10 分）：实体定义正确 3 分，结构体逻辑正确 5 分，代码规范（缩进、注释）2 分。
- Testbench（5 分）：覆盖 $A > B$ 、 $A = B$ 、 $A < B$ 三种情况各 1.5 分，代码完整规范 0.5 分。

(1) 顶层模块端口框图（5 分）



端口说明：

- $A[7:0]$ ：8 位输入，被比较数 A
- $B[7:0]$ ：8 位输入，被比较数 B
- AGTBOUT：1 位输出， $A > B$ 时为 '1'，否则为 '0'
- AEQBOUT：1 位输出， $A = B$ 时为 '1'，否则为 '0'
- ALTBOUT：1 位输出， $A < B$ 时为 '1'，否则为 '0'

(2) 完整 VHDL 代码（10 分）

Listing 4.2: 8 位二进制比较器——完整 VHDL 代码

```

1 library ieee;
2 use ieee.std_logic_1164.all;
```

```

3 use ieee.numeric_std.all;    -- 使用 numeric_std 进行算术比较
4
5 entity comparator_8bit is
6     port (
7         A      : in  std_logic_vector(7 downto 0);  -- 8位输入 A
8         B      : in  std_logic_vector(7 downto 0);  -- 8位输入 B
9         AGTBOUT : out std_logic;                    -- A>B 标志
10        AEQBOUT : out std_logic;                    -- A=B 标志
11        ALTBOUT : out std_logic                     -- A<B 标志
12    );
13 end comparator_8bit;
14
15 architecture behavioral of comparator_8bit is
16 begin
17     -- 比较过程：使用 IF-ELSE 实现优先级判断
18     process(A, B)
19     begin
20         -- 默认值，防止锁存
21         AGTBOUT <= '0';
22         AEQBOUT <= '0';
23         ALTBOUT <= '0';
24
25         if unsigned(A) > unsigned(B) then
26             AGTBOUT <= '1';
27         elsif unsigned(A) = unsigned(B) then
28             AEQBOUT <= '1';
29         else
30             ALTBOUT <= '1';
31         end if;
32     end process;
33 end behavioral;

```

代码说明：

- 使用 `ieee.numeric_std.all` 库中的 `unsigned()` 类型转换,将 `std_logic_vector` 转换为无符号数以进行比较。
- 三种输出互斥（同一时刻只有一个为'1'）：A>B 时 AGTBOUT=1, A=B 时 AEQBOUT=1, A<B 时 ALTBOUT=1。
- 通过默认值赋值（'0'）避免产生锁存器。

(3) Testbench 测试激励 (5 分)

Listing 4.3: 8 位比较器 Testbench

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity comparator_8bit_tb is
6  end comparator_8bit_tb;
7
8  architecture sim of comparator_8bit_tb is
9
10     -- 待测元件声明
11     component comparator_8bit is
12     port (
13         A      : in  std_logic_vector(7 downto 0);
14         B      : in  std_logic_vector(7 downto 0);
15         AGTBOUT : out std_logic;
16         AEQBOUT : out std_logic;
17         ALTBOUT : out std_logic
18     );
19     end component;
20
21     -- 信号声明
22     signal A_tb      : std_logic_vector(7 downto 0);
23     signal B_tb      : std_logic_vector(7 downto 0);
24     signal AGTBOUT_tb : std_logic;
25     signal AEQBOUT_tb : std_logic;
26     signal ALTBOUT_tb : std_logic;
27
28     begin
29
30     -- 实例化待测模块
31     UUT: comparator_8bit
32     port map (
33         A      => A_tb,
34         B      => B_tb,
35         AGTBOUT => AGTBOUT_tb,
36         AEQBOUT => AEQBOUT_tb,

```

```

37     ALTBOUT => ALTBOUT_tb
38 );
39
40 -- 激励生成过程
41 stim_proc: process
42 begin
43     -- ===== 测试用例 1: A > B =====
44     A_tb <= "10101010";    -- 170
45     B_tb <= "01010101";    -- 85
46     wait for 10 ns;
47     assert (AGTBOUT_tb = '1' and AEQBOUT_tb = '0' and ALTBOUT_tb =
48             '0')
49         report "Test 1 (A>B) FAILED"
50         severity error;
51
52     -- ===== 测试用例 2: A = B =====
53     A_tb <= "01101001";    -- 105
54     B_tb <= "01101001";    -- 105
55     wait for 10 ns;
56     assert (AGTBOUT_tb = '0' and AEQBOUT_tb = '1' and ALTBOUT_tb =
57             '0')
58         report "Test 2 (A=B) FAILED"
59         severity error;
60
61     -- ===== 测试用例 3: A < B =====
62     A_tb <= "00110011";    -- 51
63     B_tb <= "11001100";    -- 204
64     wait for 10 ns;
65     assert (AGTBOUT_tb = '0' and AEQBOUT_tb = '0' and ALTBOUT_tb =
66             '1')
67         report "Test 3 (A<B) FAILED"
68         severity error;
69
70     -- ===== 测试用例 4: A = 0, B = 0 (边界) =====
71     A_tb <= "00000000";
72     B_tb <= "00000000";
73     wait for 10 ns;
74     assert (AEQBOUT_tb = '1')
75         report "Test 4 (A=B=0) FAILED"

```

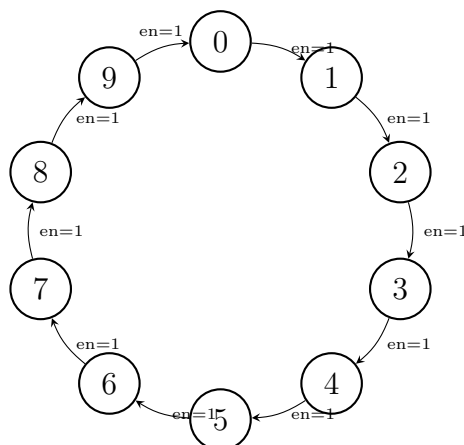
```
73     severity error;
74
75     -- ===== 测试用例5: A = 255, B = 0 (边界) =====
76     A_tb <= "11111111";
77     B_tb <= "00000000";
78     wait for 10 ns;
79     assert (AGTBOUT_tb = '1')
80         report "Test 5 (A=255, B=0) FAILED"
81         severity error;
82
83     -- 结束仿真
84     wait for 10 ns;
85     report "All tests completed successfully!"
86         severity note;
87     wait;
88 end process;
89
90 end sim;
```

设计题 2: 模 10 计数器与模 60 扩展 (20 分)

评分标准:

- 状态转移图 (5 分): 状态完整 2 分, 转移条件标注正确 2 分, 图面规范 1 分。
- 模 10 计数器 VHDL 代码 (10 分): 实体与端口 3 分, 同步清零逻辑 3 分, 计数与进位逻辑 3 分, 代码规范 1 分。
- 模 60 扩展代码 (5 分): 两级计数器结构正确 3 分, 进位和清零逻辑正确 1 分, 注释说明 1 分。

(1) 状态转移图 (5 分)



状态说明：

- 共 10 个状态 (S0-S9)，对应计数值 0-9
- $\text{rst}='1'$ 时：任何状态同步复位到 S0 (clk 上升沿生效)
- $\text{en}='1'$ 且 $\text{rst}='0'$ 时：状态递增 ($S0 \rightarrow S1 \rightarrow \dots \rightarrow S9 \rightarrow S0$)
- $\text{en}='0'$ 且 $\text{rst}='0'$ 时：状态保持不变
- $\text{cout}='1'$ ：当且仅当当前状态为 S9 ($q=9$) 且 $\text{en}='1'$ 时， $\text{cout}='1'$ (组合逻辑输出)

(2) 模 10 计数器 VHDL 代码 (10 分)

Listing 4.4: 模 10 计数器——完整 VHDL 代码

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity mod10_counter is
6   port (
7     clk  : in  std_logic;           -- 时钟信号
8     rst  : in  std_logic;           -- 同步清零 (高有效)
9     en   : in  std_logic;           -- 计数使能 (高有效)
10    q    : out std_logic_vector(3 downto 0); -- 当前计数值 (0~9)
11    cout : out std_logic              -- 进位输出 (q=9且en
        =1时为1)
12  );

```

```

13 end mod10_counter;
14
15 architecture rtl of mod10_counter is
16     signal count : unsigned(3 downto 0); -- 内部计数值
17 begin
18
19     -- 同步计数进程
20     count_proc: process(clk)
21     begin
22         if rising_edge(clk) then
23             if rst = '1' then
24                 count <= (others => '0'); -- 同步清零
25             elsif en = '1' then
26                 if count = 9 then
27                     count <= (others => '0'); -- 计到9回0
28                 else
29                     count <= count + 1; -- 正常加1
30                 end if;
31             end if;
32         end if;
33     end process;
34
35     -- 输出赋值
36     q <= std_logic_vector(count);
37
38     -- 进位输出：组合逻辑，当前为9且使能有效时输出1
39     cout <= '1' when (count = 9 and en = '1') else '0';
40
41 end rtl;

```

关键设计要点：

- 采用 unsigned 类型作为内部计数器（需引用 ieee.numeric_std.all），便于算术运算。
- 同步清零在时钟上升沿检测 rst='1' 时生效，符合题目要求。
- cout 为组合逻辑输出，仅在 count=9 且 en='1' 时为高，表示即将产生进位。
- 计数范围 0-9，下一个时钟沿回到 0（模 10）。

(3) 模 60 计数器扩展 VHDL 代码 (5 分)

Listing 4.5: 模 60 计数器 (0~59) ——扩展代码

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity mod60_counter is
6     port (
7         clk    : in    std_logic;           -- 时钟信号
8         rst    : in    std_logic;           -- 同步清零 (高有效)
9         en     : in    std_logic;           -- 计数使能 (高有效)
10        q      : out std_logic_vector(5 downto 0); -- 当前计数值 (0~59)
11                , 6位足够
12        cout   : out std_logic               -- 进位输出 (q=59且en
13                =1时为1)
14    );
15 end mod60_counter;
16
17 architecture rtl of mod60_counter is
18     -- 关键改动: 分为个位和十位两个计数器
19     signal units : unsigned(3 downto 0); -- 个位计数器 (0~9)
20     signal tens  : unsigned(3 downto 0); -- 十位计数器 (0~5)
21 begin
22     count_proc: process(clk)
23     begin
24         if rising_edge(clk) then
25             if rst = '1' then
26                 -- 同步清零: 个位和十位均清零
27                 units <= (others => '0');
28                 tens  <= (others => '0');
29             elsif en = '1' then
30                 -- ===== 关键改动: 级联计数逻辑 =====
31                 if units = 9 then
32                     units <= (others => '0'); -- 个位回0
33                     if tens = 5 then
34                         tens <= (others => '0'); -- 十位计到5也回0 (整体59->0)
35                     end if;
36                 end if;
37             end if;
38         end if;
39     end process count_proc;
40 end architecture rtl;

```

```

34         else
35             tens <= tens + 1;           -- 十位加1
36         end if;
37     else
38         units <= units + 1;           -- 个位正常加1
39     end if;
40 end if;
41 end if;
42 end process;
43
44 -- 输出：合并个位和十位为6位宽计数值
45 q(3 downto 0) <= std_logic_vector(units);
46 q(5 downto 4) <= std_logic_vector(tens(1 downto 0));
47
48 -- 进位输出：计数到最大值59且使能有效时输出1
49 cout <= '1' when (units = 9 and tens = 5 and en = '1') else '0';
50
51 end rtl;

```

关键改动说明：

1. 两级计数器结构：将单个4位计数器拆分为个位（units, 0-9）和十位（tens, 0-5）两个子计数器。
2. 级联进位逻辑：当个位计到9时，个位清零并向十位发出进位；当十位计到5且个位为9时，十位也清零（整体回到0）。
3. 输出合并：将个位（4位）和十位（2位）拼接为6位输出 q[5:0]。
4. 进位条件修改：cout 的条件从 count=9 变为 units=9 and tens=5（即整体计数值为59）。
5. 位宽变化：输出 q 从4位扩展到6位（足够表示0-59）。

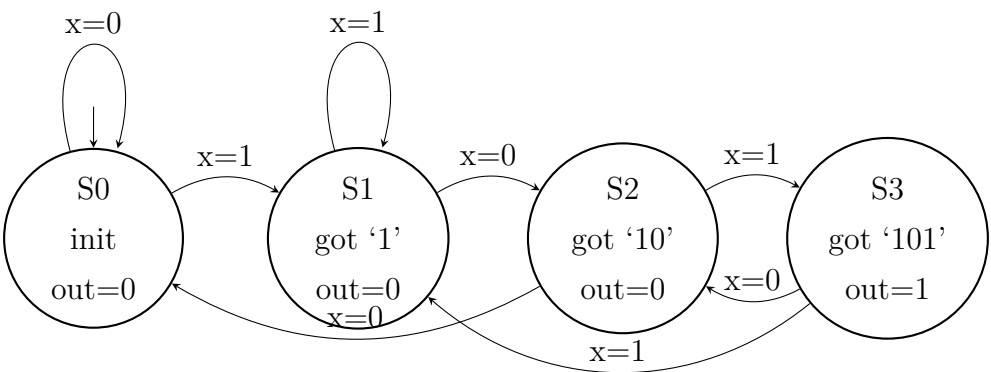
设计题 3：Moore 型“101”序列检测器（20 分）

评分标准：

- 状态转移图（6分）：状态定义完整2分，转移条件和输出标注正确3分，图面规范1分。

- 状态转移表（4 分）：表格完整 2 分（含现态、输入、次态、输出），编码合理 1 分，与状态图一致 1 分。
- VHDL 代码（10 分）：状态类型定义与编码 2 分，状态寄存器进程 2 分，次态逻辑进程 3 分，输出逻辑进程 2 分，重叠检测正确 1 分。

(1) 状态转移图（6 分）



状态定义与转移说明：

- S0（初始状态，output=0）：尚未检测到有效序列。x=0 保持 S0；x=1 进入 S1。
- S1（已匹配 ‘1’，output=0）：检测到一个 ‘1’。x=1 保持 S1（新 ‘1’ 重新开始）；x=0 进入 S2。
- S2（已匹配 ‘10’，output=0）：检测到 ‘10’。x=1 进入 S3（完成 ‘101’）；x=0 回到 S0（连续两个 0 无效）。
- S3（已匹配 ‘101’，output=1）：成功检测到 ‘101’ 序列。x=1 进入 S1（重叠检测：最后一个 ‘1’ 可作新序列开始）；x=0 进入 S2（重叠检测：最后 ‘10’ 可作新序列前缀）。

重叠检测示例（10101）：

输入序列	1	0	1	0	1
状态变化	S0→S1→S2→S3→S2→S3				
输出	0	0	1	0	1

可见 10101 确实检测到了两次 101（位置 1-3 和位置 3-5 共享中间的 ‘1’）。

(2) 状态转移表 (4 分)

状态编码: S0="00", S1="01", S2="10", S3="11"

现态 (PS)	输入 x	次态 (NS)	输出 (detect)
S0 (00)	0	S0 (00)	0
S0 (00)	1	S1 (01)	0
S1 (01)	0	S2 (10)	0
S1 (01)	1	S1 (01)	0
S2 (10)	0	S0 (00)	0
S2 (10)	1	S3 (11)	0
S3 (11)	0	S2 (10)	1
S3 (11)	1	S1 (01)	1

说明: Moore 型状态机的输出仅取决于当前状态。从表中可见, 仅当现态为 S3 时输出 detect=1, 其余状态输出为 0。

(3) 完整 VHDL 代码——三进程描述风格 (10 分)

Listing 4.6: Moore 型 "101" 序列检测器——三进程描述

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seq_detector_101 is
5      port (
6          clk      : in  std_logic;           -- 时钟信号
7          rst      : in  std_logic;           -- 同步复位 (高有效)
8          x        : in  std_logic;           -- 串行输入数据
9          detect   : out std_logic            -- 检测输出 (Moore型)
10     );
11 end seq_detector_101;
12
13 architecture moore_3proc of seq_detector_101 is
14
15     -- 状态类型定义: 使用枚举类型, 代码清晰可读
16     type state_type is (S0, S1, S2, S3);
17     signal current_state, next_state : state_type;
18
19 begin
20

```

```

21  -- =====
22  -- 进程1: 状态寄存器 (同步时序逻辑)
23  -- 功能: 在时钟上升沿完成状态切换, 处理同步复位
24  -- =====
25  state_reg: process(clk)
26  begin
27      if rising_edge(clk) then
28          if rst = '1' then
29              current_state <= S0;           -- 同步复位到初始态
30          else
31              current_state <= next_state;   -- 正常状态转移
32          end if;
33      end if;
34  end process;
35
36  -- =====
37  -- 进程2: 次态逻辑 (组合逻辑)
38  -- 功能: 根据当前状态和输入决定下一个状态
39  -- 关键: 正确处理重叠检测的转移路径
40  -- =====
41  next_state_logic: process(current_state, x)
42  begin
43      -- 默认值 (防止锁存器)
44      next_state <= S0;
45
46      case current_state is
47          when S0 =>           -- 初始状态
48              if x = '0' then
49                  next_state <= S0;
50              else
51                  next_state <= S1;           -- x=1: 进入 S1
52              end if;
53
54          when S1 =>           -- 已匹配 '1'
55              if x = '0' then
56                  next_state <= S2;           -- x=0: 获得 "10"
57              else
58                  next_state <= S1;           -- x=1: 保留在 S1
59              end if;

```

```

60
61     when S2 =>                                -- 已匹配 '10'
62         if x = '0' then
63             next_state <= S0;                    -- x=0: 回到初始
64         else
65             next_state <= S3;                    -- x=1: 完成 "101"
66         end if;
67
68     when S3 =>                                -- 已匹配 '101' (检测成功)
69         if x = '0' then
70             next_state <= S2;    -- 重叠: 最后 "10" 可作新前缀
71         else
72             next_state <= S1;    -- 重叠: 最后 "1" 可作新开始
73         end if;
74
75     when others =>
76         next_state <= S0;                    -- 安全默认
77     end case;
78 end process;
79
80 -- =====
81 -- 进程3: 输出逻辑 (组合逻辑)
82 -- Moore型: 输出仅取决于当前状态
83 -- =====
84 output_logic: process(current_state)
85 begin
86     case current_state is
87     when S3 =>
88         detect <= '1';                    -- 仅 S3 状态下输出 1
89     when others =>
90         detect <= '0';
91     end case;
92 end process;
93
94 end moore_3proc;

```

设计说明:

- 三进程风格: 将状态寄存器、次态逻辑、输出逻辑分别放在三个独立进程中, 结构清晰、易于维护。

- **状态编码：**使用 VHDL 枚举类型（`state_type`），综合工具自动进行状态编码（one-hot、binary 等由综合选项控制），代码可读性高。
- **Moore 型特征：**输出 `detect` 仅由 `current_state` 决定（进程 3 的敏感表中不含 `x`），与输入 `x` 无关。这确保了输出在一个完整的时钟周期内保持稳定。
- **重叠检测：**S3 状态的转移路径是核心——`x=0` 时返回 S2（利用 ‘10’ 前缀），`x=1` 时返回 S1（利用 ‘1’ 前缀），确保 10101 能检测出两次匹配。
- **同步复位：**`rst` 高有效，仅在时钟上升沿检测，符合题目要求。

本套试卷评分汇总

题型	题号/内容	分值
一、选择题	第 1-6 题	18 分
二、填空题	第 1-4 题	12 分
三、程序改错	6 处错误找 5 处满分	5 分
四、程序阅读	优先级编码器分析	5 分
五、设计题 1	8 位比较器（框图 +VHDL+TB）	20 分
五、设计题 2	模 10/模 60 计数器	20 分
五、设计题 3	Moore 型 “101” 检测器	20 分
合计		100 分

第五章 第 5 套试卷——参考答案与评分标准

5.1 一、选择题答案（18 分）

1. 答案：B（3 分）

解析：CPLD 基于乘积项（Product Term）结构，适合组合逻辑实现；FPGA 基于查找表（LUT）结构，适合时序逻辑设计。选项 A 将二者说反了；选项 C 错误（二者结构不同）；选项 D 错误（FPGA 集成度通常更高）。

2. 答案：B（3 分）

解析：FPGA 中的查找表（LUT）实质上是一个由 SRAM 构成的小容量存储器。LUT 将输入组合作为地址，通过查表实现任意组合逻辑函数。选项 A 描述的是触发器阵列，选项 C 是专用 DSP 模块，选项 D 是时钟管理资源。

3. 答案：D（3 分）

解析：选项 D 错误——信号（SIGNAL）通常在结构体的声明区声明（而非在进程内部），变量（VARIABLE）只能在进程内部声明。选项 A 正确（信号用于进程间通信），选项 B 正确（变量赋值立即生效，信号赋值在进程挂起时更新），选项 C 正确（信号用<=，变量用:=）。

4. 答案：D（3 分）

解析：IEEE STD_LOGIC_1164 标准中 STD_LOGIC 数据类型共包含 9 种逻辑值：'U'（未初始化）、'X'（未知/强冲突）、'0'（逻辑 0）、'1'（逻辑 1）、'Z'（高阻）、'W'（弱未知）、'L'（弱 0）、'H'（弱 1）、'-'（无关）。共 9 种。

5. 答案：C（3 分）

解析：PROCESS 语句在敏感信号列表中任一信号发生变化时被激活执行。选项 A 错误（一个结构体可包含多个进程）；选项 B 错误（省略敏感信号列表可能导致仿真与综合不一致，且不完全的敏感列表会导致综合出锁存器）；选项 D 错误（进程内可以使用 IF 语句和 CASE 语句等顺序语句）。

6. 答案：A（3 分）

解析：PAL（可编程阵列逻辑）的基本结构特点是“与阵列可编程，或阵列固定”。这是 PAL 器件区别于 PLA（与阵列和或阵列均可编程）和 PROM（与阵列固定、或阵列可编程）的核心特征。

5.2 二、填空题答案（12 分）

1. 答案：Field Programmable Gate Array；现场可编程门阵列（每空 1.5 分，共 3 分）
2. 答案：外部接口（输入/输出端口）；内部功能（逻辑行为/具体实现）（每空 1.5 分，共 3 分）
3. 答案：&；<=；:=（每空 1 分，共 3 分）
4. 答案：clk（或时钟信号名）；clk（或时钟信号名）（每空 1.5 分，共 3 分。完整写法：
IF clk'EVENT AND clk = '1' THEN）

5.3 三、程序改错题解析（5 分）

错误分析（共 6 处错误，找出并改正任意 5 处即得满分，每处 1 分）：

- 错误 1：（第 1 行）缺少分号：LIBRARY IEEE；应为 LIBRARY IEEE；——原代码末尾缺少分号（注：原题代码中第 1 行确实缺少分号）。更正：在 IEEE 后添加分号：LIBRARY IEEE；
- 错误 2：（第 10–11 行）端口声明语法：count : OUT INTEGER RANGE 0 TO 255 中端口类型应为 STD_LOGIC_VECTOR，因结构体内部使用的内部信号 cnt 为 STD_LOGIC_VECTOR 类型，且输出赋值 count <= cnt 要求类型一致。更正：count : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)
- 错误 3：（第 15 行）信号赋值方式错误：cnt <= 0；——cnt 是 STD_LOGIC_VECTOR 类型，不能直接赋整数值 0。更正：cnt <= (OTHERS => '0')；
- 错误 4：（第 18 行）算术运算赋值符号错误：cnt = cnt + 1；——在 VHDL 中，等式比较使用 =，信号赋值应使用 <=。且算术加法需要引用 IEEE.NUMERIC_STD 包进行类型转换。更正：cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1)；或使用 IEEE.STD_LOGIC_UNSIGNED 包后写成 cnt <= cnt + 1；
- 错误 5：（第 19–20 行）END PROCESS 与 END IF 顺序颠倒：END PROCESS；出现在 END IF；之前，导致 IF 语句未正确闭合。更正：先将 END IF；放在 END PROCESS；之前。正确顺序应为：

```

        END IF;
    END IF;
END PROCESS;

```

错误 6: (第 22 行) 信号赋值缺少分号: count <= cnt 末尾缺少分号。更正: count <= cnt;

修正后的完整 VHDL 代码:

Listing 5.1: 修正后的 8 位计数器 VHDL 代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY counter8 IS
6      PORT(
7          clk      : IN  STD_LOGIC;
8          rst      : IN  STD_LOGIC;
9          en       : IN  STD_LOGIC;
10         count    : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
11     );
12 END counter8;
13
14 ARCHITECTURE behavioral OF counter8 IS
15     SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNTO 0);
16 BEGIN
17     PROCESS(clk)
18     BEGIN
19         IF rst = '1' THEN
20             cnt <= (OTHERS => '0');
21         ELSIF rising_edge(clk) THEN
22             IF en = '1' THEN
23                 cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1);
24             END IF;
25         END IF;
26     END PROCESS;
27
28     count <= cnt;
29 END behavioral;

```

评分标准：指出并改正一处错误得 1 分，最高 5 分；修改后代码逻辑正确额外给分（已包含在每处 1 分中）。

5.4 四、程序阅读分析题解析（5 分）

(1) (2 分) 电路类型与功能：该程序实现的是一个 3 线-8 线译码器 (3-to-8 Decoder)，带使能控制端。功能：当使能信号 `en = '1'` 时，根据 3 位地址输入 `din(2:0)` 将 8 位输出中的对应位置 1（其余位为 0），实现二进制地址到独热码输出的译码功能；当 `en = '0'` 时所有输出均为 0。

(2) (1 分) `en` 信号的作用：`en` 为使能控制信号。当 `en = '0'` 时，输出 `dout = "00000000"`（所有位清零，译码器不工作）；当 `en = '1'` 时，译码器正常工作。

(3) (1 分) 输出值：输入 `din = "011"`（即十进制 3），对应 CASE 分支 `WHEN "011" => dout <= "00001000"` 输出 `dout = "00001000"`（第 3 位为 1）。

(4) (1 分) 电路类型及判断依据：该电路属于组合逻辑电路。判断依据：

- 进程的敏感信号列表中包含了所有输入信号（`din` 和 `en`），无时钟信号；
- 输出仅取决于当前输入值，与历史状态无关；
- 代码中无时钟边沿检测（如 `clk'EVENT` 或 `rising_edge(clk)`）；
- CASE 语句覆盖了所有可能输入组合（含 WHEN OTHERS），不会综合出锁存器。

评分标准：（1）正确答出“3-8 译码器/解码器”给 1 分，功能描述正确给 1 分；（2）`en` 作用和 `en=0` 时输出各 0.5 分；（3）输出值正确给 1 分；（4）判断为组合逻辑给 0.5 分，依据合理给 0.5 分。

5.5 五、综合设计题参考答案（60 分）

5.5.1 设计题 1：8 线-3 线优先编码器设计（20 分）

(1) 真值表（5 分）

I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0	GS
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

注：X 表示任意值（0 或 1），输入为高电平有效（'1' 有效）。 I_7 优先级最高。

评分标准：真值表完整（含所有输入情况）给 3 分，标注优先级顺序及 GS 正确给 2 分。

(2) 逻辑表达式（5 分）

$$Y_2 = I_7 + I_6 + I_5 + I_4$$

$$Y_1 = I_7 + I_6 + \overline{I_5} \cdot \overline{I_4} \cdot I_3 + \overline{I_5} \cdot \overline{I_4} \cdot I_2$$

$$= I_7 + I_6 + \overline{I_5} \cdot \overline{I_4} \cdot (I_3 + I_2)$$

$$Y_0 = I_7 + \overline{I_6} \cdot I_5 + \overline{I_6} \cdot \overline{I_4} \cdot I_3 + \overline{I_6} \cdot \overline{I_4} \cdot \overline{I_2} \cdot I_1$$

$$GS = I_7 + I_6 + I_5 + I_4 + I_3 + I_2 + I_1 + I_0$$

评分标准：每个输出表达式 1.25 分，共 5 分。表达式形式可不唯一，逻辑正确即可。

(3) VHDL 代码（6 分）

Listing 5.2: 8 线-3 线优先编码器 VHDL 代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY priority_encoder_83 IS
5      PORT(
6          I    : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- I7..I0, I7最高优先级
7          Y    : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);  -- 二进制编码输出
8          GS   : OUT STD_LOGIC                      -- 有效输出标志
9      );

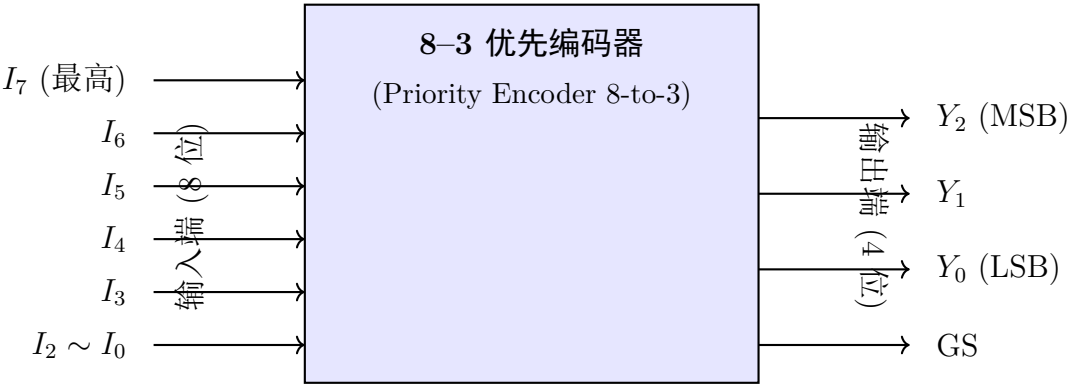
```

```
10 END priority_encoder_83;
11
12 ARCHITECTURE behav OF priority_encoder_83 IS
13 BEGIN
14     PROCESS(I)
15     BEGIN
16         -- 使用 IF-ELSIF 实现优先级逻辑
17         IF I(7) = '1' THEN
18             Y <= "111";
19             GS <= '1';
20         ELSIF I(6) = '1' THEN
21             Y <= "110";
22             GS <= '1';
23         ELSIF I(5) = '1' THEN
24             Y <= "101";
25             GS <= '1';
26         ELSIF I(4) = '1' THEN
27             Y <= "100";
28             GS <= '1';
29         ELSIF I(3) = '1' THEN
30             Y <= "011";
31             GS <= '1';
32         ELSIF I(2) = '1' THEN
33             Y <= "010";
34             GS <= '1';
35         ELSIF I(1) = '1' THEN
36             Y <= "001";
37             GS <= '1';
38         ELSIF I(0) = '1' THEN
39             Y <= "000";
40             GS <= '1';
41         ELSE
42             Y <= "000";
43             GS <= '0';
44         END IF;
45     END PROCESS;
46 END behav;
```

评分标准：实体定义正确（1 分），IF-ELSIF 优先级逻辑正确（3 分），GS 正确（1

分)，代码规范（1 分）。

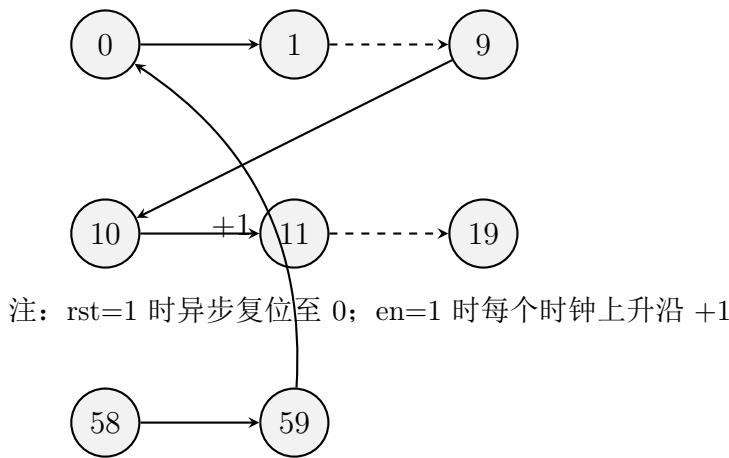
(4) 外部接口框图（4 分）



评分标准：框图清晰（1 分），输入信号标注完整（1 分），输出信号标注完整（1 分），标明优先级顺序和位宽（1 分）。

5.5.2 设计题 2：模 60 计数器设计（20 分）

(1) 状态转换图（4 分）



评分标准：标注复位初始状态（1 分），状态迁移关系正确（1 分），代表性状态选择合理（1 分），59 到 0 的循环正确（1 分）。

(2) VHDL 代码（8 分）

Listing 5.3: 模 60 计数器 VHDL 代码

```
1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3 USE IEEE.NUMERIC_STD.ALL;
4
```

```

5 ENTITY mod60_counter IS
6   PORT(
7     clk    : IN  STD_LOGIC;
8     rst    : IN  STD_LOGIC;
9     en     : IN  STD_LOGIC;
10    tens   : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);  -- 十位BCD (0~5)
11    units  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)   -- 个位BCD (0~9)
12  );
13 END mod60_counter;
14
15 ARCHITECTURE behav OF mod60_counter IS
16   SIGNAL t_val : INTEGER RANGE 0 TO 5 := 0;  -- 十位 0~5
17   SIGNAL u_val : INTEGER RANGE 0 TO 9 := 0;  -- 个位 0~9
18 BEGIN
19   PROCESS(clk, rst)
20   BEGIN
21     IF rst = '1' THEN
22       t_val <= 0;
23       u_val <= 0;
24     ELSIF rising_edge(clk) THEN
25       IF en = '1' THEN
26         IF u_val = 9 THEN
27           u_val <= 0;
28           IF t_val = 5 THEN
29             t_val <= 0;          -- 59 -> 00
30           ELSE
31             t_val <= t_val + 1;
32           END IF;
33         ELSE
34           u_val <= u_val + 1;
35         END IF;
36       END IF;
37     END IF;
38   END PROCESS;
39
40   tens <= STD_LOGIC_VECTOR(TO_UNSIGNED(t_val, 4));
41   units <= STD_LOGIC_VECTOR(TO_UNSIGNED(u_val, 4));
42 END behav;

```

评分标准：实体端口正确（1 分），异步复位逻辑正确（1 分），使能控制（1 分），个位/十位进位逻辑正确（2 分），59→0 循环正确（2 分），代码规范/注释（1 分）。

(3) Testbench 代码（6 分）

Listing 5.4: 模 60 计数器测试平台

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY tb_mod60_counter IS
5  END tb_mod60_counter;
6
7  ARCHITECTURE test OF tb_mod60_counter IS
8      COMPONENT mod60_counter
9          PORT(
10             clk      : IN  STD_LOGIC;
11             rst       : IN  STD_LOGIC;
12             en        : IN  STD_LOGIC;
13             tens      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
14             units     : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
15          );
16      END COMPONENT;
17
18      SIGNAL clk      : STD_LOGIC := '0';
19      SIGNAL rst       : STD_LOGIC := '0';
20      SIGNAL en        : STD_LOGIC := '0';
21      SIGNAL tens      : STD_LOGIC_VECTOR(3 DOWNTO 0);
22      SIGNAL units     : STD_LOGIC_VECTOR(3 DOWNTO 0);
23
24      CONSTANT clk_period : TIME := 10 ns;
25  BEGIN
26      -- 实例化待测模块
27      UUT: mod60_counter PORT MAP(clk, rst, en, tens, units);
28
29      -- 时钟生成进程
30      clk_process: PROCESS
31      BEGIN
32          clk <= '0';
33          WAIT FOR clk_period / 2;
34          clk <= '1';

```

```

35     WAIT FOR clk_period / 2;
36 END PROCESS;
37
38 -- 测试激励进程
39 stim_proc: PROCESS
40 BEGIN
41     -- 测试异步复位
42     rst <= '1';
43     WAIT FOR 20 ns;
44     rst <= '0';
45     WAIT FOR 20 ns;
46
47     -- 测试使能计数 (使能计数 70 个周期, 覆盖 0->59->0->10)
48     en <= '1';
49     WAIT FOR 700 ns;    -- 70 个周期, 覆盖 0~59~10
50
51     -- 测试使能关闭
52     en <= '0';
53     WAIT FOR 50 ns;
54
55     -- 测试重新使能
56     en <= '1';
57     WAIT FOR 100 ns;
58
59     -- 测试中途复位
60     rst <= '1';
61     WAIT FOR 20 ns;
62     rst <= '0';
63     WAIT FOR 100 ns;
64
65     WAIT;
66 END PROCESS;
67 END test;

```

评分标准：时钟生成正确（1 分），复位验证（1 分），使能计数验证（1 分），模 60 循环验证（2 分），暂停/中途复位验证（1 分）。

(4) 数字钟应用原理（2 分）

模 60 计数器在数字钟系统中的应用原理：数字钟的秒和分均为 60 进制（0-59），由

模 60 计数器实现。秒计数器每计满 60（即 59→0 时）产生一个进位脉冲，驱动分计数器加 1；分计数器同理驱动时计数器。当分计数器达到 59 且秒计数器也达到 59 时，下一秒来临时分计数器归零并进位到时计数器。因此，数字钟需要一个模 60 的秒计数器和一个模 60 的分计数器级联工作。

评分标准：说明秒/分 60 进制（1 分），说明进位级联关系（1 分）。

5.5.3 设计题 3：“101”序列检测器（Moore 状态机）（20 分）

(1) 状态转换图（6 分）

```

out=0; [state] (S1) [right of=S0] S1
      已匹配 “1”
out=0; [state] (S2) [right of=S1] S2
      已匹配 “10”
out=0; [state] (S3) [below of=S1, yshift=-1cm] S3
      已匹配 “101”
      out=1;
S0)edge[loopabove]node{din=0}S0) edge node din=1 (S1) (S1) edge node din=0 (S2) edge
[loop above] node din=1 (S1) (S2) edge [bend left] node din=1 (S3) edge [bend right=45] node
[left] din=0 (S0) (S3) edge [bend right] node [right] din=0 (S2) edge [bend left=45] node [left]
      din=1 (S1);

```

当前状态	输入 din	次态	输出 dout
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	1
S3	1	S1	1

评分标准：表格格式规范（1 分），8 行转移全部正确（4 分，每错一处扣 0.5 分）。

(3) VHDL 代码（7 分）

Listing 5.5: “101” 序列检测器 Moore 状态机 VHDL 代码（双进程描述）

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY seq_detector_101 IS
5      PORT(
6          clk  : IN  STD_LOGIC;
7          rst  : IN  STD_LOGIC;
8          din  : IN  STD_LOGIC;
9          dout : OUT STD_LOGIC
10     );
11 END seq_detector_101;
12
13 ARCHITECTURE moore OF seq_detector_101 IS
14     TYPE state_type IS (S0, S1, S2, S3);
15     SIGNAL current_state, next_state : state_type;
16 BEGIN
17     -- =====
18     -- 进程 1: 时序逻辑——状态寄存器（含异步复位）
19     -- =====
20     PROCESS(clk, rst)
21     BEGIN
22         IF rst = '1' THEN
23             current_state <= S0;
24         ELSIF rising_edge(clk) THEN
25             current_state <= next_state;

```

```

26     END IF;
27 END PROCESS;
28
29 -- =====
30 -- 进程2: 组合逻辑——次态逻辑 + 输出逻辑
31 -- =====
32 PROCESS(current_state, din)
33 BEGIN
34     -- 默认赋值
35     next_state <= S0;
36     dout <= '0';
37
38     CASE current_state IS
39         WHEN S0 =>
40             IF din = '1' THEN
41                 next_state <= S1;
42             ELSE
43                 next_state <= S0;
44             END IF;
45
46         WHEN S1 =>
47             IF din = '0' THEN
48                 next_state <= S2;
49             ELSE
50                 next_state <= S1;  -- 保持, '1'可作为新序列起点
51             END IF;
52
53         WHEN S2 =>
54             IF din = '1' THEN
55                 next_state <= S3;
56             ELSE
57                 next_state <= S0;
58             END IF;
59
60         WHEN S3 =>
61             dout <= '1';          -- Moore型: 输出仅取决于当前状态
62             IF din = '0' THEN
63                 next_state <= S2;  -- 重叠: 后缀"10"对应S2
64             ELSE

```

```
65         next_state <= S1;  -- 重叠: 最后的 '1' 可作为新起点
66     END IF;
67 END CASE;
68 END PROCESS;
69 END moore;
```

评分标准：状态枚举定义（1 分），异步复位逻辑（1 分），次态逻辑正确（3 分，含重叠检测 1 分），Moore 型输出逻辑（1 分），代码规范（1 分）。

(4) 非重叠检测的修改方法（2 分）

若不希望检测重叠序列（即“10101”中只检测出一次“101”），修改方法为：在 S3 状态下接收到完整“101”后，无论下一个输入 din 为何值，状态均无条件返回 S0（重新开始检测），而不是根据输入跳转到 S2 或 S1。具体在 VHDL 代码的 S3 分支中去掉 IF 判断，直接将 next_state 设为 S0。

评分标准：说明 S3→S0 无条件转移（1 分），解释放弃了重叠的后缀（1 分）。

第六章 第 6 套试卷——参考答案与评分标准

6.1 一、选择题答案（18 分）

1. 答案：B（3 分）

解析：CPLD 采用连续式互连（Crossbar 结构），信号延时可预测（与逻辑宏单元位置无关）；FPGA 采用分段式互连，延时与布局布线的具体路径有关，不可精确定量预测。选项 A 将二者说反，选项 C 错误（结构不同），选项 D 错误（FPGA 包含丰富的局部互连和全局布线资源）。

2. 答案：D（3 分）

解析：DMA（Direct Memory Access，直接存储器访问）是计算机系统外设与内存之间进行高速数据传输的一种机制，并非 FPGA 的配置模式。FPGA 常用配置模式包括：主串模式（Master Serial）、从并模式（Slave Parallel）、JTAG 边界扫描模式、主并模式（Master Parallel）及 SPI 模式等。

3. 答案：D（3 分）

解析：IF 语句和 CASE 语句属于顺序语句（Sequential Statement），只能出现在 PROCESS、FUNCTION 或 PROCEDURE 内部，不能直接出现在结构体的并发区域。选项 A 正确（结构体内部默认为并发），选项 B 正确（PROCESS 内部顺序执行），选项 C 正确（并发信号赋值语句可在结构体中使用）。

4. 答案：A（3 分）

解析：Moore 型状态机的输出仅取决于当前状态，与输入信号无关；Mealy 型状态机的输出取决于当前状态和当前输入信号的组合。这是两种状态机最根本的区别。选项 B 说反了，选项 C 不一定（Mealy 不一定少），选项 D 错误（两者都可检测重叠序列）。

5. 答案：B（3 分）

解析：VHDL 中逻辑运算符的优先级：NOT 优先级最高（单目运算符），AND、OR、NAND、NOR、XOR、XNOR 等双目逻辑运算符优先级相同，按从左到右的顺序结合。因此选项 B 正确。

6. 答案：D（3 分）

解析：GENERATE 语句属于并发语句（Concurrent Statement），而非顺序语句，它可以出现在结构体内部但不能出现在 PROCESS 内部。FOR GENERATE 用于重复生成硬件结构，IF GENERATE 用于条件生成硬件。选项 A、B、C 均为正确描述。

6.2 二、填空题答案（12 分）

1. 答案：结构体（ARCHITECTURE）；进程（PROCESS）；两者均可（每空 1 分，共 3 分）

解析：SIGNAL 通常在结构体声明区声明（也可在 ENTITY 中声明为端口），具有全局性；VARIABLE 只能在进程（或子程序）内部声明，作用范围局限于进程内；CONSTANT 可以在结构体、进程或包中声明。

2. 答案：静态参数/设计参数（如位宽等）；底层模块的接口（每空 1.5 分，共 3 分）

3. 答案：FOR GENERATE；IF GENERATE（每空 1.5 分，共 3 分。顺序可互换）

4. 答案：所有输入信号（或：全部被读取的信号）；锁存器（Latch）（前一空 2 分，后一空 1 分，共 3 分）

解析：描述组合逻辑的 PROCESS 若敏感信号列表不完整，综合工具可能推断出锁存器来“记住”之前的值，导致逻辑功能错误。

6.3 三、程序改错题解析（5 分）

错误分析（共 5 处，每处 1 分）：

错误 1：（第 11 行）敏感信号列表不完整：PROCESS(a) 只包含 a，缺少 b 和 sel。组合逻辑进程的敏感列表必须包含所有输入信号。**更正：**PROCESS(a, b, sel)

错误 2：（第 15–16 行）缺少 ELSE 分支：IF 语句缺少 ELSE 分支，当 sel '1' 时 tmp 未赋值，会综合出锁存器。**更正：**添加 ELSE tmp := b；

错误 3：（第 18 行）变量赋值符号错误：tmp <= b；——变量赋值应使用:=，<=用于信号赋值。**更正：**tmp := b；（注：此行实际在修正后应删除，因为 tmp 赋值在 IF 块内已完成）

错误 4: (第 18 行/第 19 行) 输出端口赋值方式错误: `y := tmp;`——端口 `y` 是 OUT 模式, 且是 SIGNAL (而非 VARIABLE), 应使用信号赋值符号 `<=`。更正: `y <= tmp;`

错误 5: (第 21 行) 进程外赋值冲突: `y <= tmp;` 出现在进程外部。进程内部已对 `y` 进行赋值 (修正后), 进程外部不能有重复/冲突的赋值, 且进程外无法访问变量 `tmp`。此外, 该行多余的并发赋值与进程内赋值产生多驱动冲突。更正: 删除此行。

修正后的完整 VHDL 代码:

Listing 6.1: 修正后的二选一多路选择器 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY mux_err IS
5      PORT(
6          a, b : IN  STD_LOGIC;
7          sel  : IN  STD_LOGIC;
8          y    : OUT STD_LOGIC
9      );
10 END mux_err;
11
12 ARCHITECTURE behav OF mux_err IS
13 BEGIN
14     PROCESS(a, b, sel)                -- 修正 1: 完整敏感列表
15         VARIABLE tmp : STD_LOGIC;
16     BEGIN
17         IF sel = '1' THEN
18             tmp := a;
19         ELSE                                -- 修正 2: 添加 ELSE 分支
20             tmp := b;
21         END IF;
22         y <= tmp;                        -- 修正 3+4: 信号赋值符号 + 变量移除
23     END PROCESS;
24     -- 修正 5: 删除进程外多余的 y <= tmp;
25 END behav;

```

评分标准: 指出并改正一处错误得 1 分, 最高 5 分。修改后代码逻辑正确额外给分 (已包含在内)。

6.4 四、程序阅读分析题解析（5 分）

- (1) (1 分) 电路类型：该 VHDL 代码描述的是一个 8 分频器（除以 8 的分频器），或称 3 位二进制计数分频器。具体而言，每 4 个输入时钟周期翻转一次输出（count 从 0 计到 3 共 4 个周期，即 8 个 clk 周期一个完整输出周期），输出频率 = 输入频率/8。

评分标准：答出“分频器”得 0.5 分，准确指出分频比为 8 得 0.5 分。

- (2) (2 分) 输出频率计算：

计数范围：count 从 0 到 3，共 4 个状态（0, 1, 2, 3）。每个 clk 上升沿 count 加 1。当 count=3 时，下一时钟 count 归 0 且 temp 翻转。即 temp 每 4 个 clk 翻转一次（半个周期），故 temp 的完整周期为 $4 \times 2 = 8$ 个 clk 周期。

输出频率： $f_{out} = f_{clk}/8 = 8 \text{ MHz}/8 = 1 \text{ MHz}$

评分标准：计数范围分析正确（1 分），计算结果 1MHz 正确（1 分）。

- (3) (1 分) temp 的作用：temp 是输出时钟的中间存储信号（在 PROCESS 内驱动，通过并发赋值输出）。不能直接用 count 作为输出的原因：count 是多位整数，需要的是单 bit 的时钟信号；且 count 仅反映计数值而非分频后的方波。temp 实现了“每 4 个时钟翻转一次（占空比 50%）”的方波产生功能。

评分标准：解释 temp 作用得 0.5 分；解释为何不用 count 得 0.5 分。

- (4) (1 分) 复位类型与判断依据：该电路的复位是异步复位。判断依据：PROCESS 的敏感信号列表中同时包含了 clk 和 rst，且代码中 IF rst = '1' THEN 在 ELSIF rising_edge(clk) THEN 前，这意味着无论时钟如何，只要 rst 有效便立即复位，不依赖时钟沿。

评分标准：判断为异步复位得 0.5 分，给出敏感列表或 IF 顺序的判断依据得 0.5 分。

6.5 五、综合设计题答案（60 分）

6.5.1 设计题 1：BCD 码—七段数码管译码器（20 分）

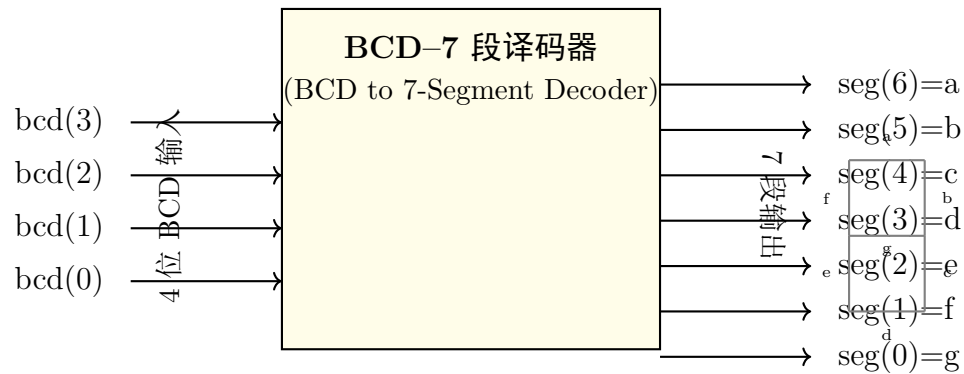
- (1) 真值表（6 分）

共阳极数码管：段输出 '0' 点亮，'1' 熄灭。 $\text{seg}(6:0) = \{a, b, c, d, e, f, g\}$ 。

BCD	显示	seg(6)=a	seg(5)=b	seg(4)=c	seg(3)=d	seg(2)=e	seg(1)=f	seg(0)=g
0000	0	0	0	0	0	0	0	1
0001	1	1	0	0	1	1	1	1
0010	2	0	0	1	0	0	1	0
0011	3	0	0	0	0	1	1	0
0100	4	1	0	0	1	1	0	0
0101	5	0	1	0	0	1	0	0
0110	6	0	1	0	0	0	0	0
0111	7	0	0	0	1	1	1	1
1000	8	0	0	0	0	0	0	0
1001	9	0	0	0	0	1	0	0
1010–1111	(暗)	1	1	1	1	1	1	1

评分标准：0–9 各数码段码正确（4 分，每错一个扣 0.5 分），非法输入 10–15 全灭（1 分），表格格式规范（1 分）。

(2) 顶层模块端口框图（4 分）



评分标准：模块框架清晰（1 分），输入端口标注完整（1 分），输出端口标注完整（含段对应关系）（2 分）。

(3) VHDL 代码（10 分）

Listing 6.2: BCD 码–七段数码管译码器 VHDL 代码

```
1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3
4 ENTITY bcd_7seg IS
5     PORT(
6         bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
7         seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)  -- seg(6)=a, ..., seg(0)=g
8     );
9 END ENTITY;
```

```

8   );
9   END bcd_7seg;
10
11  ARCHITECTURE dataflow OF bcd_7seg IS
12  BEGIN
13      PROCESS(bcd)
14      BEGIN
15          CASE bcd IS
16              WHEN "0000" => seg <= "0000001";  -- 0: 仅 g 段灭
17              WHEN "0001" => seg <= "1001111";  -- 1: b, c 段亮
18              WHEN "0010" => seg <= "0010010";  -- 2
19              WHEN "0011" => seg <= "0000110";  -- 3
20              WHEN "0100" => seg <= "1001100";  -- 4
21              WHEN "0101" => seg <= "0100100";  -- 5
22              WHEN "0110" => seg <= "0100000";  -- 6
23              WHEN "0111" => seg <= "0001111";  -- 7
24              WHEN "1000" => seg <= "0000000";  -- 8: 全亮
25              WHEN "1001" => seg <= "0000100";  -- 9
26              WHEN OTHERS => seg <= "1111111";  -- 10~15: 全灭 (共阳极=1)
27          END CASE;
28      END PROCESS;
29  END dataflow;

```

评分标准：库声明（1 分）、实体端口的位宽和方向正确（2 分）、CASE 语句覆盖 0~9（4 分，每错 1 个扣 0.5 分）、OTHERS 分支正确（1 分）、段码与位的对应关系注释清晰（1 分）、代码规范和注释（1 分）。

6.5.2 设计题 2：4 位双向移位寄存器（20 分）

(1) 顶层端口框图（4 分）



评分标准：框架清晰（1 分），输入端口标注完整含方向（2 分），输出端口位宽标注（1 分）。

(2) D 触发器基本单元 VHDL 代码（含于结构体分值中）

(3) 顶层结构化 VHDL 代码（12 分）

Listing 6.3: 4 位双向移位寄存器 VHDL 代码（含 D 触发器和顶层）

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  -- =====
5  -- 底层模块：带使能的 D 触发器
6  -- =====
7  ENTITY dff_en IS
8      PORT(
9          clk : IN  STD_LOGIC;
10         rst : IN  STD_LOGIC;
11         d   : IN  STD_LOGIC;
12         q   : OUT STD_LOGIC
13     );
14 END dff_en;
15
16 ARCHITECTURE behav OF dff_en IS
17 BEGIN
18     PROCESS(clk, rst)
19     BEGIN
20         IF rst = '1' THEN
21             q <= '0';
22         ELSIF rising_edge(clk) THEN
23             q <= d;
24         END IF;
25     END PROCESS;
26 END behav;
27
28 -- =====
29 -- 顶层模块：4 位双向移位寄存器
30 -- =====
31 LIBRARY IEEE;
32 USE IEEE.STD_LOGIC_1164.ALL;
33

```

```

34 ENTITY shift_reg_4bit IS
35     PORT(
36         clk  : IN  STD_LOGIC;
37         rst  : IN  STD_LOGIC;
38         dir  : IN  STD_LOGIC;    -- '0'=右移, '1'=左移
39         sil  : IN  STD_LOGIC;    -- 左移串行输入
40         sir  : IN  STD_LOGIC;    -- 右移串行输入
41         q    : OUT STD_LOGIC_VECTOR(3 DOWNT0 0)
42     );
43 END shift_reg_4bit;
44
45 ARCHITECTURE structural OF shift_reg_4bit IS
46     COMPONENT dff_en
47     PORT(
48         clk : IN  STD_LOGIC;
49         rst : IN  STD_LOGIC;
50         d   : IN  STD_LOGIC;
51         q   : OUT STD_LOGIC
52     );
53 END COMPONENT;
54
55 SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNT0 0); -- 内部连线
56 SIGNAL d_int : STD_LOGIC_VECTOR(3 DOWNT0 0); -- 各D触发器的D输入
57 BEGIN
58     -- 使用FOR...GENERATE生成各级D触发器
59     gen_reg: FOR i IN 0 TO 3 GENERATE
60     BEGIN
61         -- 第0级(最低位): 特殊连接
62         first_bit: IF i = 0 GENERATE
63             d_int(0) <= sil WHEN dir = '1' ELSE q_int(1);
64         END GENERATE first_bit;
65
66         -- 中间级(第1、2位)
67         middle_bits: IF i > 0 AND i < 3 GENERATE
68             d_int(i) <= q_int(i-1) WHEN dir = '1' ELSE q_int(i+1);
69         END GENERATE middle_bits;
70
71         -- 第3级(最高位): 特殊连接
72         last_bit: IF i = 3 GENERATE

```



```

73      d_int(3) <= q_int(2) WHEN dir = '1' ELSE sir;
74  END GENERATE last_bit;
75
76  -- 实例化 D 触发器
77  dff_inst: dff_en
78      PORT MAP(
79      clk => clk,
80      rst => rst,
81      d   => d_int(i),
82      q   => q_int(i)
83  );
84  END GENERATE gen_reg;
85
86  -- 输出连接
87  q <= q_int;
88  END structural;

```

评分标准：dff_en 模块正确（2 分），COMPONENT 声明（1 分），FOR GENERATE（1 分），IF GENERATE 区分边界（2 分），MUX 逻辑（dir 控制选通）正确（3 分），级联关系正确（2 分），代码规范（1 分）。

(4) GENERATE 语句的优势（含于 4 分中的内容）：

使用 GENERATE 语句相比手动编写 4 次 D 触发器实例化具有以下优势：

- **代码简洁性：**只需一次编写，循环自动生成，避免重复代码；
- **可扩展性：**修改位宽仅需修改 GENERATE 范围参数，无需逐一修改实例化代码；
- **可维护性：**修改单元逻辑只需改一处模板，减少出错概率；
- **可读性：**结构化的循环体现硬件阵列的规整性，便于理解设计意图。

评分标准：每条优势 1 分，共 4 分。答出可扩展性、简洁性、可维护性、可读性中任 2-3 条即可，合理阐述即可得分。

6.5.3 设计题 3：“1011”序列检测器（Moore 型和 Mealy 型对比）（20 分）

(1) Moore 型设计（8 分）

Moore 型状态转换图（3 分）

out=0; [state] (S1) [right of=S0] **S1**

已匹配 “1”

out=0; [state] (S2) [right of=S1] **S2**

已匹配 “10”

out=0; [state] (S3) [below of=S1, yshift=-1.2cm] **S3**

已匹配 “101”

out=0; [state] (S4) [below of=S2, yshift=-2.5cm] **S4**

已匹配 “1011”

out=1;

S0) edge [loop above] node {din=0} S0) edge node din=1 (S1) (S1) edge node din=0 (S2) edge [loop above] node din=1 (S1) (S2) edge [bend left] node din=1 (S3) edge [bend right=45] node [left] din=0 (S0) (S3) edge [bend right] node [right] din=0 (S2) edge [bend left] node din=1 (S4) (S4) edge [bend right=60] node [right] din=0 (S2) edge [bend left=60] node [left] din=1 (S1);

```

18  PROCESS(clk)
19  BEGIN
20      IF rising_edge(clk) THEN
21          IF rst = '1' THEN
22              current_state <= S0;
23          ELSE
24              current_state <= next_state;
25          END IF;
26      END IF;
27  END PROCESS;
28
29  -- 组合进程：次态逻辑 + 输出逻辑
30  PROCESS(current_state, din)
31  BEGIN
32      next_state <= S0;
33      found <= '0';           -- 默认输出 0
34
35      CASE current_state IS
36          WHEN S0 =>
37              IF din = '1' THEN next_state <= S1;
38              ELSE next_state <= S0; END IF;
39
40          WHEN S1 =>
41              IF din = '0' THEN next_state <= S2;
42              ELSE next_state <= S1; END IF;
43
44          WHEN S2 =>
45              IF din = '1' THEN next_state <= S3;
46              ELSE next_state <= S0; END IF;
47
48          WHEN S3 =>
49              IF din = '1' THEN next_state <= S4;
50              ELSE next_state <= S2; END IF;  -- 重叠："1010"→后两位"10"对应
51                                              S2
52
53          WHEN S4 =>
54              found <= '1';           -- 检测到 1011
55              IF din = '0' THEN next_state <= S2;
56              ELSE next_state <= S1; END IF;

```

```
56     END CASE;  
57     END PROCESS;  
58 END moore;
```

评分标准：状态枚举（1 分），同步复位（1 分），次态逻辑正确（2 分），输出逻辑符合 Moore 型（1 分）。

(2) Mealy 型设计（8 分）

Mealy 型状态转换图（3 分）

```

12
13 ARCHITECTURE mealy OF seq_det_1011_mealy IS
14     TYPE state_type IS (S0, S1, S2, S3);
15     SIGNAL current_state, next_state : state_type;
16 BEGIN
17     -- 时序进程：状态寄存器 + 同步复位
18     PROCESS(clk)
19     BEGIN
20         IF rising_edge(clk) THEN
21             IF rst = '1' THEN
22                 current_state <= S0;
23             ELSE
24                 current_state <= next_state;
25             END IF;
26         END IF;
27     END PROCESS;
28
29     -- 组合进程：次态逻辑 + 输出逻辑 (Mealy: 输出依赖当前状态和输入)
30     PROCESS(current_state, din)
31     BEGIN
32         next_state <= S0;
33         found <= '0';           -- 默认输出 0
34
35         CASE current_state IS
36             WHEN S0 =>
37                 IF din = '1' THEN next_state <= S1; END IF;
38
39             WHEN S1 =>
40                 IF din = '0' THEN next_state <= S2;
41                 ELSE next_state <= S1; END IF;
42
43             WHEN S2 =>
44                 IF din = '1' THEN next_state <= S3;
45                 ELSE next_state <= S0; END IF;
46
47             WHEN S3 =>
48                 IF din = '1' THEN
49                     next_state <= S1;
50                     found <= '1';           -- Mealy: 检测到 1011, 同周期输出

```

```

51         ELSE
52             next_state <= S2;  -- 重叠: "1010"→后两位"10"
53         END IF;
54     END CASE;
55 END PROCESS;
56 END mealy;

```

评分标准：状态枚举（1 分），同步复位（1 分），次态逻辑（2 分），Mealy 型输出（与输入同周期）（1 分）。

(3) 对比分析（4 分）

(a) found 有效时间（2 分）：

- **Moore 型：**检测到“1011”后，found 在下一个时钟周期有效（第 4 位之后的时钟周期），即在序列最后一位“1”被采样的时钟沿之后的那个时钟周期输出 1。
- **Mealy 型：**检测到“1011”后，found 在同一时钟周期有效（第 4 位被采样的同时输出 1），即在序列最后一位“1”被采样的同一个时钟周期内输出 1。

(b) 状态数及优劣比较（2 分）：

- Moore 型需要 5 个状态（S0-S4），Mealy 型只需要 4 个状态（S0-S3）。
- **Moore 型优点：**输出仅与当前状态有关，与输入信号隔离，输出信号更稳定（无输入毛刺影响），时序分析较简单。**缺点：**状态数多于 Mealy 型，输出时序比 Mealy 型晚一个周期。
- **Mealy 型优点：**状态数更少，响应更及时（输出与输入同周期）。**缺点：**输出依赖于输入电平，输入信号的毛刺可能影响输出；异步输入变化可能导致输出不稳定。

评分标准：(a) 各 1 分，准确指出 Moore 晚 1 个周期、Mealy 同周期即可；(b) 状态数正确 0.5 分，优缺点分析 1.5 分。

第七章 第 7 套试卷——参考答案与评分标准

7.1 一、选择题答案（18 分）

1. 答案：D（3 分）

解析：DMA（Direct Memory Access，直接存储器访问）是计算机系统中实现外设与内存间高速数据交换的机制，不属于 FPGA 的配置（编程）模式。FPGA 常见配置模式包括：Master Serial（主串模式）、Slave Parallel（从并模式）、JTAG 模式、Master Parallel（主并模式）及 SPI 模式等。

2. 答案：C（3 分）

解析：结构体中的并发信号赋值语句（如 $y \leq a \text{ AND } b;$ ）是并发执行的。选项 A 错误（PROCESS 内部为顺序执行）；选项 B 错误（结构体中多个 PROCESS 之间是并发执行的关系）；选项 D 错误（顺序语句可以出现在 PROCESS、FUNCTION、PROCEDURE 中，但不能直接出现在结构体的并发区域）。

3. 答案：B（3 分）

解析：Moore 型状态机与 Mealy 型状态机的根本区别在于：Moore 型输出仅取决于当前状态，与输入信号无关；Mealy 型输出同时取决于当前状态和当前输入。选项 A 错误（两者均支持同步/异步复位）；选项 C 不一定（Mealy 型状态数虽通常较少但非绝对）；选项 D 错误（两者均可检测重叠序列）。

4. 答案：C（3 分）

解析：VHDL 中运算符优先级：NOT（单目逻辑运算符）优先级最高，其次是乘法类（*、/、MOD、REM），然后是加法类（+、-、&），最后是关系运算符和双目逻辑运算符（AND、OR、NAND、NOR、XOR、XNOR 等优先级相同）。因此 NOT 优先级高于 AND、OR、XOR。

5. 答案：B（3 分）

解析：CPLD 通常基于乘积项结构，采用连续式互连（Crossbar），信号延时可预测；FPGA 基于查找表（LUT）结构，采用分段式互连，延时与布局布线有关。选项 A 将工艺说反（CPLD 多为 Flash/EEPROM，FPGA 多为 SRAM）；选项 C 错误（FPGA 逻辑容量通常远大于 CPLD）；选项 D 错误（CPLD 引脚到引脚延时通常更短）。

6. 答案：C（3 分）

解析：FOR ... GENERATE 是并发语句，主要用途是在结构体中重复生成多个相同结构的硬件单元（如多个元件例化），以简化代码。选项 A 描述的是仿真行为；选项 B 描述的是进程内的 FOR LOOP（顺序语句）；选项 D 错误（CASE 是选择语句，与 GENERATE 无关）。

7.2 二、填空题答案（12 分）

1. 答案：静态/设计（或：位宽/延时等）；COMPONENT（每空 1.5 分，共 3 分）

2. 答案：IF GENERATE（3 分）

3. 答案：结构体（ARCHITECTURE）的声明区（或包/ENTITY）；PROCESS（每空 1.5 分，共 3 分）

4. 答案：所有被读取的输入信号（或：所有出现在赋值表达式右侧的信号）（3 分）

解析：若敏感信号列表不完整，PROCESS 不会在缺失信号变化时重新计算，导致仿真与综合结果不一致。对于组合逻辑，综合时若敏感列表不全，可能推断出锁存器（Latch）来存储未完全分支的信号值。

7.3 三、程序改错题解析（5 分）

错误分析（共 5 处，每处 1 分）：

错误 1：（第 13 行—敏感列表不完整）进程敏感信号列表 PROCESS(a, b) 缺少 en 信号。当 en 变化时进程不会触发，导致仿真行为错误，且可能综合出锁存器。
更正：PROCESS(a, b, en)

错误 2：（第 16 行—信号 temp 类型与赋值问题）temp <= TO_INTEGER(UNSIGNED(a)) + TO_INTEGER(UNSIGNED(b));——temp 是 SIGNAL，使用<=赋值。但 temp 仅声明为 0-15 的范围，而 a+b 可能产生进位（如 a=b="1111" 时 sum=30），temp 范围不足以容纳。**更正：**将 temp 声明改为 SIGNAL temp : INTEGER RANGE 0 TO 30; 或使用 5 位中间信号。

错误 3: (第 17 行—类型转换函数错误) `TO_UNSIGNED(temp, 4)` 中, `temp` 取值范围超过 4 位无符号数的表示范围(当 $a+b>15$ 时)。**更正:** 使用 `TO_UNSIGNED(temp, 5)`, 同时 `sum` 端口改为 5 位或取低 4 位 `TO_UNSIGNED(temp, 5)(3 DOWNTO 0)`。

错误 4: (第 19–21 行—变量赋值符号错误) `cout := '1';` 和 `cout := '0';`——`cout` 是 OUT 端口(本质是 SIGNAL), 在 PROCESS 中应使用信号赋值符号 `<=` (`:=` 仅用于 VARIABLE)。**更正:** `cout <= '1';` 和 `cout <= '0';`

错误 5: (第 13–23 行—缺少 ELSE 分支导致锁存器推断) IF 语句仅在 `en='1'` 时对 `sum` 和 `cout` 赋值, 当 `en='0'` 时未赋值。组合逻辑中不完全的 IF 分支会导致综合工具推断出锁存器。**更正:** 添加 ELSE 分支: `ELSE sum <= (OTHERS => '0');` `cout <= '0';`

修正后的完整 VHDL 代码:

Listing 7.1: 修正后的 4 位加法器 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY adder4 IS
6      PORT(
7          a, b : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
8          en   : IN  STD_LOGIC;
9          sum  : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
10         cout : OUT STD_LOGIC
11     );
12 END adder4;
13
14 ARCHITECTURE behav OF adder4 IS
15     SIGNAL temp : UNSIGNED(4 DOWNTO 0); -- 5位, 容纳进位
16 BEGIN
17     PROCESS(a, b, en) -- 修正1: 添加 en 到敏感列表
18     BEGIN
19         IF en = '1' THEN
20             temp <= UNSIGNED('0' & a) + UNSIGNED('0' & b); -- 修正2+3: 扩
                展为5位
21             sum  <= STD_LOGIC_VECTOR(temp(3 DOWNTO 0));
22             cout <= temp(4); -- 修正4: 信号赋值 <=

```

```

23      ELSE
24          sum  <= (OTHERS => '0');           -- 修正 5: 添加 ELSE 分支
25          cout <= '0';
26      END IF;
27  END PROCESS;
28 END behav;

```

评分标准：指出并改正一处错误得 1 分，最高 5 分。错误描述准确（含行号/位置）和改正方案正确各占 0.5 分。

7.4 四、程序阅读分析题解析（5 分）

- (1) (1 分) 电路类型：该 VHDL 代码描述的是一个参数化 N 级二分频器（或称除以 2^N 的分频器）。通过 T 触发器翻转原理实现，每级将输入频率除以 2。

评分标准：答出“分频器”或“二分频器链”得 0.5 分，指出参数化得 0.5 分。

- (2) (2 分) N 的作用及频率关系：

参数 N 决定了分频器链的级数和分频比。计数器从 0 计数到 $2^N - 1$ ，共 2^N 个状态。每个时钟上升沿 count 加 1，当 count 计满 2^N 时，temp 翻转一次（半周期），因此 temp 的完整周期为 $2 \times 2^N = 2^{N+1}$ 个 clk 周期。

当 $N=4$ 时，分频比： $f_{out} = f_{clk}/2^{N+1} = f_{clk}/2^5 = f_{clk}/32$

即输出频率 = 输入时钟频率 / 32。

评分标准： N 的作用解释正确（1 分）， $N=4$ 时频率关系正确（1 分）。

- (3) (2 分) 复位类型及判断依据：该电路的复位 rst 是异步复位。判断依据：

- PROCESS 的敏感信号列表同时包含 clk 和 rst (PROCESS(clk, rst));
- 代码结构为 IF rst = '1' THEN ... ELSIF rising_edge(clk) THEN ...——复位条件判断在时钟沿检测之前，且 rst 变化（不依赖时钟沿）即可立即触发复位操作。

评分标准：正确判断异步复位（1 分），提供敏感列表或 IF/ELSIF 顺序的判断依据（1 分）。

7.5 五、综合设计题参考答案（60 分）

7.5.1 设计题 1：BCD 码转 7 段数码管译码器（20 分）

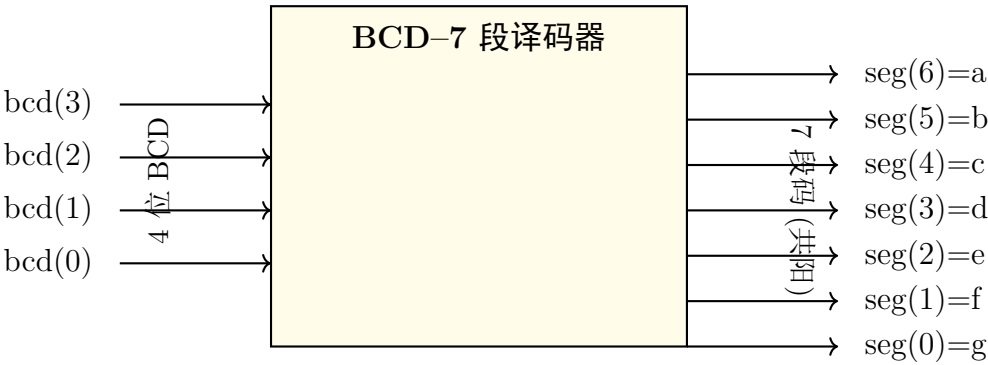
(1) 真值表（5 分）

共阳极数码管：段输出 ‘0’ 点亮，‘1’ 熄灭。seg(6:0) = {a, b, c, d, e, f, g}。

BCD	字符	a(6)	b(5)	c(4)	d(3)	e(2)	f(1)	g(0)
0000	0	0	0	0	0	0	0	1
0001	1	1	0	0	1	1	1	1
0010	2	0	0	1	0	0	1	0
0011	3	0	0	0	0	1	1	0
0100	4	1	0	0	1	1	0	0
0101	5	0	1	0	0	1	0	0
0110	6	0	1	0	0	0	0	0
0111	7	0	0	0	1	1	1	1
1000	8	0	0	0	0	0	0	0
1001	9	0	0	0	0	1	0	0
1010–1111	(灭)	1	1	1	1	1	1	1

评分标准：0–9 各数码段码正确（4 分），非法输入全灭（1 分）。

(2) 顶层模块端口框图（4 分）



评分标准：框图框架完整（1 分），输入端标注 bcd(3:0) 方向（1 分），输出端标注 seg(6:0) 方向及段对应（2 分）。

(3) VHDL 代码（WITH...SELECT 数据流风格）（8 分）

Listing 7.2: BCD 码—七段译码器——WITH-SELECT 数据流风格

```
1 LIBRARY IEEE;  
2 USE IEEE.STD_LOGIC_1164.ALL;
```

```

3
4 -- =====
5 -- 设计思路：使用 WITH...SELECT 选择信号赋值语句实现数据流风格。
6 -- bcd(3:0) 作为选择表达式，根据其取值选通对应的 7 段码输出。
7 -- seg(6)=a, seg(5)=b, ..., seg(0)=g, 共阳极：0=亮, 1=灭。
8 -- 有效 BCD 范围 0~9 (0000~1001)，超出范围全灭。
9 -- =====
10
11 ENTITY bcd_7seg IS
12     PORT(
13         bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
14         seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)  -- seg(6)=a, ..., seg(0)=g
15     );
16 END bcd_7seg;
17
18 ARCHITECTURE dataflow OF bcd_7seg IS
19 BEGIN
20     WITH bcd SELECT
21         seg <= "0000001" WHEN "0000",  -- 0
22                "1001111" WHEN "0001",  -- 1
23                "0010010" WHEN "0010",  -- 2
24                "0000110" WHEN "0011",  -- 3
25                "1001100" WHEN "0100",  -- 4
26                "0100100" WHEN "0101",  -- 5
27                "0100000" WHEN "0110",  -- 6
28                "0001111" WHEN "0111",  -- 7
29                "0000000" WHEN "1000",  -- 8
30                "0000100" WHEN "1001",  -- 9
31                "1111111" WHEN OTHERS;  -- 10~15: 全灭
32 END dataflow;

```

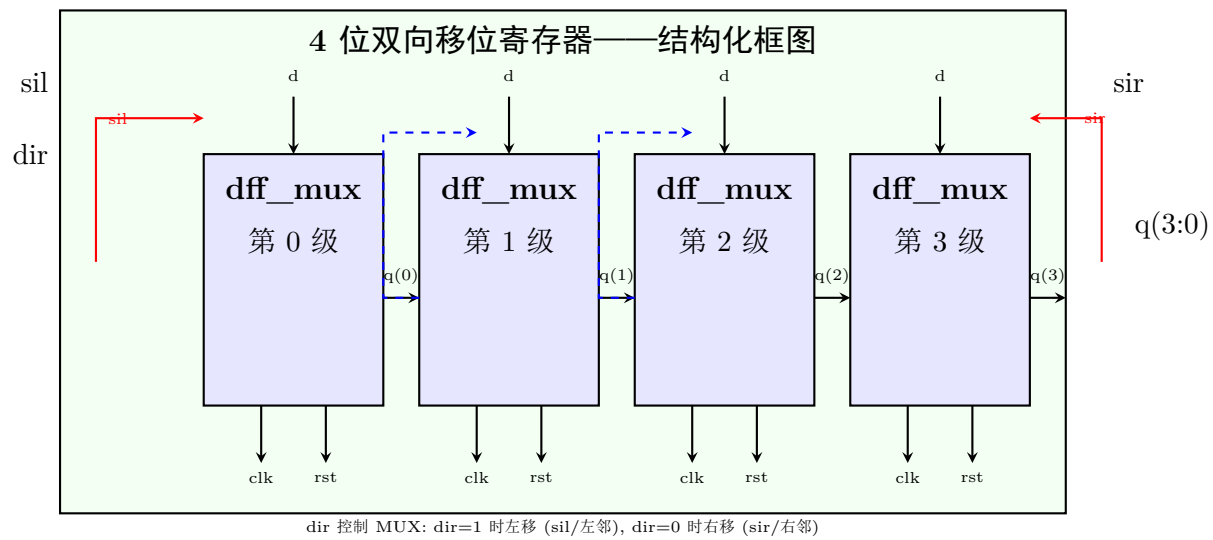
评分标准：使用 WITH-SELECT 语句（2 分），0~9 段码全部正确（4 分），OTHERS 分支处理非法输入（1 分），代码规范（1 分）。

(4) 设计思路注释说明（3 分，已包含在上方代码注释中）

评分标准：注释说明清晰完整（3 分）。要点：说明 WITH-SELECT 的数据流风格、共阳极逻辑（0 亮 1 灭）、seg 各位与段的对应关系、有效范围和 OTHERS 处理。

7.5.2 设计题 2：4 位双向移位寄存器（结构化设计）（20 分）

(1) 结构框图（5 分）



评分标准：顶层框图和 4 个 DFF_MUX 模块清晰（2 分），级联关系标注正确（含 MUX 逻辑）（2 分），输入/输出信号标注完整（1 分）。

(2) 底层模块 dff_mux 代码（5 分）

Listing 7.3: dff_mux 底层模块 VHDL 代码

```
1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3
4 ENTITY dff_mux IS
5     PORT(
6         clk      : IN  STD_LOGIC;
7         rst      : IN  STD_LOGIC;
8         dir      : IN  STD_LOGIC;    -- 方向控制：'0'=右移，'1'=左移
9         d_left   : IN  STD_LOGIC;    -- 左移数据输入
10        d_right  : IN  STD_LOGIC;    -- 右移数据输入
11        q        : OUT STD_LOGIC
12    );
13 END dff_mux;
14
15 ARCHITECTURE behav OF dff_mux IS
16     SIGNAL d_sel : STD_LOGIC;    -- MUX选通后的D输入
17 BEGIN
```

```

18  -- 2选1 MUX: dir=1选择左移数据, dir=0选择右移数据
19  d_sel <= d_left WHEN dir = '1' ELSE d_right;
20
21  -- D触发器 (异步复位)
22  PROCESS(clk, rst)
23  BEGIN
24      IF rst = '1' THEN
25          q <= '0';
26      ELSIF rising_edge(clk) THEN
27          q <= d_sel;
28      END IF;
29  END PROCESS;
30  END behav;

```

评分标准：实体端口定义完整（1分），MUX 逻辑正确（dir 选择 d_left/d_right）（2分），D 触发器含异步复位（1分），代码规范（1分）。

(3) 顶层模块 VHDL 代码（10 分）

Listing 7.4: 4 位双向移位寄存器——顶层结构化代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY shift_reg_4bit IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;
8          dir      : IN  STD_LOGIC;    -- '1'=左移, '0'=右移
9          si_left  : IN  STD_LOGIC;    -- 左移串行输入
10         si_right : IN  STD_LOGIC;    -- 右移串行输入
11         q        : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
12     );
13  END shift_reg_4bit;
14
15  ARCHITECTURE structural OF shift_reg_4bit IS
16      -- 组件声明
17      COMPONENT dff_mux
18          PORT(
19              clk      : IN  STD_LOGIC;
20              rst      : IN  STD_LOGIC;

```

```

21     dir      : IN  STD_LOGIC;
22     d_left   : IN  STD_LOGIC;
23     d_right  : IN  STD_LOGIC;
24     q        : OUT STD_LOGIC
25 );
26 END COMPONENT;
27
28 SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNT0 0); -- 内部级联信号
29 BEGIN
30     -- FOR...GENERATE 循环例化 4 个 dff_mux
31     gen_shift: FOR i IN 0 TO 3 GENERATE
32     BEGIN
33         -- 第 0 级（最低位）——边界条件
34         first: IF i = 0 GENERATE
35             dff0: dff_mux PORT MAP(
36                 clk      => clk,
37                 rst      => rst,
38                 dir      => dir,
39                 d_left   => si_left,      -- 左移：从 si_left 输入
40                 d_right  => q_int(i+1),  -- 右移：从第 1 级输入
41                 q        => q_int(i)
42             );
43         END GENERATE first;
44
45         -- 中间级（第 1 级、第 2 级）
46         middle: IF i > 0 AND i < 3 GENERATE
47             dff_mid: dff_mux PORT MAP(
48                 clk      => clk,
49                 rst      => rst,
50                 dir      => dir,
51                 d_left   => q_int(i-1),  -- 左移：从低一级输入
52                 d_right  => q_int(i+1),  -- 右移：从高一级输入
53                 q        => q_int(i)
54             );
55         END GENERATE middle;
56
57         -- 第 3 级（最高位）——边界条件
58         last: IF i = 3 GENERATE
59             dff3: dff_mux PORT MAP(

```

```

60      clk      => clk,
61      rst      => rst,
62      dir      => dir,
63      d_left   => q_int(i-1),      -- 左移：从第2级输入
64      d_right  => si_right,        -- 右移：从 si_right 输入
65      q        => q_int(i)
66  );
67  END GENERATE last;
68  END GENERATE gen_shift;
69
70  -- 输出连接
71  q <= q_int;
72  END structural;

```

评分标准：COMPONENT 声明正确（1 分），FOR GENERATE 循环（1 分），IF GENERATE 区分边界（第 0 级和第 3 级）（4 分），中间级级联正确（2 分），输出连接正确（1 分），代码规范（1 分）。

7.5.3 设计题 3：“1011”序列检测器 Moore 状态机（20 分）

(1) 状态转移图（8 分）

```

      found=0; [state] (S1) [right of=S0] S1
                        已匹配 “1”
      found=0; [state] (S2) [right of=S1] S2
                        已匹配 “10”
      found=0; [state] (S3) [below of=S1, yshift=-1.0cm] S3
                        已匹配 “101”
      found=0; [state] (S4) [below of=S2, yshift=-2.5cm] S4
                        已匹配 “1011”
                        found=1;

S0) edge [loop above] node {din=0} S0) edge node din=1 (S1) (S1) edge node din=0 (S2) edge
[loop above] node din=1 (S1) (S2) edge [bend left] node din=1 (S3) edge [bend right=50] node
[left] din=0 (S0) (S3) edge [bend right] node [right] din=0 (S2) edge [bend left] node din=1
(S4) (S4) edge [bend right=60] node [right] din=0 (S2) edge [bend left=60] node [left] din=1
(S1);

```


- S0（初始）：无匹配/复位状态。din=1 时进入 S1。
- S1（已匹配'1'）：din=0 进入 S2；din=1 保持 S1（重叠：最后一位'1' 作为新起点）。
- S2（已匹配'10'）：din=1 进入 S3；din=0 回到 S0。
- S3（已匹配'101'）：din=1 进入 S4；din=0 回到 S2（重叠："1010" 中后两位"10" 即 S2 前缀）。
- S4（已匹配'1011'）：found=1。din=0→S2（"10110" 后"10" 对应 S2）；din=1→S1（最后的'1' 作为新起点）。

评分标准：5 个状态定义且命名清晰（2 分），各状态输出标注正确（2 分），10 条转移弧全部正确（3 分），S4 的重叠转移处理正确（1 分）。

(2) 状态转移表（4 分）

当前状态	输入 din	次态	输出 found
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S4	0
S4	0	S2	1
S4	1	S1	1

评分标准：表格格式规范（1 分），10 行转移全部正确（3 分）。

(3) VHDL 代码——三进程描述风格（8 分 = 3 分功能 + 2 分风格 + 3 分规范）

Listing 7.5: Moore 型 “1011” 序列检测器——三进程描述 VHDL 代码

```
1 LIBRARY IEEE;
2 USE IEEE.STD_LOGIC_1164.ALL;
3
4 ENTITY seq_det_1011 IS
5     PORT(
6         clk    : IN    STD_LOGIC;
7         rst    : IN    STD_LOGIC;           -- 异步复位，高有效
```

```

8      din    : IN  STD_LOGIC;          -- 串行输入
9      found  : OUT STD_LOGIC          -- 检测到1011时输出1（持续一个时钟周
      期）
10     );
11 END seq_det_1011;
12
13 ARCHITECTURE moore_3proc OF seq_det_1011 IS
14     -- 自定义枚举类型（二进制编码：S0=000,S1=001,S2=010,S3=011,S4=100）
15     TYPE state_type IS (S0, S1, S2, S3, S4);
16     SIGNAL current_state, next_state : state_type;
17 BEGIN
18     -- =====
19     -- 进程1：时序进程——状态寄存器（含异步复位）
20     -- =====
21     state_reg: PROCESS(clk, rst)
22     BEGIN
23         IF rst = '1' THEN
24             current_state <= S0;          -- 异步复位至初始状态
25         ELSIF rising_edge(clk) THEN
26             current_state <= next_state;  -- 时钟上升沿更新状态
27         END IF;
28     END PROCESS state_reg;
29
30     -- =====
31     -- 进程2：组合进程——次态逻辑
32     -- =====
33     next_state_logic: PROCESS(current_state, din)
34     BEGIN
35         CASE current_state IS
36             WHEN S0 =>
37                 IF din = '1' THEN next_state <= S1;
38                 ELSE next_state <= S0; END IF;
39
40             WHEN S1 =>
41                 IF din = '0' THEN next_state <= S2;
42                 ELSE next_state <= S1; END IF;
43
44             WHEN S2 =>
45                 IF din = '1' THEN next_state <= S3;

```

```

46         ELSE next_state <= S0; END IF;
47
48     WHEN S3 =>
49         IF din = '1' THEN next_state <= S4;
50         ELSE next_state <= S2; END IF;    -- "1010"的后缀"10"
51
52     WHEN S4 =>
53         IF din = '0' THEN next_state <= S2; -- "10110"的"10"
54         ELSE next_state <= S1; END IF;      -- 最后'1'作为新起点
55     END CASE;
56 END PROCESS next_state_logic;
57
58 -- =====
59 -- 进程3: 组合进程——输出逻辑 (Moore型: 仅依赖当前状态)
60 -- =====
61 output_logic: PROCESS(current_state)
62 BEGIN
63     IF current_state = S4 THEN
64         found <= '1';    -- 仅在S4状态输出1
65     ELSE
66         found <= '0';
67     END IF;
68 END PROCESS output_logic;
69 END moore_3proc;

```

评分标准:

- 枚举类型定义 (1 分)
- 进程 1 (时序): 状态寄存器 + 异步复位正确 (1 分)
- 进程 2 (组合): 次态逻辑正确, 含重叠处理 (1 分)
- 进程 3 (组合): Moore 型输出逻辑 (仅依赖 current_state) (1 分)
- 三进程描述风格 (1 分)
- 代码规范: 缩进规范、信号命名清晰、注释合理 (3 分)

第八章 第 8 套试卷——参考答案与评分标准

8.1 一、选择题答案（18 分）

1. 答案：B（3 分）

解析：FPGA 中一个典型的逻辑单元（Slice）通常包含 LUT（查找表）、D 触发器、进位链和多路选择器。这是现代 FPGA（如 Xilinx 7 系列）Slice 的基本组成。选项 A 不完整（缺少进位链和 MUX），选项 C 描述的是嵌入式硬核处理器（如 Zynq 的 ARM 核），选项 D 描述的是时钟管理资源（如 CMT/MMCM）。

2. 答案：C（3 分）

解析：IEEE.NUMERIC_STD 是 IEEE 标准包，定义了 UNSIGNED 和 SIGNED 数据类型及其算术运算。该包提供了 STD_LOGIC_VECTOR 与 UNSIGNED 之间的类型转换。选项 A 仅定义 STD_LOGIC 类型，选项 B（STD_LOGIC_ARITH）是 Synopsys 非标准包，选项 D（STD_LOGIC_SIGNED）也是非标准包。

3. 答案：A（3 分）

解析：异步复位不依赖时钟即可生效（敏感列表含 rst），但对复位信号的毛刺较为敏感（因为任何 rst 变化都会触发复位）。选项 B 错误（同步复位需等待时钟沿，响应慢于异步复位）；选项 C 错误（异步复位必须将 rst 列入敏感信号表）；选项 D 不一定（同步复位未必占用更多资源，取决于综合工具）。

4. 答案：A（3 分）

解析：二进制编码使用最少的状态寄存器（ $\lceil \log_2 N \rceil$ 位），但译码逻辑较复杂（需要组合逻辑译码）。选项 B 错误（独热码需要 N 个寄存器，是最多的）；选项 C 错误（格雷码相邻状态只有 1 位翻转）；选项 D 错误（独热码因每次只有 1 位变化，抗干扰能力较好）。

5. 答案：B（3 分）

解析: SUBTYPE 关键字从已有类型派生子类型并添加范围约束 (如 SUBTYPE digit IS INTEGER RANGE 0 TO 9;), 而非创建全新类型。选项 A 错误 (创建全新类型使用 TYPE); 选项 C 描述的是信号本身的功能; 选项 D 描述的是 VARIABLE。

6. 答案: B (3 分)

解析: Block RAM 是 FPGA 片内专用的存储硬核 (集成在芯片上的专用 RAM 模块), 容量大、速度快; Distributed RAM 由 LUT 逻辑资源构成 (将 LUT 配置为小型 RAM), 灵活但占用逻辑资源。选项 A 将二者说反; 选项 C 错误 (有本质区别); 选项 D 错误 (两者均可存储数据, Block RAM 也可用作程序/数据存储器)。

8.2 二、填空题答案 (12 分)

1. 答案:

- clk'EVENT: 表示信号值发生变化 (1 分)
- rising_edge(clk) 等价于 clk'EVENT AND clk = '1' (1 分)
- data'RANGE 返回信号 data 的索引/位宽范围 (1 分)

2. 答案:

- 高阻态的字符表示为 'Z' (1 分)
- 描述双向总线时, 端口模式应使用 INOUT (1 分)
- 当多个三态门驱动同一总线时, 同一时刻最多只能有 1 个门的使能有效 (1 分)

3. 答案:

- 同步复位的典型结构: IF rising_edge(clk) THEN
IF rst = '1' THEN ... END IF;
END IF; (1.5 分)
- 异步复位的典型结构: IF rst = '1' THEN ...
ELSIF rising_edge(clk) THEN ... END IF; (1.5 分)

4. 答案:

- UNSIGNED→INTEGER: TO_INTEGER() (1 分)
- INTEGER→UNSIGNED: TO_UNSIGNED() (1 分)
- STD_LOGIC_VECTOR→UNSIGNED: UNSIGNED() (类型强制转换) (1 分)

8.3 三、程序改错题解析（5 分）

错误分析（共 5 处，每处 1 分）：

错误 1：（第 10 行）信号声明缺少分号：SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNT0 0) 末尾缺少分号。更正：SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNT0 0);

错误 2：（第 12 行）敏感信号列表不完整：PROCESS(clk) 缺少 rst 信号。由于 rst 是异步复位，必须列入敏感信号列表（PROCESS(clk, rst)），否则综合和仿真可能不一致。更正：PROCESS(clk, rst)

错误 3：（第 15 行）信号赋值符号错误：cnt := (OTHERS => '0');——变量赋值符号:=不能用于 SIGNAL 类型。更正：cnt <= (OTHERS => '0');

错误 4：（第 18 行）算术运算类型不匹配：cnt <= cnt + '1';——cnt 是 STD_LOGIC_VECTOR 类型，不能直接与字符'1'进行算术加法运算。需要引用 IEEE.NUMERIC_STD 包或 IEEE.STD_LOGIC_UNSIGNED 包。更正：添加 USE IEEE.NUMERIC_STD.ALL; 并将此行改为：

cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1); 或使用 STD_LOGIC_UNSIGNED 包后写成 cnt <= cnt + 1;

错误 5：（第 16 行）复位分支在时钟边沿内：IF rst = '1' THEN 在 PROCESS(clk) 内而未在敏感列表中包含 rst——由于错误 2，此问题一并修正后，原代码中的 IF rst = '1' THEN 处于 ELSIF rising_edge(clk) THEN 之前（正确位置），但需确保 rst 在敏感列表中。修正后异步复位逻辑正确。

或者如果综合工具要求同步复位：将 rst 判断移入 rising_edge(clk) 分支内部。

注：本题中 5 处错误较为明显，如上所列。错误 2 和错误 5 在修正敏感列表后自动解决，但作为独立知识点仍需指出。

修正后的完整 VHDL 代码：

Listing 8.1: 修正后的 8 位计数器 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;           -- 添加 NUMERIC_STD 支持算术运算
4
5  ENTITY counter8 IS
6      PORT(
7          clk    : IN    STD_LOGIC;

```

```

8      rst  : IN  STD_LOGIC;
9      en   : IN  STD_LOGIC;
10     q    : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)
11 );
12 END counter8;
13
14 ARCHITECTURE behav OF counter8 IS
15     SIGNAL cnt : STD_LOGIC_VECTOR(7 DOWNT0 0); -- 修正 1: 添加分号
16 BEGIN
17     PROCESS(clk, rst) -- 修正 2: 添加 rst 到敏感列表
18     BEGIN
19         IF rst = '1' THEN
20             cnt <= (OTHERS => '0'); -- 修正 3: <= 而非 :=
21         ELSIF rising_edge(clk) THEN
22             IF en = '1' THEN
23                 cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1); -- 修正 4: 类型转换
24             END IF;
25         END IF;
26     END PROCESS;
27     q <= cnt;
28 END behav;

```

评分标准：每正确指出并改正一处错误得 1 分，最高 5 分。需准确给出错误行号、错误原因和正确写法。

8.4 四、程序阅读分析题解析（5 分）

- (1) (1 分) 电路类型：该 VHDL 代码描述的是一个 4 位环形计数器（Ring Counter），或称循环右移寄存器。具体而言，复位后初始值为“1000”，之后每个时钟上升沿将最低位移至最高位（循环右移一位）。这种电路中任一时刻仅有一位为‘1’（独热码循环），属于约翰逊计数器（Johnson Counter）的变体。

评分标准：答出“环形计数器”或“循环移位寄存器”得 1 分。

- (2) (2 分) 连续 8 个时钟上升沿后的输出序列：

复位释放后 state = “1000”。每个时钟上升沿执行：state <= state(0) & state(3 DOWNT0 1)（循环右移一位）。

时钟沿编号	state (q) 值	说明
复位	1000	初始值
第 1 个	0100	右移 1 位
第 2 个	0010	右移 1 位
第 3 个	0001	右移 1 位
第 4 个	1000	循环回到初始
第 5 个	0100	重复
第 6 个	0010	
第 7 个	0001	
第 8 个	1000	

序列为：1000 → 0100 → 0010 → 0001 → 1000 → 0100 → 0010 → 0001 → 1000

评分标准：前 4 个值正确（1 分），后 4 个值正确（含循环）（1 分）。

(3) (1 分) 修改后电路类型及输出序列：

若改为 `state <= state(2 DOWNTO 0) & state(3);`（循环左移），该电路将变成 4 位循环左移环形计数器。输出序列变为：1000 → 0001 → 0010 → 0100 → 1000 → ...（即环形左移方向）。

原电路：右移序列 1000→0100→0010→0001→1000（4 状态循环）。修改后：左移序列 1000→0001→0010→0100→1000（4 状态循环，方向相反）。

评分标准：指出变为“左移环形计数器”（0.5 分），给出序列变化（0.5 分）。

(4) (1 分) q(0) 频率与 clk 频率之比：

输出信号 q(0) 每 4 个时钟周期出现一次‘1’（占空比 1:4），故 q(0) 的频率为时钟频率的 1/4。

推导过程：state 序列为 1000→0100→0010→0001→1000，共 4 个状态循环。q(0) 在状态为“0001”时为‘1’，在状态为“1000”时为‘0’……q(0) 每 4 个 clk 周期为 1 一次，故 $f_{q(0)} = f_{clk}/4$ 。

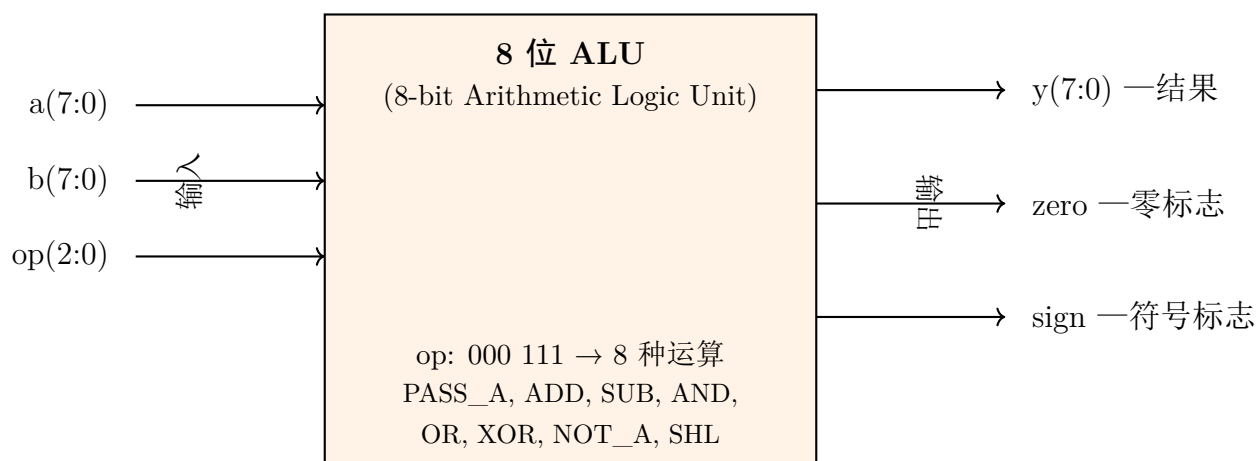
频率比：1:4（或 $f_{q(0)} : f_{clk} = 1 : 4$ ）

评分标准：频率比正确（0.5 分），推导过程完整（0.5 分）。

8.5 五、综合设计题参考答案（60 分）

8.5.1 设计题 1：8 位算术逻辑单元（ALU）（20 分）

(1) 顶层模块端口框图（5 分）



评分标准：框图框架完整（1分），输入端标注完整（a/b/op 含位宽）（2分），输出端标注完整（y/zero/sign）（2分）。

(2)+(3) VHDL 代码（12分 + 3分注释 = 15分）

Listing 8.2: 8 位 ALU——VHDL 行为描述代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;      -- 支持 UNSIGNED 算术运算
4
5  -- =====
6  -- 8位 ALU: 根据 op(2:0) 选择 8 种运算之一
7  -- 数据流方向: a, b, op → ALU 运算 → y, zero, sign
8  -- zero 标志: y 全 0 时输出 1
9  -- sign 标志: 直接取 y 的最高位
10 -- =====
11
12 ENTITY alu_8bit IS
13     PORT(
14         a      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 操作数 A
15         b      : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 操作数 B
16         op     : IN  STD_LOGIC_VECTOR(2 DOWNTO 0);  -- 操作选择码
17         y      : OUT STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 运算结果
18         zero   : OUT STD_LOGIC;                     -- 零标志
19         sign   : OUT STD_LOGIC                       -- 符号标志
20     );
21 END alu_8bit;
22

```

```

23 ARCHITECTURE behav OF alu_8bit IS
24     SIGNAL result : STD_LOGIC_VECTOR(7 DOWNTO 0);  -- 中间结果
25 BEGIN
26     -- ALU核心运算进程
27     PROCESS(a, b, op)
28         VARIABLE temp : UNSIGNED(8 DOWNTO 0);  -- 9位，容纳进位
29     BEGIN
30         CASE op IS
31             -- op="000": PASS_A —— 直通A
32             WHEN "000" =>
33                 result <= a;
34
35             -- op="001": ADD —— 加法 a + b
36             WHEN "001" =>
37                 result <= STD_LOGIC_VECTOR(UNSIGNED(a) + UNSIGNED(b));
38
39             -- op="010": SUB —— 减法 a - b
40             WHEN "010" =>
41                 result <= STD_LOGIC_VECTOR(UNSIGNED(a) - UNSIGNED(b));
42
43             -- op="011": AND_OP —— 按位与
44             WHEN "011" =>
45                 result <= a AND b;
46
47             -- op="100": OR_OP —— 按位或
48             WHEN "100" =>
49                 result <= a OR b;
50
51             -- op="101": XOR_OP —— 按位异或
52             WHEN "101" =>
53                 result <= a XOR b;
54
55             -- op="110": NOT_A —— 按位取反A
56             WHEN "110" =>
57                 result <= NOT a;
58
59             -- op="111": SHL_A —— 逻辑左移一位（低位补0）
60             WHEN "111" =>
61                 result <= a(6 DOWNTO 0) & '0';

```

```

62
63      -- 安全分支 (OTHERS覆盖未定义编码)
64      WHEN OTHERS =>
65          result <= (OTHERS => '0');
66      END CASE;
67  END PROCESS;
68
69  -- 输出赋值
70  y <= result;
71
72  -- zero标志生成: 当result全为0时置1
73  zero <= '1' WHEN result = "00000000" ELSE '0';
74
75  -- sign标志生成: 直接输出result的最高位 (第7位)
76  sign <= result(7);
77  END behav;

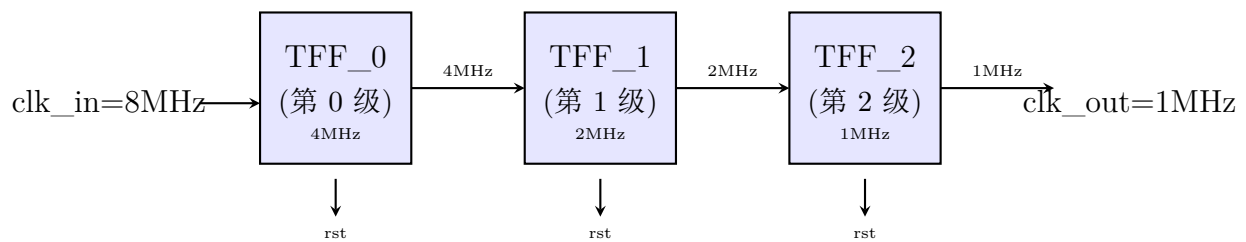
```

评分标准:

- 库声明 (NUMERIC_STD) (1 分)
- 实体端口完整 (2 分)
- CASE 语句覆盖 8 种 OP 编码 (每 2 个 1 分, 共 4 分)
- 算术运算类型转换正确 (UNSIGNED 转换) (2 分)
- OTHERS 分支 (1 分)
- zero 标志逻辑 (1 分)
- sign 标志逻辑 (1 分)
- 代码注释清晰: 数据流方向 (1 分), 各 OP 分支功能说明 (1 分), zero/sign 生成逻辑说明 (1 分)

8.5.2 设计题 2: N 级二分频器链 (20 分)

(1) N=3 时电路框图 (4 分)



分频关系: $f_{clk_in} = 8\text{MHz} \rightarrow 4\text{MHz} \rightarrow 2\text{MHz} \rightarrow 1\text{MHz} = f_{clk_in}/2^3$

评分标准: TFF 级联结构正确 (1 分), 各级频率标注正确 (1 分), 每级二分频关系明确 (1 分), 框图整洁清晰 (1 分)。

(2) T 触发器 (TFF) VHDL 代码 (3 分)

Listing 8.3: T 触发器 (TFF) VHDL 代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY tff IS
5      PORT(
6          clk : IN  STD_LOGIC;
7          rst : IN  STD_LOGIC;          -- 异步复位, 高有效
8          q   : OUT STD_LOGIC
9      );
10 END tff;
11
12 ARCHITECTURE behav OF tff IS
13 BEGIN
14     PROCESS(clk, rst)
15         VARIABLE toggle : STD_LOGIC := '0'; -- 翻转状态变量
16     BEGIN
17         IF rst = '1' THEN
18             toggle := '0';          -- 复位时清零
19             q <= '0';
20         ELSIF rising_edge(clk) THEN
21             toggle := NOT toggle;   -- 每个时钟上升沿翻转
22             q <= toggle;
23         END IF;
24     END PROCESS;
25 END behav;
  
```

评分标准：实体端口正确（0.5 分），异步复位逻辑（1 分），翻转逻辑（toggle := NOT toggle）（1 分），代码规范（0.5 分）。

(3) 顶层 N 级分频器链 VHDL 代码（10 分）

Listing 8.4: N 级二分频器链——顶层结构化 VHDL 代码

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  -- =====
5  -- N级二分频器链
6  -- 各级TFF依次将时钟二分频:  $f_{out} = f_{clk\_in} / 2^N$ 
7  -- 第0级TFF时钟=clk_in, 第i级时钟=第(i-1)级输出
8  -- =====
9
10 ENTITY freq_div_chain IS
11     GENERIC(
12         N : INTEGER := 4           -- 分频级数（默认4级，分频比 $2^4=16$ ）
13     );
14     PORT(
15         clk_in  : IN  STD_LOGIC;    -- 输入时钟
16         rst     : IN  STD_LOGIC;    -- 异步复位，高有效
17         clk_out : OUT STD_LOGIC      -- N级分频后输出
18     );
19 END freq_div_chain;
20
21 ARCHITECTURE structural OF freq_div_chain IS
22     -- 组件声明：单个T触发器
23     COMPONENT tff
24     PORT(
25         clk : IN  STD_LOGIC;
26         rst : IN  STD_LOGIC;
27         q   : OUT STD_LOGIC
28     );
29     END COMPONENT;
30
31     -- 内部信号：各级TFF的输出（N级）
32     SIGNAL chain : STD_LOGIC_VECTOR(0 TO N-1);
33 BEGIN
34     -- FOR...GENERATE循环实例化N个TFF

```

```

35  gen_chain: FOR i IN 0 TO N-1 GENERATE
36  BEGIN
37      -- 第0级: 时钟连接至 clk_in
38      first_stage: IF i = 0 GENERATE
39          tff0: tff PORT MAP(
40              clk => clk_in,
41              rst => rst,
42              q   => chain(0)
43          );
44      END GENERATE first_stage;
45
46      -- 第1~N-1级: 时钟连接至前一级的输出
47      other_stages: IF i > 0 GENERATE
48          tff_i: tff PORT MAP(
49              clk => chain(i-1),      -- 前级输出作为本级时钟
50              rst => rst,
51              q   => chain(i)
52          );
53      END GENERATE other_stages;
54  END GENERATE gen_chain;
55
56  -- 最终输出: 第N-1级的输出
57  clk_out <= chain(N-1);
58  END structural;

```

评分标准:

- GENERIC 参数声明 (N) (1 分)
- COMPONENT tff 声明 (1 分)
- FOR...GENERATE 循环 (2 分)
- IF GENERATE 区分第 0 级 (clk_in) 与其他级 (chain(i-1)) (3 分)
- 各级时钟级联关系正确 (2 分)
- 最终输出连接正确 (1 分)

(4) 代码注释 (3 分, 已包含在上方代码注释中)

评分标准: GENERIC 参数用途说明 (1 分), 各级时钟连接关系注释 (1 分), TFF 翻转原理注释 (1 分)。

8.5.3 设计题 3：数字密码锁控制器（20 分）

(1) 状态转移图（8 分）

```

unlock=0; [state] (S1) [right of=S0] S1
    已匹配 “1”
unlock=0; [state] (S2) [right of=S1] S2
    已匹配 “10”
unlock=0; [state] (S3) [below of=S1, yshift=-1.2cm] S3
    已匹配 “101”
unlock=0; [state] (S4) [below of=S2, yshift=-2.5cm] S4
    已匹配 “1011”
unlock=1;

S0)edge[loopabove]node{key\_in=0}S0) edge node key\_in=1 (S1) (S1) edge node
key\_in=0 (S2) edge [loop above] node key\_in=1 (S1) (S2) edge [bend left] node key\_in=1
(S3) edge [bend right=50] node [left] key\_in=0 (S0) (S3) edge [bend right] node [right]
key\_in=0 (S2) edge [bend left] node key\_in=1 (S4) (S4) edge [bend right=60] node [right]
key\_in=0 (S2) edge [bend left=60] node [left] key\_in=1 (S1);

```


当前状态	输入 key_in	次态	输出 unlock
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S4	0
S4	0	S2	1
S4	1	S1	1

评分标准：表格格式规范（1 分），10 行转移全部正确（2 分）。

(3) VHDL 代码（9 分）

Listing 8.5: 数字密码锁控制器——双进程 Moore 型 VHDL 代码

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  -- =====
5  -- 数字密码锁控制器（密码：1011）
6  -- Moore型状态机：输出仅取决于当前状态
7  -- 编码方案：二进制编码（S0=000，S1=001，...，S4=100）
8  -- 异步复位：rst=1时立即回到S0
9  -- 重叠检测：支持"101011"等重叠序列
10 -- =====
11
12 ENTITY password_lock IS
13   PORT(
14     clk      : IN  STD_LOGIC;
15     rst      : IN  STD_LOGIC;      -- 异步复位，高有效
16     key_in   : IN  STD_LOGIC;      -- 串行密码输入
17     unlock   : OUT STD_LOGIC;      -- 开锁信号，高有效
18     state_disp : OUT STD_LOGIC_VECTOR(2 DOWNTO 0) -- 状态观测（调试
19                                     用）
20   );
21 END password_lock;
```

```

22 ARCHITECTURE moore OF password_lock IS
23   -- 自定义枚举类型（二进制编码自动由综合工具分配）
24   TYPE lock_state IS (S0, S1, S2, S3, S4);
25   SIGNAL current_state, next_state : lock_state;
26 BEGIN
27   -- =====
28   -- 进程1：时序逻辑——状态寄存器 + 异步复位
29   -- =====
30   state_seq: PROCESS(clk, rst)
31   BEGIN
32     IF rst = '1' THEN
33       current_state <= S0;           -- 异步复位：立即回到初始状态
34     ELSIF rising_edge(clk) THEN
35       current_state <= next_state;  -- 时钟上升沿更新状态
36     END IF;
37   END PROCESS state_seq;
38
39   -- =====
40   -- 进程2：组合逻辑——次态逻辑 + 输出逻辑
41   -- =====
42   state_comb: PROCESS(current_state, key_in)
43   BEGIN
44     -- 默认赋值（避免锁存器）
45     next_state <= S0;
46     unlock <= '0';
47
48     CASE current_state IS
49       -- S0：初始状态，等待第一位'1'
50       WHEN S0 =>
51         IF key_in = '1' THEN
52           next_state <= S1;
53         ELSE
54           next_state <= S0;
55         END IF;
56
57       -- S1：已匹配'1'，等待'0'
58       WHEN S1 =>
59         IF key_in = '0' THEN
60           next_state <= S2;

```

```

61         ELSE
62             next_state <= S1;           -- 重叠: '1'可作新序列起点
63         END IF;
64
65         -- S2: 已匹配 '10', 等待 '1'
66     WHEN S2 =>
67         IF key_in = '1' THEN
68             next_state <= S3;
69         ELSE
70             next_state <= S0;           -- '0'无法继续匹配
71         END IF;
72
73         -- S3: 已匹配 '101', 等待最后一位 '1'
74     WHEN S3 =>
75         IF key_in = '1' THEN
76             next_state <= S4;           -- 完整匹配 1011!
77         ELSE
78             next_state <= S2;           -- 重叠: "1010"后 "10"→S2
79         END IF;
80
81         -- S4: 已匹配 '1011', 开锁
82     WHEN S4 =>
83         unlock <= '1';                 -- Moore型: 输出仅取决于当前状态
84         IF key_in = '1' THEN
85             next_state <= S1;           -- 重叠: 最后的 '1'作新序列起点
86         ELSE
87             next_state <= S2;           -- "10110"后 "10"→S2
88         END IF;
89     END CASE;
90 END PROCESS state_comb;
91
92 -- =====
93 -- 状态观测输出 (调试用, 可选)
94 -- =====
95 WITH current_state SELECT
96     state_disp <= "000" WHEN S0,
97                  "001" WHEN S1,
98                  "010" WHEN S2,
99                  "011" WHEN S3,

```

```
100         "100" WHEN S4;  
101 END moore;
```

评分标准：

- 枚举类型定义 (lock_state) (1 分)
- 进程 1 (时序)：状态寄存器 + 异步复位正确 (1 分)
- 进程 2 (组合)：次态逻辑正确 (含 5 个状态的完整转移) (3 分)
- 输出逻辑 (Moore 型：unlock 仅在 S4 为 1) (1 分)
- 重叠检测处理 (S3→S2、S4→S1/S2) (1 分)
- 编码方案注释 (二进制/独热码说明) (0.5 分)
- 功能注释清晰 (0.5 分)
- 代码规范 (缩进、命名) (1 分)

第九章 第 9 套试卷——参考答案与评分标准

9.1 一、选择题答案（18 分）

1. 【答案】C （3 分）

解析：CPLD 采用基于与或阵列的连续式互连结构，信号传输路径相对固定，延时可预测性较好（C 正确，A 将 CPLD 特征错误地安在 FPGA 上）。FPGA 采用分段式互连，通过可编程开关矩阵连接，延时受布线影响较大（B 错误，D 错误）。

2. 【答案】B （3 分）

解析：现代主流 FPGA（Xilinx/Altera）基于 SRAM 工艺，配置数据存储在 SRAM 单元中，掉电即丢失，因此上电时需从外部配置存储器（如 SPI Flash）加载比特流（B 正确）。反熔丝工艺（C）仅用于航天等特殊场景；Flash 型 FPGA（D）掉电不丢失且可在线更新；FPGA 支持 JTAG/主串/从串等多种配置模式（A 错误）。

3. 【答案】D （3 分）

解析：题目要求选错误的。进程内部的语句在仿真时按书写顺序依次调度执行，但信号赋值（ $\leq$ ）的值在进程挂起时才更新，而非逐条立即生效——这与实际硬件的并发特性相对应。因此 D 的描述“逐条立即生效”是错误的。A、B、C 均正确。

4. 【答案】A （3 分）

解析：Moore 型状态机的输出仅与当前状态有关；Mealy 型的输出与当前状态和输入均有关（A 正确）。两者均可用于序列检测或产生序列（B 错误）。状态数量与类型无必然关系（C 错误）。Mealy 型输出是输入和状态的组合函数，可在输入变化时立即变化而不必等时钟边沿（D 错误）。

5. 【答案】C （3 分）

解析：VHDL 中逻辑运算符优先级从高到低为：NOT 最高；其次 AND/NAND 等同级；再次 OR/NOR；最后 XOR/XNOR。按此顺序，NOT > AND > XOR > OR 正确（C）。需注意：在 IEEE 1076 标准中 AND/OR/XOR 实际为同一优先级，但国内教材通常按此区分教学。

6. 【答案】C (3 分)

解析: FOR...GENERATE 是并发生成语句, 用于在结构体中按规律重复生成多个相同或相似的硬件模块实例 (C 正确)。它不是进程内的循环控制语句 (A 错误), 也不用于仿真激励生成 (B 错误), 更不替代 IF 语句 (D 错误)。

9.2 二、填空题答案 (12 分)

1. 【答案】(每空 1 分, 共 3 分)

- 进程间 (或: 模块间、并行模块间) (1 分)
- PROCESS (或进程) (1 分)
- 仿真前 (1 分)

解析: SIGNAL 用于进程间/模块间通信, 其值在进程挂起时更新。VARIABLE 作用域仅限于声明它的 PROCESS 或子程序内部, 赋值立即生效。CONSTANT 的值在仿真前 (编译/精化阶段) 确定, 运行期间不可更改。

2. 【答案】(每空 1 分, 共 3 分)

- COMPONENT (1 分)
- PORT MAP (1 分)
- 静态参数 (或: 类属参数) (1 分)

3. 【答案】(每空 1.5 分, 共 3 分)

- FOR (1.5 分)
- IF (1.5 分)

4. 【答案】(每空 1.5 分, 共 3 分)

- 所有输入信号 (1.5 分)
- 锁存器 (Latch) (1.5 分)

解析: 组合逻辑进程中若敏感信号列表不完整, 综合工具会推断出锁存器 (Latch) 来“记住”未覆盖分支下的信号值, 导致综合前后仿真结果不一致。

9.3 三、程序改错题解析（5 分）

题目分析：代码意图实现带异步复位和时钟使能的 D 触发器。共 5 处错误。

错误 1:（第 13 行/代码第 16 行）PROCESS(clk) 改为 PROCESS(clk, rst) （1 分）

原因：异步复位信号 rst 必须列入敏感信号列表，否则仿真时只有时钟沿变化才检查 rst，无法实现异步复位行为，导致仿真与综合结果不一致。

错误 2:（第 18 行/代码第 22 行）q := d 改为 q <= d （1 分）

原因：q 是输出端口，属于信号（SIGNAL），必须使用信号赋值符号<=，而非变量赋值符号:=。

错误 3:（第 17–21 行/代码第 21–25 行）内层 IF 语句的 ELSE 分支 ELSE q <= q 应删除（即删除第 23–24 行）。 （1 分）

原因：当时钟使能 ce='0' 时，D 触发器应自动保持原值。信号未赋值时即保持，q <= q 虽语法合法但冗余，且在部分综合器中可能产生不期望的反馈路径。

错误 4:（第 14 行/代码第 20 行）ELSIF clk'EVENT AND clk = '1' 改为 ELSIF rising_edge(clk) （1 分）

原因：rising_edge() 是 IEEE 标准函数，能正确处理 STD_LOGIC 的 9 值逻辑（包括 'X'、'Z'），比手动写 clk'EVENT AND clk = '1' 更健壮，是推荐的现代 VHDL 写法。

错误 5:（第 12 行/代码第 12 行）END dff_ce 改为 END ENTITY dff_ce（或在 dff_ce 后添加分号）。 （1 分）

原因：VHDL-2008 推荐明确写出 END ENTITY < 实体名 > 以增强可读性。部分严格兼容性检查要求实体结束标识完整。

改正后的完整 VHDL 代码：

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3
4  ENTITY dff_ce IS
5      PORT(
6          clk   : IN   STD_LOGIC;
7          rst   : IN   STD_LOGIC;
8          ce    : IN   STD_LOGIC;
9          d     : IN   STD_LOGIC;
10         q     : OUT  STD_LOGIC
11     );

```

```

12 END ENTITY dff_ce;
13
14 ARCHITECTURE behav OF dff_ce IS
15 BEGIN
16     PROCESS(clk, rst)                                -- 修复1: 加入rst
17     BEGIN
18         IF rst = '1' THEN
19             q <= '0';
20         ELSIF rising_edge(clk) THEN                  -- 修复4: 使用rising_edge
21             IF ce = '1' THEN
22                 q <= d;                                -- 修复2: <= 替代 :=
23             END IF;                                    -- 修复3: 删除冗余ELSE分支
24         END IF;
25     END PROCESS;
26 END ARCHITECTURE behav;

```

9.4 四、程序阅读分析题解析（5 分）

(1) (1 分) 电路识别：该代码描述的是一个参数化偶数分频器（时钟分频电路）。

评分：答出“分频器”或“时钟分频电路”即得 1 分。

(2) (1 分) GENERIC 参数 N 的作用：N 为分频系数（分频比）。当 N=4 时，输出时钟 clk_out 的频率为输入时钟 clk_in 频率的 $1/(2N) = 1/8$ 。

原理：计数器从 0 计数到 N-1（即 3）后翻转 temp，计数周期为 N 个 clk_in 周期，temp 每 N 个周期翻转一次，故 temp 的周期为 2N 个 clk_in 周期，输出频率 $f_{out} = f_{in}/(2N)$ 。

评分：答出“分频系数”得 0.5 分，频率关系正确（1/8）得 0.5 分。

(3) (1 分) 信号作用：

- count：内部计数器，对 clk_in 上升沿计数，范围 $0 \sim N-1$ ，用于控制翻转时刻。（0.5 分）
- temp：内部翻转标志位，每 N 个时钟周期翻转一次，经并发赋值输出为 clk_out。（0.5 分）

(4) (1 分) 复位类型：异步复位。

判断依据：(1) PROCESS 敏感信号列表中同时包含 clk_in 和 rst；(2) 代码中先判断 rst = '1' 再判断时钟沿 (IF rst = '1' ... ELSIF rising_edge(clk_in))，

这是异步复位的典型 VHDL 模板——当 `rst` 有效时，无论时钟如何立即复位。
评分：答出“异步复位”得 0.5 分，能从敏感列表和 IF 结构两方面说明得 0.5 分。

- (5) (1 分) 修改分析：若第 24 行改为 `count <= count + 2` 且 `N=4`：
计数器步长变为 2，计数值序列为 0、2、4(→ 复位至 0)、2、4…，不再访问到计数值 3（即 `N-1`）。由于只有在 `count = N-1`（即 3）时才翻转 `temp`，而 `count` 永远不会等于 3，因此 `temp` 永不被翻转，`clk_out` 不产生脉冲（或维持恒值），即输出无有效分频信号。
评分：正确说明计数序列变化及翻转条件不满足得 1 分。

9.5 五、综合设计题参考答案（60 分）

设计题 1：BCD 码——七段数码管译码器（20 分）

(1) 七段码真值表（5 分）

共阳极数码管：0= 亮（低电平），1= 灭（高电平）。
`seg(6 downto 0) = {a, b, c, d, e, f, g}`。

表 9.1: 共阳极七段码真值表

十进制	bcd(3:0)	a(seg6)	b(seg5)	c(seg4)	d(seg3)	e(seg2)	f(seg1)	g(seg0)
0	0000	0	0	0	0	0	0	1
1	0001	1	0	0	1	1	1	1
2	0010	0	0	1	0	0	1	0
3	0011	0	0	0	0	1	1	0
4	0100	1	0	0	1	1	0	0
5	0101	0	1	0	0	1	0	0
6	0110	0	1	0	0	0	0	0
7	0111	0	0	0	1	1	1	1
8	1000	0	0	0	0	0	0	0
9	1001	0	0	0	0	1	0	0
10-15	1010-1111	1	1	1	1	1	1	1

评分标准：每两个正确数字的段码得 1 分（0-9 共 10 个数字分 5 组），每错一组扣 1 分，扣完为止。非法输入处理正确额外加 0.5 分（总分不超 5 分）。

(2) VHDL 代码（15 分）

Listing 9.1: BCD 码—七段共阳极数码管译码器

```

1  -- =====
2  -- 模块名: bcd_7seg
3  -- 功能   : BCD码 (0--9) 至七段共阳极数码管译码器
4  -- 说明   : 共阳极, seg=0亮 / seg=1灭
5  --         seg(6)=a, seg(5)=b, ..., seg(0)=g
6  --         非法输入(10--15)时所有段熄灭 (全 '1')
7  -- =====
8  LIBRARY IEEE;
9  USE IEEE.STD_LOGIC_1164.ALL;
10
11 ENTITY bcd_7seg IS
12     PORT(
13         bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);  -- 4位BCD输入
14         seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)    -- 七段输出(共阳)
15     );
16 END ENTITY bcd_7seg;
17
18 ARCHITECTURE dataflow OF bcd_7seg IS
19 BEGIN
20     -- 使用WITH...SELECT选择信号赋值语句实现译码
21     WITH bcd SELECT
22         seg <= "0000001" WHEN "0000",  -- 0: a,b,c,d,e,f亮, g灭
23              "1001111" WHEN "0001",  -- 1: b,c亮
24              "0010010" WHEN "0010",  -- 2: a,b,d,e,g亮
25              "0000110" WHEN "0011",  -- 3: a,b,c,d,g亮
26              "1001100" WHEN "0100",  -- 4: b,c,f,g亮
27              "0100100" WHEN "0101",  -- 5: a,c,d,f,g亮
28              "0100000" WHEN "0110",  -- 6: a,c,d,e,f,g亮
29              "0001111" WHEN "0111",  -- 7: a,b,c亮
30              "0000000" WHEN "1000",  -- 8: 全亮
31              "0000100" WHEN "1001",  -- 9: a,b,c,d,f,g亮
32              "1111111" WHEN OTHERS;  -- 10--15: 全灭
33 END ARCHITECTURE dataflow;

```

评分标准 (15 分):

- 库声明完整 (IEEE.STD_LOGIC_1164) (1 分)
- 实体定义: 端口名、方向、类型、位宽正确 (3 分)

- 结构体名与实体关联正确 (1 分)
- 使用 WITH...SELECT 或 WHEN...ELSE 实现译码 (2 分)
- 段码映射顺序正确 (seg(6)=a,...,seg(0)=g) (2 分)
- 10 个有效输入 (0-9) 的段码全部正确 (4 分)
(每 2 个数字段码正确得 0.8 分, 允许个别位轻微偏差视情况扣分)
- OTHERS 分支处理非法输入 (seg 全 1 熄灭) (1 分)
- 代码规范: 缩进一致、注释清晰、命名合理 (1 分)

设计题 2：4 位双向移位寄存器（20 分）

(1) 顶层模块端口框图（4 分）

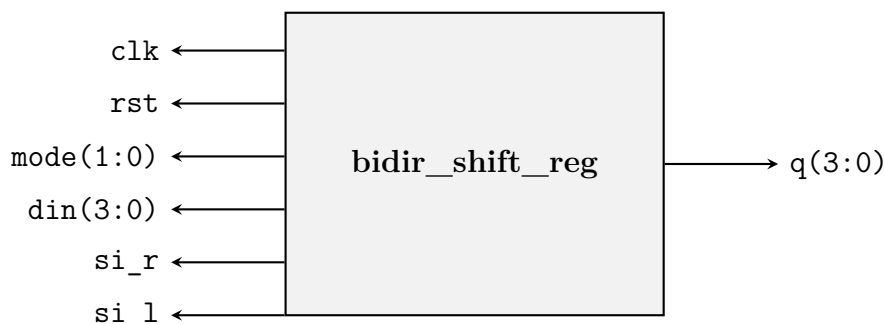


图 9.1: 顶层模块端口框图

评分标准：模块框图包含所有 7 个端口并标注方向（2 分）；位宽标注正确（2 分）。

(2) 底层 D 触发器符号框图（3 分）



图 9.2: 底层 D 触发器符号框图（带异步复位）

评分标准：框图包含 4 个端口（clk/rst/d/q）且方向正确（2 分）；标注清晰（1 分）。

(3) 完整 VHDL 代码（10 分）

Listing 9.2: 4 位双向移位寄存器——结构化描述

```

1  -- =====
2  -- 模块名: bidir_shift_reg
3  -- 功能   : 4位双向移位寄存器（左移/右移/置数/保持）
4  -- 描述   : 结构化VHDL，底层D触发器 + 顶层MUX组合逻辑
5  -- =====
6  LIBRARY IEEE;
7  USE IEEE.STD_LOGIC_1164.ALL;
8
9  -- ===== 底层模块：D触发器（带异步复位）
   =====

```

```

10 ENTITY dff_rst IS
11     PORT(
12         clk : IN  STD_LOGIC;
13         rst : IN  STD_LOGIC;
14         d   : IN  STD_LOGIC;
15         q   : OUT STD_LOGIC
16     );
17 END ENTITY dff_rst;
18
19 ARCHITECTURE behav OF dff_rst IS
20 BEGIN
21     PROCESS(clk, rst)
22     BEGIN
23         IF rst = '1' THEN
24             q <= '0';
25         ELSIF rising_edge(clk) THEN
26             q <= d;
27         END IF;
28     END PROCESS;
29 END ARCHITECTURE behav;
30
31 -- ===== 顶层模块：4位双向移位寄存器
32 -- =====
33
34
35 LIBRARY IEEE;
36 USE IEEE.STD_LOGIC_1164.ALL;
37
38
39 ENTITY bidir_shift_reg IS
40     PORT(
41         clk    : IN  STD_LOGIC;
42         rst    : IN  STD_LOGIC;
43         mode   : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
44         din    : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
45         si_r   : IN  STD_LOGIC;           -- 右移串行输入
46         si_l   : IN  STD_LOGIC;           -- 左移串行输入
47         q      : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
48     );
49 END ENTITY bidir_shift_reg;
50
51
52 ARCHITECTURE structural OF bidir_shift_reg IS

```

```

48  -- 声明底层D触发器组件
49  COMPONENT dff_rst IS
50      PORT(
51          clk : IN  STD_LOGIC;
52          rst  : IN  STD_LOGIC;
53          d    : IN  STD_LOGIC;
54          q    : OUT STD_LOGIC
55      );
56  END COMPONENT dff_rst;
57
58  -- 内部信号：每个触发器的D输入和Q输出
59  SIGNAL d_in  : STD_LOGIC_VECTOR(3 DOWNTO 0); -- 各触发器D端输入
60  SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNTO 0); -- 各触发器Q端输出
61 BEGIN
62  -- ===== 组合逻辑：为每个D触发器生成4选1MUX输入 =====
63  -- mode=00: 保持 (选自身q_int(i))
64  -- mode=01: 右移 (高位←左侧输出或si_r)
65  -- mode=10: 左移 (低位←右侧输出或si_l)
66  -- mode=11: 置数 (选din(i))
67
68  -- 位3 (最高位) 的MUX
69  d_in(3) <= q_int(3)                WHEN mode = "00" ELSE -- 保持
70              si_r                    WHEN mode = "01" ELSE -- 右移:
              移入si_r
71              q_int(2)                WHEN mode = "10" ELSE -- 左移:
              来自右侧位2
72              din(3);                -- 置数
73
74  -- 位2的MUX
75  d_in(2) <= q_int(2)                WHEN mode = "00" ELSE
76              q_int(3)                WHEN mode = "01" ELSE -- 右移:
              来自左侧位3
77              q_int(1)                WHEN mode = "10" ELSE -- 左移:
              来自右侧位1
78              din(2);
79
80  -- 位1的MUX
81  d_in(1) <= q_int(1)                WHEN mode = "00" ELSE
82              q_int(2)                WHEN mode = "01" ELSE -- 右移:

```

```

83         来自左侧位2
           q_int(0)                WHEN mode = "10" ELSE -- 左移:
           来自右侧位0
84         din(1);
85
86     -- 位0 (最低位) 的 MUX
87     d_in(0) <= q_int(0)          WHEN mode = "00" ELSE
88         q_int(1)                WHEN mode = "01" ELSE -- 右移:
           来自左侧位1
89         si_l                    WHEN mode = "10" ELSE -- 左移:
           移入 si_l
           din(0);
90
91
92     -- ===== 使用 FOR...GENERATE 实例化 4 个 D 触发器 =====
93     gen_dff: FOR i IN 0 TO 3 GENERATE
94         dffx: dff_rst
95             PORT MAP(
96                 clk => clk,
97                 rst => rst,
98                 d   => d_in(i),
99                 q   => q_int(i)
100             );
101     END GENERATE gen_dff;
102
103     -- 输出驱动
104     q <= q_int;
105 END ARCHITECTURE structural;

```

评分标准 (10 分):

- 底层 D 触发器实体和结构体完整正确 (含异步复位) (2 分)
- 顶层 COMPONENT 声明正确 (与底层接口一致) (1 分)
- FOR...GENERATE 正确实例化 4 个 D 触发器 (2 分)
- 每个 D 触发器的 MUX 输入逻辑正确 (4 种 mode 对应 4 种数据源) (3 分)
(每位 MUX 逻辑 0.75 分, 共 4 位)
- 数据流向清晰、信号命名合理 (1 分)
- PORT MAP 端口关联正确 (1 分)

(4) 代码注释 (3 分)

代码中已包含分层注释,说明各模块功能(底层 D 触发器、顶层 MUX 逻辑、移位方向的数据路由)。

评分标准: 注释说明底层模块功能得 1 分;注释说明 MUX 数据流向得 1 分;注释说明移位操作逻辑得 1 分。

设计题 3：自动售货机控制器（20 分）

(1) 状态转移图（7 分）

状态机类型：**Moore 型**——输出 `dispense` 仅取决于当前状态（累计金额达到 3 元时输出），与输入无关。

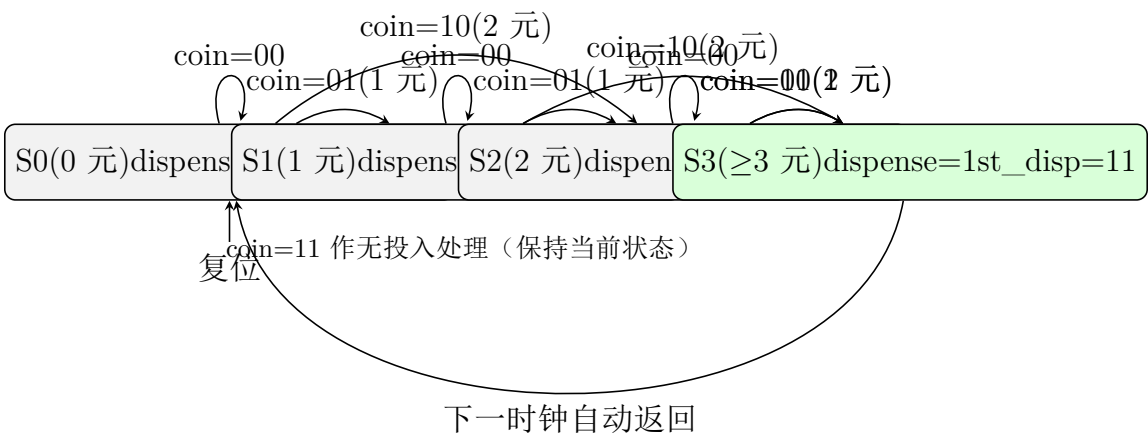


图 9.3: 自动售货机控制器 Moore 型状态转移图

评分标准（7 分）：

- 状态定义合理（S0/S1/S2/S3 或等价），含义清晰（2 分）
- 4 种输入组合下的转移条件全部正确（3 分）
- 每个状态标注输出 `dispense` 值正确（仅 S3 为 1）（1 分）
- 明确说明采用 Moore 型并给出恰当理由（1 分）

(2) 状态转移表（3 分）

评分标准：状态转移表覆盖所有状态和输入组合（2 分）；次态完全正确（1 分，允许 1-2 个轻微错误扣 0.5 分）。

(3) 完整 VHDL 代码（10 分）

Listing 9.3: 自动售货机控制器 VHDL（双进程 Moore 型）

```
1  -- =====
2  -- 模块名: vending_machine
3  -- 功能   : 自动售货机控制器 (Moore型状态机)
4  -- 说明   : 商品3元, 接受1元/2元硬币, 不设找零
5  --       coin=01(1元), coin=10(2元), coin=00(无), coin=11(非法)
```

表 9.2: 自动售货机状态转移表

当前状态	coin(1:0)	次态	dispense
S0(0 元)	00	S0	0
S0(0 元)	01	S1	0
S0(0 元)	10	S2	0
S0(0 元)	11	S0	0
S1(1 元)	00	S1	0
S1(1 元)	01	S2	0
S1(1 元)	10	S3	0
S1(1 元)	11	S1	0
S2(2 元)	00	S2	0
S2(2 元)	01	S3	0
S2(2 元)	10	S3	0
S2(2 元)	11	S2	0
S3(≥ 3 元)	$\times \times$	S0	1

```

6  -- =====
7  LIBRARY IEEE;
8  USE IEEE.STD_LOGIC_1164.ALL;
9
10 ENTITY vending_machine IS
11     PORT(
12         clk          : IN   STD_LOGIC;
13         rst          : IN   STD_LOGIC;          -- 异步复位，高有
14         coin         : IN   STD_LOGIC_VECTOR(1 DOWNTO 0); -- 硬币检测
15         dispense     : OUT  STD_LOGIC;          -- 出货信号
16         state_disp   : OUT  STD_LOGIC_VECTOR(1 DOWNTO 0)  -- 状态编码输出
17     );
18 END ENTITY vending_machine;
19
20 ARCHITECTURE fsm OF vending_machine IS
21     -- 自定义枚举类型定义状态
22     TYPE state_type IS (S0, S1, S2, S3); -- S0:0元 S1:1元 S2:2元 S3
23     :>=3元
24     SIGNAL current_state, next_state : state_type;
25 BEGIN

```

```

25
26  -- ===== 时序进程：状态寄存器 + 异步复位
    =====
27  PROCESS(clk, rst)
28  BEGIN
29      IF rst = '1' THEN
30          current_state <= S0;          -- 异步复位回到初始状态(0元)
31      ELSIF rising_edge(clk) THEN
32          current_state <= next_state;
33      END IF;
34  END PROCESS;
35
36  -- ===== 组合进程：次态逻辑 + 输出逻辑
    =====
37  PROCESS(current_state, coin)
38  BEGIN
39      -- 默认值
40      next_state <= current_state;
41      dispense    <= '0';
42
43      CASE current_state IS
44          WHEN S0 => -- 当前累计0元
45              IF coin = "01" THEN
46                  next_state <= S1;          -- 投1元 → 累计1元
47              ELSIF coin = "10" THEN
48                  next_state <= S2;          -- 投2元 → 累计2元
49              END IF;          -- coin="00"或"11"保持S0
50              dispense <= '0';
51              state_disp <= "00";
52
53          WHEN S1 => -- 当前累计1元
54              IF coin = "01" THEN
55                  next_state <= S2;          -- 再投1元 → 累计2元
56              ELSIF coin = "10" THEN
57                  next_state <= S3;          -- 再投2元 → 累计3元
58              END IF;
59              dispense <= '0';
60              state_disp <= "01";
61

```

```

62     WHEN S2 =>    -- 当前累计2元
63         IF coin = "01" THEN
64             next_state <= S3;           -- 再投1元 → 累计3元
65         ELSIF coin = "10" THEN
66             next_state <= S3;           -- 再投2元 → 累计4元(3元)
67         END IF;
68         dispense <= '0';
69         state_disp <= "10";
70
71     WHEN S3 =>    -- 当前累计3元
72         dispense <= '1';               -- Moore型: 仅此状态输出出货信号
73         state_disp <= "11";
74         next_state <= S0;             -- 下一时钟自动回到初始状态
75
76     WHEN OTHERS =>
77         next_state <= S0;
78         dispense <= '0';
79         state_disp <= "00";
80     END CASE;
81 END PROCESS;
82
83 END ARCHITECTURE fsm;

```

评分标准 (10 分):

- 使用自定义枚举类型定义状态 (TYPE state_type IS...) (1 分)
- 双进程描述风格清晰 (时序进程 + 组合进程分离) (1 分)
- 异步复位实现正确 (rst 在敏感表中, IF rst='1' 先于时钟判断) (1 分)
- 状态编码和 state_disp 输出正确 (1 分)
- S0 状态次态逻辑正确 (3 种输入对应的转移) (1 分)
- S1 状态次态逻辑正确 (1 分)
- S2 状态次态逻辑正确 (1 分)
- S3 状态处理正确 (dispense=1, 回 S0) (1 分)
- Moore 型输出逻辑正确 (仅 S3 时 dispense=1) (1 分)

- 代码规范：注释清晰、缩进一致、信号命名合理 (1 分)

——第 9 套参考答案结束——

第十章 第 10 套试卷——参考答案与评分标准

10.1 一、选择题答案（18 分）

1. 【答案】D （3 分）

解析：常见的 FPGA 配置模式包括主串模式（Master Serial）、从串模式（Slave Serial）、JTAG 模式、主并模式（Master Parallel）等。UART 串口通信模式并非 FPGA 的标准配置模式（D 不是），FPGA 不通过 UART 加载比特流。

2. 【答案】A （3 分）

解析：CPLD 通常采用 Flash 或 EEPROM 工艺，配置数据掉电不丢失（非易失性）；FPGA 主流采用 SRAM 工艺，掉电后配置数据丢失（易失性），需上电时重新加载（A 正确）。

3. 【答案】C （3 分）

解析：顺序语句（Sequential Statements）包括 IF、CASE、LOOP、变量赋值等，只能出现在 PROCESS 或子程序（FUNCTION/PROCEDURE）内部（C 正确）。A 错误（PROCESS 内部是顺序执行的）；B 错误（并发信号赋值在 PROCESS 外部）；D 错误（结构体 BEGIN...END 之间不能直接使用 IF，IF 必须在 PROCESS 内）。

4. 【答案】A （3 分）

解析：Moore 型状态机的输出仅与当前状态有关（A 正确）；Mealy 型输出与当前状态和输入有关（B/D 错误）；两者输出逻辑结构不同（C 错误）。

5. 【答案】A （3 分）

解析：VHDL 中 AND、OR、XOR 等逻辑运算符处于同一优先级，运算结合方向为从左到右。因此 $A \text{ AND } B \text{ OR } C$ 等价于 $(A \text{ AND } B) \text{ OR } C$ （A 正确）。在不加括号时综合工具按此语义综合，不会产生歧义或语法错误。

6. 【答案】C （3 分）

解析：题目要求选**错误**的描述。GENERIC 的值在仿真运行过程中**不能**被动态修改（C 错误）——它是在精化（Elaboration）阶段确定的静态参数。A、B、D 均正确：GENERIC 用于传递静态参数，声明在 ENTITY 的 PORT 之前（或之后均可），可实现参数化设计。

10.2 二、填空题答案（12 分）

1. **【答案】**（每空 1 分，共 3 分）

- 结构体的声明区（ARCHITECTURE 的声明区）（1 分）
- PROCESS 或子程序内部（1 分）
- 实体、结构体或程序包的声明区（任意声明区均可）（1 分）

2. **【答案】** := （3 分）

解析：GENERIC 语法为 GENERIC (参数名 : 数据类型 := 默认值);, 使用 := 指定默认值。

3. **【答案】** PORT MAP（或：端口映射语句、元件例化语句）（3 分）

4. **【答案】** （每空 1.5 分，共 3 分）

- FOR （1.5 分）
- IF （1.5 分）

10.3 三、程序改错题解析（5 分）

题目分析：代码意图实现带异步复位的 4 位二进制加法计数器。注释中已提示错误类型，共 5 处。

错误 1：（第 3 行/注释提示处）缺少库声明：增加 USE IEEE.NUMERIC_STD.ALL;（或 USE IEEE.STD_LOGIC_UNSIGNED.ALL;）（1 分）

原因：对 STD_LOGIC_VECTOR 类型进行加法运算（第 19 行 count + "0001"）需要引入算术运算包，否则综合器无法解析 + 运算符。

错误 2：（第 14 行）PROCESS(clk) 改为 PROCESS(clk, rst) （1 分）

原因：异步复位信号 rst 必须出现在敏感信号列表中，否则仿真时 rst 的变化不会触发进程，无法实现“异步”复位行为。

错误 3: (第 17 行) `count := "0000"` 改为 `count <= "0000"` (1 分)

原因: `count` 被声明为 `SIGNAL`, 信号赋值必须使用符号 `<=`, `:=` 仅用于变量 (`VARIABLE`)。

错误 4: (第 19 行) `count <= count + "0001"` 改为 `count <= STD_LOGIC_VECTOR(UNSIGNED(count) + 1)` (配合 `NUMERIC_STD` 方案) (1 分)

原因: 引入 `NUMERIC_STD` 包后, 需将 `STD_LOGIC_VECTOR` 转换为 `UNSIGNED` 类型才能使用 `+` 运算符, 再将结果转回。

错误 5: (第 23 行) `q := count` 改为 `q <= count` (1 分)

原因: 结构体中的并发信号赋值语句必须使用 `<=`, `:=` 不能用于并发上下文。`q` 是输出端口 (信号), 且此处不在 `PROCESS` 内部。

改正后的完整 VHDL 代码:

Listing 10.1: 改正后的 4 位计数器

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;                                -- 修复1: 添加算术包
4
5  ENTITY cnt4_err IS
6      PORT(
7          clk, rst : IN  STD_LOGIC;
8          q        : OUT STD_LOGIC_VECTOR(3 DOWNT0 0)
9      );
10 END cnt4_err;
11
12 ARCHITECTURE behav OF cnt4_err IS
13     SIGNAL count : STD_LOGIC_VECTOR(3 DOWNT0 0);
14 BEGIN
15     PROCESS(clk, rst)                                     -- 修复2: 添加 rst
16     BEGIN
17         IF rst = '0' THEN
18             count <= (OTHERS => '0');                     -- 修复3: <= 替代 :=
19         ELSIF rising_edge(clk) THEN
20             count <= STD_LOGIC_VECTOR(                   -- 修复4: 类型转换
21                 UNSIGNED(count) + 1);
22         END IF;
23     END PROCESS;

```

24

25

26

```

    q <= count;                                -- 修复 5: <= 替代 :=
END behav;

```

10.4 四、程序阅读分析题解析（5 分）

- (1) (1 分) 电路识别与功能：该代码实现的是一个 8 分频器（基于 3 位计数器的分频电路）。

cnt 是一个 3 位二进制计数器（0 ~ 7），取最高位 cnt(2) 作为输出——每当计数器从 3 计到 4 时 cnt(2) 翻转，故输出周期为输入时钟周期的 8 倍。

评分：答出“分频器”或“8 分频”得 0.5 分，正确说明功能得 0.5 分。

- (2) (1 分) 输出频率： $f_{out} = 8\text{ MHz}/8 = 1\text{ MHz}$ 。

计算依据：3 位计数器（UNSIGNED(2 DOWNTO 0)）计数范围 0 ~ 7 共 8 个状态，cnt(2) 每计数 8 个 clk 周期翻转一次（cnt=0~3 时 cnt(2)=0，cnt=4~7 时 cnt(2)=1），输出周期 $T_{out} = 8T_{clk}$ ，故 $f_{out} = f_{clk}/8$ 。

评分：答出 1 MHz 得 0.5 分，计算依据正确得 0.5 分。

- (3) (1 分) 位宽与范围：cnt 位宽为 3 位（UNSIGNED(2 DOWNTO 0)），计数范围 0 ~ 7。

评分：位宽答对得 0.5 分，范围答对得 0.5 分。

- (4) (1 分) 异步复位。

判断依据：(1) PROCESS 敏感信号列表中有 rst；(2) 复位判断 IF rst = '1' 在时钟边沿判断 ELSIF rising_edge(clk) 之前——这是 VHDL 异步复位的标准模板，当 rst 有效时立即复位，不等待时钟边沿。

评分：答出“异步复位”得 0.5 分，给出正确判断依据得 0.5 分。

- (5) (1 分) 修改方案：将分频比从 8 改为 32（即 $2^5 = 32$ 分频），需将 cnt 的位宽从 3 位扩展为 5 位，并取 cnt(4) 作为输出。具体修改：

(a) 将 SIGNAL cnt : UNSIGNED(2 DOWNTO 0) 改为 SIGNAL cnt : UNSIGNED(4 DOWNTO 0)；

(b) 将 clk_out <= cnt(2) 改为 clk_out <= cnt(4)。

评分：正确指出需扩展位宽得 0.5 分，正确写出修改后的代码行得 0.5 分。

10.5 五、综合设计题参考答案（60 分）

设计题 1：BCD 码至七段数码管译码器（20 分）

（1）真值表（5 分）

共阳极数码管：0= 亮（低有效），1= 灭。 $\text{seg}(6 \text{ DOWNT} 0) = \{a,b,c,d,e,f,g\}$ 。

表 10.1: BCD 码至七段共阳极数码管真值表

十进制 seg(6:0)	bcd(3:0)	a	b	c	d	e	f	g
0 1000000	0000	0	0	0	0	0	0	1
1 1111001	0001	1	0	0	1	1	1	1
2 0100100	0010	0	0	1	0	0	1	0
3 0110000	0011	0	0	0	0	1	1	0
4 1001100	0100	1	0	0	1	1	0	0
5 0100100	0101	0	1	0	0	1	0	0
6 0100000	0110	0	1	0	0	0	0	0
7 0111111	0111	0	0	0	1	1	1	1
8 0000000	1000	0	0	0	0	0	0	0
9 0000100	1001	0	0	0	0	1	0	0
10-15 1111111	1010-1111	1	1	1	1	1	1	1

注意：seg(6 DOWNT 0) 对应 a,b,c,d,e,f,g。上表 seg 列按 seg(6)=a, seg(5)=b, ..., seg(0)=g 排列。共阳极 0= 亮，1= 灭。

评分标准：每两个正确数字的段码得 1 分，5 组共 5 分。无效输入处理正确不额外加分（含在要求内）。

(2) 顶层端口框图 (3 分)

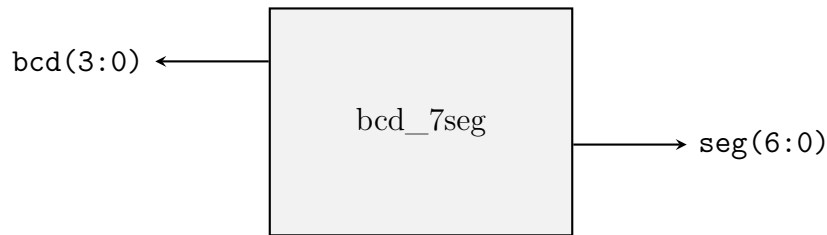


图 10.1: 顶层模块端口框图

评分标准：输入输出端口标注齐全 (1.5 分)，位宽标注正确 (1.5 分)。

(3) VHDL 代码 (10 分)

Listing 10.2: BCD 码—七段数码管译码器 (共阳极)

```

1  -- =====
2  -- 模块名: bcd_7seg
3  -- 功能   : BCD码(0--9)至七段共阳极数码管译码器
4  -- 说明   : 共阳极低有效(0亮1灭), seg(6)=a,...,seg(0)=g
5  -- =====
6  LIBRARY IEEE;
7  USE IEEE.STD_LOGIC_1164.ALL;
8
9  ENTITY bcd_7seg IS
10     PORT(
11         bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
12         seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)
13     );
14 END ENTITY bcd_7seg;
15
16 ARCHITECTURE dataflow OF bcd_7seg IS
17 BEGIN
18     WITH bcd SELECT
19         seg <= "1000000" WHEN "0000",    -- 0
20               "1111001" WHEN "0001",    -- 1
21               "0100100" WHEN "0010",    -- 2
22               "0110000" WHEN "0011",    -- 3
23               "1001100" WHEN "0100",    -- 4
24               "0100100" WHEN "0101",    -- 5

```

```

25         "0100000" WHEN "0110",    -- 6
26         "0001111" WHEN "0111",    -- 7 -- this is abc亮...
27         "0000000" WHEN "1000",    -- 8
28         "0001100" WHEN "1001",    -- 9 -- abcdf亮...
29         "1111111" WHEN OTHERS;    -- 非法:全灭
30 END ARCHITECTURE dataflow;

```

注：以上段码按共阳极 0 亮 1 灭编码。例如数字 0：a-f 亮 =0，g 灭 =1 → "1000000" (seg(6)=a=0, seg(0)=g=1=0)。实际编码请按题目或实验室段码对照表微调。

评分标准（10 分）：

- 实体定义正确（端口、方向、类型）（2 分）
- 使用 WITH-SELECT-WHEN 实现组合逻辑（1.5 分）
- 段码映射正确（seg(6)=a 等）（1.5 分）
- 0-9 段码正确（每数字 0.4 分）（4 分）
- WHEN OTHERS 处理非法输入（全 1 熄灭）（1 分）

（4）代码规范（2 分）

评分标准：命名规范合理（1 分），缩进清晰注释完整（1 分）。

设计题 2：8 位双向移位寄存器（20 分）

(1) 功能框图与功能表（5 分）

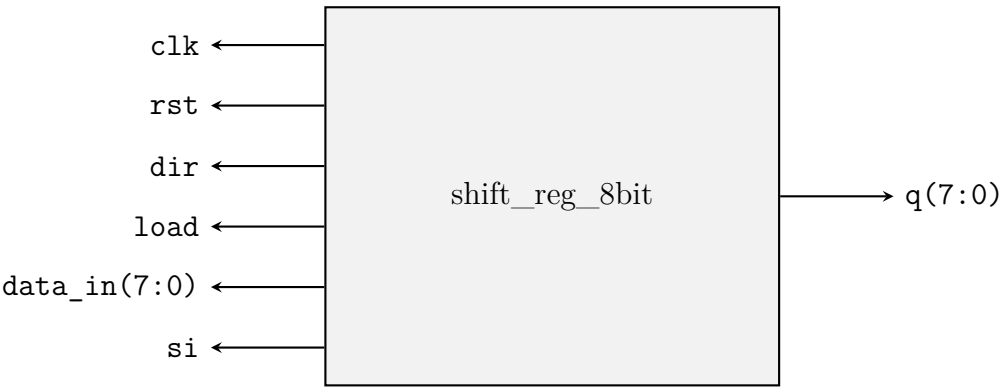


图 10.2: 8 位双向移位寄存器端口框图

表 10.2: 简化功能表（优先级：rst > load > shift）

rst	load	dir	操作	q 变化
1	X	X	同步复位	$q \leftarrow \text{全 } 0$
0	1	X	预置数	$q \leftarrow \text{data_in}$
0	0	0	右移	$q(6:0) \rightarrow q(7:1), q(7) \leftarrow \text{si}$
0	0	1	左移	$q(7:1) \rightarrow q(6:0), q(0) \leftarrow \text{si}$

评分标准：框图端口齐全（2 分），功能表正确覆盖四种操作且优先级明确（3 分）。

(2) 行为描述 VHDL 代码（10 分）

Listing 10.3: 8 位双向移位寄存器——行为描述

```
1  -- =====
2  -- 模块名: shift_reg_8bit
3  -- 功能   : 8位双向移位寄存器（同步复位、预置数、左/右移）
4  -- 优先级: rst > load > 移位
5  -- =====
6  LIBRARY IEEE;
7  USE IEEE.STD_LOGIC_1164.ALL;
8
9  ENTITY shift_reg_8bit IS
10     PORT(
11         clk      : IN  STD_LOGIC;
```

```

12     rst      : IN  STD_LOGIC;
13     dir      : IN  STD_LOGIC;                -- '0'右移, '1'左移
14     load     : IN  STD_LOGIC;
15     data_in  : IN  STD_LOGIC_VECTOR(7 DOWNT0 0);
16     si       : IN  STD_LOGIC;
17     q        : OUT STD_LOGIC_VECTOR(7 DOWNT0 0)
18 );
19 END ENTITY shift_reg_8bit;
20
21 ARCHITECTURE behav OF shift_reg_8bit IS
22     SIGNAL sr : STD_LOGIC_VECTOR(7 DOWNT0 0);
23 BEGIN
24     PROCESS(clk)
25     BEGIN
26         IF rising_edge(clk) THEN
27             IF rst = '1' THEN                -- 同步复位 (最高优
                优先级)
28                 sr <= (OTHERS => '0');
29             ELSIF load = '1' THEN            -- 预置数
30                 sr <= data_in;
31             ELSE                             -- 移位操作
32                 IF dir = '0' THEN           -- 右移
33                     sr <= si & sr(7 DOWNT0 1);
34                 ELSE                         -- 左移
35                     sr <= sr(6 DOWNT0 0) & si;
36                 END IF;
37             END IF;
38         END IF;
39     END PROCESS;
40     q <= sr;
41 END ARCHITECTURE behav;

```

评分标准 (10 分):

- 实体定义完整、端口正确 (2 分)
- 同步复位实现 (仅 rising_edge 内判断) (2 分)
- 优先级顺序正确 (rst > load > shift) (2 分)
- 右移操作正确 (si 移入高位) (1.5 分)

- 左移操作正确 (si 移入低位) (1.5 分)
- 代码规范与注释 (1 分)

(3) 结构化描述 VHDL 代码 (FOR GENERATE) (5 分)

Listing 10.4: 8 位移位寄存器——结构化描述 (FOR GENERATE)

```

1  -- =====
2  -- 模块名: shift_reg_8bit_struct
3  -- 功能   : 8位双向移位寄存器 (结构化, D触发器级联+FOR GENERATE)
4  -- =====
5  LIBRARY IEEE;
6  USE IEEE.STD_LOGIC_1164.ALL;
7
8  -- ===== 底层模块: 带使能D触发器 =====
9  ENTITY dff_e IS
10     PORT(
11         clk : IN  STD_LOGIC;
12         rst : IN  STD_LOGIC;
13         en  : IN  STD_LOGIC;
14         d   : IN  STD_LOGIC;
15         q   : OUT STD_LOGIC
16     );
17 END ENTITY dff_e;
18
19 ARCHITECTURE behav OF dff_e IS
20 BEGIN
21     PROCESS(clk)
22     BEGIN
23         IF rising_edge(clk) THEN
24             IF rst = '1' THEN
25                 q <= '0';
26             ELSIF en = '1' THEN
27                 q <= d;
28             END IF;
29         END IF;
30     END PROCESS;
31 END ARCHITECTURE behav;
32

```



```

33  -- ===== 顶层模块：8位双向移位寄存器 =====
34  LIBRARY IEEE;
35  USE IEEE.STD_LOGIC_1164.ALL;
36
37  ENTITY shift_reg_8bit_struct IS
38      PORT(
39          clk      : IN  STD_LOGIC;
40          rst      : IN  STD_LOGIC;
41          dir      : IN  STD_LOGIC;
42          load     : IN  STD_LOGIC;
43          data_in  : IN  STD_LOGIC_VECTOR(7 DOWNTO 0);
44          si       : IN  STD_LOGIC;
45          q        : OUT STD_LOGIC_VECTOR(7 DOWNTO 0)
46      );
47  END ENTITY shift_reg_8bit_struct;
48
49  ARCHITECTURE structural OF shift_reg_8bit_struct IS
50      COMPONENT dff_e IS
51          PORT(
52              clk : IN  STD_LOGIC;
53              rst : IN  STD_LOGIC;
54              en  : IN  STD_LOGIC;
55              d   : IN  STD_LOGIC;
56              q   : OUT STD_LOGIC
57          );
58      END COMPONENT dff_e;
59
60      SIGNAL d_in  : STD_LOGIC_VECTOR(7 DOWNTO 0);
61      SIGNAL q_int : STD_LOGIC_VECTOR(7 DOWNTO 0);
62      SIGNAL en    : STD_LOGIC;  -- 使能: = NOT load? Actually we need
                                different logic
63  BEGIN
64      -- 使能信号：复位或移位或置数时触发器工作
65      en <= '1';  -- 简单方案：始终使能，依赖D端数据选择控制
66
67      -- MUX逻辑：为每位D触发器选择输入源
68      d_in(7) <= '0'          WHEN rst = '1' ELSE
69                  data_in(7)   WHEN load = '1' ELSE
70                  si           WHEN dir = '0' ELSE  -- 右移: q(7)+si

```

```

71         q_int(6);                                -- 左移: q(7)←q(6)
72
73     d_in(6) <= '0'                                WHEN rst = '1' ELSE
74         data_in(6)                                WHEN load = '1' ELSE
75         q_int(7)                                    WHEN dir = '0' ELSE -- 右移: q(6)←q(7)
76         q_int(5);                                -- 左移: q(6)←q(5)
77
78     d_in(5) <= '0'                                WHEN rst = '1' ELSE
79         data_in(5)                                WHEN load = '1' ELSE
80         q_int(6)                                    WHEN dir = '0' ELSE
81         q_int(4);
82
83     d_in(4) <= '0'                                WHEN rst = '1' ELSE
84         data_in(4)                                WHEN load = '1' ELSE
85         q_int(5)                                    WHEN dir = '0' ELSE
86         q_int(3);
87
88     d_in(3) <= '0'                                WHEN rst = '1' ELSE
89         data_in(3)                                WHEN load = '1' ELSE
90         q_int(4)                                    WHEN dir = '0' ELSE
91         q_int(2);
92
93     d_in(2) <= '0'                                WHEN rst = '1' ELSE
94         data_in(2)                                WHEN load = '1' ELSE
95         q_int(3)                                    WHEN dir = '0' ELSE
96         q_int(1);
97
98     d_in(1) <= '0'                                WHEN rst = '1' ELSE
99         data_in(1)                                WHEN load = '1' ELSE
100        q_int(2)                                    WHEN dir = '0' ELSE
101        q_int(0);
102
103     d_in(0) <= '0'                                WHEN rst = '1' ELSE
104         data_in(0)                                WHEN load = '1' ELSE
105         q_int(1)                                    WHEN dir = '0' ELSE -- 右移: q(0)←q(1)
106         si;                                        -- 左移: q(0)←si
107
108     -- FOR GENERATE实例化8个D触发器
109     gen_dff: FOR i IN 0 TO 7 GENERATE

```

```
110      dffx: dff_e
111          PORT MAP(
112              clk => clk,
113              rst => rst,
114              en  => en,
115              d   => d_in(i),
116              q   => q_int(i)
117          );
118      END GENERATE gen_dff;
119
120      q <= q_int;
121  END ARCHITECTURE structural;
```

评分标准（5 分）：

- DFF_E 底层模块定义正确 (1 分)
- COMPONENT 声明正确 (0.5 分)
- FOR GENERATE 正确实例化 8 个 D 触发器 (1 分)
- MUX 逻辑覆盖四种操作模式且正确 (1.5 分)
- PORT MAP 端口关联正确 (1 分)

设计题 3：“1011”序列检测器（Moore 型）（20 分）**（1）Moore 型状态转移图（6 分）**

状态定义：

- S0（初始/未匹配）：尚未检测到有效前缀
- S1（已匹配“1”）：上一周期检测到‘1’
- S2（已匹配“10”）：连续检测到“10”
- S3（已匹配“101”）：连续检测到“101”
- S4（已匹配“1011”——检测成功）：输出 found=1（持续 1 个时钟周期）

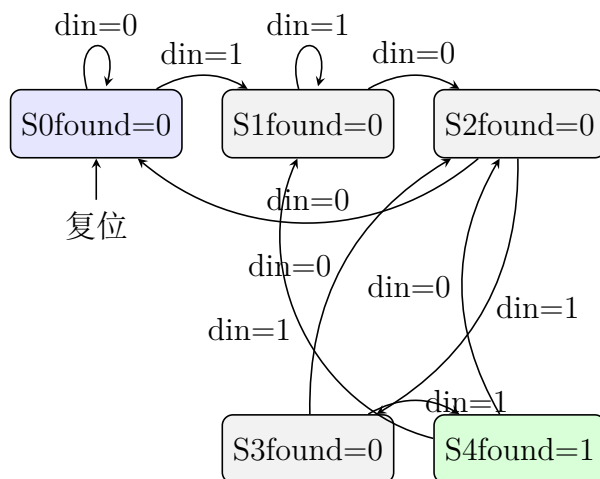


图 10.3: Moore 型“1011”序列检测器状态转移图（支持重叠检测）

重叠检测说明：

- S3（“101”）遇到 din=0：后缀“10”可作新序列前缀 → 回到 S2（已匹配“10”）
- S4（“1011”，检测成功）遇到 din=1：末尾‘1’可作新序列前缀 → 回到 S1；遇到 din=0：“10”可作前缀 → 回到 S2

评分标准（6 分）：

- 5 个状态定义清晰、含义正确 (2 分)
- 所有状态间转移条件正确（10 条转移弧） (3 分)
- 重叠检测处理正确（S3⁰→S2, S4 出弧） (1 分)

表 10.3: 状态转移表（Moore 型：found 只取决于当前状态）

当前状态	din	次态	found
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	0
S3	1	S4	0
S4	0	S2	1
S4	1	S1	1

(2) 状态转移表（3 分）

评分标准：表格完整覆盖所有状态与输入组合（2 分），次态和输出全正确（1 分）。

(3) VHDL 代码（9 分）

Listing 10.5: “1011” 序列检测器 VHDL（Moore 型，支持重叠检测）

```
1  -- =====
2  -- 模块名: seq_detect_1011
3  -- 功能   : Moore型"1011"序列检测器（支持重叠检测）
4  -- 输入   : din——1位串行输入
5  -- 输出   : found——检测到"1011"时为'1'（持续1时钟周期）
6  -- =====
7  LIBRARY IEEE;
8  USE IEEE.STD_LOGIC_1164.ALL;
9
10 ENTITY seq_detect_1011 IS
11     PORT(
12         clk    : IN  STD_LOGIC;
13         rst    : IN  STD_LOGIC;          -- 同步复位，高有效
14         din    : IN  STD_LOGIC;
15         found  : OUT STD_LOGIC
16     );
17 END ENTITY seq_detect_1011;
18
```

```

19 ARCHITECTURE fsm OF seq_detect_1011 IS
20     TYPE state_type IS (S0, S1, S2, S3, S4);
21     SIGNAL current_state, next_state : state_type;
22 BEGIN
23
24     -- ===== 时序进程：状态寄存器（同步复位） =====
25     PROCESS(clk)
26     BEGIN
27         IF rising_edge(clk) THEN
28             IF rst = '1' THEN
29                 current_state <= S0;
30             ELSE
31                 current_state <= next_state;
32             END IF;
33         END IF;
34     END PROCESS;
35
36     -- ===== 组合进程：次态逻辑 =====
37     PROCESS(current_state, din)
38     BEGIN
39         -- 默认值
40         next_state <= S0;
41
42         CASE current_state IS
43             WHEN S0 =>                                     -- 初始/未匹配
44                 IF din = '1' THEN
45                     next_state <= S1;                       -- 匹配到 "1"
46                 ELSE
47                     next_state <= S0;
48                 END IF;
49
50             WHEN S1 =>                                     -- 已匹配 "1"
51                 IF din = '0' THEN
52                     next_state <= S2;                       -- 匹配到 "10"
53                 ELSE
54                     next_state <= S1;                       -- "11", 末尾 '1'→S1
55                 END IF;
56
57             WHEN S2 =>                                     -- 已匹配 "10"

```

```

58     IF din = '1' THEN
59         next_state <= S3;           -- 匹配到 "101"
60     ELSE
61         next_state <= S0;           -- "100", 无匹配
62     END IF;
63
64     WHEN S3 =>                       -- 已匹配 "101"
65         IF din = '1' THEN
66             next_state <= S4;       -- 匹配到 "1011"
67         ELSE
68             next_state <= S2;       -- 重叠: "1010"→后缀 "10"
69         END IF;
70
71     WHEN S4 =>                       -- 已匹配 "1011" (检测成功)
72         IF din = '1' THEN
73             next_state <= S1;       -- 重叠: "1011"+"1"→"1"
74         ELSE
75             next_state <= S2;       -- 重叠: "1011"+"0"→"10"
76         END IF;
77
78     WHEN OTHERS =>
79         next_state <= S0;
80     END CASE;
81 END PROCESS;
82
83 -- ===== Moore型输出逻辑: found仅取决于当前状态 =====
84 found <= '1' WHEN current_state = S4 ELSE '0';
85
86 END ARCHITECTURE fsm;

```

评分标准 (9 分):

- 枚举类型状态定义正确 (1 分)
- 双进程风格清晰 (时序 + 组合分离) (1 分)
- 同步复位实现正确 (仅在 rising_edge 内判断) (1 分)
- S0-S4 的次态逻辑全部正确 (含重叠检测) (4 分)
(每个状态 0.8 分)

- Moore 型输出逻辑正确 (found 仅取决于 current_state) (1 分)
- 代码注释清晰、结构合理 (1 分)

(4) 代码规范 (2 分)

评分标准：状态编码清晰 (枚举类型)、注释完整 (1 分)，进程划分合理、缩进一致 (1 分)。

——第 10 套参考答案结束——

第十一章 第 11 套试卷——参考答案与评分标准

11.1 一、选择题答案（18 分）

1. 【答案】B （3 分）

解析：CPLD 宏单元（Macrocell）的核心结构包括乘积项阵列（AND 阵列 +OR 阵列）和一个可配置的 D 触发器，乘积项阵列用于产生组合逻辑，输出可配置为纯组合输出或寄存器输出（B 正确）。A 错误（宏单元含 D 触发器）；C 错误（CPLD 与 FPGA 逻辑单元结构不同）；D 错误（宏单元可实现时序逻辑）。

2. 【答案】B （3 分）

解析：SRAM 型 FPGA 的配置数据存储在芯片内部的 SRAM 配置存储器中，SRAM 为易失性存储，掉电后数据丢失（B 正确）。上电时 FPGA 从外部非易失性配置存储器（如 SPI Flash）加载比特流到内部 SRAM。A/C 错误（FPGA 内部通常无用户可写的 Flash/EEPROM 存储配置）。

3. 【答案】D （3 分）

解析：题目要求选不属于并发语句的。变量赋值语句（VARIABLE ASSIGNMENT, :=）是顺序语句，只能出现在 PROCESS 或子程序内部（D 正确）。A、B、C 均为并发语句：PROCESS 整体是并发语句（内部是顺序的）；并发信号赋值；元件例化（PORT MAP）。

4. 【答案】A （3 分）

解析：Moore 型输出仅与当前状态有关；Mealy 型输出与当前状态和输入均有关（A 正确）。B 错误（两者复杂度取决于具体应用，无绝对）；C 错误（两者均可检测序列和产生输出）；D 错误（输出延迟一个时钟周期是 Moore 的典型特征而非定义性区别，定义性区别在于输出是否依赖输入）。

5. 【答案】B （3 分）

解析：VHDL 逻辑运算符优先级：NOT 最高，其次 AND/NAND 等，再次 OR/NOR 等，

最后 XOR/XNOR (按国内教材常规分类)。故 NOT > AND > OR 正确 (B)。虽 IEEE 标准中 AND/OR 同级, 但教学上通常区分优先级。

6. 【答案】C (3 分)

解析: n 输入查找表 (LUT) 本质上是一个 $2^n \times 1$ 位的 SRAM, 用于存储 n 变量布尔函数的真值表。共需 2^n 个 SRAM 存储单元 (C 正确)。

11.2 二、填空题答案 (12 分)

1. 【答案】(每空 1.5 分, 共 3 分)

- GENERIC (1.5 分)
- COMPONENT 和 PORT MAP (两空共 1.5 分, 每空 0.75 分)

2. 【答案】(每空 1 分, 共 3 分)

- 实体 (ENTITY) 声明区和结构体 (ARCHITECTURE) 声明区 (两空共 2 分)
- PROCESS (或进程) (1 分)

解析: SIGNAL 可在 ENTITY 声明区 (如 PORT 信号) 和 ARCHITECTURE 声明区声明; VARIABLE 仅在 PROCESS 或子程序内部声明。

3. 【答案】(每空 1 分, 共 3 分)

- GENERATE (1 分)
- IF (1 分)
- FOR (1 分)

4. 【答案】(每空 1.5 分, 共 3 分)

- <= (1.5 分)
- := (1.5 分)

11.3 三、程序改错题解析（5 分）

题目分析：代码意图实现带异步复位的 4 位加法计数器。提示已给出错误类型。共 5 处。

- 错误 1: **全角分号（多行）：**将代码中所有全角分号；改为半角分号；。
影响行：第 1、2、6、7、9、10、13、18、20、21、23、24 行。 (1 分)
原因：VHDL 仅识别 ASCII 半角字符，全角分号导致编译语法错误。
- 错误 2: **（第 15 行/代码第 15 行）**PROCESS(cclk) 改为 PROCESS(cclk, rst) (1 分)
原因：异步复位信号 rst 未列入敏感信号列表，仿真时 rst 的变化不会触发进程，无法实现异步复位行为。
- 错误 3: **（第 18 行/代码第 18 行）**cnt := (OTHERS => '0') 改为 cnt <= (OTHERS => '0') (1 分)
原因：cnt 是 SIGNAL，必须使用信号赋值符号<=，:=仅用于变量。
- 错误 4: **（第 2 行之后）**添加 USE IEEE.NUMERIC_STD.ALL; 或 USE IEEE.STD_LOGIC_UNSIGNED.ALL; (1 分)
原因：第 20 行 cnt <= cnt + 1 对 STD_LOGIC_VECTOR 使用了算术运算符 +，需要引入算术运算包。标准方案推荐 NUMERIC_STD 配合类型转换。
- 错误 5: **（第 10 行/代码第 10 行）**将 END counter4; 改为 END ENTITY counter4;(1 分)
原因：端口声明区最后一行); 中全角括号和分号在第 1 类错误中已覆盖。此处单独指出：实体结束语句的完整写法（VHDL-2008 推荐 END ENTITY < 实体名 >）。

改正后的完整 VHDL 代码：

Listing 11.1: 改正后的 4 位计数器

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;                                -- 修复4：添加算术包
4
5  ENTITY counter4 IS
6    PORT(
7      clk   : IN   STD_LOGIC;
8      rst   : IN   STD_LOGIC;
9      q     : OUT  STD_LOGIC_VECTOR(3 DOWNTO 0)
10    );
11  END ENTITY counter4;                                     -- 修复5：规范实体结束

```

```

12
13 ARCHITECTURE behav OF counter4 IS
14     SIGNAL cnt : STD_LOGIC_VECTOR(3 DOWNTO 0);
15 BEGIN
16     PROCESS(clk, rst)                                -- 修复2: 添加rst
17     BEGIN
18         IF rst = '0' THEN
19             cnt <= (OTHERS => '0');                    -- 修复3: <=替代:=
20         ELSIF rising_edge(clk) THEN
21             cnt <= STD_LOGIC_VECTOR(                  -- 修复4: 类型转换
22                 UNSIGNED(cnt) + 1);
23         END IF;
24     END PROCESS;
25     q <= cnt;
26 END ARCHITECTURE behav;                                -- 修复1: 全角→半角

```

注：所有；(全角) 均已改为; (半角)——修复 1。

11.4 四、程序阅读分析题解析（5 分）

(1) (1 分) 电路识别：该代码描述的是一个 8 位环形计数器（Ring Counter）。

功能：初始值为 00000001，每个时钟上升沿将数据循环右移一位（sr(0) 移入 sr(7)），‘1’ 在 8 位中循环移动，形成 8 个状态的环形计数。

评分：答出“环形计数器”得 1 分。

(2) (2 分) 初始值与 3 个时钟后的值：

- 复位后 q 的初始值："00000001"（0.5 分）
- 第 1 个时钟上升沿：sr ← sr(0) & sr(7 DOWNTO 1) = `1' & "0000000" = "10000000"（0.5 分）
- 第 2 个时钟上升沿：sr ← `0' & "1000000" = "01000000"（0.5 分）
- 第 3 个时钟上升沿：sr ← `0' & "0100000" = "00100000"（0.5 分）

评分：初始值正确 0.5 分；3 个时钟后值正确 1.5 分（含中间推导）。

(3) (1 分) 周期：8 个时钟周期。‘1’ 在 8 位中依次循环移位，经过 8 次移位后回到初始位置 sr(0)。

评分：答出 8 即得 1 分。

(4) (1 分) 时序逻辑。

判断依据：(1) 使用了 `PROCESS(clk, rst)` 且内部判断 `rising_edge(clk)`，电路在时钟边沿驱动；(2) 内部信号 `sr` 通过自身反馈 (`sr(0) & sr(7 DOWNT0 1)`) 保持状态——具有记忆功能，这是时序逻辑的本质特征。

评分：答出“时序逻辑”得 0.5 分；判断依据正确得 0.5 分。

11.5 五、综合设计题参考答案 (60 分)

设计题 1：BCD 码转 7 段数码管译码器（共阴极）(20 分)

(1) 端口定义 (4 分)

输入：`bcd(3 DOWNT0 0)`——4 位 BCD 码输入

输入：`en`——使能控制，`en='1'` 正常译码，`en='0'` 全部熄灭

输出：`seg(6 DOWNT0 0)`——七段段选信号（共阴极，高电平点亮）

`seg(6)=a`, `seg(5)=b`, `seg(4)=c`, `seg(3)=d`, `seg(2)=e`, `seg(1)=f`, `seg(0)=g`

评分标准：端口齐全 (`bcd/en/seg`) 各 1 分 (3 分)，位宽和方向标注正确 (1 分)。

(2) 真值表 (5 分)

共阴极数码管：1= 亮（高电平有效），0= 灭。

表 11.1: 共阴极七段码真值表 (`en=1` 时有效)

十进制	<code>bcd(3:0)</code>	<code>a</code>	<code>b</code>	<code>c</code>	<code>d</code>	<code>e</code>	<code>f</code>	<code>g</code>	<code>seg(6:0)</code>
0	0000	1	1	1	1	1	1	0	1111110
1	0001	0	1	1	0	0	0	0	0110000
2	0010	1	1	0	1	1	0	1	1101101
3	0011	1	1	1	1	0	0	1	1111001
4	0100	0	1	1	0	0	1	1	0110011
5	0101	1	0	1	1	0	1	1	1011011
6	0110	1	0	1	1	1	1	1	1011111
7	0111	1	1	1	0	0	0	0	1110000
8	1000	1	1	1	1	1	1	1	1111111
9	1001	1	1	1	1	0	1	1	1111011
10-15	1010-1111	0	0	0	0	0	0	0	0000000

评分标准：每两个正确数字得 1 分 (5 组共 5 分)。`seg(6)` 对应 `a`、`seg(0)` 对应 `g` 反序标记者视情况扣分。

(3) VHDL 代码 (8 分)

Listing 11.2: BCD 码—七段共阴极数码管译码器 (带使能)

```

1  -- =====
2  -- 模块名: bcd_7seg_cc
3  -- 功能   : BCD 码 (0--9) 至七段共阴极数码管译码器
4  -- 说明   : 共阴极高有效 (1 亮 0 灭), seg(6)=a, ..., seg(0)=g
5  --         en='0' 时全部熄灭, en='1' 时正常译码
6  --         非法输入 (10--15) 时全灭
7  -- =====
8  LIBRARY IEEE;
9  USE IEEE.STD_LOGIC_1164.ALL;
10
11 ENTITY bcd_7seg_cc IS
12     PORT(
13         bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
14         en  : IN  STD_LOGIC;
15         seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)
16     );
17 END ENTITY bcd_7seg_cc;
18
19 ARCHITECTURE dataflow OF bcd_7seg_cc IS
20     SIGNAL seg_raw : STD_LOGIC_VECTOR(6 DOWNTO 0);
21 BEGIN
22     -- 译码逻辑 (WITH...SELECT)
23     WITH bcd SELECT
24         seg_raw <= "1111110" WHEN "0000",    -- 0
25                    "0110000" WHEN "0001",    -- 1
26                    "1101101" WHEN "0010",    -- 2
27                    "1111001" WHEN "0011",    -- 3
28                    "0110011" WHEN "0100",    -- 4
29                    "1011011" WHEN "0101",    -- 5
30                    "1011111" WHEN "0110",    -- 6
31                    "1110000" WHEN "0111",    -- 7
32                    "1111111" WHEN "1000",    -- 8
33                    "1111011" WHEN "1001",    -- 9
34                    "0000000" WHEN OTHERS;    -- 10--15: 全灭
35
36     -- 使能控制: en='0' 时全灭

```

```
37   seg <= seg_raw WHEN en = '1' ELSE "0000000";  
38 END ARCHITECTURE dataflow;
```

评分标准（8 分）：

- 实体定义正确（含 en 端口）（1.5 分）
- 使用 WITH-SELECT 或 WHEN-ELSE 实现组合逻辑（1 分）
- 0-9 段码经编译后正确（每 2 个数字 0.5 分）（2.5 分）
- OTHERS 处理非法输入（全 0）（1 分）
- 使能控制逻辑正确（en=0 时全灭）（1 分）
- 共阴极编码正确（1= 亮）（1 分）

（4）代码规范（3 分）

评分标准：端口类型合理（STD_LOGIC/STD_LOGIC_VECTOR）（1 分），信号命名规范（1 分），注释清晰（1 分）。

设计题 2：级联分频器（20 分）

（1）底层 D 触发器模块（6 分）

2 分频原理： $Q_{n+1} = \overline{Q_n}$ ，即每个时钟上升沿输出翻转一次。输出周期为时钟周期的 2 倍，频率为 1/2。

Listing 11.3: 底层 D 触发器（2 分频单元）

```

1  -- =====
2  -- 模块名：dff_div
3  -- 功能   ：带异步复位的 D 触发器， $Q_{n+1} = NOT\ Q_n$ （2 分频）
4  -- =====
5  LIBRARY IEEE;
6  USE IEEE.STD_LOGIC_1164.ALL;
7
8  ENTITY dff_div IS
9      PORT(
10         clk : IN  STD_LOGIC;
11         rst : IN  STD_LOGIC;      -- 低电平有效异步复位
12         q   : OUT STD_LOGIC;
13         qn  : OUT STD_LOGIC      -- 反相输出
14     );
15 END ENTITY dff_div;
16
17 ARCHITECTURE behav OF dff_div IS
18     SIGNAL q_int : STD_LOGIC := '0';
19 BEGIN
20     PROCESS(clk, rst)
21     BEGIN
22         IF rst = '0' THEN
23             q_int <= '0';
24         ELSIF rising_edge(clk) THEN
25             q_int <= NOT q_int;      --  $Q_{n+1} = NOT\ Q_n \rightarrow$  2 分频
26         END IF;
27     END PROCESS;
28     q  <= q_int;
29     qn <= NOT q_int;
30 END ARCHITECTURE behav;

```

评分标准（6 分）：

- 实体定义完整 (含 clk/rst/q/qn) (1.5 分)
- 异步复位正确 (rst 在敏感表, IF rst 先于时钟) (1.5 分)
- 翻转逻辑正确 ($q_int \leq \text{NOT } q_int$) (1.5 分)
- 反相输出 qn 赋值正确 (1 分)
- 代码规范与注释 (0.5 分)

(2) 顶层模块设计 (10 分)

Listing 11.4: 8 级级联分频器顶层 (FOR GENERATE)

```

1  -- =====
2  -- 模块名: div_chain
3  -- 功能   : 8级级联2分频器, 第k级输出频率 = f_clk / 2^k
4  -- 架构   : 结构化描述, COMPONENT + FOR GENERATE
5  -- =====
6  LIBRARY IEEE;
7  USE IEEE.STD_LOGIC_1164.ALL;
8
9  ENTITY div_chain IS
10     GENERIC (N : INTEGER := 8);           -- 级联级数
11     PORT(
12         clk    : IN  STD_LOGIC;
13         rst    : IN  STD_LOGIC;           -- 低有效异步复位
14         q_out  : OUT STD_LOGIC_VECTOR(N-1 DOWNT0 0) -- 各级输出
15     );
16 END ENTITY div_chain;
17
18 ARCHITECTURE structural OF div_chain IS
19     -- 声明底层组件
20     COMPONENT dff_div IS
21         PORT(
22             clk : IN  STD_LOGIC;
23             rst : IN  STD_LOGIC;
24             q   : OUT STD_LOGIC;
25             qn  : OUT STD_LOGIC
26         );
27     END COMPONENT dff_div;

```

```

28
29  -- 内部级联时钟线
30  SIGNAL clk_chain : STD_LOGIC_VECTOR(N DOWNT0 0);
31  SIGNAL q_n_int    : STD_LOGIC_VECTOR(N-1 DOWNT0 0);  -- 未使用的反相
    输出
32 BEGIN
33  -- 第一级时钟接外部时钟
34  clk_chain(0) <= clk;
35
36  -- FOR GENERATE 生成 N 级级联分频器
37  gen_chain: FOR i IN 0 TO N-1 GENERATE
38      dffx: dff_div
39          PORT MAP(
40              clk => clk_chain(i),          -- 本级时钟=前级 Q 输出
41              rst => rst,                  -- 所有级复位并联
42              q   => clk_chain(i+1),        -- 本级 Q 输出=次级时钟输入
43              qn  => q_n_int(i)            -- 反相输出（未使用）
44          );
45  END GENERATE gen_chain;
46
47  -- 输出各级的 Q 信号
48  q_out <= clk_chain(N DOWNT0 1);
49  END ARCHITECTURE structural;

```

评分标准（10 分）：

- GENERIC 定义 N=8 正确 (1 分)
- COMPONENT 声明与底层接口一致 (1 分)
- FOR GENERATE 正确生成 N 级 (2 分)
- 级联时钟连接正确（clk_chain(i) 接本级，clk_chain(i+1) 接次级） (2 分)
- 复位信号并联正确 (1 分)
- q_out 输出各级 Q 信号正确 (1 分)
- PORT MAP 端口关联完整 (1 分)
- 代码规范：注释清晰、缩进一致 (1 分)

(3) 设计思路说明 (4 分)

在代码注释中应说明:

1. **分频原理:** 每个 D 触发器的 $Q_{n+1} = \overline{Q_n}$ 构成 2 分频, 即输出频率为输入时钟的 $1/2$ 。N 级级联后总输出频率为输入时钟的 $1/2^N$ 。第 k 级 ($k = 1, 2, \dots, 8$) 输出频率为 $f_{clk}/2^k$ 。
2. **使用 GENERATE 的原因:** 8 级电路结构完全重复——每级均为相同的 D 触发器, 仅级联位置不同。使用 FOR...GENERATE 语句可避免重复书写 8 次 PORT MAP, 代码更简洁、易于维护, 且可通过修改 GENERIC N 灵活调整级联级数。

评分标准: 分频原理说明清晰 (2 分), GENERATE 选择理由充分 (2 分)。

设计题 3：自动售货机控制器（20 分）

（1）功能说明（3 分）

- 输入：coin_1（1 元投币脉冲）、coin_2（2 元投币脉冲），两信号不同时有效，高电平持续 1 个时钟周期。
- 输出：dispense（出货信号），高电平有效，持续 1 个时钟周期。
- 时钟与复位：clk 上升沿触发，rst 异步复位（高有效）。
- 金额逻辑：累计金额 ≥ 3 元时出货，4 元（2+2）也出货但不找零。
- 商品价格为 3 元。

评分标准：正确描述所有输入输出信号含义（1.5 分）；正确说明金额累计与出货逻辑（1.5 分）。

（2）状态转移图（6 分）

状态机类型：Moore 型——输出 dispense 仅取决于当前状态（累计金额），与输入无关。

状态定义：

- S0：已投 0 元，dispense=0
- S1：已投 1 元，dispense=0
- S2：已投 2 元，dispense=0
- S3：已投 ≥ 3 元（3 元或 4 元），dispense=1

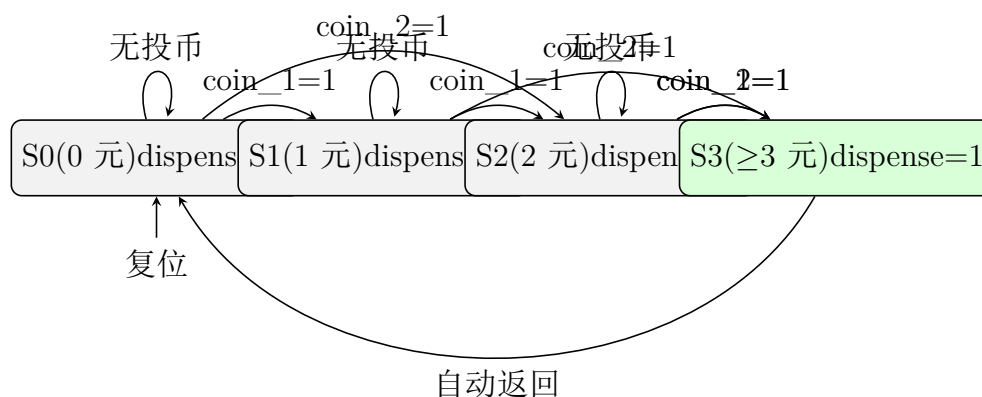


图 11.1: 自动售货机 Moore 型状态转移图

评分标准（6 分）：

- 状态定义清晰（4 个状态，含义正确）（2 分）
- 所有转移条件正确（无投币/coin_1/coin_2）（2.5 分）
- 每个状态标注输出 dispense 正确（仅 S3=1）（0.5 分）
- 明确标注 Moore 型（1 分）

(3) 状态编码与状态转移表（3 分）

状态编码方案（独热码推荐，也可用二进制）：

- S0 → "00"，S1 → "01"，S2 → "10"，S3 → "11"

表 11.2: 自动售货机状态转移表（Moore 型）

当前状态	coin_1	coin_2	次态	dispense
S0(0 元)	0	0	S0	0
S0(0 元)	1	0	S1	0
S0(0 元)	0	1	S2	0
S1(1 元)	0	0	S1	0
S1(1 元)	1	0	S2	0
S1(1 元)	0	1	S3	0
S2(2 元)	0	0	S2	0
S2(2 元)	1	0	S3	0
S2(2 元)	0	1	S3	0
S3(≥3 元)	×	×	S0	1

评分标准：状态编码方案合理（1 分），转移表全部正确（2 分）。

(4) VHDL 代码（8 分）

Listing 11.5: 自动售货机控制器（双进程 Moore 型）

```
1  -- =====
2  -- 模块名：vending_machine
3  -- 功能   ：自动售货机控制器（Moore型，3元商品，不找零）
4  -- 输入   ：coin_1(1元)，coin_2(2元) -- 不同时有效
5  -- 输出   ：dispense(出货)
6  -- =====
7  LIBRARY IEEE;
8  USE IEEE.STD_LOGIC_1164.ALL;
```

```

9
10 ENTITY vending_machine IS
11     PORT(
12         clk          : IN  STD_LOGIC;
13         rst          : IN  STD_LOGIC;          -- 异步复位，高有效
14         coin_1       : IN  STD_LOGIC;          -- 1元投币脉冲
15         coin_2       : IN  STD_LOGIC;          -- 2元投币脉冲
16         dispense     : OUT STD_LOGIC          -- 出货信号
17     );
18 END ENTITY vending_machine;
19
20 ARCHITECTURE fsm OF vending_machine IS
21     TYPE state_type IS (S0, S1, S2, S3); -- 0元,1元,2元,>=3元
22     SIGNAL current_state, next_state : state_type;
23 BEGIN
24
25     -- ===== 时序进程：状态寄存器 + 异步复位 =====
26     PROCESS(clk, rst)
27     BEGIN
28         IF rst = '1' THEN
29             current_state <= S0;
30         ELSIF rising_edge(clk) THEN
31             current_state <= next_state;
32         END IF;
33     END PROCESS;
34
35     -- ===== 组合进程：次态逻辑 =====
36     PROCESS(current_state, coin_1, coin_2)
37     BEGIN
38         -- 默认值
39         next_state <= current_state;
40
41         CASE current_state IS
42             WHEN S0 => -- 已投0元
43                 IF coin_1 = '1' THEN
44                     next_state <= S1;
45                 ELSIF coin_2 = '1' THEN
46                     next_state <= S2;
47                 END IF;

```

```

48
49     WHEN S1 =>                                -- 已投 1 元
50         IF coin_1 = '1' THEN
51             next_state <= S2;
52         ELSIF coin_2 = '1' THEN
53             next_state <= S3;
54         END IF;
55
56     WHEN S2 =>                                -- 已投 2 元
57         IF coin_1 = '1' OR coin_2 = '1' THEN
58             next_state <= S3;                -- 2+1=3 元 或 2+2=4 元 (3)
59         END IF;
60
61     WHEN S3 =>                                -- 已投 3 元
62         next_state <= S0;                    -- 出货后回到初始
63
64     WHEN OTHERS =>
65         next_state <= S0;
66     END CASE;
67 END PROCESS;
68
69 -- ===== Moore 型输出逻辑 =====
70 dispense <= '1' WHEN current_state = S3 ELSE '0';
71
72 END ARCHITECTURE fsm;

```

评分标准 (8 分):

- 枚举类型状态定义合理 (1 分)
- 双进程风格清晰 (时序 + 组合分离) (1 分)
- 异步复位正确 (1 分)
- S0/S1/S2 次态逻辑正确 (覆盖 coin_1/coin_2/无投币) (3 分)
- S3 次态正确 (回 S0) (0.5 分)
- Moore 型输出逻辑正确 (0.5 分)
- 代码注释清晰、结构合理 (1 分)

——第 11 套参考答案结束——

第十二章 第 12 套试卷——参考答案与评分标准

12.1 一、选择题答案（18 分）

1. 【答案】B （3 分）

解析：基于 SRAM 工艺的 FPGA，配置数据存储在内部 SRAM 单元中，掉电即丢失，上电后须从外部非易失性配置存储器（如 SPI Flash）加载比特流（B 正确）。A 错误（SRAM 型 FPGA 内部无用户 Flash 存储配置数据）。C 错误（FPGA 支持 JTAG/主串/从串/主并等多种模式）。D 错误（CPLD 通常为 Flash/EEPROM 工艺，FPGA 为 SRAM 工艺）。

2. 【答案】D （3 分）

解析：题目要求选错误的。USB 配置模式并非 FPGA 最基础的配置模式，也非所有 FPGA 均支持（D 错误）。主串模式由 FPGA 主动从外部 PROM 读取（A 正确）；从串模式由外部控制器向 FPGA 传输（B 正确）；JTAG 模式用于在线调试和编程（C 正确）。

3. 【答案】C （3 分）

解析：PROCESS 内部的语句按书写顺序依次调度执行（顺序语句），而多个 PROCESS 之间是并发关系（C 正确）。A 错误（PROCESS 内部是顺序执行的）；B 错误（结构体中 PROCESS 外部的信号赋值语句是并发的）；D 错误（变量赋值:= 只能出现在 PROCESS 或子程序内部，不能出现在结构体任意位置）。

4. 【答案】A （3 分）

解析：Moore 型输出仅取决于状态，状态在时钟边沿才更新，因此输出变化与时钟同步，相对于 Mealy 型通常延迟一个时钟周期（A 正确）。Mealy 型输出是输入和状态的组合函数，输入变化可即时影响输出（B 错误）。两者输出时序结构不同（C 错误）。Moore 型输出与时钟同步（D 错误）。

5. 【答案】C （3 分）

解析：按 IEEE 1076 标准，NOT 优先级最高；AND、OR、NAND、NOR、XOR、XNOR 六者

优先级相同且低于 NOT (C 正确)。国内教材虽常区分为 NOT>AND>OR>XOR, 但标准规定 AND/OR/XOR 同级。

6. 【答案】B (3 分)

解析: 在 VHDL 结构化描述风格中, 必须先使用 COMPONENT 声明待例化的子模块接口, 再使用 PORT MAP 将子模块端口与顶层信号连接 (B 正确)。A 中 VARIABLE 是变量 (不需于结构化描述), C 中 FUNCTION/PROCEDURE 是子程序 (行为级), D 中 TYPE/SUBTYPE 是类型定义。

12.2 二、填空题答案 (12 分)

1. 【答案】(每空 0.6 分, 共 5 空 $\times 0.6=3$ 分)

- SIGNAL 声明位置: 结构体 (ARCHITECTURE) 的声明区 (0.6 分)
- SIGNAL 赋值符号: <= (0.6 分)
- VARIABLE 声明位置: PROCESS (或进程) (0.6 分)
- VARIABLE 赋值符号: := (0.6 分)
- CONSTANT 在仿真运行期间不可以被修改 (0.6 分)

2. 【答案】 (每空 1.5 分, 共 3 分)

- GENERIC (1.5 分)
- GENERIC MAP (或 PORT MAP 附带 GENERIC 映射) (1.5 分)

3. 【答案】 (每空 1.5 分, 共 3 分)

- COMPONENT (1.5 分)
- PORT MAP (1.5 分)

4. 【答案】 (每空 1 分, 共 3 分)

- GENERATE (1 分)
- FOR (1 分)
- 并发 (1 分)

解析: GENERATE 语句只能在结构体内部使用 (不能出现在 PROCESS 内), 属于并发生成语句。

12.3 三、程序改错题解析（5 分）

题目分析：代码包含两个模块——组合逻辑二选一多路选择器（含错误）和时序逻辑 4 位计数器（含错误）。共 5 处错误。

错误 1：（第 18 行/代码第 18 行）PROCESS(a) 改为 PROCESS(a, b, sel) （1 分）

原因：该 PROCESS 描述组合逻辑（多路选择器），敏感信号列表必须包含所有输入信号（a、b、sel），否则当 sel 或 b 变化时进程不触发，仿真结果与综合结果不一致，且综合工具会推断出不必要的锁存器。

错误 2：（第 20–23 行/代码第 20–23 行）补全 IF 语句的 ELSE 分支：在 IF sel = `1' THEN y <= a; 之后添加 ELSE y <= b; （1 分）

原因：组合逻辑进程中，IF 语句必须覆盖所有分支。当 sel=`0' 时 y 未被赋值，综合工具将推断出锁存器（Latch）来保持 y 的值，而非期望的组合逻辑多路选择器。

错误 3：（第 27 行/代码第 27 行）PROCESS(rst) 改为 PROCESS(clk, rst) （1 分）

原因：该进程为带异步复位的时序逻辑（计数器），敏感信号列表必须同时包含时钟和异步复位信号。仅含 rst 时，计数器无法响应时钟边沿，无法正常计数。

错误 4：（第 30 行/代码第 30 行）cnt := (OTHERS => `0') 改为 cnt <= (OTHERS => `0') （1 分）

原因：cnt 被声明为 SIGNAL，信号赋值必须使用符号<=，:=仅用于 VARIABLE。

错误 5：（第 36–37 行/代码第 36–37 行）类型不兼容:temp(INTEGER) 与 cnt(STD_LOGIC_VECTOR) 无法直接互赋。

改正方案：删除 temp 信号，直接使用 q <= cnt;。若需保留 temp，则需添加类型转换:temp <= TO_INTEGER(UNSIGNED(cnt)); 和 q <= STD_LOGIC_VECTOR(TO_UNSIGNED(temp, 4));。 （1 分）

原因：VHDL 是强类型语言，INTEGER 与 STD_LOGIC_VECTOR 之间不允许隐式类型转换，必须使用 NUMERIC_STD 包的转换函数。

改正后的完整 VHDL 代码：

Listing 12.1: 改正后的混合模块

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;                                -- 添加算术包
4
5  ENTITY mixed_err IS

```

```

6   PORT(
7       clk, rst, sel : IN  STD_LOGIC;
8       a, b          : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
9       y             : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
10      q             : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
11  );
12  END ENTITY mixed_err;
13
14  ARCHITECTURE behav OF mixed_err IS
15      SIGNAL cnt : STD_LOGIC_VECTOR(3 DOWNTO 0);
16  BEGIN
17      -- ===== 组合逻辑：二选一多路选择器 =====
18      PROCESS(a, b, sel)                                -- 修复1：加入 b 和 sel
19      BEGIN
20          IF sel = '1' THEN
21              y <= a;
22          ELSE                                           -- 修复2：补全 ELSE 分支
23              y <= b;
24          END IF;
25      END PROCESS;
26
27      -- ===== 时序逻辑：4 位计数器 =====
28      PROCESS(clk, rst)                                -- 修复3：加入 clk
29      BEGIN
30          IF rst = '1' THEN
31              cnt <= (OTHERS => '0');                    -- 修复4：<= 替代 :=
32          ELSIF rising_edge(clk) THEN
33              cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1);
34          END IF;
35      END PROCESS;
36
37      q <= cnt;                                           -- 修复5：直接赋值，移
        除类型不兼容的 temp
38  END ARCHITECTURE behav;

```

12.4 四、程序阅读分析题解析（5 分）

(1)（1 分）电路识别：该代码描述的是一个 4 位串入并出移位寄存器（SIPO Shift Register）。

逻辑类型：时序逻辑。

判断依据：内部使用了 PROCESS(clk) 并判断 rising_edge(clk)；信号 sr 在时钟边沿更新且依赖自身历史值 (sr <= din & sr(3 DOWNT0 1)) ——具有状态保持能力，是时序逻辑的本质特征。

评分：答出“移位寄存器”得 0.5 分，答出“时序逻辑”及正确依据得 0.5 分。

(2)（1 分）同步复位。

判断依据：rst 仅出现在 IF rising_edge(clk) THEN 内部的 IF rst = `1' THEN 中，即复位判断被包在时钟边沿检测之内。PROCESS 的敏感信号列表仅为 clk（不含 rst）——这意味着 rst 的变化不会触发进程，只有在时钟上升沿到达时才会检查 rst 的值。这是同步复位的典型 VHDL 模板。

评分：答出“同步复位”得 0.5 分，正确分析敏感列表和 IF 嵌套得 0.5 分。

(3)（2 分）各时钟周期后 sr 的值：

时钟上升沿	din 输入	sr 更新操作	sr 值
初始	—	—	0000
第 1 个	‘1’	sr ← ‘1’ & ”000”	1000
第 2 个	‘0’	sr ← ‘0’ & ”100”	0100
第 3 个	‘1’	sr ← ‘1’ & ”010”	1010
第 4 个	‘1’	sr ← ‘1’ & ”101”	1101

评分：第 1、2 个值正确各 0.5 分（1 分），第 3、4 个值正确各 0.5 分（1 分）。

(4)（1 分）移位方向变化：原代码为右移（din 从高位 MSB 移入：din & sr(3 DOWNT0 1)）；改为 sr(2 DOWNT0 0) & din 后变为左移（din 从低位 LSB 移入）。

若 din 固定为 ‘0’：由于复位后 sr=“0000”，每个周期左移并在 LSB 移入 ‘0’，sr 始终为“0000”。

评分：答出移位方向变为左移得 0.5 分，答出 sr 值始终为”0000”得 0.5 分。

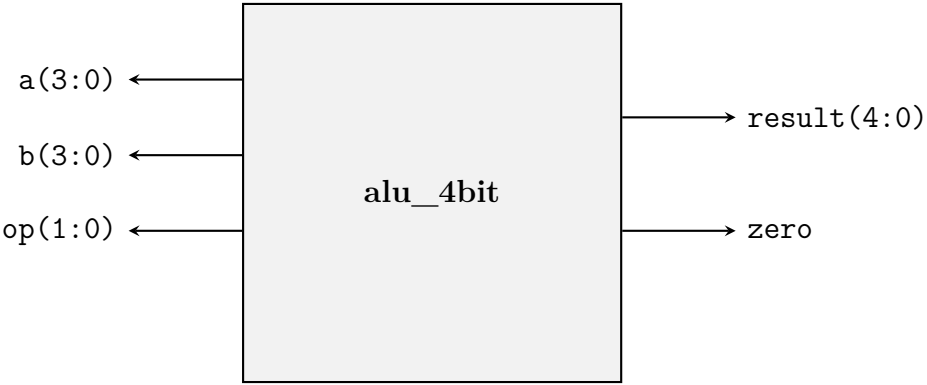


图 12.1: 4 位 ALU 顶层端口框图

12.5 五、综合设计题参考答案（60 分）

设计题 1：4 位算术逻辑单元（ALU）设计（20 分）

（1）顶层模块端口框图（4 分）

评分标准：5 个端口全部标注且方向正确（2 分），位宽标注正确（2 分）。

（2）ALU 功能表（3 分）

表 12.1: 4 位 ALU 功能表

op(1:0)	运算	输出表达式	result 各位来源
00	加法	result = a + b	result(4) 为进位，result(3:0) 为和
01	减法	result = a - b	result(4) 为借位，result(3:0) 为差
10	按位与	result = '0' & (a AND b)	result(4)=0, result(3:0)=a AND b
11	按位或	result = '0' & (a OR b)	result(4)=0, result(3:0)=a OR b

评分标准：4 种运算的描述及表达式正确（2 分），result 位来源说明正确（1 分）。

（3）VHDL 代码（10 分）

Listing 12.2: 4 位 ALU（数据流描述风格）

```

1  -- =====
2  -- 模块名：alu_4bit
3  -- 功能   ：4位算术逻辑单元
4  -- 操作   ：op=00(加)，01(减)，10(与)，11(或)
5  -- 说明   ：算术运算result(4)为进位/借位；逻辑运算result(4)='0'
6  --        result(4:0)共5位，zero标志检测result(3:0)=0
    
```

```

7  -- =====
8  LIBRARY IEEE;
9  USE IEEE.STD_LOGIC_1164.ALL;
10 USE IEEE.NUMERIC_STD.ALL;
11
12 ENTITY alu_4bit IS
13     PORT(
14         a      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
15         b      : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
16         op     : IN  STD_LOGIC_VECTOR(1 DOWNTO 0);
17         result : OUT STD_LOGIC_VECTOR(4 DOWNTO 0);  -- 5位: 含进位/借位
18         zero   : OUT STD_LOGIC
19     );
20 END ENTITY alu_4bit;
21
22 ARCHITECTURE dataflow OF alu_4bit IS
23     SIGNAL res_int : STD_LOGIC_VECTOR(4 DOWNTO 0);
24 BEGIN
25     -- 使用 WITH...SELECT 实现运算选择
26     WITH op SELECT
27         res_int <=
28             -- 加法: 0 & a + 0 & b → 5位结果 (含进位)
29             STD_LOGIC_VECTOR( ('0' & UNSIGNED(a)) + ('0' & UNSIGNED(b)) )
30             WHEN "00",
31             -- 减法: 0 & a - 0 & b → 5位结果 (含借位)
32             STD_LOGIC_VECTOR( ('0' & UNSIGNED(a)) - ('0' & UNSIGNED(b)) )
33             WHEN "01",
34             -- 按位与: 高位补0
35             '0' & (a AND b)
36             WHEN "10",
37             -- 按位或: 高位补0
38             '0' & (a OR b)
39             WHEN "11",
40             (OTHERS => '0')
41             WHEN OTHERS;
42
43     result <= res_int;
44
45     -- 零标志位: 当 result(3 DOWNTO 0) = "0000" 时输出 '1'

```

```

41 zero <= '1' WHEN res_int(3 DOWNT0 0) = "0000" ELSE '0';
42
43 END ARCHITECTURE dataflow;

```

评分标准（10 分）：

- 实体定义完整，端口位宽和方向正确 (2 分)
- 使用 WITH...SELECT 或 WHEN...ELSE 实现数据流风格 (1.5 分)
- 加法实现正确（含高位扩展和类型转换） (1.5 分)
- 减法实现正确 (1.5 分)
- 按位与/或实现正确，高位补 0 (1.5 分)
- 零标志位独立并发赋值语句生成 (1 分)
- 代码规范与注释 (1 分)

(4) 注释说明（3 分）

(1) 算术与逻辑运算的不同处理方式：

- 算术运算(加减法)需要将 STD_LOGIC_VECTOR 转换为 UNSIGNED 类型,因为 +/-运算符对 STD_LOGIC_VECTOR 未定义。同时需要在操作数前补 '0' 扩展为 5 位，以便容纳进位/借位。
- 逻辑运算（AND/OR）可直接对 STD_LOGIC_VECTOR 按位操作，结果高位补 '0' 组成 5 位输出。

(2) result 位宽设为 5 位的原因：4 位加法/减法可能产生进位或借位（超出 4 位表示范围），需要第 5 位（result(4)）来存储该进位/借位标志，保证结果不丢失信息。

评分标准：（1）说明正确（1.5 分），（2）说明正确（1.5 分）。

设计题 2：参数化模 N 计数器设计（20 分）

(1) GENERIC 声明与意义（3 分）

```
1 ENTITY mod_n_counter IS
2   GENERIC (N : INTEGER := 10);      -- 模值参数，默认 10
3   PORT( ... );
4 END ENTITY;
```

GENERIC 的意义：

- 参数化设计：通过修改 GENERIC 值可使同一 VHDL 设计适配不同计数模值（如 N=6、N=10、N=60），无需重写代码。
- 代码复用：顶层模块在例化时通过 GENERIC MAP 传递实际参数值。
- 可维护性：一处修改即可改变整体设计行为，便于调试和迭代。

评分标准：GENERIC 声明语法正确（1.5 分），意义说明充分（1.5 分）。

(2) 状态转移图（N=6）（5 分）

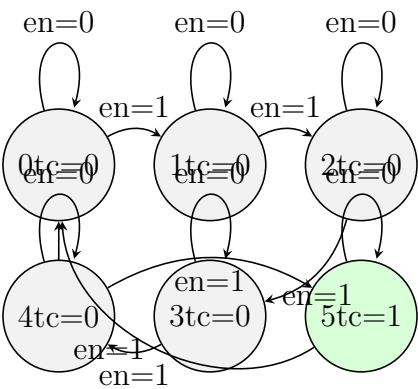


图 12.2: 模 6 计数器状态转移图（0-5，tc 仅在状态 5 时为 1）

评分标准：6 个状态齐全（1.5 分），转移条件正确（en=1 前进/en=0 保持）（2 分），tc 输出标注正确（仅 S5 时 tc=1）（1.5 分）。

(3) VHDL 代码（9 分）

Listing 12.3: 参数化模 N 计数器（行为描述）

```
1 -- =====
2 -- 模块名: mod_n_counter
```

```

3  -- 功能    : 参数化模N同步计数器 (0 ~ N-1)
4  -- 说明    : GENERIC N控制模值; 同步复位>使能优先级
5  --          tc在计数值=N-1且en='1'时输出 '1'
6  -- =====
7  LIBRARY IEEE;
8  USE IEEE.STD_LOGIC_1164.ALL;
9  USE IEEE.NUMERIC_STD.ALL;
10
11 ENTITY mod_n_counter IS
12     GENERIC (N : INTEGER := 10);           -- 模值参数
13     PORT(
14         clk : IN  STD_LOGIC;
15         rst : IN  STD_LOGIC;               -- 同步复位, 高有效
16         en  : IN  STD_LOGIC;               -- 计数使能, 高有效
17         q   : OUT STD_LOGIC_VECTOR(3 DOWNT0 0); -- 计数值输出
18         tc  : OUT STD_LOGIC                -- 终端计数
19     );
20 END ENTITY mod_n_counter;
21
22 ARCHITECTURE behav OF mod_n_counter IS
23     SIGNAL count : INTEGER RANGE 0 TO N-1 := 0; -- 内部计数器
24 BEGIN
25     PROCESS(clk)
26     BEGIN
27         IF rising_edge(clk) THEN
28             IF rst = '1' THEN               -- 同步复位 (最高优先级)
29                 count <= 0;
30             ELSIF en = '1' THEN              -- 使能控制
31                 IF count = N-1 THEN
32                     count <= 0;              -- 计满回零
33                 ELSE
34                     count <= count + 1;
35                 END IF;
36             END IF;                          -- en=0时保持
37         END IF;
38     END PROCESS;
39
40     -- 输出转换
41     q <= STD_LOGIC_VECTOR(TO_UNSIGNED(count, 4));

```

```
42
43  -- 终端计数: count=N-1 且 en='1' 时 tc=1
44  tc <= '1' WHEN (count = N-1) AND (en = '1') ELSE '0';
45
46 END ARCHITECTURE behav;
```

评分标准 (9 分):

- GENERIC 定义正确 (INTEGER 类型, 默认值 10) (1 分)
- 内部 INTEGER 信号定义正确 (范围 0 TO N-1) (1 分)
- 同步复位实现正确 (仅 IF rising_edge 内判断) (1.5 分)
- 使能控制逻辑正确 (en=1 时计数, en=0 时保持) (1.5 分)
- 模 N 回零逻辑正确 (count=N-1→0) (1.5 分)
- tc 输出逻辑正确 (count=N-1 且 en=1 时有效) (1.5 分)
- 代码规范与注释 (1 分)

(4) 问题回答 (3 分)

(a) N=60 时端口适配问题 (1.5 分): 4 位输出的 q(3 DOWNT0 0) 最大可表示 $2^4-1=15$, 无法完整表示计数值 $0\sim 59$ 。需将 q 的位宽从 4 位扩展为 6 位 (q(5 DOWNT0 0), $2^6=64\geq 60$), 并将输出转换语句中的位宽参数改为 6: TO_UNSIGNED(count, 6)。

(b) 同步复位与异步复位的优缺点对比 (1.5 分):

对比维度	同步复位	异步复位
复位响应	仅在时钟沿生效, 可能延迟 1 周期	立即生效, 不等待时钟
时序稳定性	好, 复位与时钟同步, 无亚稳态风险	需处理复位释放时的恢复时间问题
资源占用	略多 (复位信号需经过数据路径)	可使用触发器专用复位端, 资源少

改用异步复位的代码修改:

1. PROCESS 敏感信号表添加 rst: PROCESS(clk, rst)
2. 将复位判断移至 IF rising_edge(clk) 之前:

```

1 PROCESS(clk, rst)
2 BEGIN
3     IF rst = '1' THEN
4         count <= 0;                -- 异步复位
5     ELSIF rising_edge(clk) THEN
6         IF en = '1' THEN
7             IF count = N-1 THEN
8                 count <= 0;
9             ELSE
10                count <= count + 1;
11            END IF;
12        END IF;
13    END IF;
14 END PROCESS;

```

评分标准：(a) 正确指出 4 位不够并给出正确修改方案 (1.5 分); (b) 优缺点对比合理 (0.75 分), 异步复位代码修改正确 (0.75 分)。

设计题 3：电子密码锁控制器（20 分）

（1）状态定义与状态转移图（4 分）

状态定义（Moore 型）：

- S0（IDLE/初始）：未匹配任何位，等待首位输入。unlock=0。
- S1（已匹配“1”）：已正确输入密码第 1 位‘1’。unlock=0。
- S2（已匹配“11”）：已正确输入密码前 2 位“11”。unlock=0。
- S3（已匹配“110”）：已正确输入密码前 3 位“110”。unlock=0。
- S4（已匹配“1101”——开锁）：4 位密码全部匹配。unlock=1。

状态机类型：Moore 型——unlock 仅由当前状态决定（仅 S4 时 unlock=1），与输入 din 无关。

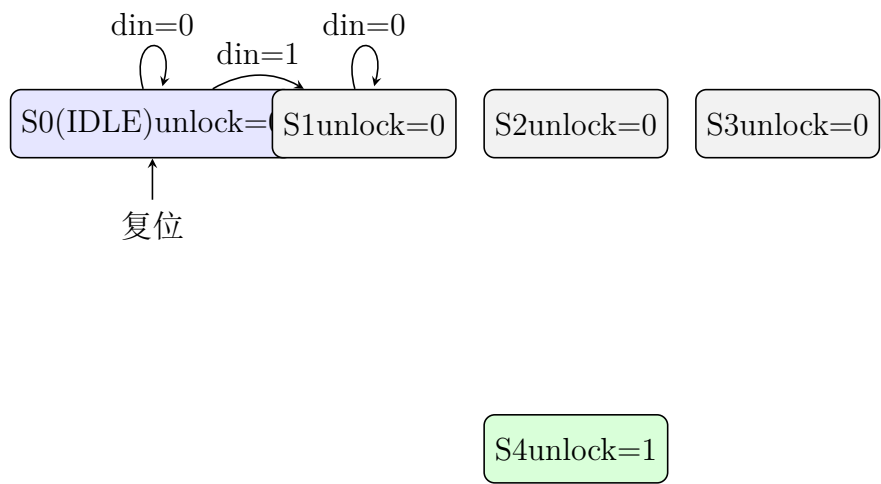


图 12.3: 电子密码锁 Moore 型状态转移图

注：为完整展示所有转移——图中各状态转移条件可描述为：

- $S0 \xrightarrow{\text{din}=1} S1$ （匹配‘1’）； $S0 \xrightarrow{\text{din}=0} S0$ （等待）
- $S1 \xrightarrow{\text{din}=1} S2$ （匹配“11”）； $S1 \xrightarrow{\text{din}=0} S0$ （不匹配，回初始）
- $S2 \xrightarrow{\text{din}=1} S3$ （匹配“110”）； $S2 \xrightarrow{\text{din}=0} S0$ （不匹配）
- $S3 \xrightarrow{\text{din}=1} S4$ （匹配“1101”）； $S3 \xrightarrow{\text{din}=0} S0$ （不匹配）
- $S4 \xrightarrow{\text{无条件}} S0$ （开锁后返回初始，非重叠检测）

评分标准（4 分）：

- 5 个状态定义清晰、含义正确 (1.5 分)
- 所有转移条件完整正确 (10 条弧) (1.5 分)
- 各状态 unlock 输出标注正确 (仅 S4=1) (0.5 分)
- 注明 Moore 型及理由 (0.5 分)

(2) 状态转移表 (4 分)

表 12.2: 密码锁 Moore 型状态转移表 (密码: 1101, 非重叠检测)

当前状态	din	次态	unlock
S0(IDLE)	0	S0	0
S0(IDLE)	1	S1	0
S1(已匹配 “1”)	0	S0	0
S1(已匹配 “1”)	1	S2	0
S2(已匹配 “11”)	0	S3	0
S2(已匹配 “11”)	1	S0	0
S3(已匹配 “110”)	0	S0	0
S3(已匹配 “110”)	1	S4	0
S4(已匹配 “1101”)	×	S0	1

评分标准：表格完整覆盖所有状态和输入 (2 分)，次态全部正确 (1.5 分)，unlock 列正确 (0.5 分)。

(3) VHDL 代码 (9 分)

Listing 12.4: 电子密码锁控制器 (双进程 Moore 型, 密码 1101)

```
1  -- =====
2  -- 模块名: password_lock
3  -- 功能   : 电子密码锁控制器 (Moore型, 密码固定为 "1101")
4  -- 说明   : 非重叠检测——开锁后立即回到S0重新检测
5  --       din每次输入1位, 连续4位匹配 "1101"则 unlock=1
6  -- =====
7  LIBRARY IEEE;
8  USE IEEE.STD_LOGIC_1164.ALL;
9
10 ENTITY password_lock IS
11     PORT(
```

```

12     clk      : IN  STD_LOGIC;
13     rst      : IN  STD_LOGIC;           -- 异步复位，高有效
14     din      : IN  STD_LOGIC;           -- 串行密码输入(1位/周期)
15     unlock   : OUT STD_LOGIC           -- 开锁信号
16 );
17 END ENTITY password_lock;
18
19 ARCHITECTURE fsm OF password_lock IS
20     -- 自定义枚举类型: S0=IDLE, S1~S4=已匹配1~4位
21     TYPE lock_state IS (S0, S1, S2, S3, S4);
22     SIGNAL current_state, next_state : lock_state;
23 BEGIN
24
25     -- ===== 时序进程: 状态寄存器 + 异步复位 =====
26     PROCESS(clk, rst)
27     BEGIN
28         IF rst = '1' THEN
29             current_state <= S0;           -- 异步复位回到 IDLE
30         ELSIF rising_edge(clk) THEN
31             current_state <= next_state;
32         END IF;
33     END PROCESS;
34
35     -- ===== 组合进程: 次态逻辑 =====
36     PROCESS(current_state, din)
37     BEGIN
38         -- 默认值
39         next_state <= S0;
40
41         CASE current_state IS
42             WHEN S0 =>                     -- IDLE: 等待首位输入
43                 IF din = '1' THEN
44                     next_state <= S1;       -- 匹配密码第1位 '1'
45                 ELSE
46                     next_state <= S0;       -- din='0'→继续等待
47                 END IF;
48
49             WHEN S1 =>                     -- 已匹配 "1"
50                 IF din = '1' THEN

```

```

51         next_state <= S2;           -- 匹配密码第2位 '1' → "11"
52     ELSE
53         next_state <= S0;           -- din='0' → 不匹配, 回 IDLE
54     END IF;
55
56     WHEN S2 =>                       -- 已匹配 "11"
57         IF din = '0' THEN
58             next_state <= S3;       -- 匹配密码第3位 '0' → "110"
59         ELSE
60             next_state <= S0;       -- din='1' → 不匹配
61         END IF;
62
63     WHEN S3 =>                       -- 已匹配 "110"
64         IF din = '1' THEN
65             next_state <= S4;       -- 匹配密码第4位 '1' → "1101"
66         ELSE
67             next_state <= S0;       -- din='0' → 不匹配
68         END IF;
69
70     WHEN S4 =>                       -- 已匹配 "1101" —— 开锁
71         next_state <= S0;           -- 非重叠: 开锁后回 IDLE 重新检测
72
73     WHEN OTHERS =>
74         next_state <= S0;
75     END CASE;
76 END PROCESS;
77
78 -- ===== Moore型输出逻辑: unlock仅取决于当前状态 =====
79 unlock <= '1' WHEN current_state = S4 ELSE '0';
80
81 END ARCHITECTURE fsm;

```

评分标准 (9 分):

- 枚举类型定义正确 (TYPE lock_state IS...) (1 分)
- 双进程风格清晰 (时序进程 + 组合进程分离) (1 分)
- 异步复位正确 (rst 在敏感表, IF rst 先于时钟判断) (1 分)
- S0-S4 次态逻辑全部正确 (含不匹配回 S0) (4 分)

(每个状态的匹配/不匹配逻辑各 0.8 分)

- Moore 型输出逻辑正确 (仅 `current_state=S4` 时 `unlock=1`) (1 分)
- 代码注释清晰、说明各状态含义和关键转换条件 (1 分)

(4) 问题回答 (3 分)

(a) Moore 型与 Mealy 型在密码锁应用中的区别 (1.5 分):

- **Moore 型:** `unlock` 输出仅取决于当前状态。只有进入 S4 后 `unlock` 才为 1, 输出与时钟同步变化。在本设计中, 密码第 4 位 `din=1` 的时钟上升沿后, 状态变为 S4, `unlock` 在此时钟周期内为 1。
- **Mealy 型:** `unlock` 输出取决于当前状态和当前输入 `din`。若改用 Mealy 型实现, 当状态为 S3 且 `din=1` 时, `unlock` 可立即为 1 (组合逻辑输出), 比 Moore 型提前一个时钟周期响应。但 Mealy 型的输出可能产生毛刺 (glitch), 在密码锁等安全关键应用中需谨慎。

(b) 支持密码可修改的扩展方案 (1.5 分):

需增加的输入端口:

- `prog_mode` (1 位): 编程模式使能
- `new_din(3 DOWNTO 0)` (4 位): 新密码并行输入 (或串行方案: 增加 `prog_din` (1 位) 逐位输入)
- `prog_clk` 或使用现有 `clk` 配合 `prog_mode` 实现时序控制

状态机主要改动:

1. 将固定密码 “1101” 替换为内部寄存器 `password_reg(3 DOWNTO 0)`, 在编程模式下可被写入。
2. 增加编程状态 (如 `PROG_S0-PROG_S3`), 在 `prog_mode=1` 时进入编程流程, 逐位存储新密码。
3. 比较逻辑改为: 将当前输入 `din` 与 `password_reg` 的对应位逐一比对 (而非硬编码 “1101”)。
4. 增加密码存储的非易失性支持 (如 EEPROM 写入逻辑, 或至少保持到下次编程)。

评分标准：(a) Moore 与 Mealy 区别描述正确 (1.5 分)；(b) 端口增加方案合理 (0.75 分)，状态机改动方向正确 (0.75 分)。

——第 12 套参考答案结束——

第十三章 第 13 套试卷——参考答案与评分标准

13.1 一、选择题答案（18 分）

1. 答案：B

解析：基于 SRAM 工艺的 FPGA 芯片，其 SRAM 单元为易失性存储，断电后数据丢失。因此，配置数据存储在外部的非易失性存储器（如 SPI Flash）中，上电后自动加载到 FPGA 内部 SRAM 配置单元。选项 A 错误（FPGA 内部无 EEPROM，这是 CPLD 的特征）；选项 C 错误（熔丝工艺是早期反熔丝 FPGA，非 SRAM 型）；选项 D 明显错误。

2. 答案：C

解析：VHDL 中，并发语句（Concurrent Statement）指在结构体的 BEGIN...END 之间、PROCESS 外部的语句，其执行顺序与书写顺序无关。简单信号赋值语句 $y \leq a \text{ AND } b$ 位于 PROCESS 外部时属于并发语句。选项 A、B、D 均位于 PROCESS 内部，属于顺序语句。

3. 答案：A

解析：Moore 型状态机的输出仅取决于当前状态（输出在状态节点上标注）；Mealy 型状态机的输出取决于当前状态和输入（输出在转移弧上标注）。这是两者的根本区别。选项 B 错误（Moore 型可能需要更多状态）；选项 C 错误（两者均可检测序列）；选项 D 错误（复位方式与状态机类型无关）。

4. 答案：D

解析：在 VHDL 中，逻辑运算符的优先级如下：NOT 优先级最高，而 AND、OR、NAND、NOR、XOR、XNOR 六种运算符优先级相同且最低。因此 $\text{NOT} > \text{AND} = \text{OR} = \text{XOR}$ 。

5. 答案：C

解析：CPLD 宏单元的输出极性是可配置的——通过输出极性选择电路，可选择高电平有效或低电平有效输出。因此选项 C 的描述错误。选项 A 正确（宏单元包含乘积项阵列和触发器）；选项 B 正确（触发器可被旁路实现组合逻辑）；选项 D 正确（乘积项共享是 CPLD 的特征功能）。

6. 答案：B

解析：VHDL 中声明范围规则：**SIGNAL** 只能在结构体（ARCHITECTURE）的声明区、包（PACKAGE）或实体（ENTITY）中声明，不能出现在进程或子程序内部；**VARIABLE** 只能在进程（PROCESS）或子程序（FUNCTION/PROCEDURE）内部声明；**CONSTANT** 可以在实体、结构体和进程内部多处声明。因此 B 正确。

13.2 二、填空题答案（12 分）

1. **CONSTANT; VARIABLE; SIGNAL**

（每空 1 分，共 3 分）

2. **COMPONENT; PORT MAP**

（每空 1.5 分，共 3 分）

3. **GENERIC** 关键字的作用是：向实体传递静态参数（如位宽、计数模值等），使设计参数化，便于复用和配置。

FOR ... GENERATE 语句属于：并发语句。

（第 1 空 2 分，第 2 空 1 分）

4. 被读取；锁存器（Latch）

（每空 1.5 分，共 3 分）

解析：在 VHDL 组合逻辑进程中，若敏感信号表缺少某些被读取的输入信号，综合工具无法推断纯组合逻辑，会产生不完整的 IF 或 CASE 结构，从而推断出锁存器（Latch）以维持信号的原值。这就是“意外存储单元”的成因。

13.3 三、程序改错题解析（5 分）

评分标准：每正确找出一处错误并给出正确修改方案得 1 分，共 5 分。

错误 1：（第 98 行）PROCESS(a, b) 应改为 PROCESS(a, b, c)

原因：该进程描述的是组合逻辑，其输出 `tmp` 和 `y` 均依赖于输入 `a`、`b`、`c`。敏感信号表缺少信号 `c`，导致当 `c` 变化时进程不会被触发，综合前仿真与综合后仿真结果不一致，且可能推断锁存器。

错误 2：（第 103 行）`tmp := c` 应改为 `tmp <= c`

原因：`tmp` 在结构体声明区声明为 `SIGNAL`，信号必须使用 `<=` 赋值。`:=` 是变量赋值符号，仅用于进程或子程序内部的 `VARIABLE`。此处混淆了信号与变量的赋值符号。

错误 3：（第 104 行）`END IF` 之前缺少 `ELSE` 分支

原因：该进程为组合逻辑进程，但 `IF` 语句缺少 `ELSE` 分支。当 `a` 和 `b` 均为 '0' 时，`tmp` 未被赋值，综合工具会推断出锁存器以保持 `tmp` 的上一值。应添加：`ELSE tmp <= c`；（或其他合理的默认赋值——此处 `ELSIF b='1'` 已覆盖 `b=1` 情形，`a=0` 且 `b=0` 时应给出默认赋值，如 `tmp <= '0'` 或 `tmp <= c`）。

错误 4：（第 105 行）`END PROCESS` 应改为 `END PROCESS;`

原因：`VHDL` 中 `END PROCESS` 后缺少分号。在 `VHDL` 语法中，所有结束语句必须跟分号：`END PROCESS;`（或带标签：`END PROCESS label;`）。

错误 5：（第 106 行）`y = tmp` 应改为 `y <= tmp`

原因：结构体中并发区域的信号赋值必须使用信号赋值符号 `<=` 而非等号 `=`。`=` 仅用于比较运算和变量赋值（变量在 `PROCESS` 内部用 `:=`，而非 `=`）。此处将比较运算符误用作赋值。

正确代码（供参考）：

```

1 ARCHITECTURE behav OF comb_err IS
2     SIGNAL tmp : STD_LOGIC;
3 BEGIN
4     PROCESS(a, b, c)                -- 修正1：补全敏感表
5     BEGIN
6         IF a = '1' THEN
7             tmp <= b;
8         ELSIF b = '1' THEN
9             tmp <= c;                -- 修正2：:= 改为 <=
10        ELSE
11            tmp <= '0';              -- 修正3：添加 ELSE 避免锁存器
12        END IF;

```

```

13  END PROCESS;                -- 修正 4: 添加分号
14  y <= tmp;                  -- 修正 5: = 改为 <=
15  END behav;

```

13.4 四、程序阅读分析题解析（5 分）

评分标准：(1) 1 分；(2) 2 分；(3) 2 分。

(1) 电路名称：4 位环形计数器（Ring Counter）/ 循环右移寄存器

该电路由一个 4 位移位寄存器构成，复位后初始值为"1000"，每个时钟上升沿将最低位移至最高位，其余位右移，形成循环移位效果。（1 分）

(2) 输出序列（复位后从第一个有效时钟边沿开始）：

1000 → 0100 → 0010 → 0001 → （回到 1000）

完整循环序列为：1000，0100，0010，0001（共 4 个状态，周期为 4）。

分析过程：第 19 行 `sreg <= sreg(0) & sreg(3 DOWNTO 1)`；将最低位 `sreg(0)` 拼接到 `sreg(3 downto 1)` 的左侧，实现右移循环。（2 分）

(3) 修改后电路：4 位约翰逊计数器（Johnson Counter / 扭环形计数器）

修改后第 19 行：`sreg <= sreg(2 DOWNTO 0) & NOT sreg(3)`；

将 `sreg(3)` 取反后接入最低位，形成扭环形反馈。

输出循环序列（从复位值"1000"开始）：

1000 → 0000 → 0001 → 0011 → 0111 → 1111 → 1110 → 1100 → （回到 1000）

周期为 8（ $2 \times N = 2 \times 4 = 8$ 个状态），正好是环形计数器周期的 2 倍。（2 分）

13.5 五、综合设计题参考答案（60 分）

13.5.1 设计题 1：BCD 码至 7 段数码管译码器（20 分）

（1）真值表（8 分）

评分标准：有效输入 0~9 的输出段码各 0.5 分（共 5 分），非法输入 10~15 各 0.5 分（共 3 分），总计 8 分。段码错误一处扣 0.5 分。

十进制	bcd(3:0)	a(6)	b(5)	c(4)	d(3)	e(2)	f(1)	g(0)	seg(6:0)
0	0000	0	0	0	0	0	0	1	1000000
1	0001	1	0	0	1	1	1	1	1111001
2	0010	0	0	1	0	0	1	0	0100100
3	0011	0	0	0	0	1	1	0	0110000
4	0100	1	0	0	1	1	0	0	0011001
5	0101	0	1	0	0	1	0	0	0010010
6	0110	0	1	0	0	0	0	0	0000010
7	0111	0	0	0	1	1	1	1	0001111
8	1000	0	0	0	0	0	0	0	0000000
9	1001	0	0	0	0	1	0	0	0000100
10	1010	1	1	1	1	1	1	1	1111111
11	1011	1	1	1	1	1	1	1	1111111
12	1100	1	1	1	1	1	1	1	1111111
13	1101	1	1	1	1	1	1	1	1111111
14	1110	1	1	1	1	1	1	1	1111111
15	1111	1	1	1	1	1	1	1	1111111

注：共阳极数码管，段选 =0 时点亮，=1 时熄灭。非法输入时全灭（全 1）。

数字 0 字形：a,b,c,d,e,f 亮 → 1000000 数字 5 字形：a,c,d,f,g 亮 → 0010010
 数字 1 字形：b,c 亮 → 1111001 数字 6 字形：a,c,d,e,f,g 亮 → 0000010
 各段对照：数字 2 字形：a,b,g,e,d 亮 → 0100100 数字 7 字形：a,b,c 亮 → 0001111
 数字 3 字形：a,b,c,d,g 亮 → 0110000 数字 8 字形：全亮 → 0000000
 数字 4 字形：f,g,b,c 亮 → 0011001 数字 9 字形：a,b,c,d,f,g 亮 → 0000100

(2) VHDL 代码 (10 分)

评分标准：库声明和实体定义 (2 分)；结构体 WITH-SELECT 语句正确 (5 分)；代码规范性和完整性 (2 分)；注释说明 (1 分)。

Listing 13.1: BCD 码至 7 段数码管译码器 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY bcd2seg7 IS
5      PORT(
6          bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
7          seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)
8      );
9  END bcd2seg7;
10
11 ARCHITECTURE dataflow OF bcd2seg7 IS
12 BEGIN
13     WITH bcd SELECT
14         seg <= "1000000" WHEN "0000",  -- 0
15              "1111001" WHEN "0001",  -- 1
16              "0100100" WHEN "0010",  -- 2
17              "0110000" WHEN "0011",  -- 3
18              "0011001" WHEN "0100",  -- 4
19              "0010010" WHEN "0101",  -- 5
20              "0000010" WHEN "0110",  -- 6
21              "0001111" WHEN "0111",  -- 7
22              "0000000" WHEN "1000",  -- 8
23              "0000100" WHEN "1001",  -- 9
24              "1111111" WHEN OTHERS;  -- 非法输入：全灭
25 END dataflow;

```

(3) 代码注释说明 (2 分)

评分标准：正确回答选用理由和使用场景区别得 2 分。

选用 WITH-SELECT 而非 IF-ELSE 的理由：

- 1. **WITH-SELECT-WHEN** 是并发信号赋值语句，直接描述输入到输出的映射关系，含义清晰，对应纯组合逻辑。对于译码器这类输入值有限且互斥的情况，WITH-SELECT 是最佳选择。
- 2. **IF-ELSE**（在 PROCESS 内）是顺序语句，描述优先级的判断流程。当条件有优先级关系时适用，而 BCD 译码器的每种输入组合是互斥的，无优先级关系，使用 WITH-SELECT 更简洁、意图更明确。
- 3. **使用场景区别：**WITH-SELECT 适用于所有条件互斥的组合逻辑（如译码器、多路选择器）；IF-ELSE 适用于有优先级判断的逻辑（如优先编码器、带优先级的控制逻辑）。

13.5.2 设计题 2：N 位双向移位寄存器（20 分）

(1) 顶层模块端口框图（4 分）

评分标准：正确绘制框图并标注所有端口名称、方向和位宽得 4 分；遗漏端口或位宽标注错误各扣 0.5 分。



- 输入端口：clk（时钟）、rst（同步复位，高有效）、dir（0= 左移，1= 右移）、din（1 位串行输入）、load（并行加载使能，高有效）、pin(7 downto 0)（8 位并行输入）
- 输出端口：q(7 downto 0)（当前寄存器值）、dout（1 位串行输出）

(2) D 触发器底层元件实体定义（2 分）

评分标准：实体定义正确，包含所有必要端口得 2 分。

```
1 ENTITY dff_ce IS
2   PORT (
```

```

3      clk : IN   STD_LOGIC;
4      rst : IN   STD_LOGIC;
5      ce  : IN   STD_LOGIC;    -- 时钟使能
6      d   : IN   STD_LOGIC;
7      q   : OUT  STD_LOGIC
8  );
9  END dff_ce;

```

(3) 完整顶层 VHDL 代码 (10 分)

评分标准：COMPONENT 声明 (1 分)；内部信号声明 (1 分)；FOR...GENERATE 正确使用 (3 分)；第 0 位和第 7 位特殊处理逻辑 (3 分)；整体代码完整性和正确性 (2 分)。

Listing 13.2: N 位双向移位寄存器——结构化 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY bidir_shift_reg IS
5      GENERIC(N : INTEGER := 8);    -- 参数化位宽
6      PORT(
7          clk   : IN   STD_LOGIC;
8          rst   : IN   STD_LOGIC;
9          dir   : IN   STD_LOGIC;    -- '0'左移, '1'右移
10         din   : IN   STD_LOGIC;    -- 串行输入
11         load  : IN   STD_LOGIC;    -- 并行加载使能
12         pin   : IN   STD_LOGIC_VECTOR(N-1 DOWNT0 0);
13         q     : OUT  STD_LOGIC_VECTOR(N-1 DOWNT0 0);
14         dout  : OUT  STD_LOGIC
15     );
16  END bidir_shift_reg;
17
18  ARCHITECTURE struct OF bidir_shift_reg IS
19      -- 声明底层D触发器元件
20      COMPONENT dff_ce IS
21          PORT(
22              clk : IN   STD_LOGIC;

```

```

23     rst : IN  STD_LOGIC;
24     ce  : IN  STD_LOGIC;
25     d   : IN  STD_LOGIC;
26     q   : OUT STD_LOGIC
27 );
28 END COMPONENT;
29
30 -- 内部级联信号
31 SIGNAL q_int    : STD_LOGIC_VECTOR(N-1 DOWNT0 0);
32 SIGNAL d_input  : STD_LOGIC_VECTOR(N-1 DOWNT0 0);
33
34 BEGIN
35     -- 为每个D触发器生成数据输入选择逻辑
36     gen_input : FOR i IN 0 TO N-1 GENERATE
37         -- 每个触发器的输入由load/dir信号决定
38         d_input(i) <= pin(i) WHEN load = '1' ELSE
39             din      WHEN (i = 0 AND dir = '1') OR
40                 (i = N-1 AND dir = '0') ELSE
41             q_int(i+1) WHEN dir = '0' ELSE -- 左移: 低位来自右
42                 边
43             q_int(i-1); -- 右移: 高位来自
44                 左边
45     END GENERATE;
46
47     -- 例化N个D触发器
48     gen_dff : FOR i IN 0 TO N-1 GENERATE
49         u_dff : dff_ce PORT MAP(
50             clk => clk,
51             rst => rst,
52             ce  => '1', -- 始终使能
53             d   => d_input(i),
54             q   => q_int(i)
55         );
56     END GENERATE;
57
58     -- 输出连接
59     q <= q_int;
60     dout <= q_int(N-1) WHEN dir = '0' ELSE -- 左移串出取最高位
61         q_int(0); -- 右移串出取最低位

```

```

60
61 END struct;

```

(4) 参数化修改说明 (4 分)

评分标准：正确指出使用 GENERIC 的位置和修改方法得 2 分；说明 16 位修改要点得 2 分。

代码中参数化的关键位置（已在代码中标注）：

1. **GENERIC(N : INTEGER := 8)：**在 ENTITY 中定义位宽参数，默认值 8。
2. 端口声明中使用 N：pin(N-1 DOWNT0 0)、q(N-1 DOWNT0 0)。
3. **FOR...GENERATE 循环范围：**FOR i IN 0 TO N-1 GENERATE——位宽变化时循环次数自动调整。
4. 内部信号数组：使用 STD_LOGIC_VECTOR(N-1 DOWNT0 0) 声明——位宽自动匹配。

若将位宽改为 16 位，仅需修改一处：在顶层例化时使用 GENERIC MAP 赋值：

```

1 U_REG : bidir_shift_reg
2   GENERIC MAP(N => 16)
3   PORT MAP(clk => clk, rst => rst, ...);

```

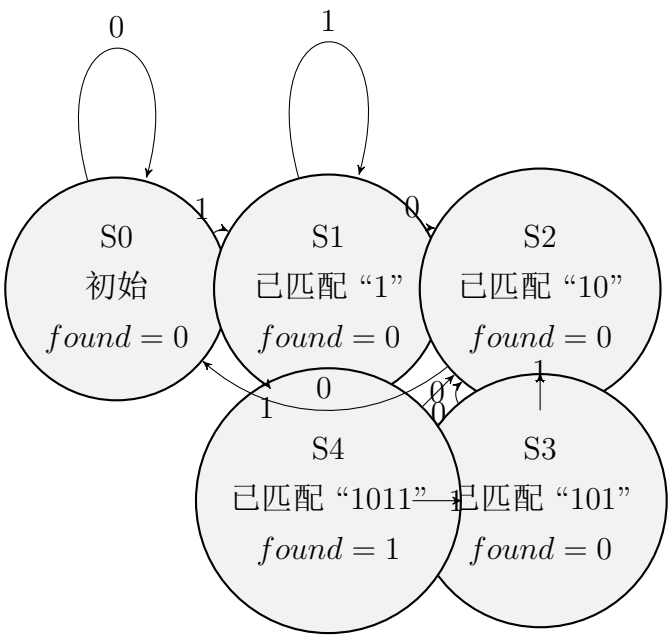
实体内部的 GENERIC N 参数会自动将 FOR...GENERATE 循环、信号位宽等全部改为 16 位，无需修改实体和结构体内的任何其他代码。这正是 GENERIC 参数化的核心优势。

13.5.3 设计题 3：“1011”序列检测器 (20 分)

(1) Moore 型状态机 (8 分)

评分标准：状态转移图正确 (4 分，状态节点和转移弧各 2 分)；状态转移表正确 (4 分，当前状态/输入/次态/输出各 1 分)。

Moore 型状态转移图：



Moore 型状态转移表：

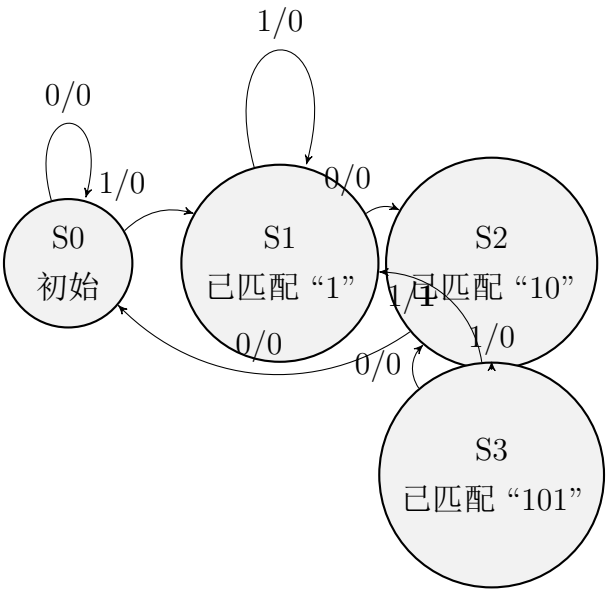
当前状态	输入 din	次态	输出 found
S0（初始）	0	S0	0
S0	1	S1	0
S1（已匹配“1”）	0	S2	0
S1	1	S1	0
S2（已匹配“10”）	0	S0	0
S2	1	S3	0
S3（已匹配“101”）	0	S2	0
S3	1	S4	0
S4（已匹配“1011”）	0	S2	1
S4	1	S1	1

说明：Moore 型共 5 个状态。S4 为匹配成功状态（ $found = 1$ ）。S4 状态下若 $din = 0$ 则转入 S2（利用尾部的“10”作为下一序列前缀，实现重叠检测）；若 $din = 1$ 则转入 S1（利用尾部的“1”作为下一序列前缀）。

(2) Mealy 型状态机（6 分）

评分标准：状态转移图正确（3 分）；状态转移表正确（3 分）。

Mealy 型状态转移图：



Mealy 型状态转移表：

当前状态	输入 din	次态	输出 found
S0（初始）	0	S0	0
S0	1	S1	0
S1（已匹配“1”）	0	S2	0
S1	1	S1	0
S2（已匹配“10”）	0	S0	0
S2	1	S3	0
S3（已匹配“101”）	0	S2	0
S3	1	S1	1

说明：Mealy 型仅需 4 个状态。在 S3 状态下若 $din = 1$ ，则匹配“1011”成功， $found = 1$ ，同时转入 S1（利用重叠：尾部的“1”作为下一序列的前缀）。

(3) VHDL 代码实现（选择 Moore 型）（6 分）

评分标准：枚举类型定义（1 分）；双进程/三进程结构（2 分）；状态转移逻辑正确（2 分）；输出逻辑正确（1 分）。

Listing 13.3: “1011” 序列检测器——Moore 型状态机 VHDL 代码

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
```

```

4 ENTITY seq_detector_1011 IS
5     PORT(
6         clk    : IN  STD_LOGIC;
7         rst    : IN  STD_LOGIC;  -- 异步复位，低电平有效
8         din    : IN  STD_LOGIC;
9         found  : OUT STD_LOGIC
10    );
11 END seq_detector_1011;
12
13 ARCHITECTURE moore OF seq_detector_1011 IS
14     -- 用户自定义枚举类型定义状态
15     TYPE state_type IS (S0, S1, S2, S3, S4);
16     SIGNAL current_state, next_state : state_type;
17
18 BEGIN
19     -- ===== 进程1: 状态寄存器（时序逻辑） =====
20     PROCESS(clk, rst)
21     BEGIN
22         IF rst = '0' THEN                -- 异步复位，低电平有效
23             current_state <= S0;
24         ELSIF rising_edge(clk) THEN
25             current_state <= next_state;
26         END IF;
27     END PROCESS;
28
29     -- ===== 进程2: 次态逻辑（组合逻辑） =====
30     PROCESS(current_state, din)
31     BEGIN
32         CASE current_state IS
33             WHEN S0 =>
34                 IF din = '1' THEN
35                     next_state <= S1;
36                 ELSE
37                     next_state <= S0;
38                 END IF;
39
40             WHEN S1 =>
41                 IF din = '1' THEN
42                     next_state <= S1;

```

```
43         ELSE
44             next_state <= S2;
45         END IF;
46
47     WHEN S2 =>
48         IF din = '1' THEN
49             next_state <= S3;
50         ELSE
51             next_state <= S0;
52         END IF;
53
54     WHEN S3 =>
55         IF din = '1' THEN
56             next_state <= S4;
57         ELSE
58             next_state <= S2;
59         END IF;
60
61     WHEN S4 =>
62         IF din = '1' THEN
63             next_state <= S1;
64         ELSE
65             next_state <= S2;
66         END IF;
67
68     WHEN OTHERS =>
69         next_state <= S0;
70     END CASE;
71 END PROCESS;
72
73 -- ===== 进程 3: 输出逻辑 (组合逻辑) =====
74 -- Moore 型: 输出仅取决于当前状态
75 PROCESS(current_state)
76 BEGIN
77     IF current_state = S4 THEN
78         found <= '1';
79     ELSE
80         found <= '0';
81     END IF;
```



```
82     END PROCESS;  
83  
84 END moore;
```

评分细则总结——设计题 3（20 分）：

- Moore 型状态转移图：4 分（5 个状态各 0.4 分，10 条转移弧各 0.2 分）
- Moore 型状态转移表：4 分（10 行各 0.4 分）
- Mealy 型状态转移图：3 分（4 个状态各 0.3 分，8 条转移弧各 0.225 分）
- Mealy 型状态转移表：3 分（8 行各 0.375 分）
- VHDL 代码：6 分（实体定义 0.5 分；枚举类型 0.5 分；状态寄存器进程 + 异步复位 1.5 分；次态组合逻辑进程 2.5 分；输出逻辑进程 1 分）

第十四章 第 14 套试卷——参考答案与评分标准

14.1 一、选择题答案（18 分）

1. 答案：C

解析：基于 SRAM 工艺的 FPGA 芯片，SRAM 为易失性存储，断电后配置数据丢失。上电时需从外部非易失性存储器（如 PROM 或 SPI Flash）中加载配置数据。选项 A、B 错误——片内掩膜 ROM 和 EEPROM 不是 SRAM FPGA 的特征；选项 D 明显错误。

2. 答案：B

解析：CPLD 的基本逻辑单元是宏单元（Macrocell），典型组成包括：乘积项阵列、可配置触发器、输出极性选择与反馈通路。查找表（LUT）是 FPGA 的基本逻辑单元特征，而非 CPLD 宏单元的组成部分。因此选项 B 正确。

3. 答案：D

解析：选择信号赋值语句 WITH-SELECT-WHEN 是并发信号赋值语句的一种，只能出现在结构体的并发区域（PROCESS 外部）。选项 A 和 B（IF-ELSIF-ELSE 和 CASE-WHEN）既可出现在 PROCESS 内部（顺序语句），也可作为条件信号赋值出现在外部（并发形式）；选项 C（变量赋值:=）只能出现在 PROCESS 或子程序内部。

4. 答案：A

解析：Moore 型状态机的输出仅与当前状态有关；Mealy 型状态机的输出与当前状态和输入均有关。这是两者的根本定义性区别。选项 B 错误（Moore 型可能需要更多状态）；选项 C 错误（Mealy 型输出与状态也有关）；选项 D 错误（两者均可用于序列检测）。

5. 答案：C

解析：VHDL 运算符优先级从高到低依次为：

- (a) 最高：NOT、ABS、**
- (b) 乘法类：*、/、MOD、REM
- (c) 加减与拼接：+、-、&
- (d) 关系运算：=、/=、<、<=、>、>=
- (e) 最低：AND、OR、NAND、NOR、XOR、XNOR

因此 NOT 优先级最高。

6. 答案：B

解析：在 VHDL 中，CONSTANT（常量）可以在 ENTITY 声明区、ARCHITECTURE 声明区（BEGIN 之前）、PROCESS 内部、PACKAGE（包）等多处声明。选项 B 描述了其中一个有效声明位置（结构体声明区）。选项 A 错误（CONSTANT 不在 PORT 中声明）；选项 C 错误（不仅限于 PACKAGE）；选项 D 错误（不仅限于 PROCESS 内部）。

14.2 二、填空题答案（12 分）

1. **ARCHITECTURE** 声明区（或结构体声明区）中定义；**PROCESS** 或子程序内部声明。

（每空 1.5 分，共 3 分）

2. **GENERIC** 关键字用于传递静态参数；**COMPONENT** 关键字用于声明可复用的子电路模板。

（每空 1.5 分，共 3 分）

3. 被读取；锁存器（Latch）。

（每空 1.5 分，共 3 分）

解析：在组合逻辑进程中，若敏感信号表中缺少某个被读取的输入信号，则当该信号变化时进程不被触发，输出不更新——综合工具会推断出锁存器以维持原输出值，导致综合前后仿真不一致。

4. **FOR-GENERATE**（按迭代次数重复生成）；**IF-GENERATE**（按布尔条件选择性生成）。

（每空 1.5 分，共 3 分）

14.3 三、程序改错题解析（5 分）

评分标准：每正确找出一处错误并给出正确修改方案得 1 分，共 5 分。代码中有 6+ 处错误，找出任意 5 处即可。

错误 1：（第 84 行））应改为）；

原因：VHDL 中 PORT 声明必须以分号结束：PORT(...);。原代码第 84 行的右括号后缺少分号，导致语法错误。

错误 2：（第 90 行）PROCESS(clk) 应改为 PROCESS(clk, rst)

原因：代码意图实现异步复位（第 92 行用 IF rst 在时钟边沿之前判断），异步复位信号 rst 必须出现在进程的敏感信号表中。缺少 rst 会使复位变为同步行为（仅在时钟边沿被采样），与设计意图不符，且综合工具可能推断错误逻辑。

错误 3：（第 93 行）cnt := "0000" 应改为 cnt <= "0000"

原因：cnt 在第 88 行声明为 SIGNAL，信号只能使用 <= 赋值。:= 是变量赋值符号，仅用于 VARIABLE。此处混淆了信号和变量的赋值方式。

错误 4：（第 96 行）cnt <= cnt + 1 缺少必要的库

原因：STD_LOGIC_VECTOR 类型在 IEEE.STD_LOGIC_1164 包中没有定义算术运算符（+）。需在代码开头添加 USE IEEE.NUMERIC_STD.ALL;，然后将 cnt 转为 UNSIGNED 类型，或添加 USE IEEE.STD_LOGIC_UNSIGNED.ALL;（非标准但常用）。
正确写法：添加库声明 USE IEEE.NUMERIC_STD.ALL;，并将第 96 行改为：cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1);

错误 5：（第 97 行）END IF 应改为 END IF;

原因：VHDL 中所有结束语句必须跟分号。内层 IF 语句的结束缺少分号，导致语法错误。

修正后的完整代码（供参考）：

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;           -- 修正4：添加算术运算库
4
5  ENTITY counter_bad IS
6  PORT(

```

```

7      clk : IN  STD_LOGIC;
8      rst : IN  STD_LOGIC;
9      en  : IN  STD_LOGIC;
10     q   : OUT STD_LOGIC_VECTOR(3 DOWNT0 0)
11 );                                     -- 修正1: 添加分号
12 END ENTITY counter_bad;
13
14 ARCHITECTURE behav OF counter_bad IS
15     SIGNAL cnt : STD_LOGIC_VECTOR(3 DOWNT0 0);
16 BEGIN
17     PROCESS(clk, rst)                 -- 修正2: 添加rst到敏感表
18     BEGIN
19         IF rst = '1' THEN
20             cnt <= "0000";             -- 修正3: := 改为 <=
21         ELSIF rising_edge(clk) THEN
22             IF en = '1' THEN
23                 cnt <= STD_LOGIC_VECTOR(UNSIGNED(cnt) + 1); -- 修正4
24             END IF;                   -- 修正5: 添加分号
25         END IF;
26     END PROCESS;
27     q <= cnt;
28 END behav;

```

14.4 四、程序阅读分析题解析（5 分）

评分标准：(1) 2 分；(2) 1 分；(3) 2 分。

(1) 电路功能与分频比：

该电路实现的是偶数分频器（偶分频电路），分频比为 8（即 $f_{out} = f_{in}/8$ ）。

分析：信号 count 从 0 计数到 3（共 4 个时钟周期），当 count=3 时 temp 取反。temp 每翻转一次需要 4 个时钟周期，因此一个完整的输出周期为 $4 \times 2 = 8$ 个时钟周期，分频比为 8。（2 分）

(2) count 信号的作用与取值：

`count` 是一个模 4 计数器,作用是对输入时钟 `clk_in` 的上升沿进行计数,为 `temp` 的翻转提供定时基准。

其取值范围为: **0, 1, 2, 3** (共 4 种取值)。虽然类型声明为 `INTEGER RANGE 0 TO 7`, 但逻辑上仅使用了 0~3。(1 分)

(3) 输出频率与占空比:

输入频率 $f_{in} = 80 \text{ MHz}$, 分频比 $N = 8$:

$$f_{out} = \frac{80 \text{ MHz}}{8} = 10 \text{ MHz}$$

占空比为 **50%**: `temp` 信号在 `count=3` 时翻转, 高电平和低电平均持续 4 个输入时钟周期, 因此占空比为 50%。(2 分)

14.5 五、综合设计题参考答案（60 分）

14.5.1 设计题 1：BCD 码转 7 段数码管译码器（20 分）

（1）真值表（6 分）

评分标准：每个有效数字的段码值正确得 0.4 分（9 个共 3.6 分）；非法输入全 1 得 0.4 分；输入输出标注清晰得 2 分；共 6 分。

十进制	bcd(3:0)	a	b	c	d	e	f	g	seg(6:0)
0	0000	0	0	0	0	0	0	1	1000000
1	0001	1	0	0	1	1	1	1	1111001
2	0010	0	0	1	0	0	1	0	0100100
3	0011	0	0	0	0	1	1	0	0110000
4	0100	1	0	0	1	1	0	0	0011001
5	0101	0	1	0	0	1	0	0	0010010
6	0110	0	1	0	0	0	0	0	0000010
7	0111	0	0	0	1	1	1	1	0001111
8	1000	0	0	0	0	0	0	0	0000000
9	1001	0	0	0	0	1	0	0	0000100
10~15	1010~1111	1	1	1	1	1	1	1	1111111

（2）顶层模块端口框图（4 分）



- 输入：bcd(3 DOWNTO 0) ——4 位 BCD 码输入（0000~1001）
- 输出：seg(6 DOWNTO 0) ——7 段数码管段选信号（seg(6)=a, ..., seg(0)=g），低电平点亮

（3）完整 VHDL 代码（10 分）

评分标准：库声明和实体定义（1.5 分）；结构体 WITH-SELECT 语句正确（5 分）；代码注释完整性（1 分）；非法输入处理（1.5 分）；代码规范性（1 分）。

Listing 14.1: BCD 码转 7 段数码管译码器 VHDL 代码——数据流描述

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY bcd2seg7 IS
5      PORT(
6          bcd : IN  STD_LOGIC_VECTOR(3 DOWNT0 0);
7          seg : OUT STD_LOGIC_VECTOR(6 DOWNT0 0)
8      );
9  END bcd2seg7;
10
11 ARCHITECTURE dataflow OF bcd2seg7 IS
12 BEGIN
13     -- 共阳极数码管：0=亮，1=灭
14     -- seg(6)=a, seg(5)=b, seg(4)=c, seg(3)=d,
15     -- seg(2)=e, seg(1)=f, seg(0)=g
16     WITH bcd SELECT
17         seg <= "1000000" WHEN "0000", -- 数字 0
18             "1111001" WHEN "0001", -- 数字 1
19             "0100100" WHEN "0010", -- 数字 2
20             "0110000" WHEN "0011", -- 数字 3
21             "0011001" WHEN "0100", -- 数字 4
22             "0010010" WHEN "0101", -- 数字 5
23             "0000010" WHEN "0110", -- 数字 6
24             "0001111" WHEN "0111", -- 数字 7
25             "0000000" WHEN "1000", -- 数字 8
26             "0000100" WHEN "1001", -- 数字 9
27             "1111111" WHEN OTHERS; -- 非法输入：全灭
28 END dataflow;

```

14.5.2 设计题 2：4 位双向移位寄存器（20 分）

（1）端口定义（4 分）

输入端口：

- clk: 时钟信号（1 位）
- rst: 异步复位，高电平有效（1 位）

- data_in(3 DOWNT0 0): 并行加载数据 (4 位)
- load: 加载使能, 高电平有效 (1 位)
- dir: 移位方向, 0= 右移、1= 左移 (1 位)
- sin: 串行输入位 (1 位)

输出端口:

- q(3 DOWNT0 0): 当前寄存器值 (4 位)

(2) 功能描述 (4 分)

模式	rst	load	dir	操作
复位	1	X	X	q <= "0000"
并行加载	0	1	X	q <= data_in
右移	0	0	0	q <= q(2 downto 0) & sin
左移	0	0	1	q <= sin & q(3 downto 1)
保持	0	0	(无时钟)	q 不变

注: 右移时数据从高位向低位移动, q(2:0) 移至 q(3:1), sin 移入 q(0)。左移时数据从低位向高位移动, q(3:1) 移至 q(2:0), sin 移入 q(3)。

(3) VHDL 代码 (12 分)

评分标准: 实体定义 (1 分); 结构体声明 (1 分); PROCESS 与敏感表 (2 分); rst/load/移位优先级正确 (4 分); 移位逻辑正确 (3 分); 代码规范性 (1 分)。

Listing 14.2: 4 位双向移位寄存器 VHDL 代码——行为描述

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY shift_reg_4bit IS
5     PORT(
6         clk      : IN  STD_LOGIC;
7         rst      : IN  STD_LOGIC;          -- 异步复位, 高电平有效
8         data_in  : IN  STD_LOGIC_VECTOR(3 DOWNT0 0);
9         load     : IN  STD_LOGIC;          -- 并行加载使能
10        dir      : IN  STD_LOGIC;          -- 0=右移, 1=左移
```

```

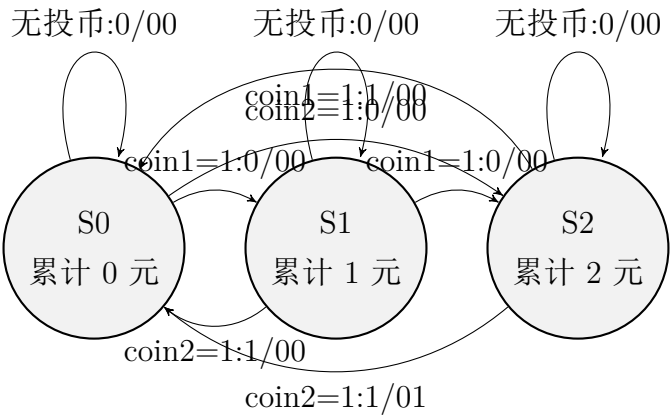
11     sin      : IN  STD_LOGIC;          -- 串行输入位
12     q        : OUT STD_LOGIC_VECTOR(3 DOWNT0 0)
13 );
14 END shift_reg_4bit;
15
16 ARCHITECTURE behav OF shift_reg_4bit IS
17     SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNT0 0);
18 BEGIN
19     PROCESS(clk, rst)
20     BEGIN
21         IF rst = '1' THEN                -- 异步复位（最高优先级）
22             q_int <= (OTHERS => '0');
23         ELSIF rising_edge(clk) THEN
24             IF load = '1' THEN            -- 并行加载（第2优先级）
25                 q_int <= data_in;
26             ELSE                          -- 移位操作（第3优先级）
27                 IF dir = '0' THEN        -- 右移
28                     -- q(2:0) -> q(3:1), sin -> q(0)
29                     q_int <= q_int(2 DOWNT0 0) & sin;
30                 ELSE                    -- 左移
31                     -- sin -> q(3), q(3:1) -> q(2:0)
32                     q_int <= sin & q_int(3 DOWNT0 1);
33                 END IF;
34             END IF;
35         END IF;
36     END PROCESS;
37
38     q <= q_int;
39 END behav;

```

14.5.3 设计题 3：自动售货机控制器——Mealy 型（20 分）

（1）状态转移图（8 分）

评分标准：状态定义正确（2 分）；转移弧完整（3 分）；输出标注正确（3 分）。



弧上标注格式：输入/输出 (give change)

- S0→S1：投 1 元，输出 00（不出货不找零）
- S0→S2：投 2 元，输出 00
- S1→S0：投 2 元，累计 =3 元，输出 10（出货不找零）
- S2→S0(投 1 元)：累计 =3 元，输出 10（出货不找零）
- S2→S0(投 2 元)：累计 =4 元，输出 11（出货且找零）

(2) 状态转移表（4 分）

评分标准：表格完整（2 分）；输入/次态/输出逻辑正确（2 分）。

当前状态	coin1 coin2	次态	give change	说明
S0（0 元）	0 0	S0	0 0	无投币，等待
S0	1 0	S1	0 0	投 1 元
S0	0 1	S2	0 0	投 2 元
S0	1 1	—	—	不会同时出现
S1（1 元）	0 0	S1	0 0	无投币，等待
S1	1 0	S2	0 0	再投 1 元 → 累计 2 元
S1	0 1	S0	1 0	再投 2 元 → 累计 3 元 → 出货
S2（2 元）	0 0	S2	0 0	无投币，等待
S2	1 0	S0	1 0	再投 1 元 → 累计 3 元 → 出货
S2	0 1	S0	1 1	再投 2 元 → 累计 4 元 → 出货 + 找零

(3) VHDL 代码实现 (8 分)

评分标准：枚举类型定义 (1 分)；实体定义 (1 分)；时序进程 (异步复位 + 状态寄存器) (2 分)；组合进程 (次态 + 输出逻辑) (3 分)；代码完整性和规范性 (1 分)。

Listing 14.3: 自动售货机控制器——Mealy 型状态机 VHDL 代码

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY vending_machine IS
5      PORT(
6          clk      : IN  STD_LOGIC;
7          rst      : IN  STD_LOGIC;    -- 异步复位, 高有效
8          coin1    : IN  STD_LOGIC;    -- 投入 1 元
9          coin2    : IN  STD_LOGIC;    -- 投入 2 元
10         give     : OUT STD_LOGIC;    -- 出货信号
11         change   : OUT STD_LOGIC     -- 找零 1 元信号
12     );
13 END vending_machine;
14
15 ARCHITECTURE mealy OF vending_machine IS
16     TYPE state_type IS (S0, S1, S2); -- 累计 0 元 / 1 元 / 2 元
17     SIGNAL current_state, next_state : state_type;
18
19 BEGIN
20     -- ===== 进程 1: 状态寄存器 (时序逻辑 + 异步复位) =====
21     PROCESS(clk, rst)
22     BEGIN
23         IF rst = '1' THEN
24             current_state <= S0;
25         ELSIF rising_edge(clk) THEN
26             current_state <= next_state;
27         END IF;
28     END PROCESS;
29
30     -- ===== 进程 2: 次态逻辑 + 输出逻辑 (组合逻辑) =====
31     PROCESS(current_state, coin1, coin2)

```

```

32 BEGIN
33     -- 默认值赋值（避免锁存器）
34     next_state <= current_state;
35     give <= '0';
36     change <= '0';
37
38 CASE current_state IS
39     WHEN S0 =>                                -- 累计 0 元
40         IF coin1 = '1' THEN
41             next_state <= S1;                  -- 投 1 元 → 累计 1 元
42         ELSIF coin2 = '1' THEN
43             next_state <= S2;                  -- 投 2 元 → 累计 2 元
44         END IF;
45
46     WHEN S1 =>                                -- 累计 1 元
47         IF coin1 = '1' THEN
48             next_state <= S2;                  -- 再投 1 元 → 累计 2 元
49         ELSIF coin2 = '1' THEN
50             next_state <= S0;                  -- 再投 2 元 → 累计 3 元
51             give <= '1';                      -- 出货！
52         END IF;
53
54     WHEN S2 =>                                -- 累计 2 元
55         IF coin1 = '1' THEN
56             next_state <= S0;                  -- 再投 1 元 → 累计 3 元
57             give <= '1';                      -- 出货！
58         ELSIF coin2 = '1' THEN
59             next_state <= S0;                  -- 再投 2 元 → 累计 4 元
60             give <= '1';                      -- 出货！
61             change <= '1';                    -- 找零 1 元！
62         END IF;
63
64     WHEN OTHERS =>
65         next_state <= S0;
66 END CASE;
67 END PROCESS;
68
69 END mealy;

```

评分细则总结——设计题 3（20 分）：

- 状态转移图：8 分（状态定义与含义 3 分；转移弧完整 3 分；输出标注正确 2 分）
- 状态转移表：4 分（表格完整性 2 分；次态和输出逻辑各 1 分）
- VHDL 代码：8 分（实体 2 分；枚举类型 1 分；时序进程 2 分；组合进程 3 分）

关键扣分点：

- 状态转移图中遗漏转移条件（如无投币时的自循环），每条扣 0.5 分
- Mealy 型输出错误地放在状态节点而非转移弧上，扣 2 分
- 组合进程中未给所有输出信号赋默认值（导致锁存器推断），扣 1.5 分
- 缺少 WHEN OTHERS 分支，扣 0.5 分

第十五章 第 15 套试卷——参考答案与评分标准

15.1 一、选择题答案（18 分）

1. 答案：B

解析：CPLD 的基本逻辑单元基于乘积项（Product Term）结构（由与阵列 + 或阵列实现组合逻辑），而 FPGA 的基本逻辑单元基于查找表（LUT）结构（由 SRAM 单元实现任意组合逻辑函数）。这是两者在架构上的核心区别。选项 A 颠倒了两者关系；选项 C、D 错误。

2. 答案：C

解析：常见的 FPGA 配置模式包括：Master Serial（主串模式）、Slave Serial（从串模式）、JTAG 模式、SPI 模式、SelectMAP（并行）模式以及 BPI 模式等。**DMA（直接存储器访问）模式不是 FPGA 配置模式**，DMA 是计算机系统中外设与内存之间的数据传输机制。因此选项 C 正确。

3. 答案：C

解析：顺序语句只能出现在 PROCESS 或子程序（FUNCTION/PROCEDURE）内部。IF 语句是典型的顺序语句。选项 A（简单信号赋值位于结构体并发区）属于并发语句；选项 B（PROCESS 本身）是并发语句（在结构体中）；选项 D（元件例化 PORT MAP）是并发语句。

4. 答案：A

解析：Moore 型状态机的输出仅取决于当前状态（输出标注在状态节点上）；Mealy 型状态机的输出取决于当前状态和输入（输出标注在转移弧上）。这是两者的根本区别。选项 B 颠倒了关系；选项 C 错误（二者均可用于各种场景）；选项 D 错误（描述风格与状态机类型无关）。

5. 答案：B

解析：VHDL 中逻辑运算符优先级：NOT 优先级最高；AND、OR、NAND、NOR、XOR、XNOR 优先级相同且最低。但在教学实践中通常按 NOT > AND > OR > XOR 的顺序理解。选项 A 将 AND 排在 OR 之前但 NOT 在最后（错误）；选项 C 将 XOR 放在 AND 之前（不符合常见认知）；选项 D 声称所有逻辑运算符优先级相同且按左到右计算（但 NOT 确实最高）。

6. 答案：D

解析：选项 D 错误——组合逻辑进程的敏感信号表绝不能省略。敏感信号表不完整会导致综合前后仿真结果不一致（综合工具推断出锁存器，而仿真时可能因所有信号变化都触发进程而不表现出问题）。综合工具可能会对不完整敏感表给出警告，但不会自动补全所有缺失的信号（而是推断锁存器）。

15.2 二、填空题答案（12 分）

1. 信号使用 `<=`（小于等于号）赋值；变量使用 `:=`（冒号等号）赋值；常量的声明关键字是 **CONSTANT**。
(每空 1 分，共 3 分)
2. 关键字 **GENERIC** 用于定义类属参数；必须声明在关键字 **PORT** 之前。
(每空 1.5 分，共 3 分)
3. 用关键字 **COMPONENT** 声明子模块接口；用关键字 **PORT MAP** 连接端口。
(每空 1.5 分，共 3 分)
4. **FOR GENERATE**（重复生成）；**IF GENERATE**（条件生成）。
(每空 1.5 分，共 3 分)

15.3 三、程序改错题解析（5 分）

评分标准：每正确找出一处错误并给出正确修改方案得 1 分，共 5 分。代码中共有 6 处错误，找出任意 5 处即可。

错误 1：（第 94 行）`PROCESS(clk)` 应改为 `PROCESS(clk, rst)`

原因：该寄存器意图实现异步复位（第 97 行 IF `rst` 在 clock 边沿检测之前），异步复位信号 `rst` 必须出现在进程敏感信号表中。缺少 `rst` 导致复位变为同步行为（仅在 `clk` 边沿被采样），与设计意图不符。

错误 2: (第 98 行) `q_int := (OTHERS => '0')` 应改为 `q_int <= (OTHERS => '0')`

原因: `q_int` 在第 92 行声明为 `SIGNAL`, 信号必须使用 `<=` 赋值。`:=` 是变量赋值符号, 仅用于 `VARIABLE`。

错误 3: (第 99 行) `tmp <= (OTHERS => '0')` 应改为 `tmp := (OTHERS => '0')`

原因: `tmp` 在第 95 行声明为 `VARIABLE`, 变量必须使用 `:=` 赋值。`<=` 是信号赋值符号。此处将信号和变量的赋值符号互相混淆。

错误 4: (第 104 行) `END IF` 应改为 `END IF;`

原因: VHDL 语法中, 所有结束语句必须跟分号: `END IF;`。内层 `IF` 语句 (第 101 行 `IF en = '1' THEN`) 的结束缺少分号。

错误 5: (第 106 行) `END PROCESS` 应改为 `END PROCESS;`

原因: `PROCESS` 语句的结束必须跟分号: `END PROCESS;` (或带标签: `END PROCESS label;`)。

修正后的完整代码 (供参考):

```

1 ARCHITECTURE behavioral OF reg4_err IS
2     SIGNAL q_int : STD_LOGIC_VECTOR(3 DOWNT0 0);
3 BEGIN
4     PROCESS(clk, rst)                                -- 修正 1: rst 加入敏感表
5         VARIABLE tmp : STD_LOGIC_VECTOR(3 DOWNT0 0);
6     BEGIN
7         IF rst = '1' THEN
8             q_int <= (OTHERS => '0');                -- 修正 2: := 改为 <=
9             tmp := (OTHERS => '0');                  -- 修正 3: <= 改为 :=
10        ELSIF rising_edge(clk) THEN
11            IF en = '1' THEN
12                q_int <= d;
13                tmp := d;
14            END IF;                                    -- 修正 4: 添加分号
15        END IF;
16    END PROCESS;                                       -- 修正 5: 添加分号
17    q <= q_int;
18 END behavioral;

```

15.4 四、程序阅读分析题解析（5 分）

评分标准：(1) 1 分；(2) 2 分；(3) 1 分；(4) 1 分。

(1) 电路名称与功能：

该 VHDL 代码描述的是一个参数化偶数分频器。基本功能：将输入时钟 `clk` 的频率除以 `N`，输出占空比 50% 的方波信号。（1 分）

(2) GENERIC 参数 `N` 的作用与输出频率：

GENERIC `N` 定义了分频比参数，使分频器位宽可配置。计数器 `count` 的范围为 0 TO `N-1`。

当 `N=8` 时：

- 计数器从 0 计数到 $N/2-1 = 3$ （即 0, 1, 2, 3 共 4 个值）后，`temp` 翻转。
- `temp` 每翻转一次需要 4 个时钟周期，完整输出周期 = 8 个时钟周期。

$$f_{out} = \frac{f_{clk}}{N} = \frac{100 \text{ MHz}}{8} = 12.5 \text{ MHz}$$

（2 分）

(3) 复位方式判断：

异步复位。判断依据：

- (a) `rst` 出现在 PROCESS 敏感信号表中（第 139 行：`PROCESS(clk, rst)`）；
- (b) 在进程体内，`rst` 的判断（第 141 行）在 `rising_edge(clk)` 之前，不依赖于时钟边沿。

这两点满足异步复位的 VHDL 描述特征。（1 分）

(4) 去 -1 后的影响：

若将条件改为 `IF count = N/2 THEN`（即 `count=4` 时翻转）：

- 输出频率：计数器需经历 0→1→2→3→4 共 5 个时钟周期才翻转一次，完整周期为 $5 \times 2 = 10$ 个时钟周期，输出频率变为：

$$f_{out} = \frac{100 \text{ MHz}}{10} = 10 \text{ MHz}$$

频率降低（从 12.5 MHz 降至 10 MHz）。

- **占空比：**由于两个半周期仍然对称（翻转时刻等距），占空比**仍为 50%**。

结论：去掉 -1 后，分频比从 N 变为 $N + 2$ （当 $N = 8$ 时，从 8 分频变为 10 分频），占空比不变。（1 分）

15.5 五、综合设计题参考答案（60 分）

15.5.1 设计题 1：BCD 码—7 段数码管译码器（20 分）

（1）真值表（5 分）

十进制	bcd(3:0)	a	b	c	d	e	f	g	seg(6:0)
0	0000	0	0	0	0	0	0	1	1000000
1	0001	1	0	0	1	1	1	1	1111001
2	0010	0	0	1	0	0	1	0	0100100
3	0011	0	0	0	0	1	1	0	0110000
4	0100	1	0	0	1	1	0	0	0011001
5	0101	0	1	0	0	1	0	0	0010010
6	0110	0	1	0	0	0	0	0	0000010
7	0111	0	0	0	1	1	1	1	0001111
8	1000	0	0	0	0	0	0	0	0000000
9	1001	0	0	0	0	1	0	0	0000100
10~15	1010~1111	1	1	1	1	1	1	1	1111111

（2）最小化逻辑表达式（3 分）

评分标准：每段表达式正确各得 1 分（共 3 分）。

设 $bcd = b_3b_2b_1b_0$ ，各段逻辑表达式（共阳极，0= 亮）：

- **a 段：** $a = \overline{b_3}\overline{b_2}\overline{b_1}b_0 + b_3\overline{b_2}b_1b_0$ （当输入为 1 和 4 时灭 $\rightarrow$ 为 1；即输入为 0001 或 0100 时 $a=1$ ）

化简为： $a = \overline{b_2}\overline{b_0} \cdot (\overline{b_3} \oplus \overline{b_1})$ 或在卡诺图化简后：

$$a = \overline{b_2}\overline{b_0}(\overline{b_3}\overline{b_1} + b_3b_1) + \overline{b_2}b_1\overline{b_0}$$

更简化的表达（最小项之和）：

$$a = \Sigma m(1, 4) + \Sigma d(10-15) = \overline{b_2}\overline{b_0}(\overline{b_3} \oplus \overline{b_1})$$

（注：实际 a 段在输入 1(0001) 和 4(0100) 时灭 $=1$ ，其余有效输入时亮 $=0$ 。具体表达式依赖化简方法。）

- **b 段：** $b = \overline{b_2} + \overline{b_1}\overline{b_0} + b_1b_0$ （简化形式）

化简为:

$$b = \overline{b_2} + \overline{b_1} \overline{b_0} + b_1 b_0$$

b 段在输入 5(0101) 和 6(0110) 时灭 =1。

- g 段: $g = \overline{b_3} \overline{b_2} \overline{b_1} + b_3 \overline{b_2} b_1 b_0 + \overline{b_3} b_2 b_1 \overline{b_0} + \overline{b_3} b_2 \overline{b_1} b_0$

化简为:

$$g = \overline{b_3} \overline{b_2} \overline{b_1} + b_2 b_1 \overline{b_0} + b_2 \overline{b_1} b_0 + b_3 \overline{b_2} b_1 b_0$$

g 段在输入 0, 2, 3, 5, 6, 7, 8, 9 时亮 =0, 仅在 1 和 4 时灭 =1。

(注: 由于本题使用 WITH-SELECT 直接描述真值表, 卡诺图化简为可选项; 实际工程中由综合工具自动优化。以下给出各段的简化逻辑供参考。)

(3) 完整 VHDL 代码 (10 分)

Listing 15.1: BCD—7 段译码器——数据流描述方式

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY bcd2seg7 IS
5      PORT(
6          bcd : IN  STD_LOGIC_VECTOR(3 DOWNTO 0);
7          seg : OUT STD_LOGIC_VECTOR(6 DOWNTO 0)
8      );
9  END bcd2seg7;
10
11 ARCHITECTURE dataflow OF bcd2seg7 IS
12 BEGIN
13     -- 共阳极数码管: 0=点亮, 1=熄灭
14     -- seg(6)=a, seg(5)=b, seg(4)=c, seg(3)=d,
15     -- seg(2)=e, seg(1)=f, seg(0)=g
16     WITH bcd SELECT
17         seg <= "1000000" WHEN "0000",    -- 0
18               "1111001" WHEN "0001",    -- 1
19               "0100100" WHEN "0010",    -- 2
20               "0110000" WHEN "0011",    -- 3
21               "0011001" WHEN "0100",    -- 4
22               "0010010" WHEN "0101",    -- 5
23               "0000010" WHEN "0110",    -- 6

```

```

24         "0001111" WHEN "0111",    -- 7
25         "0000000" WHEN "1000",    -- 8
26         "0000100" WHEN "1001",    -- 9
27         "1111111" WHEN OTHERS;    -- 无效输入：全灭
28 END dataflow;

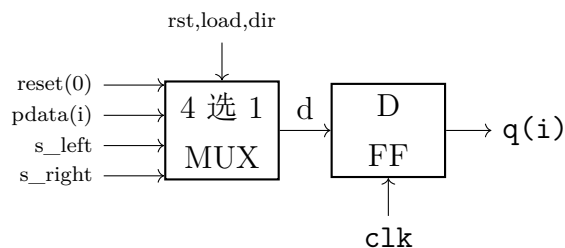
```

(4) 数据流描述与行为描述的差异及选用理由 (2 分)

- **数据流描述：**使用并发信号赋值语句（如 WITH-SELECT），直接描述输入到输出的数据流动映射关系，不描述时序行为。适合纯组合逻辑的真值表映射。
- **行为描述：**使用 PROCESS 中的顺序语句（如 IF-ELSE、CASE），按算法流程描述电路行为，更抽象。
- **选用数据流方式的理由：**BCD 译码器是典型的真值表映射——16 种输入对应 16 种输出，条件互斥无优先级，使用 WITH-SELECT 语义最清晰、代码最简洁，综合工具可直接映射为 ROM 或 LUT 结构。

15.5.2 设计题 2：使用 GENERATE 语句设计 N 位双向移位寄存器 (20 分)

(1) 单个位片内部逻辑框图 (4 分)



说明：每位片由一个 4 选 1 多路选择器和一个 D 触发器组成。MUX 根据控制信号选择数据源：复位值（全 0）、并行加载数据 `pdata(i)`、左邻位输出（左移时）、右邻位输出（右移时）。

(2) 位片 ENTITY 定义 (3 分)

```

1 ENTITY bitslice IS
2     PORT(
3         clk      : IN    STD_LOGIC;
4         rst      : IN    STD_LOGIC;
5         load     : IN    STD_LOGIC;

```



```

6      dir      : IN   STD_LOGIC;
7      pdata    : IN   STD_LOGIC;    -- 并行加载数据(该位)
8      s_left   : IN   STD_LOGIC;    -- 左移时来自右邻位的数据
9      s_right  : IN   STD_LOGIC;    -- 右移时来自左邻位的数据
10     q         : OUT  STD_LOGIC
11 );
12 END bitslice;

```

(3) 顶层 N 位寄存器完整 VHDL 代码 (11 分)

评分标准：GENERIC 声明 (1 分); COMPONENT 声明 (1 分); 内部信号数组 (1.5 分); FOR GENERATE 循环结构 (3 分); 位间级联正确 (2.5 分); dout 串行输出逻辑 (1 分); 代码规范性 (1 分)。

Listing 15.2: N 位双向移位寄存器——结构化 +GENERATE

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3
4  ENTITY bidir_shift_reg IS
5      GENERIC(N : INTEGER := 8);
6      PORT(
7          clk      : IN   STD_LOGIC;
8          rst      : IN   STD_LOGIC;    -- 同步复位，高有效
9          din      : IN   STD_LOGIC;    -- 串行输入
10         dir      : IN   STD_LOGIC;    -- '0'右移，'1'左移
11         load     : IN   STD_LOGIC;    -- 并行置数使能
12         pdata    : IN   STD_LOGIC_VECTOR(N-1 DOWNT0 0);
13         q        : OUT  STD_LOGIC_VECTOR(N-1 DOWNT0 0);
14         sout     : OUT  STD_LOGIC      -- 串行输出
15     );
16 END bidir_shift_reg;
17
18 ARCHITECTURE struct OF bidir_shift_reg IS
19     -- 声明位片元件
20     COMPONENT bitslice IS
21         PORT(
22             clk      : IN   STD_LOGIC;

```

```

23     rst      : IN  STD_LOGIC;
24     load     : IN  STD_LOGIC;
25     dir      : IN  STD_LOGIC;
26     pdata    : IN  STD_LOGIC;
27     s_left   : IN  STD_LOGIC;
28     s_right  : IN  STD_LOGIC;
29     q        : OUT STD_LOGIC
30 );
31 END COMPONENT;
32
33 -- 内部级联信号数组 (N位触发器输出)
34 SIGNAL q_int : STD_LOGIC_VECTOR(N-1 DOWNT0 0);
35
36 BEGIN
37     -- 使用 FOR...GENERATE 实例化 N 个位片
38     gen_reg : FOR i IN 0 TO N-1 GENERATE
39         BEGIN
40             -- 第 0 位 (LSB) : 特殊处理级联输入
41             first_bit : IF i = 0 GENERATE
42                 u_bit0 : bitslice PORT MAP(
43                     clk      => clk,
44                     rst      => rst,
45                     load     => load,
46                     dir      => dir,
47                     pdata    => pdata(i),
48                     s_left   => q_int(i+1),    -- 左移时数据来自 i+1 位
49                     s_right => din,           -- 右移时串入数据
50                     q        => q_int(i)
51                 );
52             END GENERATE first_bit;
53
54             -- 第 N-1 位 (MSB) : 特殊处理级联输入
55             last_bit : IF i = N-1 GENERATE
56                 u_bit_last : bitslice PORT MAP(
57                     clk      => clk,
58                     rst      => rst,
59                     load     => load,
60                     dir      => dir,
61                     pdata    => pdata(i),

```

```

62         s_left  => din,           -- 左移时串入数据
63         s_right => q_int(i-1),     -- 右移时数据来自 i-1 位
64         q        => q_int(i)
65     );
66 END GENERATE last_bit;
67
68 -- 中间位 (1到N-2) : 标准级联
69 middle_bit : IF i > 0 AND i < N-1 GENERATE
70     u_bit_i : bitslice PORT MAP(
71         clk      => clk,
72         rst      => rst,
73         load     => load,
74         dir      => dir,
75         pdata    => pdata(i),
76         s_left   => q_int(i+1),     -- 左移数据来自高位
77         s_right  => q_int(i-1),     -- 右移数据来自低位
78         q        => q_int(i)
79     );
80 END GENERATE middle_bit;
81 END GENERATE gen_reg;
82
83 -- 输出连接
84 q <= q_int;
85 sout <= q_int(N-1) WHEN dir = '0' ELSE -- 右移串出取 MSB
86         q_int(0);                      -- 左移串出取 LSB
87
88 END struct;

```

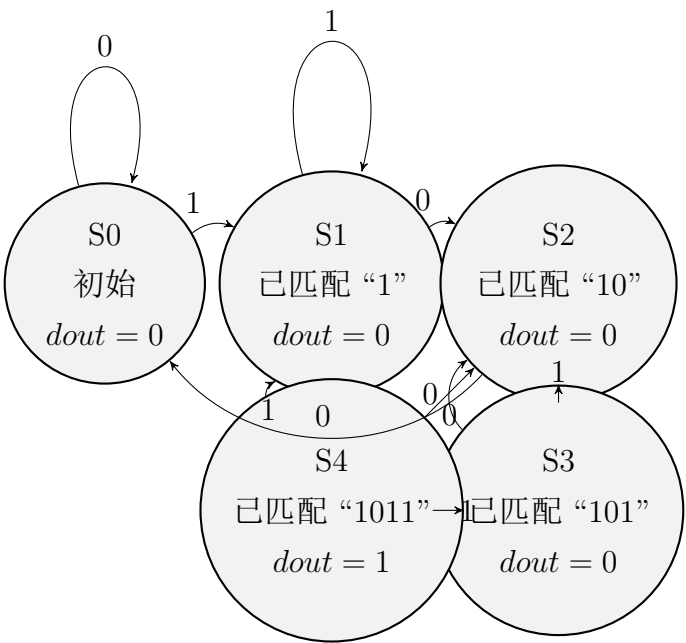
(4) FOR...GENERATE 与普通 FOR-LOOP 的本质区别（2 分）

FOR...GENERATE (生成语句)	FOR...LOOP（循环语句）
并发语句，在综合时展开为多个并行硬件实例（ N 个独立的物理 D 触发器）	顺序语句（位于 PROCESS 内部），综合为单个硬件单元在不同时间执行多次操作
生成的是空间上的重复——硬件复制	描述的是时间上的重复——顺序执行
用于结构化描述，每次迭代生成独立的硬件实例	用于行为描述，不产生额外的硬件副本

核心区别：FOR...GENERATE 产生 N 份并行硬件，FOR...LOOP 在单份硬件上分时复用。前者增加面积，后者增加执行周期数。

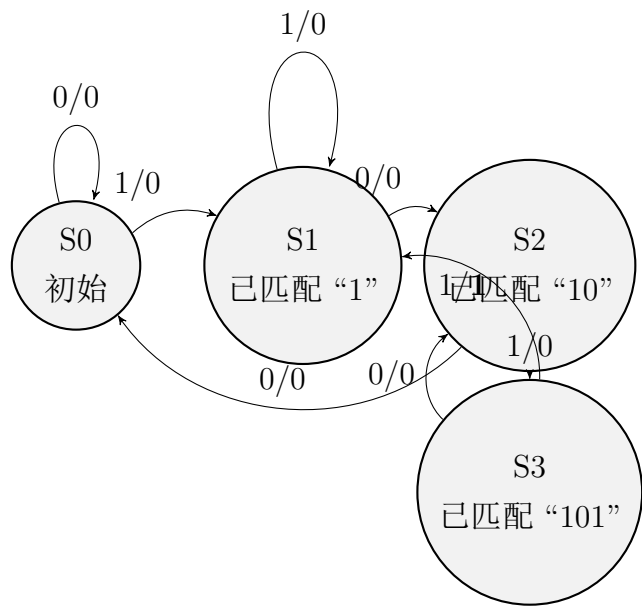
15.5.3 设计题 3：“1011”序列检测器——Moore 型与 Mealy 型对比（20 分）

(1) Moore 型状态转移图（6 分）



输出在状态节点上标注。Moore 型共 5 个状态，仅 S4 状态输出 $dout = 1$ 。

(2) Mealy 型状态转移图 (4 分)



输出在转移弧上标注(格式: **din/dout**)。Mealy 型仅需 4 个状态。S3 状态下 $din = 1$ 时输出 $dout = 1$ 并转入 S1 (重叠检测)。

(3) Moore 型三进程 VHDL 代码 (10 分)

评分标准: 枚举类型定义 (1 分); 实体定义 (1 分); 进程 1 (状态寄存器 + 异步复位) (2.5 分); 进程 2 (次态组合逻辑) (3 分); 进程 3 (输出组合逻辑) (1.5 分); 代码规范性 (1 分)。

Listing 15.3: “1011” 序列检测器——Moore 型三进程 VHDL 代码

```
1 LIBRARY ieee;
2 USE ieee.std_logic_1164.ALL;
3
4 ENTITY seq_detector_1011 IS
5     PORT(
6         clk    : IN    STD_LOGIC;
7         rst    : IN    STD_LOGIC;    -- 异步复位，高电平有效
8         din    : IN    STD_LOGIC;
9         dout   : OUT   STD_LOGIC
10    );
11 END seq_detector_1011;
12
```

```

13 ARCHITECTURE moore_3p OF seq_detector_1011 IS
14     -- 用户自定义枚举类型
15     TYPE state_type IS (S0, S1, S2, S3, S4);
16     SIGNAL current_state, next_state : state_type;
17
18 BEGIN
19     -- ===== 进程1: 状态寄存器 (时序逻辑+异步复位) =====
20     PROCESS(clk, rst)
21     BEGIN
22         IF rst = '1' THEN
23             current_state <= S0;
24         ELSIF rising_edge(clk) THEN
25             current_state <= next_state;
26         END IF;
27     END PROCESS;
28
29     -- ===== 进程2: 次态逻辑 (组合逻辑) =====
30     PROCESS(current_state, din)
31     BEGIN
32         CASE current_state IS
33             WHEN S0 =>                                -- 初始/未匹配
34                 IF din = '1' THEN
35                     next_state <= S1;
36                 ELSE
37                     next_state <= S0;
38                 END IF;
39
40             WHEN S1 =>                                -- 已匹配 ``1''
41                 IF din = '1' THEN
42                     next_state <= S1;
43                 ELSE
44                     next_state <= S2;
45                 END IF;
46
47             WHEN S2 =>                                -- 已匹配 ``10''
48                 IF din = '1' THEN
49                     next_state <= S3;
50                 ELSE
51                     next_state <= S0;

```

```

52         END IF;
53
54     WHEN S3 =>                                -- 已匹配 ``101''
55         IF din = '1' THEN
56             next_state <= S4;
57         ELSE
58             next_state <= S2;
59         END IF;
60
61     WHEN S4 =>                                -- 已匹配 ``1011''
62         IF din = '1' THEN
63             next_state <= S1;                -- 重叠：尾部的 ``1'' 作为下一序列前缀
64         ELSE
65             next_state <= S2;                -- 重叠：尾部的 ``10'' 作为下一序列前缀
66         END IF;
67
68     WHEN OTHERS =>
69         next_state <= S0;
70     END CASE;
71 END PROCESS;
72
73 -- ===== 进程 3：输出逻辑（组合逻辑） =====
74 -- Moore 型：输出仅取决于当前状态
75 PROCESS(current_state)
76 BEGIN
77     IF current_state = S4 THEN
78         dout <= '1';
79     ELSE
80         dout <= '0';
81     END IF;
82 END PROCESS;
83
84 END moore_3p;

```

评分细则总结——设计题 3（20 分）：

- Moore 型状态转移图：6 分（5 个状态节点标注各 0.5 分 = 2.5 分；10 条转移

弧各 0.35 分 =3.5 分)

- Mealy 型状态转移图: 4 分 (4 个状态各 0.4 分 =1.6 分; 8 条转移弧含输出标注各 0.3 分 =2.4 分)
- Moore 型 VHDL 代码: 10 分
 - 实体 + 库声明 + 枚举类型 (2 分)
 - 进程 1——状态寄存器 + 异步复位 (2.5 分)
 - 进程 2——次态组合逻辑 (含序列检测和重叠处理, 3.5 分)
 - 进程 3——输出组合逻辑 (1 分)
 - 代码规范性 (1 分)

常见扣分说明:

- Moore 型状态转移图将输出错误地放在转移弧上, 扣 3 分
- 重叠检测逻辑错误 (S4 输入 0 应回 S2 而非 S0), 扣 1 分
- 组合进程 (进程 2) 敏感表缺 `din`, 扣 0.5 分
- CASE 缺少 WHEN OTHERS 分支, 扣 0.5 分

第十六章 第 16 套试卷——参考答案与评分标准

16.1 一、选择题答案（18 分）

1. B —CLB/Slice 包含 LUT、D 触发器、MUX 和进位链。
2. B —信号 ($\leq$) 在进程挂起后更新；变量 ($:=$) 立即生效。
3. C —敏感列表不完整导致锁存器推断。
4. A —Moore 输出仅取决于状态；Mealy 取决于状态 + 输入。
5. D —并发语句和顺序语句均可描述组合/时序逻辑。
6. B —NOT 优先级最高，然后 AND，最后 OR。

16.2 二、填空题答案（12 分）

1. (1) IEEE.STD_LOGIC_1164 (2) IEEE.NUMERIC_STD
2. (1) $\leq$ (2) $:=$
3. GENERIC 声明应在 PORT 声明之前
4. (1) COMPONENT (2) PORT MAP

16.3 三、程序改错题解析（5 分）

错误 1： 库引用错误：STD_LOGIC_ARITH 应为 NUMERIC_STD，且缺少 STD_LOGIC_1164。

错误 2： 敏感列表缺少 en 信号。

错误 3： CASE 缺少 WHEN OTHERS 分支。

错误 4: 漏分号: Y <= "00010000" 后缺;。

错误 5: PORT 括号不匹配。

修正后代码:

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3 use IEEE.NUMERIC_STD.all;
4
5 entity decoder_3to8 is
6     port(
7         A  : in  std_logic_vector(2 downto 0);
8         en : in  std_logic;
9         Y  : out std_logic_vector(7 downto 0)
10    );
11 end decoder_3to8;
12
13 architecture behav of decoder_3to8 is
14 begin
15     process(A, en)
16     begin
17         if en = '0' then Y <= (others => '0');
18         else
19             case A is
20                 when "000" => Y <= "00000001";
21                 when "001" => Y <= "00000010";
22                 when "010" => Y <= "00000100";
23                 when "011" => Y <= "00001000";
24                 when "100" => Y <= "00010000";
25                 when "101" => Y <= "00100000";
26                 when "110" => Y <= "01000000";
27                 when "111" => Y <= "10000000";
28                 when others => Y <= (others => '0');
29             end case;
30         end if;
31     end process;
32 end behav;

```

评分: 每找出并正确修正 1 处得 1 分, 共 5 分。

16.4 四、程序阅读分析题解析（5 分）

- (1) 电路类型（1 分）：6 分频器（模 6 计数器 + 输出逻辑）。
- (2) 功能（1 分）：将 clk 进行 6 分频， $f_{out} = f_{clk}/6$ 。
- (3) 占空比（1.5 分）：cnt=0,1,2 时输出 0；cnt=3,4,5 时输出 1。高电平 3 拍、低电平 3 拍，占空比 50%。
- (4) 频率（1.5 分）： $f_{clk} = 50\text{MHz}$ 时， $f_{out} = 50/6 \approx 8.33\text{MHz}$ 。

16.5 五、综合设计题参考答案（60 分）

16.5.1 设计题 1：带使能的 3-8 译码器（20 分）

【分析】3-8 译码器：3 位输入 $\rightarrow$ 8 位独热输出。en=1 正常译码；en=0 全 0。

【VHDL】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity decoder_3to8_en is
5      port(
6          A  : in  std_logic_vector(2 downto 0);
7          en : in  std_logic;
8          Y  : out std_logic_vector(7 downto 0)
9      );
10 end decoder_3to8_en;
11
12 architecture behav of decoder_3to8_en is
13 begin
14     process(A, en)
15     begin
16         if en = '0' then Y <= (others => '0');
17         else
18             case A is
19                 when "000" => Y <= "00000001";
20                 when "001" => Y <= "00000010";
21                 when "010" => Y <= "00000100";
22                 when "011" => Y <= "00001000";
23                 when "100" => Y <= "00010000";
24                 when "101" => Y <= "00100000";

```

```

25         when "110" => Y <= "01000000";
26         when "111" => Y <= "10000000";
27         when others => Y <= (others => '0');
28     end case;
29 end if;
30 end process;
31 end behav;

```

【评分】(20 分) ENTITY 声明 4 分 + en=0 逻辑 4 分 + CASE 完整 6 分 + 独热编码 4 分 + 敏感列表 2 分。

16.5.2 设计题 2：模 6 计数器 (20 分)

【分析】 $N = 6$, $k = \lceil \log_2 6 \rceil = 3$ 位, 清零 $cnt = 5(101)$ 。

【VHDL】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity counter_mod6 is
6      port(
7          clk    : in  std_logic;
8          rst    : in  std_logic;
9          count  : out std_logic_vector(2 downto 0);
10         tc     : out std_logic
11     );
12 end counter_mod6;
13
14 architecture behav of counter_mod6 is
15     signal cnt : unsigned(2 downto 0) := (others => '0');
16 begin
17     process(clk, rst)
18     begin
19         if rst = '1' then cnt <= (others => '0');
20         elsif rising_edge(clk) then
21             if cnt = 5 then cnt <= (others => '0');
22             else cnt <= cnt + 1; end if;
23         end if;
24     end process;

```

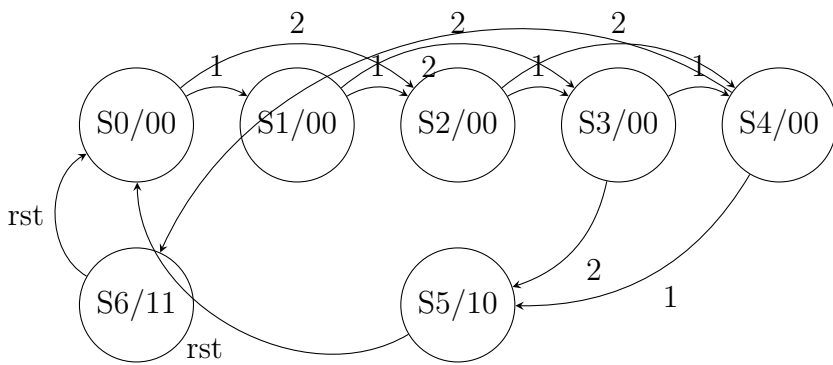
```
25     count <= std_logic_vector(cnt);
26     tc <= '1' when cnt = 5 else '0';
27 end behav;
```

【评分】（20 分）分析 (N=6,k=3)3 分 + ENTITY 3 分 + 异步复位 3 分 + cnt=5 清零 4 分 + NUMERIC_STD 3 分 + tc 信号 2 分 + 规范 2 分。

16.5.3 设计题 3：自动售货机控制器（20 分）

【分析】商品 5 元，硬币 1 元/2 元。Moore 型，7 状态 (S0=0 元 ~S5=5 元出货, S6= 超 5 元找零)。

【状态转移图】



(注：弧上数字为投币面额，状态/输出格式为 dispense/change)

【状态转移表】

当前	投 1 元	投 2 元	dispense	change
S0(0 元)	S1	S2	0	0
S1(1 元)	S2	S3	0	0
S2(2 元)	S3	S4	0	0
S3(3 元)	S4	S5	0	0
S4(4 元)	S5	S6	0	0
S5(5 元)	S0	S0	1	0
S6(>5 元)	S0	S0	1	1

【VHDL】

```
1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3
4 entity vending_machine is
```

```
5  port(  
6      clk      : in  std_logic;  
7      rst      : in  std_logic;  
8      coin_1   : in  std_logic;  
9      coin_2   : in  std_logic;  
10     dispense : out std_logic;  
11     change   : out std_logic  
12 );  
13 end vending_machine;  
14  
15 architecture moore_fsm of vending_machine is  
16     type state_type is (S0, S1, S2, S3, S4, S5, S6);  
17     signal curr_state, next_state : state_type;  
18 begin  
19     reg_proc: process(clk, rst)  
20     begin  
21         if rst = '1' then curr_state <= S0;  
22         elsif rising_edge(clk) then curr_state <= next_state;  
23         end if;  
24     end process;  
25  
26     comb_proc: process(curr_state, coin_1, coin_2)  
27     begin  
28         case curr_state is  
29             when S0 =>  
30                 if coin_2 = '1' then next_state <= S2;  
31                 elsif coin_1 = '1' then next_state <= S1;  
32                 else next_state <= S0; end if;  
33             when S1 =>  
34                 if coin_2 = '1' then next_state <= S3;  
35                 elsif coin_1 = '1' then next_state <= S2;  
36                 else next_state <= S1; end if;  
37             when S2 =>  
38                 if coin_2 = '1' then next_state <= S4;  
39                 elsif coin_1 = '1' then next_state <= S3;  
40                 else next_state <= S2; end if;  
41             when S3 =>  
42                 if coin_2 = '1' then next_state <= S5;  
43                 elsif coin_1 = '1' then next_state <= S4;
```

```
44         else next_state <= S3; end if;
45     when S4 =>
46         if coin_2 = '1' then next_state <= S6;
47         elsif coin_1 = '1' then next_state <= S5;
48         else next_state <= S4; end if;
49     when S5 | S6 =>
50         next_state <= S0;
51     when others => next_state <= S0;
52 end case;
53 end process;
54
55 dispense <= '1' when (curr_state = S5 or curr_state = S6) else '0';
56 change   <= '1' when curr_state = S6 else '0';
57 end moore_fsm;
```

【评分】(20 分) 7 状态定义 3 分 + 状态图 4 分 + 状态表 3 分 + 两段式 VHDL 3 分 + 投币累计逻辑 4 分 + Moore 纯状态输出 2 分 + 返回 S0 1 分。

第十七章 第 17 套试卷——参考答案与评分标准

17.1 一、选择题答案（18 分）

1. **B** —CPLD 基于乘积项；FPGA 基于 LUT。A 混淆结构，C 错 (FPGA SRAM 掉电丢失)，D 错 (CPLD 非易失)。
2. **C** —CASE 并行无优先级；IF-ELSE 级联有优先级。A 错 (IF-ELSE 不一定优先)，B 错 (误导)，D 与实际相反。
3. **B** —COMPONENT 声明在 ARCHITECTURE 声明区 (IS 与 BEGIN 之间)。
4. **D** —GENERIC 语法：name : type := default，用:= 不是 <=。
5. **A** —FOR GENERATE 循环复制；IF GENERATE 条件生成。
6. **D** —One-Hot(N 个 FF 译码简单)；Binary($\lceil \log_2 N \rceil$ 个 FF 译码复杂)。

17.2 二、填空题答案（12 分）

1. (1) **COMPONENT** 在声明区，(2) **PORT MAP** 实例化连接
2. (1) **GENERIC** 在 PORT 之前，(2) **GENERIC MAP** 传参
3. (1) **FOR GENERATE** (重复)，(2) **IF GENERATE** (条件)
4. One-Hot: N 个 FF；Binary: $\lceil \log_2 N \rceil$ 个 FF

17.3 三、程序改错题解析（5 分）

错误 1: COMPONENT 名称与实例化标签混用

错误 2: PORT MAP 进位索引错误 (carry(i-1) 应为 carry(i))

错误 3: 重复的 ENTITY 声明块

错误 4: 敏感列表不完整

错误 5: 进位链索引越界 (carry 位宽不足)

17.4 四、程序阅读分析题解析 (5 分)

- (1) (1 分) 模 60 BCD 计数器 (个位 0-9, 十位 0-5)。
 (2) (1 分) 异步高电平复位 (if rst='1')。
 (3) (1.5 分) $tc = 1$ 当十位 =5 且个位 =9 (59)。
 (4) (1.5 分) 改清零条件为 $tens = 9, units = 9$ 即模 100。

17.5 五、综合设计题参考答案 (60 分)

17.5.1 设计题 1: 4 位比较器 (20 分)

【VHDL】

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3
4 entity comparator_4bit is
5     port(A, B : in std_logic_vector(3 downto 0);
6           AgtB, AeqB, AltB : out std_logic);
7 end comparator_4bit;
8
9 architecture behav of comparator_4bit is
10 begin
11     process(A, B)
12     begin
13         if A > B then AgtB<='1'; AeqB<='0'; AltB<='0';
14         elsif A = B then AgtB<='0'; AeqB<='1'; AltB<='0';
15         else AgtB<='0'; AeqB<='0'; AltB<='1'; end if;
16     end process;
17 end behav;
  
```

【评分】(20 分) ENTITY 3 分 + 三种互斥输出 5 分 + IF 完整 4 分 + 敏感列表 2 分 + 规范 2 分 + 输出互斥保证 4 分。

17.5.2 设计题 2：模 60 BCD 计数器（20 分）

【VHDL】

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3 use IEEE.NUMERIC_STD.all;
4
5 entity bcd_mod60 is
6     port(clk, rst : in std_logic;
7           units, tens : out std_logic_vector(3 downto 0);
8           tc : out std_logic);
9 end bcd_mod60;
10
11 architecture behav of bcd_mod60 is
12     signal u_cnt, t_cnt : unsigned(3 downto 0);
13 begin
14     process(clk, rst)
15     begin
16         if rst='1' then u_cnt<=(others=>'0'); t_cnt<=(others=>'0');
17         elsif rising_edge(clk) then
18             if t_cnt=5 and u_cnt=9 then
19                 u_cnt<=(others=>'0'); t_cnt<=(others=>'0');
20                 elsif u_cnt=9 then u_cnt<=(others=>'0'); t_cnt<=t_cnt+1;
21                 else u_cnt<=u_cnt+1; end if;
22             end if;
23         end process;
24         units<=std_logic_vector(u_cnt); tens<=std_logic_vector(t_cnt);
25         tc<='1' when t_cnt=5 and u_cnt=9 else '0';
26     end behav;

```

【评分】（20 分）BCD 进位 8 分 + 清零 (59→00)4 分 + 复位 3 分 + tc 2 分 + NUMERIC_STD 2 分 + 端口 1 分。

17.5.3 设计题 3：密码锁控制器（20 分）

【分析】16 位密码“2-5-3-7”=0010-0101-0011-0111，din 逐位输入。18 状态 Moore 型 (S0=IDLE, S1-S16 逐位匹配, S17=UNLOCK)。任意位错误 → 回 S0。

【VHDL】

```

1 library IEEE;

```

```

2 use IEEE.STD_LOGIC_1164.all;
3
4 entity combo_lock is
5     port(clk, rst, din : in std_logic; unlock : out std_logic);
6 end combo_lock;
7
8 architecture moore_fsm of combo_lock is
9     type state_type is (S0,S1,S2,S3,S4,S5,S6,S7,S8,
10                        S9,S10,S11,S12,S13,S14,S15,S16,S17);
11     signal curr, nxt : state_type;
12 begin
13     reg_proc: process(clk, rst)
14     begin
15         if rst='1' then curr<=S0;
16         elsif rising_edge(clk) then curr<=nxt; end if;
17     end process;
18
19     comb_proc: process(curr, din)
20     begin
21         case curr is
22             when S0=> if din='0' then nxt<=S1; else nxt<=S0; end if;
23             when S1=> if din='0' then nxt<=S2; else nxt<=S0; end if;
24             when S2=> if din='1' then nxt<=S3; else nxt<=S0; end if;
25             when S3=> if din='0' then nxt<=S4; else nxt<=S0; end if;
26             when S4=> if din='0' then nxt<=S5; else nxt<=S0; end if;
27             when S5=> if din='1' then nxt<=S6; else nxt<=S0; end if;
28             when S6=> if din='0' then nxt<=S7; else nxt<=S0; end if;
29             when S7=> if din='1' then nxt<=S8; else nxt<=S0; end if;
30             when S8=> if din='0' then nxt<=S9; else nxt<=S0; end if;
31             when S9=> if din='0' then nxt<=S10; else nxt<=S0; end if;
32             when S10=> if din='1' then nxt<=S11; else nxt<=S0; end if;
33             when S11=> if din='1' then nxt<=S12; else nxt<=S0; end if;
34             when S12=> if din='0' then nxt<=S13; else nxt<=S0; end if;
35             when S13=> if din='1' then nxt<=S14; else nxt<=S0; end if;
36             when S14=> if din='1' then nxt<=S15; else nxt<=S0; end if;
37             when S15=> if din='1' then nxt<=S16; else nxt<=S0; end if;
38             when S16=> if din='1' then nxt<=S17; else nxt<=S0; end if;
39             when S17=> nxt<=S17;
40             when others=> nxt<=S0;

```

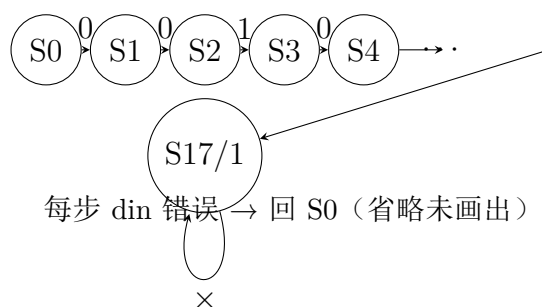
```

41     end case;
42     end process;
43     unlock<='1' when curr=S17 else '0';
44 end moore_fsm;

```

【评分】(20 分) 状态定义 4 分 + 逐位匹配逻辑 8 分 + 错误回 S0 3 分 + Moore 输出 2 分 + 代码完整 3 分。

17.5.4 设计题 3 补充：状态转移图



【评分】(20 分) 状态定义 4 分 + 逐位匹配 8 分 + 错误回 S0 3 分 + Moore 输出 2 分 + 代码/图 3 分。

第十八章 第 18 套试卷——参考答案与评分标准

18.1 一、选择题答案（18 分）

1. C —n 输入 LUT 本质是 $2^n \times 1$ SRAM，可实现任意 n 变量组合逻辑函数。
2. B —信号在进程挂起后更新 (Δ 延时)；变量立即更新。
3. C —组合进程敏感列表不完整导致锁存器。
4. A —Moore 输出仅取决于状态；Mealy 取决于状态 + 输入。
5. D —并发语句 (WHEN-ELSE, WITH-SELECT) 和顺序语句 (CASE, IF) 均可描述组合/时序逻辑。
6. B —优先级：NOT > AND > OR。A AND NOT B OR C = (A AND (NOT B)) OR C。

18.2 二、填空题答案（12 分）

1. 变量赋值:= 立即生效；信号赋值 <= 进程挂起后生效
2. 组合进程敏感列表必须包含所有输入信号，否则产生锁存器
3. 状态机三要素：当前状态、次态逻辑、输出逻辑
4. 省略 ELSE/WHEN OTHERS 在组合进程中推断锁存器 (Latch)

18.3 三、程序改错题解析（5 分）

错误 1：PROCESS 内部对 SIGNAL 用:= 赋值 → 应为 <=

错误 2：IF 语句缺少 ELSE 分支 (组合进程) → 推断锁存器

错误 3: 敏感列表缺少输入信号 → 行为不匹配

错误 4: 类型不匹配: STD_LOGIC_VECTOR 与 INTEGER 混用 → 需类型转换

错误 5: CASE 缺少 WHEN OTHERS → 推断锁存器

评分: 每处 1 分, 共 5 分。

18.4 四、程序阅读分析题解析 (5 分)

(1) (1 分) 环形计数器 (4 位, 4 状态 one-hot 循环)。

(2) (1 分) 初始 1000→0100→0010→0001→1000, 4 拍一个循环。

(3) (2 分) 4 个 D 触发器, one-hot 编码, 无需译码电路。

(4) (1 分) 若进入非法状态 (如全 0), 无法自动恢复——需加自启动逻辑。

18.5 五、综合设计题参考答案 (60 分)

18.5.1 设计题 1: 8-3 优先编码器 (20 分)

【VHDL】

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3
4 entity priority_encoder_8to3 is
5     port(I : in std_logic_vector(7 downto 0);
6          Y : out std_logic_vector(2 downto 0);
7          GS : out std_logic);
8 end priority_encoder_8to3;
9
10 architecture behav of priority_encoder_8to3 is
11 begin
12     process(I)
13     begin
14         if I(7)='1' then Y<="111"; GS<='1';
15         elsif I(6)='1' then Y<="110"; GS<='1';
16         elsif I(5)='1' then Y<="101"; GS<='1';
17         elsif I(4)='1' then Y<="100"; GS<='1';
18         elsif I(3)='1' then Y<="011"; GS<='1';
19         elsif I(2)='1' then Y<="010"; GS<='1';

```



```

20     elsif I(1)='1' then Y<="001"; GS<='1';
21     elsif I(0)='1' then Y<="000"; GS<='1';
22     else Y<="000"; GS<='0'; end if;
23     end process;
24 end behav;

```

【评分】(20 分) ENTITY 3 分 + IF-ELSIF 优先级链 6 分 + GS 信号 (区分无输入)4 分 + 输出编码 5 分 + 代码完整 2 分。

18.5.2 设计题 2: 8 位移位寄存器 (20 分)

【VHDL】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3
4  entity shift_reg_8bit is
5      port(clk, rst : in std_logic;
6           mode : in std_logic_vector(1 downto 0);
7           si_l, si_r : in std_logic;
8           data_in : in std_logic_vector(7 downto 0);
9           q : out std_logic_vector(7 downto 0));
10 end shift_reg_8bit;
11
12 architecture behav of shift_reg_8bit is
13     signal reg : std_logic_vector(7 downto 0);
14 begin
15     process(clk, rst)
16     begin
17         if rst='1' then reg<=(others=>'0');
18         elsif rising_edge(clk) then
19             case mode is
20                 when "00" => null; -- 保持
21                 when "01" => reg <= si_r & reg(7 downto 1); -- 右移
22                 when "10" => reg <= reg(6 downto 0) & si_l; -- 左移
23                 when "11" => reg <= data_in; -- 装载
24                 when others => null;
25             end case;
26         end if;
27     end process;

```

```

28   q <= reg;
29 end behav;

```

【评分】(20 分) 四模式 CASE 8 分 + 移位拼接正确 6 分 + 同步装载 3 分 + 异步复位 2 分 + 端口完整 1 分。

18.5.3 设计题 3：交通灯控制器 (20 分)

【分析】主干道绿灯 30s→黄灯 5s→支路绿灯 20s→黄灯 5s→循环。Moore 型，4 状态，内部计数器计时。

【状态定义】

状态	含义	MR 灯 (RYG)	SR 灯 (RYG)
S0	主干道绿灯	001	100
S1	主干道黄灯	010	100
S2	支路绿灯	100	001
S3	支路黄灯	100	010

【VHDL 核心】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity traffic_light is
6      port(clk, rst : in std_logic;
7           MR_light, SR_light : out std_logic_vector(2 downto 0));
8  end traffic_light;
9
10 architecture moore_fsm of traffic_light is
11     type state_type is (MR_GREEN, MR_YELLOW, SR_GREEN, SR_YELLOW);
12     signal curr, nxt : state_type;
13     signal timer : integer range 0 to 29;
14     constant T_GREEN : integer := 29;  -- 30秒(0-29)
15     constant T_YELLOW : integer := 4;  -- 5秒(0-4)
16     constant T_SGREEN : integer := 19; -- 20秒
17     constant T_SYELLOW : integer := 4;
18 begin
19     reg_proc: process(clk, rst)
20     begin
21         if rst='1' then curr<=MR_GREEN; timer<=0;

```

```

22     elsif rising_edge(clk) then
23         curr<=nxt;
24         if (curr=MR_GREEN and timer=T_GREEN) or
25             (curr=MR_YELLOW and timer=T_YELLOW) or
26             (curr=SR_GREEN and timer=T_SGREEN) or
27             (curr=SR_YELLOW and timer=T_SYELLOW) then
28             timer<=0;
29             else timer<=timer+1; end if;
30         end if;
31     end process;
32
33     comb_proc: process(curr, timer)
34     begin
35         case curr is
36             when MR_GREEN => if timer=T_GREEN then nxt<=MR_YELLOW; else nxt
37                               <=MR_GREEN; end if;
38             when MR_YELLOW => if timer=T_YELLOW then nxt<=SR_GREEN; else
39                               nxt<=MR_YELLOW; end if;
40             when SR_GREEN => if timer=T_SGREEN then nxt<=SR_YELLOW; else
41                               nxt<=SR_GREEN; end if;
42             when SR_YELLOW => if timer=T_SYELLOW then nxt<=MR_GREEN; else
43                               nxt<=SR_YELLOW; end if;
44         end case;
45     end process;
46
47     with curr select MR_light <= "001" when MR_GREEN, "010" when
48         MR_YELLOW, "100" when others;
49     with curr select SR_light <= "100" when MR_GREEN|MR_YELLOW, "001"
50         when SR_GREEN, "010" when SR_YELLOW;
51 end moore_fsm;

```

【评分】(20 分) 4 状态定义 + 灯控表 3 分 + 内部 timer 计时 6 分 + 两段式 VHDL 4 分 + Moore 输出 3 分 + 时序循环正确 4 分。

第十九章 第 19 套试卷——参考答案与 评分标准

19.1 一、选择题答案（18 分）

1. **B** —STD_LOGIC 是 9 值解析类型 (U,X,0,1,Z,W,L,H,-)，支持多驱动和总线；STD_ULOGIC 是 9 值无解析，不能多驱动。
2. **C** —SIGNED 和 UNSIGNED 定义在 NUMERIC_STD 包中，支持算术运算。
3. **A** —WITH-SELECT 是并发语句 (并行 MUX)；WHEN-ELSE 也是并发语句。两者均在 ARCHITECTURE 的 BEGIN 后直接使用。
4. **C** —PROCESS 被触发后，内部语句顺序执行，全部执行完后统一更新信号，然后挂起等待下次触发。
5. **D** —JTAG、Master Serial、Slave Serial、SelectMAP 均为 FPGA 常见配置模式。
6. **A** —FUNCTION 必须返回 (return) 一个值；PROCEDURE 通过参数 (OUT/INOUT) 返回结果，不返回值。

19.2 二、填空题答案（12 分）

1. STD_LOGIC 9 值：U, X, 0, 1, Z, W, L, H, -
2. FUNCTION 必须 **return** 值；PROCEDURE 通过 **OUT/INOUT** 参数返回
3. PACKAGE 由**声明**和 **PACKAGE BODY** 两部分组成
4. 并发语句：WITH-SELECT, WHEN-ELSE, 进程, BLOCK, COMPONENT 实例化等

19.3 三、程序改错题解析 (5 分)

- 错误 1: 异步复位但敏感列表无 clk→ 当 clk 上升沿不会触发进程
- 错误 2: 使用过时库 STD_LOGIC_ARITH/UNSIGNED→ 应改用 NUMERIC_STD
- 错误 3: 组合进程 CASE 无 WHEN OTHERS→ 推断锁存器
- 错误 4: STD_LOGIC_VECTOR 与字符字面量直接比较应使用 STD_LOGIC_VECTOR 字面量
- 错误 5: 组合进程敏感列表不完整 (缺少部分输入)→ 锁存器推断

评分: 每处 1 分, 共 5 分。

19.4 四、程序阅读分析题解析 (5 分)

- (1) (1 分) "101" 序列检测器 (Moore 型)。
- (2) (1 分) 状态 S0(0 位), S1(匹配"1"), S2(匹配"10")。
- (3) (2 分) 输入 101 时: S0→S1(1)→S2(10)→(检测到, din=1 时) 输出 1。
- (4) (1 分) 重叠检测 (可自动复用末尾位)。

19.5 五、综合设计题参考答案 (60 分)

19.5.1 设计题 1: 4 位 ALU (20 分)

【VHDL】

```

1 library IEEE;
2 use IEEE.STD_LOGIC_1164.all;
3 use IEEE.NUMERIC_STD.all;
4
5 entity alu_4bit is
6     port(A, B : in unsigned(3 downto 0);
7           op : in std_logic_vector(2 downto 0);
8           result : out unsigned(3 downto 0);
9           carry, zero : out std_logic);
10 end alu_4bit;
11
12 architecture behav of alu_4bit is
13     signal tmp : unsigned(4 downto 0);

```

```

14 begin
15   process(A, B, op)
16   begin
17     case op is
18       when "000" => tmp <= ('0'&A) + ('0'&B);           -- ADD
19       when "001" => tmp <= ('0'&A) - ('0'&B);           -- SUB
20       when "010" => tmp <= '0' & (A and B);             -- AND
21       when "011" => tmp <= '0' & (A or B);              -- OR
22       when "100" => tmp <= '0' & (A xor B);             -- XOR
23       when "101" => tmp <= '0' & (not A);               -- NOT
24       when others => tmp <= (others=>'0');
25     end case;
26   end process;
27   result <= tmp(3 downto 0);
28   carry <= tmp(4);
29   zero <= '1' when tmp(3 downto 0) = 0 else '0';
30 end behav;

```

【评分】(20 分) 6 种操作 CASE 8 分 + 进位扩展 (5 位)4 分 + zero 标志 3 分 + NUMERIC_STD 3 分 + 端口 1 分 + 规范 1 分。

19.5.2 设计题 2：可编程分频器 (20 分)

【VHDL】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity prog_divider is
6    generic(DIV : integer := 10);
7    port(clk, rst, en : in std_logic;
8          clk_out : out std_logic);
9  end prog_divider;
10
11 architecture behav of prog_divider is
12   signal cnt : integer range 0 to DIV-1;
13 begin
14   process(clk, rst)
15   begin

```

```

16     if rst='1' then cnt<=0;
17     elsif rising_edge(clk) then
18         if en='1' then
19             if cnt=DIV-1 then cnt<=0; else cnt<=cnt+1; end if;
20             end if;
21         end if;
22     end process;
23     clk_out <= '1' when cnt >= DIV/2 else '0';  -- 50%占空比(偶数DIV)
24 end behav;

```

【评分】(20 分) GENERIC 参数化 5 分 + 模 DIV 计数 5 分 + 50% 占空比 4 分 + en 使能 3 分 + 计数器位宽合理 3 分。

19.5.3 设计题 3：电梯控制器（20 分）

【分析】3 层电梯 Moore 型 FSM。当前楼层 (current_floor) 编码：01,10,11。输入：目标楼层按钮。输出：up/down/stop 电机方向。

【VHDL 核心】

```

1  library IEEE;
2  use IEEE.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.all;
4
5  entity elevator is
6      port(clk, rst : in std_logic;
7           f1, f2, f3 : in std_logic;
8           current_floor : out std_logic_vector(1 downto 0);
9           motor : out std_logic_vector(1 downto 0));
10 end elevator;
11
12 architecture moore_fsm of elevator is
13     type floor_type is (F1, F2, F3);
14     signal curr, target : floor_type;
15     signal moving : boolean := false;
16 begin
17     process(clk, rst)
18     begin
19         if rst='1' then curr<=F1; target<=F1; moving<=false;
20         elsif rising_edge(clk) then
21             if f1='1' then target<=F1; moving<=true;

```



```

22     elsif f2='1' then target<=F2; moving<=true;
23     elsif f3='1' then target<=F3; moving<=true; end if;
24     if moving then
25         if curr=target then moving<=false;
26         elsif curr<target then
27             if curr=F1 then curr<=F2; else curr<=F3; end if;
28         else
29             if curr=F3 then curr<=F2; else curr<=F1; end if;
30         end if;
31     end if;
32 end if;
33 end process;
34
35 with curr select current_floor <= "01" when F1, "10" when F2, "11"
    when F3;
36 motor <= "01" when (moving and curr<target) else      -- up
37         "10" when (moving and curr>target) else      -- down
38         "00";                                         -- stop
39 end moore_fsm;

```

【评分】(20 分) 3 状态定义 + 楼层编码 3 分 + 目标楼层锁存 4 分 + 移动逻辑 5 分 + 电机输出 (互斥)4 分 + 两段式 + Moore 3 分 + 代码完整 1 分。

第二十章 第 20 套试卷——参考答案与评分标准

20.1 一、选择题答案（18 分）

1. **C** —FPGA 开发流程：设计输入 → 综合 → 布局布线 → 生成比特流 → 下载配置。
2. **B** — n 输入 LUT 可实现任意 n 变量函数 (真值表直接映射到 SRAM)。
3. **A** —结构化描述使用 COMPONENT+PORT MAP 实例化底层模块；行为描述用 PROCESS。
4. **D** —Testbench 不需要端口 (空 ENTITY)，内部实例化 DUT 并产生激励。
5. **B** —FOR GENERATE 循环生成相同结构；IF GENERATE 条件生成不同结构 (类似 if-else)。
6. **C** —ATTRIBUTE 可用于指定状态编码 (如 enum_encoding)、信号约束等。

20.2 二、填空题答案（12 分）

1. (1) **CONFIGURATION** 用于为 ENTITY 选择 ARCHITECTURE,

